

9

Actas

En este capítulo, analizaremos en profundidad las transacciones ACID multidocumento de MongoDB, una función que amplió drásticamente las capacidades de MongoDB en su lanzamiento. Las transacciones son una herramienta importante en su base de datos, pero al igual que no usaría un destornillador para clavar un clavo, es fundamental usarlas solo cuando sean la herramienta adecuada. Si bien sus capacidades son potentes, las transacciones pueden generar cuellos de botella en el rendimiento si se usan incorrectamente.

Quizás le sorprenda saber que las transacciones en MongoDB no siempre son la solución que necesita. De hecho, ¡a veces son justo lo contrario de lo que debería usar! A lo largo de este capítulo, comprenderá a fondo las capacidades transaccionales de MongoDB, que van más allá de la implementación básica.

Aprenderá a equilibrar la necesidad de consistencia de los datos con los requisitos de rendimiento, tomando decisiones informadas sobre cuándo las transacciones son realmente necesarias. Exploraremos ejemplos prácticos que muestran las API principales y de devolución de llamadas, analizaremos las implicaciones de rendimiento de diferentes patrones de transacción y le proporcionaremos estrategias para optimizar sus cargas de trabajo transaccionales. Al finalizar este capítulo, estará capacitado para implementar transacciones que

Mantener la integridad de los datos sin sacrificar las ventajas de rendimiento que hacen de MongoDB una plataforma de base de datos tan poderosa.

Este capítulo cubrirá los siguientes temas:

- Descubra cómo MongoDB admite transacciones ACID de múltiples documentos
- Utilice la API de transacciones y administre sesiones de manera efectiva
- Optimice el rendimiento de las transacciones a través de las mejores prácticas
- Identificar y evitar antipatrones de transacciones comunes

Comprensión de múltiples documentos Transacciones ACID

Profundicemos en las transacciones ACID multidocumento de MongoDB, una capacidad que marcó un hito importante en la evolución de MongoDB, desde una base de datos básica orientada a documentos hasta una plataforma capaz de gestionar cargas de trabajo complejas y críticas. Si bien MongoDB siempre ha proporcionado una potente atomicidad a nivel de documento, las transacciones multidocumento amplían estas garantías a múltiples documentos, colecciones y bases de datos.

Si bien muchas aplicaciones pueden utilizar una estructura de documento enriquecida para garantizar que las escrituras relacionadas se realicen juntas, aún existen casos de uso que requieren la coordinación de escrituras entre múltiples documentos diferentes que pueden estar en la misma colección o en colecciones diferentes. Por eso, es importante comprender qué garantizan las transacciones multidocumento ACID de MongoDB y cómo utilizarlas mejor.

En esta sección, exploraremos primero la historia y la evolución del soporte de transacciones en MongoDB, desde la atomicidad de un solo documento hasta las transacciones distribuidas en clústeres fragmentados. A continuación, analizaremos las propiedades ACID fundamentales, es decir, la atomicidad y la consistencia, aislamiento y durabilidad de y cómo se aplican al modelo de transacciones de MongoDB. Finalmente, compararemos la atomicidad a nivel de documento con las transacciones multidocumento, lo que le ayudará a comprender cuándo es adecuado cada enfoque y cómo equilibrar la integridad de los datos con el rendimiento.

Historia y evolución de las transacciones en MongoDB

La trayectoria de MongoDB con el soporte de transacciones cuenta una historia interesante de cómo la base de datos ha evolucionado para satisfacer requisitos de aplicaciones cada vez más complejos:

- La era del documento único (pre-MongoDB 4.0) : Desde sus inicios, MongoDB proporcionó atomicidad a nivel de documento único. Esto fue suficiente para muchos casos de uso, especialmente al aprovechar el modelo de documento flexible de MongoDB para integrar datos relacionados en un solo documento. Los desarrolladores solían diseñar esquemas específicamente para aprovechar esta atomicidad de documento único.
- El hito del conjunto de réplicas (MongoDB 4.0, 2018) : Esta versión introdujo transacciones ACID de múltiples documentos para conjuntos de réplicas, un cambio de juego que puso a MongoDB a la par con las bases de datos relacionales para aplicaciones que requieren una fuerte consistencia en

Múltiples documentos. Esta implementación inicial admitía transacciones dentro de un único conjunto de réplicas.

- El avance en transacciones distribuidas (MongoDB 4.2, 2019) : Basándose en la versión anterior, MongoDB 4.2 amplió la compatibilidad con transacciones multidocumento a clústeres fragmentados. Este importante avance permitió transacciones distribuidas entre fragmentos, proporcionando una solución integral para aplicaciones complejas a gran escala.
- Mejoras continuas (MongoDB 4.4 y posteriores) : las versiones recientes se han centrado en mejoras de rendimiento, capacidad de gestión operativa y un soporte de API más amplio para actas.

Esta evolución refleja el compromiso de MongoDB de satisfacer diversas necesidades de aplicaciones manteniendo sus puntos fuertes en flexibilidad, escalabilidad y rendimiento.

Introducción a las propiedades ACID en MongoDB. ACID

representa las

propiedades fundamentales que garantizan el procesamiento fiable de las transacciones de la base de datos, incluso ante errores, fallos o cortes de energía. Analicemos el significado de cada una de estas propiedades en el contexto de MongoDB:

- Atomicidad : Esto garantiza que una transacción sea todo o nada; es decir, todas las operaciones de la transacción tienen éxito o ninguna lo tiene. Si alguna parte de la transacción falla, se revierte toda la transacción, dejando la base de datos en su estado original.

Estado. Si bien MongoDB siempre ha proporcionado atomicidad a nivel de documento único (es decir, si cualquier cambio en un documento falla, todos los cambios en ese documento fallan), las transacciones multidocumento extienden esta garantía a múltiples documentos.

- **Consistencia** : Esta propiedad garantiza que una transacción transforme la base de datos de un estado válido a otro, manteniendo todas las reglas y restricciones definidas. En MongoDB, las validaciones de consistencia se realizan principalmente por documento mediante funciones de validación de documentos y esquemas. Tanto las operaciones con un solo documento como las transacciones con varios documentos garantizan la consistencia. Otras validaciones de consistencia, como las restricciones de índice único, también se garantizan de la misma manera dentro y fuera de las transacciones.
- **Aislamiento** : Piense en el aislamiento como si cada transacción operara en su propia burbuja. Garantiza que las transacciones que se ejecutan simultáneamente no interfieran con los resultados de las demás, como si se ejecutaran secuencialmente. MongoDB proporciona aislamiento de instantáneas para las transacciones, lo que significa que cada transacción ve una instantánea consistente de los datos tal como existían al inicio de la transacción, independientemente de los cambios realizados por otras transacciones que se confirmen durante su ejecución. Si dos transacciones intentan realizar una modificación conflictiva, una de ellas fallará y reintentará la transacción completa.
- **Durabilidad** : Una vez confirmada una transacción, sus cambios son permanentes y sobrevivirán a fallos posteriores del sistema, como cortes de energía o caídas. En MongoDB, la durabilidad se logra mediante el registro en diario en múltiples nodos y mecanismos de escritura. Cuando una transacción se confirma con un

Con una preocupación de escritura adecuada (como la predeterminada `{w: "majority"}`), se garantiza la perdurabilidad de los cambios en la mayoría de los miembros del conjunto de réplicas. Esta es una garantía más sólida que la de que un solo nodo confirme una escritura en el disco.

En conjunto, las propiedades ACID sientan las bases para un comportamiento predecible y fiable en las transacciones de MongoDB. Garantizan que las operaciones de varios pasos se completen por completo o no tengan efecto, lo que ayuda a mantener la integridad de los datos incluso ante cargas de trabajo simultáneas, errores de aplicación o fallos del sistema. Esto puede ser especialmente importante para aplicaciones que coordinan cambios en varias colecciones, como el procesamiento de pedidos, la actualización de inventario o la gestión de cuentas de usuario.

Atomicidad a nivel de documento versus transacciones multidocumento

Comprender la diferencia entre la atomicidad a nivel de documento predeterminada de MongoDB y las transacciones de múltiples documentos es crucial para un modelado de datos efectivo y la optimización del rendimiento.

Atomicidad a nivel de documento: MongoDB siempre ha garantizado la atomicidad de las operaciones en un solo documento. Al actualizar varios campos dentro de un documento, se aplican todos los cambios o ninguno; no existe un estado intermedio donde solo se actualicen algunos campos. Además, estos cambios solo son visibles para otros hilos juntos o no son visibles en absoluto. Esta atomicidad integrada, combinada con el rico modelo de documentos de MongoDB.

que permite integrar datos relacionados, significa que muchos casos de uso en realidad no necesitan transacciones de múltiples documentos en absoluto.

Por ejemplo, si está haciendo un seguimiento de un pedido con sus líneas de artículo, incrustar las líneas de artículo dentro del documento del pedido garantiza que las actualizaciones tanto del estado del pedido como de sus artículos se realicen de manera automática:

```
{
  "_id": "order123",
  "customer": "jane_doe",
  "status": "enviado",
  "items":
    [ { "product": "widget", "quantity": 5, "price": 10 }, { "product":
      "gadget", "quantity": 1, "price": 20 }
    ]
}
```

Este enfoque mantiene los datos estrechamente vinculados y garantiza la coherencia sin la sobrecarga de coordinar cambios entre múltiples documentos.

Atomicidad de transacciones multidocumentales

Las transacciones multidocumento amplían las garantías de atomicidad en operaciones que involucran múltiples documentos, que pueden abarcar diferentes colecciones, bases de datos o fragmentos. Las transacciones multidocumento son esenciales cuando los datos relacionados no pueden integrarse razonablemente en un solo documento. Por ejemplo, procesar un pedido de comercio electrónico puede implicar la actualización conjunta de los niveles de inventario, el historial de clientes y los registros de pedidos para mantener la integridad de los datos.

```
// Comenzar a sesión y transacción
const sesión = cliente.startSession();
sesión.startTransaction();
intentar {
  // 1. Crear e insertar el documento de pedido
  orden constante = {
    ID de cliente: "cliente123",
    elementos: [
      { productId: "product456", cantidad: 2, precio: 25,9
      { productId: "product789", cantidad: 1, precio: 49,9
    ],
    ImporteTotal: 101,97
    estado: "procesando",
    createdAt: nueva fecha()
  };

  const orderResult = await db.orders.insertOne(orden, {
    constante orderId = orderResult.insertedId;

  // 2. Actualizar el inventario de varios productos
  esperar db.inventory.updateOne(
    { _id: "producto456" },
    { $inc: { stockCount: -2 } },
    { sesión }
  );

  esperar db.inventory.updateOne(
    { _id: "producto789" },
    { $inc: { stockCount: -1 } },
    { sesión }
  );

  // 3. Actualizar el historial de compras del cliente
  esperar db.clientes.updateOne(
    { _id: "cliente123" },
    {
      $inc: { gasto total: 101,97, número de pedidos: 1 }
    },
  );
```

```

    { sesión }
  );

  Si las operaciones tienen éxito, // todas intento de realizar la transacción confirmar
  esperar sesión.commitTransaction();
} captura (error) {
  un Si // Se produjo un error, abortar el transacción
  esperar sesión.abortTransaction();
  console.error("Transacción cancelada debido a un error:", erro
  arrojar error;
} finalmente {
  // Poner fin a la sesión
  sesión.endSession();
}

```

La compensación clave que hay que entender es que, si bien los documentos múltiples...

Las transacciones ofrecen garantías de consistencia más sólidas, por lo general

conllevan una mayor sobrecarga de rendimiento en comparación con las operaciones

con un solo documento. Como regla general, aproveche las operaciones con un solo documento.

Atomicidad cuando sea posible y utilizar transacciones multidocumento

con criterio, cuando sea verdaderamente necesario.

Cuándo utilizar varios documentos transacciones en MongoDB

Si bien MongoDB admite transacciones de múltiples documentos, eso no significa que...

significa que deberías usarlos por defecto. Las transacciones vienen con

compensaciones de rendimiento y, en muchos casos, las características nativas de MongoDB

El diseño ofrece alternativas mejores y más eficientes. Entonces, ¿cuándo es bueno?

¿Idea para usar transacciones multidocumentales? ¿Y cuándo es mejor?

¿sin ellos?

Veamos algunos escenarios en los que se realizan transacciones con múltiples documentos.

No sólo útil sino necesario:

- Las operaciones financieras suelen requerir una consistencia estricta para evitar discrepancias que puedan provocar errores graves. Por ejemplo, al registrar un pago, es fundamental actualizar tanto el cobro como el documento del cliente correspondiente para deducir los fondos adeudados. Si una actualización se realiza correctamente y la otra falla, el sistema podría mostrar saldos incorrectos o registros incompletos, resultados que las transacciones multidocumento están diseñadas para prevenir.
- La gestión de inventario y pedidos suele implicar la actualización simultánea de los niveles de existencias de varios productos al crear un pedido. Es crucial que todas estas actualizaciones se realicen correctamente o no como una sola unidad para evitar sobreventas o inconsistencias en el inventario. Por ejemplo, si el pedido se registra, pero las actualizaciones de inventario fallan, el sistema puede mostrar una disponibilidad de existencias incorrecta, lo que genera insatisfacción del cliente y problemas logísticos. Las transacciones multidocumento garantizan la creación del pedido y todos los procesos relacionados.

Los ajustes de inventario se aplican de forma atómica, manteniendo la integridad de los datos durante todo el proceso.

- Al gestionar las actualizaciones de versiones, es importante indicar claramente qué versión de una entidad está activa actualmente y, al mismo tiempo, marcar la versión anterior como inactiva. Esto garantiza que nunca haya ambigüedad sobre qué versión debe usar la aplicación o los usuarios. Si estos cambios se aplican por separado y una actualización falla, podría generar datos contradictorios, como varias versiones activas o ninguna, lo que causa errores o un comportamiento incoherente. Las transacciones multidocumento permiten ambas cosas.

Las actualizaciones se realizarán de forma atómica, manteniendo una información clara y confiable.
Estado de la versión.

Sin embargo, no todos los procesos de varios pasos requieren una transacción.

El modelo de documentos de MongoDB se diseñó desde cero pensando en el rendimiento, lo cual se hace más evidente al comparar las operaciones con un solo documento con las transacciones con varios documentos. La diferencia de rendimiento entre ambas existe porque las transacciones requieren un bloqueo adicional para garantizar el aislamiento, el mantenimiento de instantáneas de datos, la coordinación entre múltiples servidores (en clústeres fragmentados) y un protocolo de confirmación en dos fases (en clústeres fragmentados).

Cuando el rendimiento sea importante, siga esta regla : si puede diseñar su modelo de datos para mantener los datos relacionados en un solo documento, casi siempre obtendrá un mejor rendimiento que utilizando transacciones multidocumento. Por lo tanto, considere evitar las transacciones multidocumento en los siguientes casos:

- Los datos relacionados se pueden modelar dentro de un solo documento : Si toda la información que debe actualizarse de forma atómica cabe en un solo documento, resulta más eficiente aprovechar la atomicidad de documento único integrada en MongoDB. Este enfoque reduce la complejidad y la sobrecarga asociadas a las transacciones multidocumento, lo que resulta en un mejor rendimiento y una gestión de errores más sencilla. El uso de la atomicidad de documento único también evita los costes adicionales de bloqueo y coordinación que conllevan las transacciones que abarcan varios documentos o colecciones.
- Se acepta una consistencia más débil : no todas las aplicaciones requieren garantías estrictas de consistencia. En algunos casos, permitir...

Una consistencia relajada puede mejorar el rendimiento y la escalabilidad al reducir la sobrecarga de bloqueo y coordinación. Por ejemplo, los paneles de análisis o las redes sociales pueden tolerar pequeños retrasos o discrepancias temporales en los datos sin afectar la experiencia del usuario. Elegir una consistencia más débil cuando sea apropiado puede ayudar a optimizar el uso de recursos y la capacidad de respuesta del sistema, evitando el costo adicional de transacciones con múltiples documentos cuando no son necesarias.

- Operaciones de escritura a gran escala : Las transacciones tienen límites prácticos en la cantidad de documentos que se pueden modificar. Por ejemplo, no utilice transacciones para cargar miles de documentos.

Generar informes o paneles de análisis. Esto genera una sobrecarga innecesaria y puede provocar errores de tiempo de espera. En su lugar, utilice lecturas instantáneas con la opción `{readConcern: "snapshot"}` para lograr consistencia en un momento dado, o cree vistas de MongoDB que presenten una perspectiva consistente de sus datos sin la sobrecarga de las transacciones.

Recuerde que un diseño de esquema eficaz a menudo puede eliminar por completo la necesidad de transacciones. Antes de implementar una transacción, pregúntese: "¿ Es absolutamente necesaria la consistencia transaccional? " y "¿ Podría modelar estos datos de forma diferente para aprovechar la atomicidad de un solo documento? " .

API de transacciones y gestión de sesiones

MongoDB ofrece dos enfoques para trabajar con transacciones:

La API principal y la API de devolución de llamada. Cada una tiene su lugar, dependiendo de sus requisitos y estilo de codificación.

API principal

La API principal le brinda control explícito sobre cada paso del ciclo de vida de la transacción. Eres responsable de iniciar la sesión.

iniciar la transacción, confirmarla o abortarla y finalizarla

sesión. Este es un patrón típico que inicia una sesión, inicia una transacción, realiza una serie de operaciones e intenta confirmar la transacción. Tiene que gestionar el éxito o el fracaso de la misma.

comprometerse, y si se debe realizar otro intento, debe hacerse explícitamente. escrito en el código, como en lo siguiente:

```
// Comenzar a sesión
const sesión = cliente.startSession();
// Comenzar a transacción
sesión.startTransaction();
intentar {
  // Realizar operaciones dentro de la transacción
  await collection1.updateOne({ _id: 1 }, { $set: { estado
  esperar colección2.insertOne({ refId: 1, detalles: "..."}

  // el Confirmar transacción
  esperar sesión.commitTransaction();
} captura (error) {
  // Si un Se produjo un error, cancele la transacción
  esperar sesión.abortTransaction();
  arrojar error;
} finalmente {
  // Finalizar la sesión
```

i dS i ()

```
sesión.endSession();  
}
```

Repasemos cada parte para entender qué está pasando:

1. Creación de sesión : primero, creamos una nueva sesión con `Se transacciones client.startSession()` requiere una sesión para todas las en MongoDB.
2. Inicialización de transacción : Iniciamos una nueva transacción dentro la sesión utilizando `session.startTransaction()` .
3. Operaciones de base de datos : Dentro del bloque `try` , realizamos Múltiples operaciones (actualizar una colección e insertar en otra). Cada operación incluye el objeto de sesión para asociarlo con nuestra transacción.
4. Confirmación de transacción : si todas las operaciones tienen éxito, confirmamos la transacción. transacción con `session.commitTransaction()` , que hace visibles todos los cambios.
5. Manejo de errores : Si ocurre algún error durante las operaciones o confirmamos, lo capturamos, abortamos la transacción con `session.abortTransaction()` , y vuelva a lanzar el error.
6. Limpieza : el bloque `finally` garantiza que, independientemente del éxito o el fracaso, siempre finalizamos la sesión con `session.endSession()` para liberar recursos.

El uso de la API principal le brinda control total, pero también significa que usted es responsable de administrar cada detalle del ciclo de vida de la transacción.

Si bien este enfoque es poderoso, requiere un manejo cuidadoso de errores y limpieza para garantizar la confiabilidad y la consistencia.

API de devolución de llamada

La API de devolución de llamada (también conocida como API conveniente) simplifica la gestión de transacciones mediante el manejo de gran parte del código repetitivo para ti. En lugar de tener que escribir explícitamente todos los pasos, cada uno de los controladores también proporciona una forma menos detallada de hacer lo mismo. como gestionar automáticamente los reintentos en algunos errores. Para obtener una mejor idea, veamos un ejemplo:

```
// Usando el Llamado de vuelta API
esperar cliente.withSession(async (sesión) => {
  devolver esperar sesión.withTransaction(async () => {
    // Realizar operaciones dentro de la transacción
    esperar colección1.updateOne({ _id: 1 }, { $set: { sta
    esperar colección2.insertOne({ refId: 1, detalles: "...
  });
});
```

Este ejemplo demuestra las transacciones de MongoDB utilizando la API de devolución de llamada, que ofrece varias ventajas sobre la API principal:

- Gestión de sesiones simplificada : `client.withSession()`
El método crea automáticamente una sesión y garantiza que se ejecute correctamente. cerrado cuando se completa la operación, independientemente del éxito o falla.
- Manejo automatizado de transacciones : El
El método `session.withTransaction()` administra toda la sesión.
Ciclo de vida de la transacción. Inicia la transacción, intenta confirmarlo cuando se complete su devolución de llamada y automáticamente lo aborta si ocurre un error.

- Lógica de reintento incorporada : a diferencia del ejemplo de API principal, la API de devolución de llamada incluye lógica de reintento automática para errores transitorios, siguiendo el patrón de reintento recomendado por MongoDB.
- Código más limpio : la estructura de devolución de llamada anidada elimina la necesidad para bloques `try / catch / finally` explícitos , lo que hace que su código sea más conciso y centrado en la lógica de negocios en lugar de la gestión de transacciones.

Para la mayoría de las aplicaciones, se recomienda la API de devolución de llamada debido a su simplicidad, reintentos integrados y gestión de errores. Sin embargo, la API principal podría ser preferible en situaciones que requieren un control muy específico sobre el comportamiento de las transacciones o cuando se necesita intercalar operaciones transaccionales y no transaccionales de forma compleja.

Preocupaciones de lectura/escritura y comportamiento de las transacciones

Las preocupaciones de lectura y escritura controlan las garantías de durabilidad y consistencia de sus operaciones, incluyendo la obsolescencia y durabilidad de los datos leídos. Sin embargo, en las transacciones, las preocupaciones de lectura y escritura tienen comportamientos especiales.

Las transacciones en MongoDB emplean un comportamiento de lectura mayoritaria especulativa para garantizar una semántica de lectura propia, lo que significa que todas o la mayoría de las transacciones de lectura se comportan de la misma manera: leen los últimos datos disponibles en la instantánea y la capacidad exitosa de la transacción para confirmar significa que los datos estarán confirmados mayoritariamente para cuando la transacción se complete exitosamente.

Una inquietud de escritura determina el nivel de reconocimiento solicitado de MongoDB para operaciones de escritura. Analizamos las distintas opciones. en [Capítulo 5](#), Replicación, lo pero para las transacciones, lo importante que hay que saber es que la preocupación de escritura predeterminada, la mayoría, es la El único que tiene sentido utilizar y se aplica a la transacción como un todo y no escrituras individuales dentro de una transacción.



Preocupaciones sobre lectura y escritura a nivel de transacción

Las preocupaciones de lectura y escritura se establecen en la transacción. nivel, no en operaciones individuales dentro de la transacción, por lo que no puede especificar una lectura diferente o Escribir preocupación por operaciones individuales en una transacción.

Para establecer una preocupación de lectura o escritura en una transacción, pase los valores a la Método `startTransaction` :

```
// Ejemplo que muestra la correcta lectura/escritura de transacciones
sesión.startTransaction({
  readConcern: { nivel: "instantánea" },           // Por defecto
  writeConcern: { w: "mayoría" } });              // Por defecto
```

Como regla general, es mejor dejar los valores predeterminados para lectura y Escriba sus inquietudes sobre las transacciones si le preocupa su exactitud y durabilidad.

Consideraciones sobre el rendimiento en las transacciones

Cuando se trata de transacciones de MongoDB, el rendimiento es una consideración importante que puede determinar el éxito o el fracaso de su aplicación. Si bien las transacciones ofrecen sólidas garantías de consistencia, también generan sobrecarga que puede afectar la velocidad, la escalabilidad y el uso de recursos del sistema. En esta sección, examinaremos las consideraciones clave de rendimiento para las transacciones de MongoDB y brindaremos consejos prácticos para optimizar sus cargas de trabajo transaccionales.

Conjunto de réplicas frente a transacciones de clúster fragmentado

Las transacciones se comportan de forma diferente y tienen distintas implicaciones en el rendimiento según se ejecuten en un conjunto de réplicas o en un clúster fragmentado. En los conjuntos de réplicas, el rendimiento de las transacciones es relativamente sencillo. Todas las operaciones de una transacción son coordinadas por el nodo principal. Los principales factores que afectan al rendimiento son la cantidad de documentos afectados, su tamaño y la latencia de red entre los clientes y el nodo principal. Estos elementos influyen directamente en la rapidez con la que se ejecuta y confirma una transacción.

En los clústeres fragmentados, la complejidad aumenta. Las transacciones entre fragmentos, es decir, las que afectan a los datos de varios fragmentos, requieren un coordinador de transacciones distribuidas y un sistema de dos fases.

Protocolo de confirmación. Esto aumenta la sobrecarga y puede afectar el rendimiento de varias maneras:

- Mayor latencia : cada operación entre fragmentos agrega viajes de ida y vuelta a la red entre el coordinador y los fragmentos
- Mayor utilización de recursos : se necesitan memoria y CPU adicionales para la coordinación de transacciones
- Posibilidad de que se produzcan problemas de rendimiento en cascada : si un fragmento se vuelve lento, puede afectar el rendimiento de las transacciones en todo el clúster.

Sin embargo, no todas las transacciones fragmentadas son iguales. Las transacciones que solo interactúan con documentos en un fragmento tienen el mismo rendimiento que las transacciones del conjunto de réplicas. Las transacciones fragmentadas que leen desde varios fragmentos pero escriben solo en uno sufren una pequeña pérdida de rendimiento. Las transacciones más costosas son las que escriben en varios fragmentos. Cuantos más fragmentos se utilizan, mayor es la latencia.

En entornos fragmentados, el rendimiento se puede mejorar significativamente diseñando para la localización de fragmentos. Esto implica estructurar los datos y las claves de fragmento de modo que los documentos relacionados que se actualizan frecuentemente en una transacción residan en el mismo fragmento. Esto minimiza la necesidad de transacciones entre fragmentos.

Además, si su aplicación está distribuida globalmente, es importante tener en cuenta la latencia de las transacciones entre regiones. Para mitigarla, considere distribuir sus colecciones de forma que sus escrituras se limiten a una sola región o explore otras técnicas para equilibrar la consistencia y el rendimiento.

Consideraciones sobre el caché de WiredTiger El motor de almacenamiento WiredTiger de MongoDB utiliza un caché en memoria para mejorar el rendimiento, pero las transacciones interactúan con este caché de formas que a veces pueden causar problemas de rendimiento.

Las transacciones dependen de la caché mediante la creación de instantáneas de los documentos que leen o modifican. Estas instantáneas deben permanecer en la caché hasta que se complete la transacción. Las transacciones de larga duración pueden impedir la expulsión de la caché, lo que genera presión sobre la memoria.

El problema más grave es cuando esto conduce a la

Error `TransactionTooLargeForCache` , que se produce cuando las operaciones e

instantáneas de una transacción superan el tamaño de caché disponible. Esto suele ocurrir en los siguientes casos:

- Una transacción toca demasiados documentos
- Una transacción incluye documentos muy grandes
- Una transacción se ejecuta durante demasiado tiempo, lo que impide la expulsión de la caché
- La caché de WiredTiger está configurada demasiado pequeña para la carga de trabajo

Para prevenir estos escenarios, existen algunas estrategias dependiendo de la situación:

- Limite el tamaño de las transacciones : mantenga la cantidad de documentos por transacción por debajo de 1000
- Limitar la duración de las transacciones : Confirmar o cancelar transacciones dentro de 60 segundos (el límite predeterminado)
- Configurar el tamaño de caché adecuado : asegúrese de que el caché de WiredTiger esté configurado al menos en el 50 % de la RAM para cargas de trabajo con muchas transacciones
- Monitorear la presión de caché : estar atento a un aumento en métricas como `wiredTigerCacheSizeBytes` y `wiredTigerCacheMaxBytes`

Para garantizar un buen rendimiento de las transacciones es necesario utilizar las mejores prácticas, realizar un seguimiento diligente y minimizar el uso de transacciones cuando no sea necesario.

Gestión de errores y límites de tiempo de ejecución de transacciones

MongoDB aplica un límite de tiempo de ejecución de transacciones predeterminado de 60 segundos para evitar que las transacciones de larga duración afecten la salud del clúster. Este límite existe por varias razones importantes: las transacciones extendidas pueden bloquear documentos durante largos períodos, impidiendo que otras operaciones accedan a los datos necesarios y creando cuellos de botella en su aplicación; aumentan la presión de memoria en sus servidores, lo que puede afectar el rendimiento general de la base de datos; impiden que el caché de WiredTiger expulse datos del caché, lo que puede provocar agotamiento de la memoria y un rendimiento degradado; y aumentan significativamente la probabilidad de conflictos y abortos en las operaciones, lo que obliga a su aplicación a manejar más casos de error.

Dadas estas limitaciones, gestionar eficazmente la duración de las transacciones se vuelve crucial para la fiabilidad de la aplicación. Puede supervisar la duración de las transacciones rastreando su duración habitual y configurando alertas para aquellas que se acerquen al límite, lo que le dará tiempo para optimizar antes de que se produzcan fallos. Si bien es posible configurar el límite ajustando el parámetro del servidor `transactionLifetimeLimitSeconds`, aumentarlo debería considerarse un último recurso, ya que no soluciona el problema.

problemas de rendimiento subyacentes y pueden empeorar la salud general del sistema.

Un enfoque más sostenible consiste en optimizar las transacciones para que se completen con mayor rapidez. Esto se puede lograr dividiendo las transacciones grandes en transacciones más pequeñas y manejables que procesen menos documentos simultáneamente; garantizando la existencia de índices para todos los predicados de consulta a fin de agilizar la selección de documentos; optimizando las consultas y los canales de agregación utilizados dentro de la transacción para reducir el tiempo de procesamiento; evitando operaciones innecesarias dentro del ámbito de la transacción; y realizando operaciones de solo lectura fuera de la transacción cuando sea posible, cuando no requieran las garantías ACID que ofrecen las transacciones.

Más allá del desafío del límite de tiempo, los sistemas distribuidos como MongoDB inevitablemente experimentan errores transitorios que requieren un manejo adecuado para aplicaciones robustas. MongoDB ayuda a los desarrolladores a identificar estas situaciones mediante etiquetas de error específicas que sugieren estrategias de recuperación adecuadas. Cuando se encuentra un error, indica un problema `TransientTransactionError`, elecciones , temporal, como fallos de red primarias o limitaciones momentáneas de recursos, que podrían solucionarse si se reintenta. En estos casos, implementar un reintento de transacción completo con retroceso exponencial permite que la aplicación se recupere correctamente y evita la sobrecarga del sistema durante la inestabilidad.

De forma similar, un error `"UnknownTransactionCommitResult"` indica incertidumbre sobre si una confirmación se realizó correctamente, generalmente debido a problemas de red durante la confirmación. La respuesta correcta en este caso difiere ligeramente: debe reintentar solo la operación de confirmación en lugar de toda la transacción, ya que esta ya se realizó y solo la confirmación final es cuestionable.

Es importante destacar que monitorear las tasas de reintentos proporciona información valiosa sobre el estado del sistema. Un aumento repentino en los reintentos suele indicar un problema de rendimiento subyacente, como inestabilidad de la red, limitaciones de recursos o problemas de configuración, que requiere investigación. Al conectar la monitorización de reintentos con las métricas generales de rendimiento, puede identificar y abordar los problemas antes de que afecten significativamente la fiabilidad de su aplicación.

Adquisición de bloqueos y gestión de contenciones

MongoDB utiliza un sistema de bloqueo multigranular para las transacciones. Comprender el comportamiento de los bloqueos es crucial para optimizar el rendimiento de las transacciones. El mecanismo de bloqueo de MongoDB opera con múltiples niveles de granularidad, desde bloqueos a nivel de base de datos hasta bloqueos de documentos individuales, creando un sistema jerárquico que equilibra la consistencia de los datos con el rendimiento operativo.

La contención de bloqueos generalmente surge cuando varias transacciones compiten para modificar el mismo documento, durante operaciones administrativas como la creación de índices o colecciones, o bajo cargas de trabajo de escritura de gran volumen concentradas en un pequeño subconjunto de

Documentos. Una mitigación eficaz requiere un enfoque multifacético centrado en reducir tanto la probabilidad como la duración de los conflictos de bloqueo.

Mantener las transacciones pequeñas y enfocadas sirve como base para minimizar la duración del bloqueo, lo que permite que otras operaciones se realicen con mayor rapidez. En lugar de concentrar las escrituras en datos específicos,

Documentos: distribuir esta actividad ayuda a prevenir la formación de puntos críticos que se convierten en cuellos de botella. Las tareas administrativas deben programarse durante los períodos de menor actividad para evitar competir con las cargas de trabajo habituales de las aplicaciones.

Una indexación adecuada desempeña un papel crucial en la eficiencia operativa, ya que reduce el tiempo de ejecución de las operaciones y, por lo tanto, minimiza el tiempo que estas mantienen bloqueos. La monitorización regular mediante `db.serverStatus().locks` proporciona información valiosa sobre los tiempos de espera de los bloqueos, lo que permite la identificación proactiva de problemas emergentes antes de que afecten al rendimiento de la aplicación.

El reconocimiento de problemas de contención de bloqueos suele manifestarse mediante varios indicadores clave: las tasas de aborto de transacciones y el número de reintentos aumentan a medida que las operaciones no se completan correctamente a la primera, las colas de operaciones se alargan a medida que las solicitudes esperan la disponibilidad de recursos, los tiempos de espera de los bloqueos aumentan considerablemente y los clientes experimentan tiempos de espera agotados cuando las operaciones superan los umbrales de respuesta aceptables. Al comprender estos patrones e implementar las medidas preventivas adecuadas, los administradores de bases de datos pueden mantener un rendimiento óptimo incluso con cargas de trabajo

Optimización del tamaño y la duración de las transacciones

La forma más eficaz de garantizar un buen rendimiento de las transacciones es mantenerlas pequeñas y de corta duración. En esta sección, nos centraremos en cómo optimizar su huella transaccional limitando el tamaño y la duración de las transacciones.

Primero, intente limitar el tamaño de su transacción. Siempre que sea posible, límitela a menos de 1000 documentos modificados por transacción. A continuación, existen algunas estrategias importantes para minimizar la duración de la transacción:

- Preparar datos antes de la transacción : realizar cálculos costosos fuera de la transacción
- Utilice consultas eficientes : asegúrese de que todas las operaciones dentro de las transacciones utilicen índices
- Cambios relacionados con lotes : agrupe múltiples escrituras en una sola operación por lotes cuando sea posible
- Dividir operaciones grandes : divida operaciones muy grandes en transacciones más pequeñas y manejables

Al aplicar estas estrategias de optimización, puede mejorar significativamente el rendimiento de sus cargas de trabajo transaccionales en MongoDB.

Antipatrones de transacciones comunes y sus costos de desempeño

Incluso con las mejores intenciones, es fácil caer en trampas comunes al usar transacciones en MongoDB. Entender cómo no usar transacciones es tan crucial como saber cuándo usarlas. En esta sección, exploraremos los antipatrones más frecuentes, explicaremos por qué afectan el rendimiento y brindaremos consejos prácticos para evitarlos, para que pueda mantener su base de datos funcionando correctamente y a su equipo de operaciones satisfecho.

Transacciones de larga duración y su impacto en el rendimiento del sistema.

Al diseñar transacciones para MongoDB, el rendimiento debe ser una consideración primordial. Las transacciones eficientes garantizan que la base de datos mantenga una alta capacidad de respuesta, un alto rendimiento y una buena experiencia de usuario. Un diseño deficiente de las transacciones, especialmente cuando las operaciones se ejecutan durante demasiado tiempo, puede ejercer una presión excesiva sobre los recursos del sistema, aumentar el riesgo de conflictos y, en última instancia, provocar fallos.

Las transacciones de larga duración son como ese visitante que se queda más tiempo del debido: ocupan recursos, bloquean otras operaciones y pueden causar problemas de rendimiento importantes. Algunos ejemplos de transacciones demasiado largas son los siguientes:

- Una transacción que intenta insertar o actualizar miles o millones de documentos en una sola operación
- Operaciones que incluyen llamadas a API externas, cálculos complejos o interacciones del usuario final que provocan que la transacción permanezca abierta durante muchos segundos o minutos.
- Trabajos de procesamiento por lotes que envuelven un día entero de importaciones de datos en una sola transacción

Las transacciones de larga duración conllevan varios costos de rendimiento.

Estos incluyen lo siguiente:

- Mayor contención de bloqueo : los bloqueos se mantienen durante la duración de la transacción, bloqueando que otras operaciones accedan a los documentos bloqueados y reduciendo significativamente la concurrencia y el rendimiento.

- Presión de caché de WiredTiger : El motor de almacenamiento debe mantener todas las preimágenes y postimágenes de los datos modificados en la caché hasta que se complete la transacción. Las transacciones grandes pueden consumir una cantidad considerable de caché, lo que obliga a expulsar con frecuencia otros datos útiles.
- Riesgo de `TransactionTooLargeForCache` : Las modificaciones superan un cierto umbral del caché de WiredTiger, la transacción fallará por completo.
- Exceder el valor `transactionLifetimeLimitSeconds` : El tiempo determinado de 60 segundos para transacciones de MongoDB existe por una buena razón. Las transacciones de larga duración corren el riesgo de ser canceladas automáticamente, lo que genera trabajo desperdiciado y una gran cantidad de reintentos.
- Retraso en la replicación : las transacciones grandes pueden generar una ráfaga de entradas del registro de operaciones al confirmarse, lo que potencialmente puede provocar que las secundarias se queden atrás.

La clave es dividir las operaciones grandes en transacciones más pequeñas y manejables. Mantenga las transacciones breves y sencillas, idealmente completándolas en milisegundos en lugar de segundos. Este enfoque ayuda a mantener el rendimiento del sistema, reduce la contención de recursos y disminuye la probabilidad de fallos en las transacciones.

Uso innecesario de transacciones en las que la atomicidad de un solo documento sería suficiente

Los desarrolladores nuevos en MongoDB pueden caer en la trampa de usar excesivamente las transacciones, trayendo hábitos de bases de datos relacionales que no son

Siempre es necesario integrarlo en el modelo de documento. Examinemos algunos escenarios comunes donde las transacciones se aplican incorrectamente:

- Usar una transacción para una única actualización de documento : envolver una única actualización de documento en una transacción no es necesario ya que las operaciones de un solo documento ya son atómicas en MongoDB.
Este enfoque redundante agrega sobrecarga de procesamiento sin proporcionar ningún beneficio adicional en términos de integridad de datos.
- Usar transacciones en lugar de un mejor diseño de esquemas : Otro error común es usar transacciones cuando los datos podrían modelarse para que quepan en un solo documento. Dedicar tiempo a diseñar el esquema correctamente puede eliminar muchos requisitos de transacciones y mejorar significativamente el rendimiento.
- División de datos relacionados lógicamente entre colecciones : Los desarrolladores también suelen crear documentos separados para datos que pertenecen lógicamente juntos y luego usan transacciones para actualizarlos automáticamente.
Este enfoque no solo complica el código, sino que también introduce posibles cuellos de botella en el rendimiento y aumenta la probabilidad de conflictos entre transacciones.
- Ignorar la atomicidad de un solo documento : La atomicidad de un solo documento de MongoDB es una característica poderosa; ¡no la ignore! Usar transacciones multidocumento para operaciones que podrían gestionarse con actualizaciones de un solo documento añade sobrecarga y complejidad innecesarias.



Consejo

Cuando sea apropiado, incorpore datos relacionados en un solo documento para reducir la necesidad de varios documentos.



actas.

He aquí un ejemplo que demuestra la diferencia entre un

Enfoque relacional con gran volumen de transacciones y optimizado para documentos.

Modelo MongoDB:

```
// En lugar de colecciones separadas que requieren transacciones
// colección de usuarios: { userId: userId, nombre: "Jane", ... }
// colección de perfiles: { userId: userId, preferencias: { ... } }
// colección de configuraciones: { userId: userId, notificaciones: { ... } }
// Considere el modelo consolidado:
// colección de usuarios
{
  _id: ID de usuario,
  nombre: "Jane",
  perfil: {
    preferencias: { "..."}
  },
  ajustes: {
    notificaciones: { "..."}
  }
}
```

En el primer enfoque, la información del usuario se divide en tres partes separadas:

Colecciones: usuarios, perfiles y configuraciones. Cada vez que necesites

Para actualizar los datos relacionados en estas colecciones, se necesitarían

transacciones multidocumentales para garantizar la coherencia. Este enfoque crea...

complejidad innecesaria y sobrecarga de rendimiento.

El segundo enfoque demuestra un modelo consolidado donde todos

La información relacionada con el usuario está incrustada dentro de un solo documento en

La colección de usuarios. Con este diseño, puedes actualizar el perfil de un usuario.

Preferencias y configuraciones de notificaciones en una sola operación atómica

Sin necesidad de transacciones. Este enfoque simplifica el código, mejora el rendimiento y aprovecha el modelo de documentos de MongoDB tal como fue diseñado.

Transacciones de un solo documento

Realizar múltiples actualizaciones distintas a un solo documento en una transacción casi siempre puede reemplazarse con una única actualización atómica. A menos que haya interacción de terceros entre las dos escrituras, lo cual es problemático debido al posible aumento considerable de la latencia y al riesgo de exceder el límite de tiempo de espera de la transacción, la técnica correcta sería componer una única actualización (utilizando sintaxis de agregación expresiva si es necesario) para realizar todas las modificaciones en una sola operación.

Si es fundamental que ningún otro hilo esté leyendo un documento mientras un hilo está realizando actualizaciones en él, es posible lograrlo sin una transacción colocando un campo de bloqueo lógico en el documento con una actualización y luego, al actualizar los otros campos, desconfigurar ese campo.

Asegúrese de que el resto de operaciones comprueben que no exista ningún bloqueo cuando interactúan con el documento.

Transacciones para operaciones de solo lectura

Usar transacciones multidocumento únicamente para operaciones de lectura supone una sobrecarga innecesaria que puede afectar negativamente al rendimiento. Para garantizar una instantánea consistente para una serie de lecturas en varios documentos, utilice una instantánea de lectura en lugar de iniciar una transacción. Si necesita asegurarse de que las lecturas reflejen sus escrituras anteriores,

Utilice sesiones causalmente consistentes o utilice la lectura linealizable inquietud.

Malinterpretación del alcance y la atomicidad de las transacciones. Un

antipatrón común es incluir

muy poco o demasiado en una transacción, o malinterpretar qué operaciones son verdaderamente atómicas. Otro error común es creer que un solo comando de escritura es atómico incluso cuando afecta a varios documentos. Una operación de actualización con la opción `multi:true` o el método `updateMany` no es atómica en su conjunto; solo lo es cada escritura individual dentro de ella.

Para que una multiactualización sea atómica, debería estar encapsulada en una transacción.

Incluya únicamente operaciones que deban ser atómicas en una transacción. Las operaciones no críticas deben realizarse fuera de la transacción para reducir el alcance y mejorar el rendimiento.

Pequeñas transacciones frecuentes en documentos/cobros importantes

Las transacciones pequeñas suelen ser beneficiosas, pero ejecutarlas con demasiada frecuencia en los mismos documentos puede generar problemas de rendimiento. Esto suele ocurrir cuando distintos servicios actualizan documentos relacionados de forma independiente, como cuando varios microservicios modifican el estado del pedido y el inventario de los mismos productos. También pueden surgir problemas cuando muchas transacciones intentan actualizar un mismo documento popular, como una entrada en la clasificación durante un evento importante de ju

En algunos flujos de trabajo, como los flujos de autenticación de usuarios que actualizan con frecuencia datos de sesión, registros y registros de auditoría, las mismas colecciones se revisan una y otra vez, lo que puede acumularse.

Estas situaciones pueden causar ralentizaciones considerables. Incluso las transacciones rápidas pueden tener que esperar a que se bloqueen los documentos, lo que significa que las operaciones se procesan una a una en lugar de en paralelo. Cuanta más contención haya, mayor será la probabilidad de que se produzcan conflictos de escritura, lo que obligará a las transacciones a reintentarse.

En estos casos, otros enfoques suelen ser más eficaces. En lugar de activar muchas transacciones pequeñas, puede agrupar las actualizaciones en su aplicación y enviarlas como una sola operación. Para escenarios que requieren un alto rendimiento, un sistema basado en colas puede recopilar operaciones y procesarlas en lotes.

Si no puede evitar la contención y necesita actualizar varios documentos en una transacción, intente actualizar primero el documento con más contención. De esta forma, si la transacción necesita reintentarse, se reducirá la repetición del trabajo.

Manejo inadecuado de errores y lógica de reintento

Las transacciones pueden fallar por diversas razones: problemas transitorios de red, contención de bloqueos temporales o condiciones del servidor. Si utiliza directamente la API principal (en lugar de la API de devolución de llamada, que gestiona los reintentos automáticamente), es fundamental implementar una gestión de errores y una lógica de reintento adecuadas en el código de su aplicación.

Un error común es no reintentar una transacción, tras lo cual se un `TransientTransactionError` recuperable con un `SQLException`, suele encontrar simple reintento. Por otro lado, reintentar con demasiada intensidad, sin demora ni interrupción, puede empeorar la situación al aumentar la contención. Otro problema es reintentar indefinidamente sin establecer un número máximo de intentos, lo que puede generar desperdicio de recursos y procesos bloqueados. Los desarrolladores a veces también gestionan incorrectamente `UnknownTransactionCommitResult`, reintentando la transacción completa en lugar de sólo el comando `commitTransaction`.

Este tipo de errores pueden tener graves consecuencias. Los reintentos mal gestionados pueden sobrecargar el servidor durante periodos de alto tráfico. Además, si la lógica de reintento no es idempotente o no tiene el alcance adecuado, la misma operación podría aplicarse más de una vez.

Para evitar estos problemas, la mejor estrategia es usar la API de devolución de llamadas, que reintentará las transacciones automáticamente y de forma segura. Si usa la API principal, asegúrese de implementar una lógica de reintento con retardo y fluctuación exponenciales para evitar saturar el sistema y, al mismo tiempo, permitir que los fallos recuperables se resuelvan en un intento posterior.

Monitoreo insuficiente de las métricas de transacciones

Como dice el dicho: “ Si no lo mides , no puedes mejorarlo ” .

No supervisar el rendimiento de las transacciones es una receta para que los problemas invisibles se conviertan en interrupciones de producción.

Para estar al tanto de la salud de las transacciones, es importante realizar un seguimiento de métricas clave, como la cantidad de transacciones activas y el total.

Transacciones iniciadas, confirmadas y canceladas. Monitorear la latencia de las transacciones, es decir, cuánto tardan en completarse, ayuda a identificar ralentizaciones. Además, supervisar la presión de caché y las tasas de desalojo de WiredTiger proporciona información sobre la utilización de la caché.

Las métricas relacionadas con los bloqueos, como los tiempos de espera y los recuentos de adquisiciones, revelan si las transacciones se retrasan debido a la contención. Finalmente, el seguimiento de las tasas de error, específicamente la frecuencia con la que las transacciones se cancelan o requieren reintentos en comparación con el total de transacciones, ayuda a identificar la inestabilidad.

Para una monitorización eficaz, configure paneles en MongoDB Atlas o en su sistema preferido para visualizar estas métricas. Cree alertas para tendencias inusuales, como picos en las tasas de aborto o el aumento de la duración de las transacciones. Registrar los detalles de ejecución de las transacciones junto con las métricas de la base de datos puede ayudar a correlacionar problemas. La revisión periódica de estos patrones de transacciones permite una optimización continua y la detección temprana de posibles problemas.

Resumen

Las transacciones en MongoDB ofrecen sólidas garantías de consistencia, pero conllevan desventajas que deben considerarse cuidadosamente. Siguiendo las prácticas recomendadas descritas en este capítulo, podrá usar las transacciones eficazmente con un impacto mínimo en el rendimiento, siempre que recuerde algunos principios clave. Aproveche la atomicidad de un solo documento en lugar de las transacciones cuando su modelo de datos lo permita. Mantenga las transacciones pequeñas, enfocadas y de corta duración. Monitoree y optimice.

Continuamente a medida que su aplicación evoluciona. Y, por supuesto, siempre pruebe el comportamiento de las transacciones en condiciones de carga realistas antes de...

Despliegue de producción.

Las aplicaciones MongoDB más exitosas utilizan las transacciones de manera juiciosa: como una herramienta en una estrategia más amplia de consistencia de datos que cumple con los requisitos de su aplicación y al mismo tiempo mantiene el rendimiento y la escalabilidad que sus usuarios esperan.

Ahora que comprende bien las transacciones de MongoDB y su función para mantener la integridad de los datos en múltiples operaciones, en el siguiente capítulo, abordaremos las bibliotecas cliente. Estas son las que su aplicación utiliza para interactuar con MongoDB.

10

Bibliotecas de clientes

Las bibliotecas cliente de MongoDB, que incluyen controladores de lenguaje oficiales y mapeadores de objetos y documentos (ODM), sirven como interfaz entre los desarrolladores y la base de datos. Estas bibliotecas abstraen las interacciones de bajo nivel con la base de datos, proporcionando una experiencia idiomática y de alto rendimiento adaptada a cada lenguaje de programación. Al encapsular la complejidad de las comunicaciones de red, la optimización de consultas y la gestión de conexiones, las bibliotecas cliente permiten centrarse en escribir código de aplicación claro y fácil de mantener, en lugar de lidiar con los detalles de la implementación de la base de datos.

Este capítulo se centra en comprender, configurar y optimizar las bibliotecas cliente de MongoDB para obtener el máximo rendimiento y fiabilidad. Exploraremos las funciones de los controladores y los ODM, profundizaremos en las mejores prácticas de configuración y mostraremos cómo aprovechar sus funciones eficazmente para crear aplicaciones robustas y escalables que aprovechen al máximo las capacidades de MongoDB.

Los temas clave tratados en este capítulo incluyen los siguientes:

- Definir y explicar el papel esencial de los controladores MongoDB en la conexión de aplicaciones a la base de datos
- Características clave de los controladores MongoDB, incluida la consistencia a través de especificaciones compartidas y experiencias idiomáticas, así como

como técnicas de optimización del rendimiento, como la gestión de conexiones y la eficiencia de las consultas.

- Funciones avanzadas de los controladores, como manejo de errores, problemas de lectura y escritura y capacidades de monitoreo
- Los beneficios de los ODM en la simplificación del desarrollo de aplicaciones
- Una descripción general de los ODM, incluida la aplicación de esquemas y las consultas API y gestión de relaciones
- Temas avanzados como patrones asincrónicos y optimización del rendimiento de la red con bibliotecas de cliente

¿Qué son los controladores?

Los controladores de MongoDB son bibliotecas de software que facilitan la comunicación entre el código de su aplicación y los clústeres de MongoDB. En lugar de trabajar directamente con protocolos de red de bajo nivel, serialización de JSON binario (BSON) o detalles del protocolo de conexión, puede usar estos controladores para interactuar con MongoDB mediante API intuitivas y específicas del lenguaje. Los controladores traducen operaciones de alto nivel en comandos de base de datos, lo que le permite realizar operaciones de creación, lectura, actualización y eliminación (CRUD), administrar índices, ejecutar canalizaciones de agregación e implementar otras funciones de MongoDB sin necesidad de escribir comandos de base de datos sin procesar.

MongoDB proporciona controladores oficiales para los lenguajes de programación más populares, como C#, Go, Java, Node.js, PHP, Python, Ruby y Rust. Cada controlador cumple con un conjunto compartido de especificaciones, que se encuentra en el repositorio de especificaciones de MongoDB en <https://github.com/mongodb/specifications>.

Garantizando la consistencia y la paridad de características, a la vez que se conservan los modismos específicos del lenguaje, como las convenciones de mayúsculas y minúsculas en los nombres de métodos. Este enfoque estructurado ofrece fiabilidad en diversas tecnologías, a la vez que resulta natural para los desarrolladores que trabajan en su lenguaje preferido.

El uso eficaz de las bibliotecas cliente de MongoDB impacta directamente en el rendimiento de la aplicación. Los controladores correctamente configurados pueden minimizar la latencia, maximizar el rendimiento y ayudar a mantener la escalabilidad de la aplicación. Comprender la agrupación de conexiones, la reintento, las preferencias de lectura y las cuestiones de lectura/escritura permite ajustar las operaciones de la base de datos a cargas de trabajo específicas y requisitos de confiabilidad.

A medida que profundizamos en este capítulo, exploraremos el funcionamiento interno de estas bibliotecas, examinaremos las mejores prácticas de configuración y demostraremos cómo aprovechar sus capacidades en situaciones reales. Tanto si desarrolla una aplicación web sencilla como un sistema distribuido complejo, comprender las bibliotecas cliente de MongoDB es esencial para crear aplicaciones eficientes y robustas que aprovechen al máximo las capacidades de la base de datos.

Cómo funcionan los controladores de MongoDB

Cuando utiliza un controlador MongoDB, está aprovechando una interfaz cuidadosamente diseñada que maneja tareas complejas como las siguientes:

- Optimización del acceso a la base de datos : Los controladores gestionan los grupos de conexiones para reutilizar eficientemente las conexiones de la base de datos.

En lugar de crear nuevas conexiones para cada operación, el controlador mantiene un grupo de conexiones activas, lo que reduce la latencia y mejora la seguridad.

Rendimiento. Esto incluye establecer, mantener y cerrar conexiones según sea necesario, y a menudo permite configurar el tamaño del grupo y los parámetros de tiempo de espera de la conexión.

- Serialización y deserialización de datos : Los controladores de MongoDB gestionan la serialización de los datos de la aplicación en formato BSON antes de enviarlos a la base de datos y, posteriormente, deserializan las respuestas BSON en estructuras de datos nativas. Esta conversión automática...

garantiza que los datos se transmitan e interpreten correctamente entre su aplicación y el servidor MongoDB.

- Lógica de reintento automática : Para garantizar la confiabilidad, los controladores gestionan automáticamente la lógica de reintento ante fallos de red. Si una operación de base de datos falla debido a un problema temporal de red, el controlador puede reintentar la operación, a menudo basándose en políticas de reintento configurables, lo que garantiza la integridad de los datos y la estabilidad de la aplicación.
- Enrutamiento automático de operaciones : En los conjuntos de réplicas, los controladores enrutan las operaciones de forma inteligente a los servidores correspondientes. Los controladores conocen la topología del clúster y pueden dirigir las operaciones de lectura y escritura a los nodos correctos, lo que incluye la gestión de conmutaciones por error y el enrutamiento de lecturas a servidores secundarios mediante la configuración de preferencias de lectura.
- Monitoreo de cambios en el servidor : Los controladores monitorean continuamente los cambios en la topología del servidor dentro de los conjuntos de réplicas. Si un servidor falla o se agrega uno nuevo, el controlador actualiza su vista del clúster y ajusta las decisiones de enrutamiento según corresponda. Esta adaptación dinámica garantiza la continuidad de las aplicaciones y el correcto funcionamiento en entornos cambiantes.
- Transacciones ACID de múltiples documentos : proporcionan API para iniciar, administrar y confirmar o cancelar transacciones ACID de múltiples documentos.

transacciones, asegurando la atomicidad y la consistencia en múltiples operaciones (cubiertas con más detalle en). [Capítulo 9](#), Actas

- Flujos de cambios para notificaciones de cambios de datos en tiempo real : permitir que las aplicaciones se suscriban a flujos de cambios, lo que permite notificaciones en tiempo real de modificaciones, eliminaciones o inserciones de datos, lo cual es esencial para construir arquitecturas basadas en eventos (cubierto con más detalle en , [Flujos de cambio](#)).
- Tuberías de agregación para el procesamiento de datos complejos : Las aplicaciones pueden definir y ejecutar agregaciones sofisticadas tuberías que transforman, filtran y agrupan datos dentro MongoDB, que permite análisis complejos (cubierto con más detalle) en [Capítulo 4](#), Agregaciones).
- GridFS para almacenar y recuperar archivos grandes : Controladores Implementar la especificación GridFS, lo que permite que las aplicaciones almacenar y recuperar archivos grandes dividiéndolos en fragmentos, Superar las limitaciones de tamaño de los documentos.
- Cifrado a nivel de campo del lado del cliente para mayor seguridad : Permiten que las aplicaciones encripten campos específicos antes de... se envían a la base de datos, protegiendo también los datos confidenciales en reposo como en vuelo.
- Búsqueda Atlas y Búsqueda Vectorial : Los controladores exponen convenientemente API para permitir que las aplicaciones aprovechen Atlas Search y Vector Búsqueda, que proporciona una robusta búsqueda de texto completo y búsqueda vectorial capacidades, mejorando la búsqueda y el análisis de la aplicación funciones.

En esencia, los controladores de MongoDB son más que simples conectores; son herramientas sofisticadas diseñadas para agilizar las interacciones con la base de datos y optimizar el rendimiento. Al gestionar tareas como la gestión de conexiones, la serialización BSON y el enrutamiento inteligente, los controladores permiten centrarse en la lógica de la aplicación en lugar de en la mecánica básica de la base de datos. Comprender este funcionamiento de los controladores nos permite crear aplicaciones MongoDB robustas y escalables, y sienta las bases para el uso eficaz de abstracciones de alto nivel como los ODM.

Características clave de los controladores de MongoDB: Los controladores de MongoDB ofrecen una amplia gama de características que garantizan tanto la funcionalidad como la eficiencia. Comprender estas características es fundamental para crear aplicaciones robustas. En esta sección, profundizaremos en los aspectos clave que hacen que estos controladores sean tan eficaces. Exploraremos cómo los controladores garantizan la consistencia y la fiabilidad mediante especificaciones compartidas, ofrecen una experiencia idiomática adaptada a cada lenguaje, optimizan el rendimiento y ofrecen compatibilidad con funciones avanzadas como transacciones y flujos de cambios.

Consistencia y confiabilidad mediante especificaciones compartidas. Una de las características que definen a los controladores oficiales de MongoDB es su adhesión a especificaciones comunes. Al seguir las especificaciones básicas, todos los controladores admiten las mismas funciones fundamentales, como la agrupación de conexiones, las lecturas o escrituras reintentables y las interacciones con protocolos de cable, independientemente del lenguaje de programación. Esto...

La consistencia es particularmente valiosa para las organizaciones que trabajan en entornos multilingües, ya que los desarrolladores pueden esperar comportamientos y capacidades similares en diferentes pilas de tecnología.

Cuando MongoDB introduce nuevas características, estas especificaciones se actualizan, lo que garantiza que todos los controladores implementen la funcionalidad de forma fiable y consistente. Este enfoque basado en especificaciones reduce errores y comportamientos inesperados en diferentes controladores de lenguaje.



Controladores MongoDB mantenidos por la comunidad

Además de los controladores oficiales de MongoDB, existen controladores mantenidos por la comunidad y desarrollados con especificaciones compartidas. Es posible que estos controladores no implementen todas las especificaciones ni ofrezcan todas las funciones de los controladores oficiales; sin embargo, esto no debería disuadirle de probarlos en entornos donde no existen controladores oficiales.

Experiencia idiomática. Aunque los controladores de MongoDB comparten especificaciones comunes, cada uno está diseñado para funcionar con naturalidad dentro de su ecosistema de lenguaje de programación. Veamos los siguientes ejemplos:

- El controlador Node.js aprovecha las promesas de JavaScript y los patrones `async/await`.
- El controlador Python sigue las pautas PEP y los principios Pythonic.

- El controlador Java proporciona tipado fuerte y se integra con las herramientas de concurrencia de Java. El
- controlador C# adopta modismos .NET y consultas estilo LINQ.

Este diseño idiomático permite trabajar con MongoDB utilizando patrones y prácticas que ya conoce, lo que reduce la curva de aprendizaje y mejora la productividad. La API de cada controlador está diseñada para alinearse con las convenciones de su lenguaje anfitrión, lo que hace que las interacciones con la base de datos se sientan como una extensión natural del propio lenguaje.

Optimización del rendimiento

Los controladores MongoDB están diseñados meticulosamente para mejorar el rendimiento. Minimizan la latencia y maximizan el rendimiento incorporando numerosas optimizaciones, como las siguientes:

- Agrupación de conexiones : reutilización de conexiones de bases de datos existentes en lugar de crear nuevas para cada operación, lo que reduce la sobrecarga de la conexión y disminuye la latencia.
- Enrutamiento de operaciones inteligente : los controladores realizan selecciones de servidores inteligentes al monitorear continuamente la latencia de la red y el estado del servidor, enrutando dinámicamente las operaciones de lectura y escritura a los nodos más óptimos en un conjunto de réplicas o un clúster fragmentado, lo que garantiza una alta disponibilidad y una latencia mínima.
- Compresión de datos de red : Comprima eficientemente los datos antes de su transmisión por la red, empleando algoritmos como Snappy, zlib o zstd. Esto reduce la cantidad de datos transferidos, disminuyendo así el uso del ancho de banda y acelerando la transmisión. Esto es especialmente beneficioso para las aplicaciones.

Trabajar con documentos grandes y comprimibles o con altas cargas de red.

- Operaciones masivas de bases de datos : Permiten agrupar múltiples acciones, como inserciones, actualizaciones o eliminaciones, en una sola solicitud, lo que minimiza el número de viajes de ida y vuelta de la red y reduce drásticamente la sobrecarga general. Esto resulta especialmente beneficioso para mejorar el rendimiento de las tareas de procesamiento de datos por lotes.

Estas optimizaciones son especialmente importantes para aplicaciones de alto rendimiento donde el rendimiento de la base de datos impacta directamente la experiencia del usuario. Los controladores gestionan estas optimizaciones en segundo plano, lo que permite a los desarrolladores centrarse en la lógica de la aplicación en lugar de en el ajuste del rendimiento.

¿Qué son los mapeadores de objetos y documentos (ODM)?

Además de los controladores oficiales, los ODM ofrecen una capa de abstracción adicional que conecta el modelo de datos basado en documentos de MongoDB con los paradigmas de programación orientada a objetos. Al igual que los mapeadores objeto-relacionales (ORM) en bases de datos SQL, los ODM permiten trabajar con objetos en lenguaje nativo en lugar de documentos BSON sin formato. Los ODM populares incluyen EFCore para C#, Symphony para PHP, Django para Python, Mongoose para JavaScript, Mongoid para Ruby y Prisma para TypeScript.

Los ODM ofrecen características valiosas, incluidas las siguientes:

- Aplicación y validación de esquemas

- API convenientes para la creación de consultas
- Gestión de relaciones entre documentos
- Ganchos de middleware y ciclo de vida
- Seguridad de tipos y autocompletado de IDE

De manera similar al valor que los ORM aportan a los desarrolladores que trabajan con bases de datos SQL, los ODM proporcionan una capa de abstracción significativa entre el modelo de datos basado en documentos de MongoDB y el paradigma de programación orientada a objetos de una aplicación.

Entendiendo los ODM . En esencia, los ODM

traducen entre dos mundos: el modelo de documento flexible de MongoDB y el enfoque estructurado y basado en clases de la programación orientada a objetos. Esta capa de traducción ofrece ventajas significativas para los desarrolladores que buscan mantener una clara separación de intereses dentro de sus aplicaciones.

Para ilustrar el contraste entre usar directamente el controlador nativo de MongoDB y aprovechar un ODM, los siguientes ejemplos comparan la recuperación de un documento de usuario, el primero usando el controlador de PyMongo, mientras que el segundo demuestra la misma operación usando Motor Mongo:

```
desde pymongo importar MongoClient cliente =
MongoClient() db =
cliente.mi_base_de_datos usuarios
= db.usuarios usuario =
usuarios.find_one({"email": "john@example.com"}) imprimir(f"Nombre:
{usuario['nombre']} {usuario['apellido']}")
```

Si bien PyMongo le brinda control total sobre las consultas y las estructuras de datos, deja en manos del desarrollador la tarea de aplicar restricciones de esquema, manejar la lógica de serialización y analizar o transformar manualmente los resultados.

```

Con Motor Mongo # -----
de mongoengine importar Document, StringField, conectar connect('my_database') clase
Usuario(Documento): nombre =
StringField(requerido=Verdadero,
            longitud_máxima=50 apellido = StringField(requerido=Verdadero, longitud_máxima=50)
            correo electrónico = StringField(requerido=Verdadero, único=Verdadero)

usuario = Usuario.objects.get(email="john@example.com") print(f"Nombre:
{usuario.nombre} {usuario.apellido}")

```

Con MongoEngine, el esquema se declara una sola vez mediante clases de Python, lo que hace que el modelo de dominio sea explícito y más fácil de analizar. Las consultas resultan más naturales en un contexto orientado a objetos, y los desarrolladores se benefician de la validación integrada, una mejor finalización de código en los IDE y un menor riesgo de errores clave en tiempo de ejecución. El resultado es un código más fácil de leer, probar y desarrollar con el tiempo.

La principal diferencia entre un controlador y un ODM es que el controlador se basa en operaciones, mientras que el ODM se basa en estados. Los controladores se centran principalmente en operaciones CRUD u otras operaciones de base de datos que se deben realizar directamente.

Los ODM, por otro lado, dependen en gran medida del estado. Se carga un objeto en un contexto, se manipula mediante primitivas del lenguaje y luego se conservan los cambios. La función del ODM es traducir el estado actual del objeto a los comandos de base de datos necesarios para conservarlo.

Si bien los controladores nativos de MongoDB sientan las bases para la comunicación e interacción con la base de datos, los ODM se basan en ellos para ofrecer abstracciones de alto nivel que se alinean de forma más natural con los patrones de código de la aplicación. Este enfoque reduce el código repetitivo, mejora la organización del código y la facilidad de mantenimiento al representar las entidades de la base de datos como clases con comportamientos bien definidos.

Características clave de los ODM: Los ODM

ofrecen numerosas ventajas que simplifican las interacciones con las bases de datos y optimizan el desarrollo de aplicaciones. En esta sección, exploraremos las principales ventajas de los ODM, centrándonos específicamente en la aplicación de esquemas y la validación de datos para datos estructurados, así como en el uso de consultas intuitivas.

API para interacciones de bases de datos legibles, gestión de relaciones para definir y manejar asociaciones de datos, middleware y ganchos de ciclo de vida para ejecutar lógica personalizada en varias etapas y, finalmente, seguridad de tipos e integración de IDE, que promueve un flujo de trabajo de desarrollo más sólido con detección de errores.

Aplicación de esquemas y validación de datos

Aunque MongoDB tiene un esquema flexible por diseño, muchas aplicaciones se benefician de la consistencia estructural. Como vimos en [Capítulo 2](#), Diseño de esquemas para el rendimiento, MongoDB ofrece funciones opcionales de validación de esquemas que pueden implementarse en el servidor. Sin embargo, para la validación del lado del cliente, los ODM permiten a los desarrolladores

definir esquemas explícitos que garanticen que los documentos se ajusten a los patrones esperados.

La flexibilidad inherente de MongoDB permite almacenar datos en documentos tipo JSON sin esquemas rígidos. Si bien este enfoque ofrece gran adaptabilidad, también puede generar inconsistencias si no se gestiona la estructura de datos. Los ODM como Mongoose ofrecen una solución que permite a los desarrolladores definir esquemas, que actúan como modelos para garantizar que cada documento se ajuste a una estructura y reglas específicas mediante las siguientes acciones:

- Definición de campos obligatorios y valores predeterminados
- Especificación de tipos de datos y restricciones
- Implementación de lógica de validación personalizada
- Aplicar transformaciones al guardar o recuperar datos

Este grado de control proporciona seguridad y previsibilidad al mismo tiempo que conserva la flexibilidad por la que se conoce a MongoDB, como en el siguiente ejemplo que demuestra cómo Mongoose ayuda a aplicar los tipos de datos y las reglas de validación, lo que garantiza la integridad de los datos dentro de su base de datos.

Colecciones de MongoDB:

```
const mongoose = require('mongoose'); const
userSchema = new mongoose.Schema({ firstName:
  { tipo: String, requerido: verdadero, ajuste: verdadero } lastName:
  { tipo: String, requerido: verdadero, ajuste: verdadero }, email: { tipo:
String,
  requerido:
verdadero, único:
verdadero,
validar:
  { validador: function(v) {
    devolver /^w+([.-]?w+)*@w+([.-]?w+)*(\.w{2,3})
  }
}
```

```

    },
    mensaje: 'Ingrese un correo electrónico válido' } }, });

const user = mongoose.model('User', userSchema);

```

En el ejemplo anterior, se define un esquema Mongoose que establece varias reglas sobre cómo se manejarán los campos para los registros de usuario.

El nombre, apellido y correo electrónico del usuario son obligatorios. En los dos primeros campos, se eliminarán automáticamente los espacios sobrantes y se aplicará la unicidad y la composición de la dirección de correo electrónico. El validador personalizado del campo de correo electrónico mostrará un mensaje personalizado si la validación falla, lo que mejora la experiencia del desarrollador en comparación con simplemente no poder modificar el documento.

API de consulta intuitivas

Los ODM proporcionan métodos idiomáticos para construir consultas de bases de datos, lo que las hace más legibles y fáciles de mantener. Ejemplos de esto incluyen el uso de LINQ en C# o las API de Builders en C#, Java y PHP.

Para ilustrar la naturaleza intuitiva de la creación de consultas en ODM, esto El ejemplo de código Ruby que utiliza Mongoid demuestra cómo construir una consulta de base de datos compleja utilizando encadenamiento de métodos y sintaxis específica del lenguaje para encontrar productos en stock dentro de una categoría y rango de precio específicos y luego ordenarlos:

```

class Producto
  incluir Mongoid::Document

  campo :precio, :if => :stock?
end

```

```
campo :nombre, tipo: Cadena
campo : precio, tipo: Flotante
campo :categoría, tipo: Cadena
campo : en_stock, tipo: Booleano
fin
# todos los $100-$500, # con precio entre 100 y 500, por
Encuentra productos en stock en la categoría "Electrónica".
precio

resultados = Producto.donde(:categoría => "electrónica")
    .and(:en_stock => verdadero)
    .and(:precio.gte => 100, :precio.lte => 500)
    .order_by(:precio => :asc)
```

La mayoría de los ODM ofrecen una variedad de funciones que facilitan el trabajo con MongoDB más intuitivo y eficiente, incluyendo lo siguiente:

- API fluidas para la construcción de consultas complejas
- Operadores de filtrado que coinciden con las capacidades de consulta de MongoDB
- Mecanismos de proyección y clasificación
- Ayudantes de paginación
- Abstracciones del marco de agregación

Por lo tanto, al ofrecer una sintaxis y un lenguaje nativos del idioma, funcionalidad idiomática como el encadenamiento de métodos, los ODM son muy útiles. Simplificar y mejorar el proceso de creación de consultas, haciéndolo tanto Más accesible y más eficiente para los desarrolladores.

Gestión de relaciones

El modelo de documentos de MongoDB admite de forma inherente documentos incrustados. documentos y referencias. Los ODM amplían esto con información explícita definiciones de relación.

Para ilustrar cómo los ODM facilitan la definición de relaciones, el siguiente ejemplo de esquema Prisma en TypeScript demuestra cómo para especificar relaciones uno a uno, uno a muchos y muchos a muchos entre diferentes entidades, como autores, libros y géneros, Modelar eficazmente asociaciones de datos complejas de forma estructurada. manera:

```

modelo Autor {
  identificación Cadena @id @default(auto()) @map("_id") @db
  nombre Cadena
  libros por Cadena @unique
  correo Libro[] // // Relación de uno a muchos
  electrónico biografía ¿Bio? Relación uno a uno
}

Libro modelo {
  id Cadena @id @predeterminado(auto()) @map("_id") @db
  Título Cadena
  cadena ISBN @unique
  autorId Cadena @db.ObjectId
  autor Autor @relation(campos: [authorId], refere
  géneros Género[] // Relación de muchos a muchos
}

modelo Género {
  identificación Cadena @id @default(auto()) @map("_id") @db
  nombre Cadena @unique
  libros Libro[]
}

modelo Bio {
  id Cadena @id @predeterminado(auto()) @map("_id") @db
  texto Cadena
  autorId Cadena @unique @db.ObjectId
  autor Autor @relation(campos: [authorId], refere
}

```

Para garantizar que los datos se ajusten a patrones específicos, los ODM proporcionan la

Las siguientes capacidades permiten a los desarrolladores definir y aplicar

Reglas para la estructura y validación de documentos:

- Relaciones uno a uno, uno a muchos y muchos a muchos
- Patrones de documentos incrustados
- Patrones de referencia con población automática
- Operaciones en cascada (guardar, actualizar y eliminar)

Al proporcionar definiciones explícitas para diversas relaciones

Las configuraciones ODM permiten a los desarrolladores modelar datos complejos.

asociaciones efectivamente dentro de la estructura de documentos de MongoDB,

Cerrando la brecha entre un modelo de base de datos flexible y uno estructurado

Diseño de aplicaciones.

Ganchos de middleware y ciclo de vida

La mayoría de los ODM proporcionan ganchos para ejecutar código en puntos específicos de un

Ciclo de vida de acceso o modificación del documento. El siguiente ejemplo

Demuestra el middleware de Mongoose que se ejecuta antes de "guardar"

Operación que cifra automáticamente la contraseña de un usuario antes de almacenarla.

en la base de datos:

```
userSchema.pre('guardar', función asíncrona(siguiete) {  
  // Solo hashear la contraseña si se modifica O nuevo  
  si (!this.isModified('contraseña')) devuelve siguiete();  
  
  intentar {  
    esto.salt = await bcrypt.genSalt(10);  
    esta.contraseña = await bcrypt.hash(esta.contraseña, esta  
    próximo());  
  } captura (error) {  
    (  
    )  
  }  
})
```



```

    siguiente(error);

  } });
  userSchema.post('guardar', función(doc) {
    console.log(`El usuario ${doc.email} ha sido guardado`); });

```

Los ODM generalmente proporcionan una variedad de ganchos dentro del ciclo de vida de un documento, lo que permite a los desarrolladores ejecutar lógica personalizada en puntos estratégicos, como los siguientes ejemplos comunes:

- Validación previa y posterior
- Pre/post guardado
- Actualización previa y posterior
- Eliminación previa/posterior

Estos ganchos permiten comportamientos complejos como marcas de tiempo automáticas, registro, cifrado y seguimiento de cambios.

Seguridad de tipos e integración IDE

Los ODM modernos aprovechan las características del lenguaje para brindar seguridad en tiempo de compilación y una mejor experiencia para el desarrollador.

El siguiente ejemplo de TypeScript con Typegoose (una biblioteca para Mongoose) demuestra cómo las decoraciones de clases pueden definir esquemas y sus...

propiedades, lo que permite el autocompletado y la verificación de tipos durante el desarrollo, reduciendo así los errores y mejorando la productividad del desarrollador:

```

class Usuario
{ @prop({ requerido: verdadero })
  público firstName!: cadena;

```

```

@prop({ requerido: verdadero })
apellido público !: cadena;

@prop({ requerido: verdadero, único: verdadero })
correo electrónico público !: cadena;

@prop({ valor predeterminado: Fecha.ahora })
público registradoEn?: Fecha;

público obtener nombrecompleto(): cadena {
  devuelve `${este.nombre} ${este.apellido}`;
}
}

const UserModel = getModelForClass(Usuario);
// Autocompletar y comprobar tipos:
const nuevoUsuario = nuevo ModeloDeUsuario({
  Nombre: "Jane",
  Apellido: "Doe",
  correo electrónico: "jane@example.com"
});
// TypeScript capturaría este error:
newUser.nonExistentField // = "valor"; // ¡Error!

```

La integración perfecta con herramientas de lenguaje y sistemas de tipos ofrece a los desarrolladores muchas ventajas que mejoran la productividad y Reducir errores aprovechando funciones como la finalización de código y Comprobación de tipos:

- Autocompletado de propiedades de documentos
- Comprobación de tipos para operaciones de consulta
- Soporte de refactorización
- Documentación a través de tipos

En última instancia, la combinación de seguridad de tipos e integración IDE estrecha que ofrecen los ODM avanzados conduce a un proceso de desarrollo más eficiente y sin errores, lo que permite a los desarrolladores centrarse en crear funciones en lugar de depurar problemas básicos de manejo de datos.

Impacto en la productividad del desarrollador: Los ODM desempeñan un papel crucial en la optimización del proceso de desarrollo y el aumento de su productividad. Al ofrecer abstracciones y herramientas que se adaptan a las prácticas de programación habituales, los ODM reducen el tiempo dedicado a la gestión de detalles básicos de la base de datos, lo que permite a los desarrolladores centrarse en la creación de funcionalidades para la aplicación. A continuación, se describe cómo los ODM logran importantes mejoras de productividad:

- Reducción del código repetitivo : las operaciones de bases de datos comunes, como la validación de datos, la conversión de tipos y la gestión de relaciones, se pueden automatizar, lo que elimina la necesidad de que los desarrolladores escriban código repetitivo y detallado, lo que libera tiempo y reduce las posibilidades de errores en esas tareas repetitivas.
- Mejora de la organización del código : los ODM facilitan un enfoque más estructurado al representar las entidades de la base de datos como clases u objetos, organizando el código lógicamente en torno a conceptos comerciales, lo que simplifica el desarrollo y el mantenimiento y alinea las interacciones de la base de datos con la arquitectura de la aplicación.
- Mejorar la legibilidad : los ODM utilizan una sintaxis idiomática del lenguaje y API de consulta intuitivas, lo que hace que las operaciones de la base de datos sean más legibles y comprensibles, ya que siguen patrones de programación familiares en lugar de requerir conocimientos de programación.

comandos específicos de MongoDB, mejorando así la claridad del código y la colaboración en equipo.

- Detectar errores con antelación : Al usar la aplicación de esquemas y la verificación de tipos para detectar errores relacionados con los datos durante el desarrollo, en lugar de en la ejecución, los ODM pueden evitar que datos no válidos lleguen a la base de datos. Esto reduce la necesidad de depuración posterior, ahorrando tiempo y mejorando la calidad general del código.
- Simplificación de operaciones complejas : los ODM hacen que las operaciones de bases de datos complejas, como la gestión de relaciones, el manejo de documentos anidados y las actualizaciones condicionales, sean más sencillas al proporcionar abstracciones y métodos de alto nivel, lo que permite a los desarrolladores realizar tareas complejas con menos código y una carga cognitiva reducida.

Al automatizar tareas rutinarias, agilizar operaciones complejas y ofrecer estructuras de código más claras, los ODM mejoran significativamente la productividad de los desarrolladores, permitiéndoles centrarse más en el desarrollo de funciones y menos en la gestión de bases de datos.

Cuándo utilizar ODM

Los ODM ofrecen ventajas significativas en muchos escenarios de desarrollo, pero no siempre son necesarios para todos los proyectos. La decisión de emplear un ODM depende de varios factores, como la complejidad de la aplicación, la experiencia del equipo de desarrollo y los requisitos específicos de integridad y consistencia de los datos. En las siguientes situaciones, los ODM son especialmente valiosos y beneficiosos para su proyecto de desarrollo:

- Los equipos que estén en transición de bases de datos relacionales a MongoDB encontrarán que los ODM son útiles para adaptarse a un modelo de datos no relacionales, gracias a los patrones orientados a objetos familiares que proporcionan.
- Cuando las estructuras de datos consistentes son una prioridad, los ODM facilitan la definición y la aplicación de esquemas, lo que ayuda a prevenir inconsistencias en los datos.
- Las aplicaciones con modelos de dominio complejos, donde la gestión de relaciones entre entidades es crucial, pueden beneficiarse enormemente de la estructura que proporcionan los ODM.
- Para las bases de código que se benefician de una clara separación de preocupaciones, los ODM ayudan abstrayendo las interacciones de la base de datos en capas distintas.
- Los equipos de desarrollo que buscan reducir errores relacionados con las bases de datos en las primeras etapas del proceso de desarrollo apreciarán las funciones de verificación y validación de tipos integradas que brindan los ODM.

Sin embargo, los ODM introducen una capa de abstracción que puede afectar el rendimiento o limitar el acceso a algunas funciones específicas de MongoDB. Para aplicaciones con requisitos de rendimiento extremos o que utilizan funciones avanzadas de MongoDB, un enfoque híbrido o el uso directo de controladores podría ser adecuado en secciones críticas para el rendimiento.

En última instancia, la decisión de utilizar un ODM debe basarse en una evaluación equilibrada de la complejidad del proyecto, la familiaridad del equipo con MongoDB y la necesidad de manejo estructurado de datos y mejoras de productividad.

¿Qué son los marcos de aplicación?

Los frameworks de aplicaciones proporcionan una base estructurada para la creación de aplicaciones de software, ofreciendo una colección de componentes y bibliotecas preconstruidas que estandarizan los patrones y prácticas de desarrollo. Al abstraer tareas comunes, como la gestión de solicitudes HTTP, la administración de operaciones de bases de datos y la representación de vistas, estos frameworks permiten a los desarrolladores centrarse en la implementación de la lógica de negocio en lugar de reinventar funcionalidades básicas para cada proyecto.

El valor de los frameworks de aplicaciones. Frameworks de aplicaciones como Ruby on Rails, Django, Spring, ASP.NET MVC y Laravel impulsan significativamente los esfuerzos de desarrollo al ofrecer un enfoque estructurado para la creación de software. Ofrecen numerosos beneficios que optimizan tanto el proceso de desarrollo como el producto final. A continuación, se detallan las principales ventajas que los frameworks de aplicaciones aportan a los equipos de desarrollo y sus soluciones:

1. Estandarización : Los marcos imponen una estructura consistente y estilo de codificación en todos los proyectos, lo que facilita la colaboración entre equipos y el mantenimiento de las bases de código. Esta estandarización Reduce la curva de aprendizaje para los nuevos miembros del equipo y garantiza que se sigan las mejores prácticas durante todo el ciclo de vida del desarrollo.
2. Eficiencia : Al proporcionar componentes y bibliotecas reutilizables, los frameworks aceleran el tiempo de desarrollo. Los desarrolladores pueden integrar rápidamente funciones como autenticación, enrutamiento y validación de datos sin necesidad de escribir código repetitivo extenso, lo que permite a los equipos entregar funcionalidades con mayor rapidez.

3. Escalabilidad : Los frameworks bien diseñados están diseñados para gestionar las complejidades del escalado de aplicaciones. Ofrecen compatibilidad integrada con el equilibrio de carga, el almacenamiento en caché y la gestión de conexiones a bases de datos, lo que permite que las aplicaciones crezcan sin problemas a medida que aumenta la demanda de los usuarios.
4. Soporte de la comunidad : Los frameworks populares cuentan con comunidades grandes y activas que aportan plugins, extensiones y documentación completa. Esta amplia gama de recursos ayuda a los desarrolladores a resolver problemas rápidamente y a mantenerse al día con los últimos avances tecnológicos.

En esencia, los marcos de aplicación brindan a los desarrolladores una base estructurada, eficiente y escalable, fomentando la estandarización, la productividad y, en la mayoría de los casos, un sólido apoyo de la comunidad.

Aprovechamiento de los ODM y ORM en los marcos

La mayoría de los frameworks de aplicaciones se integran con ODM u ORM para optimizar el modelado de datos y la gestión de esquemas. Estas integraciones proporcionan un enfoque consistente para las operaciones de bases de datos, permitiendo a los desarrolladores trabajar con datos utilizando los patrones de su lenguaje de programación nativo.

El siguiente ejemplo de código ilustra cómo un ODM, en concreto Mongoose, puede integrarse en un popular framework de aplicaciones web, Express.js. Esto demuestra la fluida conexión e interacción entre la capa de enrutamiento de la aplicación.

y el modelado de datos proporcionado por el ODM, simplificando la base de datos.

operaciones dentro del contexto de una aplicación web:

```
const express = require('express');
const mangosta = require('mangosta');
const aplicación = express();
// Conectar a MongoDB
mangosta.connect('mongodb://localhost:27017/myapp');
// Definir a esquema usando Mangosta
const userSchema = nueva mangosta.Schema({
  nombre: { tipo: String, requerido: verdadero },
  correo electrónico: { tipo: Cadena, requerido: verdadero, único: verdadero },
  creado: { tipo: Fecha, predeterminado: Fecha.ahora }
});
// Crear a modelo del esquema
const Usuario = mongoose.model('Usuario', userSchema);
// Utilice el modelo en un Ruta expres
app.post('/usuarios', async (req, res) => {
  intentar {
    const usuario = nuevo Usuario(req.body);
    esperar usuario.save();
    res.status(201).send(usuario);
  } captura (error) {
    res.status(400).send(error);
  }
});
```

El ejemplo anterior demuestra una aplicación Express básica

Integrado con MongoDB mediante Mongoose. Se conecta a una base de datos local.

Instancia de MongoDB, define un esquema de usuario con el nombre requerido y

campos de correo electrónico (este último también marcado como único) y proporciona una

Marca de tiempo creada con un valor predeterminado. Un modelo Mongoose es

creado a partir del esquema para interactuar con la base de datos y expuesto

a través de una ruta personalizada, lo que permite una API REST básica para crear nuevos usuarios.

Al unificar patrones y estándares dentro de una aplicación, los frameworks mejoran la productividad del desarrollador, la mantenibilidad del código y la escalabilidad del proyecto. Además, facilitan los flujos de trabajo colaborativos, permitiendo a los equipos integrar nuevas funciones sin problemas y aprovechando las mejores prácticas establecidas.

Marcos populares compatibles con MongoDB

Varios frameworks de aplicaciones ofrecen una excelente integración con MongoDB, proporcionando un entorno robusto y eficiente para el desarrollo de aplicaciones modernas. Cada uno de estos frameworks aporta su propio conjunto de características y ventajas, adaptadas a lenguajes de programación y paradigmas de desarrollo específicos. Estos son algunos de los frameworks más populares que funcionan a la perfección con MongoDB:

- Spring Data MongoDB (Java) : Ofrece un enfoque práctico, basado en anotaciones, para la gestión de documentos MongoDB en aplicaciones Spring. Ofrece abstracciones de repositorio, mapeo de objetos y compatibilidad con transacciones, lo que lo convierte en una opción robusta para aplicaciones Java empresariales.
- Entity Framework (C#) : Ofrece un paradigma de acceso a datos familiar para desarrolladores .NET, aprovechando el patrón de repositorio y el modelado basado en código. Los proveedores de MongoDB permiten a los desarrolladores .NET usar las mismas abstracciones de Entity Framework con las que están familiarizados al trabajar con bases de datos documentales.

- Ruby on Rails (Ruby) : Con Mongoid como su ODM, Rails prioriza la convención sobre la configuración, simplificando la configuración y automatizando muchas operaciones rutinarias de datos. Su canalización de activos y herramientas de andamiaje lo hacen especialmente eficiente para el desarrollo rápido de aplicaciones basadas en MongoDB.
- Quarkus (Java) : Diseñado para entornos nativos de la nube y en contenedores, Quarkus optimiza el rendimiento con un modelo de programación reactiva que se integra a la perfección con los controladores asíncronos de MongoDB. Su bajo consumo de memoria lo hace ideal para arquitecturas de microservicios.
- Django (Python) : Django proporciona un marco integral que puede integrarse con MongoEngine, lo que permite a los desarrolladores aprovechar la flexibilidad de MongoDB mientras trabajan dentro de un ecosistema Python familiar y estructurado.
- Flask (Python) : Flask es una alternativa liviana que combina bien con las extensiones PyMongo u ODM, brindando a los desarrolladores más flexibilidad y control cuando trabajan con MongoDB.
- FastAPI (Python) : Un framework moderno y de alto rendimiento para crear API con Python, FastAPI es ideal para crear servicios RESTful con soporte de MongoDB. Admite operaciones asíncronas de bases de datos mediante Motor o Beanie, ofreciendo un excelente rendimiento.
- Laravel (PHP) : Un popular framework PHP que incluye comandos artesanales y herramientas de andamiaje para simplificar la gestión de bases de datos. Su ORM Eloquent puede ampliarse con paquetes como jenssegers/mongodb para la integración con MongoDB.

- **Symfony (PHP)** : Otro framework PHP ampliamente utilizado que ofrece potentes herramientas para la gestión de bases de datos. Symfony se integra con Doctrine MongoDB ODM para trabajar con MongoDB.

En conjunto, estos marcos brindan una variedad de opciones para que los desarrolladores utilicen MongoDB de manera efectiva dentro de sus lenguajes de programación y patrones arquitectónicos preferidos, fomentando un desarrollo de aplicaciones eficiente y escalable.

Mejores prácticas al usar frameworks con MongoDB

Al aprovechar los marcos de aplicaciones con MongoDB, tenga en cuenta estas prácticas importantes:

- **Reutilización de MongoClient** : Una instancia de MongoClient mantiene los grupos de conexiones de todos los miembros del clúster y monitoriza las conexiones. Es importante reutilizar esta instancia para evitar la duplicación innecesaria de conexiones, lo cual se logra comúnmente mediante el patrón Singleton o una configuración adecuada del contenedor IoC del framework.
- **Diseño eficiente de esquemas** : Diseñe sus esquemas para minimizar la redundancia y optimizar el rendimiento de las consultas. Utilice documentos y referencias incrustados adecuadamente para equilibrar la normalización y desnormalización de datos según los patrones de acceso de la aplicación, así como para evitar posibles problemas de N+1.
- **Evitar problemas N+1** : Tenga en cuenta los problemas de consultas N+1, donde se ejecutan varias consultas independientes para obtener datos relacionados. Utilice técnicas como la carga diligente y MongoDB.

canalización de agregación u operaciones por lotes para reducir los viajes de ida y vuelta a la base de datos.

- Estrategias de almacenamiento en caché : implemente capas de almacenamiento en caché adecuadas para reducir la carga de la base de datos y mejorar el rendimiento de la aplicación. La mayoría de los marcos ofrecen soluciones de almacenamiento en caché integradas que pueden configurarse para funcionar con resultados de consultas de MongoDB.
- Carga diferida vs. carga ansiosa : Elija entre carga diferida y carga ansiosa según las necesidades específicas de su aplicación. La carga diferida pospone la carga de los datos relacionados hasta que se accede a ellos, mientras que la carga ansiosa los recupera por adelantado. Cada enfoque tiene sus desventajas en términos de rendimiento y uso de memoria.
- Devoluciones de llamada del ciclo de vida : Aproveche los ganchos del ciclo de vida que ofrece el ODM de su framework para ejecutar lógica personalizada en distintos puntos del ciclo de vida de un documento. Estas devoluciones de llamada le permiten implementar reglas de negocio consistentes, como la validación de datos, la transformación o la activación de eventos.
- Monitoreo del rendimiento : Instrumente el código de su framework para monitorear el rendimiento de MongoDB. Comprender los patrones de consulta y los tiempos de ejecución ayuda a identificar cuellos de botella y oportunidades de optimización.

Los frameworks de aplicaciones mejoran significativamente el desarrollo de MongoDB al proporcionar componentes estructurados y reutilizables que optimizan las tareas comunes. Su integración con ODM ofrece una experiencia de desarrollo natural donde las operaciones de la base de datos se alinean con los patrones de programación idiomáticos. Al seleccionar un framework que complemente su experiencia con el lenguaje y los requisitos del proyecto, los equipos de desarrollo pueden crear aplicaciones MongoDB escalables y fáciles de mantener de forma más eficiente.

Como exploraremos en la siguiente sección, comprender los patrones y las dificultades específicos de los ODM y los marcos más populares lo ayudará a aprovechar las capacidades de MongoDB y al mismo tiempo mantener un código limpio y de alto rendimiento.

Más allá de lo básico

En esta sección, exploraremos técnicas y estrategias avanzadas que pueden mejorar drásticamente el rendimiento, la resiliencia y la escalabilidad de su aplicación MongoDB al trabajar con bibliotecas cliente. Estas prácticas marcan la diferencia entre la funcionalidad básica y las aplicaciones listas para producción, capaces de afrontar los desafíos del mundo real.

Patrones asincrónicos y no bloqueantes. Las aplicaciones modernas

suelen necesitar

gestionar múltiples operaciones simultáneamente, manteniendo la capacidad de respuesta. Los patrones de programación asincrónica son esenciales para lograrlo, especialmente al trabajar con operaciones de bases de datos que, de otro modo, podrían bloquear la ejecución de la aplicación.

Las bibliotecas cliente de MongoDB brindan un soporte sólido para operaciones asincrónicas en varios lenguajes de programación:

- Promesas y `async/await` : los controladores de JavaScript y Node.js aprovechan las promesas y la sintaxis `async/await` para lograr una legibilidad limpia.

Código asíncrono. Esto elimina las complejas cadenas de devolución de llamadas, manteniendo la ejecución sin bloqueo.

- Marcos controlados por eventos : Muchos controladores MongoDB se integran sin problemas con arquitecturas basadas en eventos, lo que permite que su Aplicación para desacoplar la lógica empresarial de las operaciones de E/S. Este enfoque permite que su aplicación continúe procesándose mientras Esperando respuestas de la base de datos.
- Implementaciones específicas del lenguaje : diferentes lenguajes
Los controladores implementan patrones asíncronos de acuerdo con convenciones del lenguaje, por ejemplo, CompletableFuture de Java ,
asyncio de Python , o async/await de C# y Node.js.

Al implementar patrones asíncronos de manera efectiva, su aplicación

Puede mantener la capacidad de respuesta incluso bajo cargas elevadas, como se ilustra en

Este ejemplo de Python:

```
importar asyncio
desde pymongo importar AsyncMongoClient
definición asíncrona principal():
    uri = "mongodb://localhost:27020/"
    cliente = AsyncMongoClient(uri)
    intentar:
        base de datos = cliente.get_database("muestra_mflix")
        películas = base de datos.get_collection("películas")
        # Consulta para a Película que tiene como título 'De vuelta a la
        consulta = { "título": "Regreso al futuro" }
        película = esperar películas.find_one(consulta)
        imprimir(película)
        esperar cliente.close()
    excepto Excepción como e:
        generar una excepción ("No se puede encontrar el documento debido

# Ejecutar la función async
asyncio.run(principal())
```

La implementación de patrones asíncronos en las bibliotecas cliente de MongoDB permite crear aplicaciones altamente responsivas y de alto rendimiento. Al aprovechar construcciones como Promises, `async/await` o, como en el ejemplo anterior, las funciones de concurrencia específicas del lenguaje, los desarrolladores pueden garantizar que las operaciones de la base de datos no bloqueen el hilo principal de la aplicación.

La ejecución simultánea, junto con E/S sin bloqueo, evita que las aplicaciones se congelen o dejen de responder, lo que en última instancia genera una experiencia de usuario más fluida y eficiente, especialmente bajo cargas elevadas.

Condiciones de falla en la superficie y manejo

Para gestionar fallos transitorios y problemas de red, se debe implementar un sistema robusto de gestión de errores. Los controladores de MongoDB proporcionan mecanismos para lecturas y escrituras reintentables, que pueden reintentar operaciones automáticamente en caso de fallos temporales:

- **Escrituras reintentables** : Los controladores de MongoDB admiten reintentos automáticos para operaciones de escritura que fallan debido a problemas de red transitorios o elecciones de conjuntos de réplicas. Esta función está habilitada por defecto, pero se puede deshabilitar mediante la opción de cadena de conexión `retryWrites=false` .
- **Lecturas reintentables** : De forma similar, las operaciones de lectura están configuradas para reintentarse automáticamente ante ciertos fallos de forma predeterminada. Puede desactivar este comportamiento con la opción `retryReads=false` .

Por más robustos que puedan ser estos reintentos automáticos, deben combinarse con patrones de manejo de errores adecuados en el código de su aplicación para manejar con elegancia las excepciones de la base de datos, como con el siguiente fragmento de código Node.js, que ilustra cómo se pueden detectar y registrar condiciones de error específicas al realizar una operación de inserción:

```
try
{ const result = await collection.insertOne(document);
  console.log(`Documento insertado con _id: ${result.insertedId}`);
} catch (err) { if
(err.code === 11000)
{
  console.error('Error de clave duplicada '); } else if
(err instanceof MongoNetworkError) { console.error('Ocurrió
un error de red'); } else { console.error('Ocurrió un
error:', err);
}
}
```

Si se detecta un error de clave duplicada, se habrá cumplido una condición de fallo, mientras que un problema de red desencadenaría otra. Los bloques try-catch permiten que la aplicación mantenga su resiliencia ante estas condiciones, que deben gestionarse con la lógica de negocio adecuada.

Algunos controladores de MongoDB, como el de Node.js, han implementado el Tiempo de Espera de la Operación del Lado del Cliente (CSOT). Esta función introduce un mecanismo de control del tiempo de vida de la operación más predecible mediante la configuración de un valor de timeoutMS :

```
const docs = await colección.find({}, {timeoutMS: 1000})
```


Este ejemplo garantiza que la inicialización del cursor y la recuperación de todos los documentos se realicen en un segundo (o 1000 milisegundos), generando un error si se excede este límite de tiempo. Este error debería gestionarse en la aplicación; sin embargo, se podría lograr un control más estricto sobre la duración de las operaciones garantizando que el control regrese a la aplicación en un segundo.

Además de garantizar que la operación no supere el umbral de `timeoutMS`, cuando CSOT está configurado y las lecturas reintentables están habilitadas, las operaciones se seguirán reintentando hasta que transcurra el `timeoutMS`. Esto proporciona una capa adicional de resiliencia a las operaciones que podrían haber fallado debido a un error transitorio de red de larga duración y que requieren reintentos adicionales para completarse.

Al implementar un manejo sólido de fallas y reintentos, los desarrolladores pueden garantizar que sus aplicaciones MongoDB permanezcan estables y confiables incluso cuando enfrentan problemas transitorios o interrupciones de la red.

Administración de conexiones. Para administrar eficientemente las conexiones a la base de datos, utilice las capacidades de agrupación de conexiones del controlador. En lugar de crear nuevas conexiones para cada operación de la base de datos, los grupos de conexiones ayudan a mantener un conjunto de conexiones reutilizables que mejoran significativamente el rendimiento. Es importante configurar el tamaño del grupo de conexiones según los requisitos de concurrencia de su aplicación y los recursos disponibles en sus servidores MongoDB.

El siguiente es un ejemplo que demuestra cómo inicializar un Cliente MongoDB en Python que utiliza el controlador PyMongo y configura parámetros específicos para el grupo de conexiones, como el número máximo

y tamaños mínimos de grupo, así como el tiempo máximo de inactividad para conexiones:

```
cliente = MongoClient('mongodb://localhost:27017', tamaño máximo del grupo  
                    = 50, tamaño mínimo  
                    del grupo = 10, tiempo  
                    máximo de inactividad MS = 30000)
```

La gestión de conexiones se analizará con más detalle en " Administración [Capítulo 11](#) de conexiones y rendimiento de red". Sin embargo, el ejemplo anterior muestra cómo crear un grupo de conexiones junto con una instancia de MongoClient . En este caso, el grupo siempre tendrá 10 conexiones abiertas y disponibles, y puede escalar hasta 50 conexiones si es necesario. Cada conexión esperará 30 segundos (el valor de `maxIdleTimeMS`) antes de cerrarse automáticamente si no se utiliza.

Una gestión eficaz de las conexiones es fundamental para el rendimiento de las aplicaciones y el uso de recursos. La siguiente lista incluye una introducción a algunos conceptos y parámetros que se pueden configurar mediante las opciones de MongoClient o directamente con la cadena de conexión:

- **Agrupación de conexiones :** Los grupos de conexiones mantienen un conjunto de conexiones de base de datos reutilizables, lo que elimina la sobrecarga de establecer nuevas conexiones repetidamente. Configure el tamaño del grupo de conexiones según las características de su carga de trabajo.

Un tamaño demasiado pequeño provoca tiempos de espera para la conexión, mientras que un tamaño demasiado grande desperdicia tiempo. recursos.
- **Monitoreo del estado de la conexión :** Implemente comprobaciones de estado para identificar y reemplazar conexiones fallidas. La mayoría de los controladores de MongoDB.

Maneja esto automáticamente pero puede resultar beneficioso ajustar los tiempos de espera y la lógica de reintento.

- Opciones de conexión : los controladores MongoDB ofrecen numerosas opciones de cadena de conexión que se pueden ajustar para un uso específico casos:
 - `maxPoolSize` : Máximo de conexiones mantenidas en el piscina
 - `minPoolSize` : Conexiones mínimas a mantener
 - `maxIdleTimeMS` : Cuánto tiempo permanecen inactivas las conexiones antes de cerrarse
 - `connectTimeoutMS` : Tiempo máximo de espera cuando estableciendo conexiones

Configurar y administrar eficazmente el pool de conexiones de su controlador MongoDB y sus opciones es crucial para mantener el rendimiento, la estabilidad y el uso eficiente de los recursos de la aplicación. Optimizar ajustes como el tamaño del pool, los tiempos de espera de conexión y los tiempos de inactividad garantiza que su aplicación escale correctamente y responda rápidamente a las solicitudes de los usuarios. Además, una sólida monitorización y gestión del ciclo de vida evitan que los problemas de conexión afecten la disponibilidad.

Para ilustrar cómo ajustar los grupos de conexiones, considere los siguientes escenarios y sus recomendaciones correspondientes:

Guión	Recomendación
Tiempos de operación lentos del lado de la aplicación que no se reflejan en el	Utilice <code>connectTimeoutMS</code> para garantizar que El controlador no espera indefinidamente durante la fase de conexión. Establecer

registros del servidor de bases de datos o el panel en tiempo real.	<code>connectTimeoutMS</code> a un valor mayor que la latencia de red más larga que tenga con un miembro del conjunto. Por ejemplo, si un miembro tiene una latencia de 10 000 milisegundos, configure <code>connectTimeoutMS</code> en 5000 (milisegundos) evita que el controlador se conecte a ese miembro.
Un firewall mal configurado cierra un socket La conexión es incorrecta y el controlador no puede detectar que el conexión cerrada incorrectamente.	Utilice <code>socketTimeoutMS</code> para garantizar que Los sockets siempre están cerrados. Configure <code>socketTimeoutMS</code> al doble o triple de la duración de la operación más lenta que ejecuta el controlador.
Los registros del servidor o el panel en tiempo real muestran que la aplicación pasa demasiado tiempo creando Nuevas conexiones.	No hay suficientes conexiones disponibles al inicio. Asigne conexiones en el pool configurando <code>minPoolSize</code> . Configure <code>minPoolSize</code> como el número de Conexiones que desea que estén disponibles al inicio. La instancia de <code>MongoClient</code> garantiza que el número de conexiones... existe en todo momento
La carga en el La base de datos es baja y	Aumentar el tamaño máximo del grupo , o aumentar la Número de hilos activos en su

Hay un pequeño número de conexiones activas en cualquier momento. La aplicación realiza menos operaciones a la vez que esperado.	aplicación o el marco que esté utilizando.
El uso de la CPU de la base de datos es mayor de lo esperado. Los registros del servidor o el panel en tiempo real muestran más intentos de conexión de lo esperado.	Disminuya el valor de <code>maxPoolSize</code> o reduzca el número de subprocesos en su aplicación. Esto puede reducir los tiempos de carga y respuesta.

Tabla 10.1: Problemas de rendimiento relacionados con la conexión y configuraciones recomendadas del controlador MongoDB

Al ajustar estos parámetros, puede mejorar significativamente la eficiencia y confiabilidad de su aplicación al interactuar con MongoDB.

Preocupaciones de lectura/escritura y preferencias de lectura

Como se discutió en [Capítulo 5](#), Replicación, MongoDB proporciona

Preferencias de lectura, preocupaciones de escritura y preocupaciones de lectura configurables que le brindan un control preciso sobre cómo las operaciones interactúan con los conjuntos de réplicas. Estos ajustes se pueden configurar directamente a través del cliente.

Las bibliotecas desempeñan un papel crucial a la hora de equilibrar la consistencia, la disponibilidad y el rendimiento:

- Preferencias de lectura : Determinan qué miembros del conjunto de réplicas gestionan las operaciones de lectura:

- principal : Esta es la configuración predeterminada. Todas las lecturas se dirigen al nodo
- principal. primario preferido : Probar primero el principal y usar el secundario si no está disponible.
- secundario : Leer solo desde los secundarios.
- secundario preferido : Probar primero los secundarios y usar el principal si no está disponible. más
- cercano : Leer desde el nodo con la red más baja.

estado latente.

- Preocupaciones de escritura : Especifican el nivel de confirmación requerido para las operaciones de escritura: w:1 : El

- principal confirma las escrituras. w: "mayoría" :
- Configuración predeterminada. Una escritura debe persistir en la mayoría de los miembros del conjunto de réplicas. w:0 : No se requiere
- confirmación (se ejecuta y se olvida; no se recomienda). wtimeout : Opción para especificar un
- límite de tiempo para evitar que las operaciones de escritura esperen indefinidamente.

- Preocupaciones de lectura : controlan las propiedades de consistencia y aislamiento de los datos leídos desde conjuntos de réplicas y clústeres fragmentados:

- local : configuración predeterminada. Devuelve los datos más recientes de instancia.

- **Disponible** : Garantías similares a las de local . Se utiliza en clústeres fragmentados para lecturas con la menor latencia posible, a costa de la consistencia, ya que una lectura disponible puede devolver documentos huérfanos al leer desde una colección fragmentada.
- **Mayoría** : Devuelve datos reconocidos por la mayoría de los miembros del conjunto de réplicas. Los documentos devueltos por la operación de lectura son duraderos, incluso en caso de fallo.
- **linealizable** : Devuelve datos que reflejan todos los resultados exitosos Escrituras con confirmación mayoritaria que se completaron antes del inicio de la operación de lectura. La consulta puede esperar a que las escrituras que se ejecutan simultáneamente se propaguen a la mayoría de los miembros del conjunto de réplicas antes
- **de devolver resultados. Instantánea** : Una consulta con la consulta de lectura de instantánea devuelve datos con confirmación mayoritaria tal como aparecen en los fragmentos desde un punto específico en el pasado reciente. La consulta de lectura de instantánea solo ofrece garantías si la transacción se confirma con la escritura mayoritaria . inquietud.

Mediante el uso eficaz de las preocupaciones de escritura y de lectura, puede ajustar el nivel de consistencia y garantías de disponibilidad según sea necesario, como esperar garantías de consistencia más fuertes o relajar los requisitos de consistencia para brindar una mayor disponibilidad.

Compresión y rendimiento de la red

Optimizar la transferencia de datos entre su aplicación y MongoDB puede afectar significativamente el rendimiento, especialmente con grandes conjuntos de datos o requisitos de alto rendimiento:

- Compresión de red : los controladores MongoDB admiten varios algoritmos de compresión: Snappy : ofrece un
 - buen equilibrio entre la relación de compresión y la velocidad zlib : relación de compresión
 - más alta pero consume más CPU zstd : algoritmo más nuevo con excelente
 - compresión y buena velocidad Minimizar la latencia de la red : implemente su aplicación y
- base de datos en la misma región o centro de datos cuando sea posible.

Considere implementaciones de borde o estrategias de almacenamiento en caché para aplicaciones distribuidas geográficamente.

- Operaciones por lotes : al insertar, actualizar o consultar varios documentos, utilice operaciones por lotes para minimizar la sobrecarga de la red, como se muestra aquí:

```
// En lugar de múltiples inserciones individuales
const bulkOp = collection.initializeUnorderedBulkOp() para (const doc
de documentos) { bulkOp.insert(doc); }
await bulkOp.execute();
```

- Proyección : Solicite sólo los campos que necesita, en lugar de recuperar documentos completos, como se muestra aquí:


```
// Recuperar únicamente los campos necesarios
const resultado = await colección.findOne(
  { _id: documentId },
  { proyección: { nombre: 1, correo electrónico:
1 } } );
```

Gestionar eficazmente la transferencia de datos mediante la compresión de red, minimizar la latencia con implementaciones estratégicas, optimizar las operaciones mediante el procesamiento por lotes y utilizar proyecciones para reducir la recuperación de datos son aspectos cruciales para mejorar el rendimiento general de las aplicaciones MongoDB. Al centrarse en estas áreas, los desarrolladores pueden mejorar significativamente la eficiencia y la velocidad del intercambio de datos, garantizando una experiencia de aplicación más fluida y con mayor capacidad de respuesta para los usuarios finales.

Resumen

Este capítulo ha proporcionado una exploración exhaustiva de las bibliotecas cliente de MongoDB, incluyendo controladores y ODM, que sirven como interfaz crítica entre las aplicaciones y la base de datos. Comenzamos definiendo el papel esencial de los controladores de MongoDB para facilitar la comunicación de alto rendimiento específica del lenguaje y exploramos características clave como la consistencia mediante especificaciones compartidas, experiencias idiomáticas adaptadas a diferentes lenguajes, técnicas de optimización del rendimiento como la gestión de conexiones y capacidades avanzadas como el soporte para transacciones. A continuación, profundizamos en el mundo de los ODM, destacando sus beneficios para simplificar el desarrollo mediante la aplicación de esquemas, API de consulta intuitivas y la gestión de relaciones.

gestión y examinó soluciones populares como Mongoose, Mongoid, Prisma, Spring Data y Django.

Analizamos la importancia de las mejores prácticas, incluida la gestión de conexiones y errores, la optimización de consultas, las preocupaciones de lectura/escritura y el monitoreo, y exploramos cómo los marcos de aplicaciones se integran con MongoDB y aprovechan los ODM para mejorar la productividad. Finalmente, profundizamos en los conceptos básicos para abordar temas avanzados como patrones asíncronos, optimización del rendimiento de la red y gestión de fallos y reintentos. Dominar estos aspectos permite a los desarrolladores crear aplicaciones MongoDB funcionales, eficientes, escalables y resilientes, aprovechando al máximo las capacidades de la base de datos para implementaciones reales.

En el próximo capítulo, ampliaremos lo que ha aprendido acerca de las bibliotecas de cliente y profundizaremos en la administración de conexiones y el rendimiento de la red, factores clave que determinan la eficacia con la que su aplicación se comunica con MongoDB detrás de escena.

11

Administrar conexiones y Rendimiento de la red

Una estrategia adecuada de gestión de conexiones puede marcar la diferencia entre una solución ultrarrápida y resiliente y una que se desmorona bajo carga. En este capítulo, exploraremos el papel de la gestión de conexiones para lograr un rendimiento y una fiabilidad óptimos.

Primero, examinaremos los fundamentos de las conexiones y los factores de red que afectan el rendimiento de la base de datos, como la latencia, la pérdida de conexiones y la saturación de la red. A continuación, exploraremos la arquitectura de conexión de MongoDB, desde la capa TCP/IP hasta la agrupación de conexiones de controladores, y profundizaremos en el ciclo de vida de las conexiones para comprender cómo se establecen, utilizan y terminan. Finalmente, abordaremos estrategias esenciales para la monitorización, la resolución de problemas y la optimización de la gestión de conexiones para evitar cascadas de fallos y garantizar que sus aplicaciones MongoDB mantengan el máximo rendimiento en condiciones reales.

Los temas tratados en este capítulo incluyen los siguientes:

- Conceptos clave sobre cómo funcionan las conexiones en MongoDB
- Cómo es el ciclo de vida de una conexión típica

- Cómo ciertas fallas de conexión pueden llevar a un sistema más amplio asuntos
- Una mirada a cómo MongoDB maneja las conexiones internamente
- Consejos para gestionar conexiones de forma eficiente

Comprender los fundamentos de la conexión

El rendimiento de la red suele convertirse en un cuello de botella oculto en las aplicaciones basadas en bases de datos. Incluso con hardware potente y consultas optimizadas, una mala conectividad de red puede afectar negativamente la capacidad de respuesta y la experiencia del usuario de la aplicación. Existen varios factores que contribuyen a este impacto, entre ellos:

- Latencia de ida y vuelta : Cada operación de base de datos requiere al menos una ida y vuelta en la red, lo que genera retrasos considerables en aplicaciones interactivas donde los usuarios esperan respuestas en segundos. Esto resulta especialmente problemático en arquitecturas de microservicios, donde una sola solicitud de usuario puede desencadenar múltiples operaciones de base de datos en cascada.
- Sobrecarga en el establecimiento de la conexión : Crear nuevas conexiones requiere protocolos de enlace TCP, negociación TLS y autenticación MongoDB. Estos pasos pueden añadir cientos de milisegundos a la primera consulta de una sesión. El impacto de esta sobrecarga se hace más evidente en entornos sin servidor o aplicaciones de alta concurrencia, donde las conexiones se establecen y finalizan con frecuencia.

- Limitaciones de ancho de banda : los resultados de consultas grandes o las operaciones masivas pueden saturar la capacidad de red disponible, lo que provoca demoras en todas las operaciones de base de datos que comparten la misma ruta de red.
Las aplicaciones que procesan archivos multimedia, generan informes con grandes conjuntos de datos o realizan tareas de migración de datos o ETL son especialmente vulnerables a las restricciones de ancho de banda.
- Estabilidad de la red : La pérdida de paquetes o los problemas de conectividad intermitente pueden provocar reintentos y tiempos de espera, lo que resulta en tiempos de respuesta impredecibles y una experiencia de usuario degradada. Estos problemas son especialmente perjudiciales en aplicaciones móviles, entornos de edge computing o al operar en regiones con infraestructura menos fiable.
- Complejidad del sistema distribuido : En las arquitecturas distribuidas modernas, las aplicaciones pueden comunicarse con múltiples instancias de bases de datos en diferentes regiones o proveedores de nube, lo que multiplica la sobrecarga de conexión y presenta desafíos de coordinación. Esta complejidad se hace más evidente durante las implementaciones globales, donde la latencia entre regiones puede afectar drásticamente el rendimiento, o durante interrupciones parciales cuando se activan los mecanismos de conmutación por error.

Si bien los desarrolladores a menudo se centran en optimizar las consultas y los índices, la capa de red con frecuencia determina el límite de rendimiento real.

Una sola conexión mal administrada puede agregar cientos de milisegundos a los tiempos de operación, lo que crea un impacto notable para usuarios.



Gestión de conexiones bajo carga de producción



Los problemas de gestión de conexión suelen aparecer solo en cargas de producción. Un sistema que funciona bien durante el desarrollo puede experimentar graves problemas de conexión al exponerse a patrones de tráfico reales.

Comprender cómo los factores de red y los procesos de conexión afectan a su aplicación le ayuda a identificar y abordar posibles cuellos de botella en el rendimiento de su implementación de MongoDB. Hay tres factores de red clave que afectan directamente el rendimiento y la capacidad de respuesta de su aplicación: latencia , pérdida de conexión y saturación de la red . Analicemos cada uno , y de ellos.

La latencia es

el retraso entre el envío de una solicitud y la recepción de una respuesta. Este retraso afecta a todas las operaciones, por pequeñas que sean. La latencia en las operaciones de MongoDB se ve influenciada por diversos factores, como la distancia geográfica, las condiciones de la red y la estructura de las operaciones. Esto hace que sea esencial que las aplicaciones distribuidas globalmente adopten estrategias como la localización de datos o el almacenamiento en caché perimetral para mantener la capacidad de respuesta. Considere los siguientes factores:

- La distancia geográfica entre los servidores de aplicaciones y las instancias de MongoDB añade una latencia base inevitable. Implemente las instancias de MongoDB en la misma región que sus servidores de aplicaciones y utilice clústeres multirregionales con preferencias de lectura adecuadas para aplicaciones globales.

- Los problemas de calidad de la red, como la congestión o los problemas de enrutamiento, aumentan la variabilidad de la latencia. Implemente la monitorización de la red con herramientas como las métricas de MongoDB Atlas o sondas personalizadas para identificar y abordar la degradación de la red antes de que afecte a los usuarios.
- Las operaciones individuales enviadas por separado incurren en penalizaciones de latencia acumulativa en comparación con las operaciones por lotes. Utilice operaciones en bloque cuando corresponda.
- Las aplicaciones con usuarios distribuidos globalmente pueden necesitar estrategias como la localidad de datos o el almacenamiento en caché de borde para mitigar los efectos de alta latencia.

En conclusión, la latencia representa un factor significativo, aunque a menudo ignorado, que afecta el rendimiento y la experiencia del usuario de MongoDB. El impacto acumulativo de incluso pequeñas mejoras en la latencia puede mejorar drásticamente la capacidad de respuesta de las aplicaciones, especialmente a gran escala. Al abordar estratégicamente la ubicación geográfica, la calidad de la red, la agrupación de operaciones y la localización de los datos, los desarrolladores pueden minimizar los cuellos de botella de la latencia y ofrecer la experiencia de respuesta que los usuarios esperan, incluso en aplicaciones distribuidas globalmente. Recuerde que la optimización de la latencia no es una tarea puntual, sino un proceso continuo que requiere medición, análisis y refinamiento regulares a medida que su aplicación y su base de usuarios evolucionan.

Rotación de conexiones

La creación y destrucción de conexiones con frecuencia genera una sobrecarga significativa, lo que puede afectar drásticamente el rendimiento de las aplicaciones y la utilización de recursos.

La pérdida de conexiones se refiere a la rápida

Establecimiento y finalización de conexiones a bases de datos, lo que implica varias operaciones costosas cada vez. A continuación, se muestran algunos ejemplos:

- Las nuevas conexiones requieren protocolos de enlace TCP, a menudo con negociación TLS. Cada protocolo de enlace TCP requiere al menos un viaje de ida y vuelta entre el cliente y el servidor, mientras que la negociación TLS añade múltiples viajes de ida y vuelta adicionales y complejas operaciones criptográficas que pueden tardar cientos de milisegundos en completarse, especialmente entre regiones geográficas.
- La autenticación SCRAM de MongoDB requiere múltiples idas y vueltas. Este mecanismo de autenticación segura ejecuta una secuencia de desafío-respuesta que normalmente requiere de 2 a 3 idas y vueltas de red antes de que la conexión se autentique y esté lista para usarse, lo que añade latencia a cada nueva conexión.
- Tanto el cliente como el servidor deben asignar memoria y recursos para cada conexión. En el lado del servidor, MongoDB asigna búferes de memoria, crea recursos de subprocesos y mantiene el estado de cada conexión. En el lado del cliente, se asignan recursos similares, incluyendo búferes de sockets y estructuras de gestión, que pueden llegar a ser considerables con cientos o miles de conexiones.
- Los protocolos de enlace TLS modernos implican operaciones criptográficas complejas, como el intercambio de claves, la validación de certificados y la negociación de cifrados. Estas operaciones consumen ciclos de CPU, lo que, a altas velocidades de conexión, puede contribuir al consumo de CPU.

La pérdida de conexión es particularmente problemática en entornos sin servidor o arquitecturas de microservicios donde los servicios

Se inician y detienen con frecuencia. En entornos sin servidor, cada invocación de función podría establecer una nueva conexión a la base de datos si no se gestiona correctamente. A gran escala, esto puede resultar en miles de intentos de conexión por segundo durante picos de tráfico, saturando tanto el servidor de base de datos como la infraestructura de la aplicación.

En secciones posteriores, exploraremos cómo abordar la pérdida de conexiones mediante el uso de agrupaciones de conexiones. También analizaremos brevemente cómo minimizarla en un entorno sin servidor.

Saturación de la red

Cuando las conexiones o el ancho de banda alcanzan sus límites, el rendimiento se degrada rápidamente, lo que puede generar una serie de posibles problemas:

- Los servidores MongoDB tienen límites configurados en conexiones simultáneas
- Demasiadas conexiones activas pueden consumir los subprocesos disponibles del servidor
- Cuando las conexiones están saturadas, las operaciones deben esperar en el cola
- Las operaciones excesivas pueden saturar el ancho de banda de red disponible

Monitorea el uso de la conexión para asegurarte de no estar cerca de los umbrales de saturación. Ser proactivo ayuda a evitar fallos en cascada que aparecen repentinamente bajo carga.

Ahora que hemos establecido la importancia de gestionar estos factores de red, profundicemos en cómo se establecen, mantienen y finalizan las conexiones de MongoDB a lo largo de su ciclo de vida. Comprender estos mecanismos proporcionará información práctica para optimizar las interacciones de su aplicación con la base de datos.

Comprender el ciclo de vida de la conexión

Las conexiones de MongoDB tienen un ciclo de vida bien definido. Comprender este ciclo de vida puede ayudarle a crear aplicaciones eficientes y escalables. El ciclo de vida de la conexión se agrupa en tres etapas, que exploraremos en las siguientes secciones.

Establecimiento de conexión

Establecer una conexión MongoDB implica una serie de protocolos de enlace de bajo nivel y a nivel de aplicación que preparan al cliente y al servidor para una comunicación segura y eficiente. Cada paso de este proceso conlleva cierta sobrecarga, pero juntos forman la base de una sesión fiable y autenticada:

1. El protocolo de enlace de tres vías TCP crea la conexión de red : este paso inicial establece la comunicación entre su aplicación y el servidor MongoDB utilizando el intercambio de paquetes estándar SYN, SYN-ACK, ACK.
2. El protocolo de enlace TLS/SSL (si está habilitado) negocia de forma segura Comunicación : Cuando el cifrado de red en tránsito está habilitado, este protocolo de enlace adicional intercambia certificados,

Verifica identidades y negocia protocolos de cifrado. Las implementaciones modernas de MongoDB suelen usar TLS 1.2 o 1.3 para una seguridad óptima.

3. El protocolo de enlace de MongoDB intercambia información de versión y autenticación : Una vez establecida la conexión de red, el controlador y el servidor intercambian información de protocolo, incluyendo las funciones compatibles, las capacidades del servidor y la compatibilidad de versiones de MongoDB. Esto garantiza que el controlador y el servidor puedan comunicarse correctamente utilizando funciones compatibles.
4. El proceso de autenticación valida las credenciales : Según su configuración de seguridad, MongoDB verifica las credenciales del usuario mediante los mecanismos de autenticación configurados. Los intentos de autenticación fallidos se registran y pueden generar alertas de seguridad según su configuración.
5. El servidor asigna recursos para la nueva conexión : Cada

La conexión consume recursos del servidor, incluyendo memoria para búferes, capacidad de programación de subprocesos y descriptores de archivos. Estos recursos son limitados en cualquier servidor, por lo que existen límites de conexión para evitar el agotamiento de recursos, lo que podría afectar el rendimiento o la estabilidad de la base de datos.

En conjunto, estos pasos garantizan que cada nueva conexión sea segura, autenticada y totalmente preparada para la interacción cliente-servidor.

Utilización y agrupación de conexiones

Los controladores de MongoDB utilizan la agrupación de conexiones para optimizar el rendimiento, reducir la latencia y gestionar los recursos de forma más eficiente. Así es como funciona la agrupación en segundo plano:

- La agrupación de conexiones mantiene las conexiones ya establecidas : los controladores MongoDB implementan agrupaciones de conexiones que preestablecen y mantienen un conjunto de conexiones. Esto elimina la sobrecarga de establecimiento para operaciones posteriores y proporciona acceso inmediato a la base de datos. recursos.
- Las operaciones adquieren conexiones del pool cuando es necesario : cuando la aplicación ejecuta una consulta, actualización o comando, el controlador obtiene automáticamente una conexión disponible del pool. Si no hay conexiones disponibles, la operación espera (con un tiempo de espera configurable) o el pool crea una nueva conexión si la capacidad está por debajo del límite.
- Las operaciones liberan las conexiones al pool al finalizar : tras completar una operación, la conexión regresa al pool en lugar de cerrarse, lo que la deja disponible de inmediato para la siguiente operación. Este enfoque de reciclaje mejora el rendimiento al evitar el coste de establecimiento de cada operación.
- La reutilización adecuada elimina los costos de establecimiento repetido : al reutilizar las conexiones, las aplicaciones evitan la sobrecarga acumulada de los protocolos de enlace y la autenticación repetidos. En aplicaciones de alto volumen, esto puede reducir la latencia y aumentar significativamente el rendimiento general.
- La monitorización del estado garantiza la validez de las conexiones : los controladores comprueban periódicamente la validez de las conexiones del pool mediante comandos de latido ligeros. Esto evita que las aplicaciones utilicen conexiones obsoletas que podrían haber sido cerradas por

firewalls, servidores proxy o problemas de red, garantizando que las operaciones no fallen debido al uso de conexiones no válidas.

La agrupación de conexiones permite un uso eficiente de MongoDB, y comprender su función puede ayudarle a administrar mejor el rendimiento de las aplicaciones, especialmente bajo carga.

Terminación de la conexión

Ya sea por diseño o debido a eventos inesperados, las conexiones de MongoDB pueden terminar, y lo harán. Comprender las causas comunes y las mejores prácticas para gestionar la terminación garantiza que su aplicación se mantenga estable y responda correctamente.

- Las conexiones inactivas pueden cerrarse después de los tiempos de espera configurados : Para ahorrar recursos, tanto los controladores como los servidores pueden cerrar las conexiones que permanecen sin usar durante períodos prolongados. Esta limpieza evita la fuga de recursos, pero requiere el restablecimiento si la actividad se reanuda después de un tiempo de espera.
- El apagado de la aplicación debería cerrar las conexiones correctamente : Las aplicaciones bien diseñadas cierran explícitamente sus clientes MongoDB durante el apagado ordenado. Esto libera recursos del servidor rápidamente y evita fugas de conexión. La mayoría de los frameworks modernos proporcionan enlaces para registrar estas operaciones de limpieza.
- Los errores de red pueden provocar la terminación abrupta de la conexión : Problemas inesperados como particiones de red, interrupciones del firewall o fallos de hardware pueden interrumpir las conexiones abruptamente. Las aplicaciones robustas implementan lógica de reintento con

retroceso exponencial para manejar estos escenarios con elegancia y sin impacto en el usuario.

- Los reinicios del servidor o el mantenimiento requieren restablecer las conexiones : durante las operaciones de mantenimiento de MongoDB o las elecciones de conjuntos de réplicas, las conexiones existentes pueden terminarse. Los controladores detectan automáticamente estos eventos y establecen nuevas conexiones, pero las aplicaciones deben implementar un manejo de errores apropiado para las operaciones durante estas transiciones.
- Una limpieza adecuada garantiza la liberación rápida de recursos : cada conexión consume memoria, descriptores de archivo y capacidad de procesamiento tanto en el cliente como en el servidor. Una terminación correcta libera estos recursos, evitando fugas de memoria y el agotamiento de los límites del sistema, como el número máximo de descriptores de archivo.

Al administrar de forma proactiva los tiempos de espera inactivos, los cierres de aplicaciones y las desconexiones inesperadas, ayuda a garantizar que los recursos se liberen de manera limpia y que su aplicación pueda recuperarse sin problemas.



Cerrar MongoClient para evitar fugas de recursos

Si el controlador que utiliza lo admite, siempre debe llamar al método `MongoClient.close()` en la rutina de apagado de la aplicación para liberar correctamente todas las conexiones y los recursos asociados. Esto evita fugas de recursos y garantiza una finalización limpia de la aplicación. Consulte la sección "Optimización de la gestión de conexiones" más adelante en este capítulo para obtener más detalles sobre las estrategias adecuadas para la limpieza de conexiones.

Comprender el ciclo de vida de la conexión MongoDB es importante para crear aplicaciones que sean eficientes y resilientes.

El establecimiento de la conexión conlleva cierta sobrecarga, por lo que la agrupación y la reutilización eficientes son importantes para los sistemas de alto rendimiento. La gestión adecuada de la terminación de la conexión evita fugas de recursos que podrían degradar la aplicación con el tiempo.

Arquitectura de conexión de MongoDB

Comprender cómo MongoDB gestiona las conexiones del controlador al servidor es importante para diseñar aplicaciones resilientes. Esta sección abarca los mecanismos de conexión fundamentales y cómo...

Interactúan en toda la pila de aplicaciones, incluido TCP/IP y el manejo de conexiones del protocolo , de conexiones del controlador del servidor y agrupación MongoDB Wire .

La comunicación cliente-servidor de MongoDB se basa en TCP/IP, pero con varias mejoras y optimizaciones específicas de MongoDB.

Comprender estos detalles técnicos ayuda a los desarrolladores a diagnosticar problemas de conexión y optimizar el rendimiento de la red.

El protocolo MongoDB Wire es un protocolo basado en sockets, de tipo solicitud-respuesta, que define cómo se comunican clientes y servidores a través de sockets TCP/IP estándar. Tiene varias características que lo definen:

- Formato JSON binario (BSON) : a diferencia de los protocolos basados en texto (como HTTP), MongoDB utiliza BSON para una serialización y deserialización eficientes con una sobrecarga de CPU mínima.

- Formato de mensaje moderno : desde MongoDB 5.1, OP_MSG ha sido el código de operación principal utilizado tanto para solicitudes de clientes como para respuestas de bases de datos, reemplazando códigos de operación más antiguos como OP_QUERY y OP_REPLY .
- Soporte de compresión : OP_COMPRESSED envuelve otros códigos de operación utilizando algoritmos de compresión como snappy, zlib o zstd para reducir el ancho de banda de la red.
- Sumas de comprobación de mensajes : las sumas de comprobación CRC-32C opcionales garantizan la integridad de los datos, lo que es especialmente importante para conexiones que no son TLS.

Cuando un cliente se conecta a MongoDB, se producen varias operaciones secuencialmente, con posibles implicaciones en el rendimiento en cada etapa:

1. Etapa de resolución de DNS : El proceso de conexión comienza con Resolución de direcciones de servidor MongoDB. La resolución DNS traduce los nombres de host a direcciones IP, y los registros SRV permiten la detección inteligente de servicios para conjuntos de réplicas y clústeres fragmentados. La configuración del TTL en los registros DNS afecta la rapidez con la que los cambios de topología se propagan a los clientes. Las resoluciones fallidas pueden activar algoritmos de reintento de retroceso exponencial en los controladores, lo que podría causar retrasos en la conexión. Una vez resueltas las direcciones, la conexión pasa a la etapa de establecimiento de TCP.
2. Etapa de conexión TCP : Después de la resolución DNS, el cliente Establece las bases de la red. Los paquetes de mantenimiento de conexión ayudan a mantener conexiones de larga duración y a detectar conexiones inactivas y recursos libres. El escalado de la ventana TCP permite un mayor rendimiento en conexiones de alta latencia al permitir mayores cantidades de datos en tránsito antes de requerir confirmación.

Una vez establecida la conexión TCP, se aplican las capas de seguridad.

3. Etapa de seguridad TLS/SSL : Con la conexión TCP establecida, se establece el cifrado de seguridad. La validación del certificado introduce un pequeño retraso mientras se verifica la autenticidad del servidor.

La reanudación de sesión TLS puede reducir la sobrecarga del protocolo de enlace en las reconexiones al reutilizar los parámetros de sesiones anteriores. Con TLS 1.3 (compatible con las versiones más recientes de MongoDB), los viajes de ida y vuelta del protocolo de enlace se reducen de dos a uno, lo que mejora la velocidad de conexión. El conjunto de cifrado seleccionado influye tanto en la seguridad como en la sobrecarga de la CPU. Una vez establecido el canal cifrado, comienza la negociación del protocolo específico de MongoDB.

4. Etapa de protocolo de enlace específico de MongoDB : La conexión intercambia información del protocolo MongoDB. El cliente envía comandos hello (o isMaster) para intercambiar documentos de capacidad (isMaster se mantiene para compatibilidad con versiones anteriores). Durante esta etapa, los controladores detectan las versiones del servidor y ajustan la compatibilidad de las funciones según corresponda. El descubrimiento de topología identifica los servidores principal, secundario y árbitros del clúster. Parámetros del servidor como maxWireVersion determinan las funciones disponibles y la compatibilidad con el protocolo Wire. Tras establecer la compatibilidad del protocolo, la conexión pasa a la autenticación.

5. Etapa de autenticación : La etapa final verifica la identidad del cliente.

Antes de permitir operaciones con la base de datos. SCRAM-SHA-256 (predeterminado) proporciona seguridad sólida, pero requiere múltiples viajes de ida y vuelta.

La autenticación del certificado X.509 vincula la identidad a los certificados TLS

Para mayor seguridad, la autenticación mediante proxy LDAP y Kerberos añade saltos de red y latencia adicionales. Una vez completada la autenticación, la conexión queda completamente establecida y lista para usar.

Cada uno de estos pasos contribuye a la latencia total de la conexión, que puede variar desde unos pocos milisegundos en implementaciones locales hasta varios cientos de milisegundos en diferentes regiones geográficas o con esquemas de autenticación complejos. Esta sobrecarga de conexión subraya la importancia de la agrupación de conexiones para las aplicaciones de alto rendimiento.

Agrupación de conexiones del controlador. Los controladores

MongoDB son bibliotecas específicas del lenguaje que gestionan las complejidades de la comunicación con los servidores MongoDB. Una de sus funciones más importantes es la agrupación de conexiones. Cuando la aplicación inicializa el cliente MongoDB, el controlador crea un grupo de conexiones (o varios, según la topología). A continuación, el controlador toma prestada una conexión disponible del grupo cuando el código ejecuta una operación MongoDB. Envía el comando a través de la conexión y espera una respuesta. Una vez completada la operación, la conexión se devuelve al grupo para su reutilización. El controlador también mantiene el grupo de conexiones supervisando el estado de las conexiones y puede probarlas o reemplazarlas periódicamente.

Los grupos de conexiones tienen los siguientes parámetros que se pueden ajustar:

- **maxPoolSize** cantidad máxima de conexiones que el grupo puede mantener (valor predeterminado: 100 en la mayoría de los controladores).

- **minPoolSize** El número mínimo de conexiones que el grupo puede mantener, incluso cuando está inactivo (predeterminado: 0 en la mayoría de los controladores).
- **maxConnecting** Limita la cantidad de conexiones que se pueden establecer simultáneamente (predeterminado: 2 en la mayoría de los controladores). Este parámetro no está disponible en el controlador Rust.
- **maxIdleTimeMS** El tiempo que una conexión inactiva permanece en el Pool antes de ser cerrada.



Optimización de maxPoolSize para aplicaciones asincrónicas

Para aplicaciones asincrónicas, configurar **maxPoolSize** para que coincida con la simultaneidad esperada en lugar del tamaño de su grupo de subprocesos a menudo da como resultado un mejor rendimiento.

Para obtener más información sobre los parámetros de agrupación de conexiones anteriores, asegúrese de consultar la documentación de MongoDB en

<https://www.mongodb.com/docs/manual/tutorial/connection-pool-performance-tuning/>.

El comportamiento del grupo de conexiones varía significativamente según si el controlador es síncrono o asíncrono. En los controladores síncronos, cuando no hay conexiones disponibles, el subproceso que realiza la llamada bloquea las operaciones hasta que una conexión esté disponible o hasta que expire el tiempo de espera de la cola. Por otro lado, los controladores asíncronos devuelven un futuro o una promesa que se resuelve cuando una conexión está disponible, en lugar de bloquearse.

Gestión de conexiones del servidor MongoDB. La gestión eficiente de conexiones permite implementaciones MongoDB escalables y de alto rendimiento. En el servidor, MongoDB utiliza una combinación de parámetros configurables y recursos del sistema operativo para gestionar las conexiones entrantes de los clientes:

- **net.maxIncomingConnections** : el número máximo de conexiones de cliente simultáneas (predeterminado: 1 millón en Windows; en Linux, el valor predeterminado es el 80 % del límite máximo de descriptores de archivo). Dado que cada conexión requiere un descriptor de archivo, este límite ayuda a evitar el agotamiento de recursos en el servidor.
- Límites del sistema operativo : Los límites del descriptor de archivo (`ulimit -n` en sistemas Linux/Unix) también pueden restringir la cantidad de conexiones.

Los detalles de la administración de la conexión también dependen de la topología de implementación (ya sea una instancia independiente, un conjunto de réplicas o un clúster fragmentado):

- Independiente : las aplicaciones se conectan directamente a un único servidor MongoDB con un único grupo de conexiones
- Conjunto de réplicas : los controladores descubren a todos los miembros y mantienen grupos de conexiones separados para cada uno, enrutando las operaciones según las preferencias de lectura.
- Clúster fragmentado : las aplicaciones se conectan a enrutadores mongoes , que mantienen sus propias conexiones a los fragmentos, creando una arquitectura de conexión de múltiples niveles.

Ahora que está familiarizado con la arquitectura de conexión de MongoDB, analicemos cómo facilita la supervisión y la resolución de problemas.

conexiones.

Monitoreo y resolución de problemas de conexiones

Una gestión eficaz de las conexiones requiere una monitorización continua y la capacidad de diagnosticar rápidamente los problemas cuando ocurren. Al implementar prácticas de monitorización proactiva y contar con estrategias claras de resolución de problemas, puede identificar posibles problemas antes de que afecten el rendimiento y la fiabilidad de su aplicación.

Los controladores de MongoDB exponen métricas detalladas sobre sus grupos de conexiones, lo que le brinda visibilidad sobre cómo su aplicación utiliza las conexiones de base de datos.

La mayoría de los controladores de MongoDB implementan la especificación de Monitoreo y Agrupamiento de Conexiones (CMAP), aunque cada controlador de lenguaje ofrece diferentes métodos para acceder a estas métricas. Por ejemplo, en Node.js, se pueden usar escuchas de eventos para supervisar la actividad del grupo de conexiones:

```
// Ejemplo de Node.js que utiliza escuchas de eventos para eventos CMAP
const uri = "mongodb+srv://usuario:pass@abcdef.mongodb.net/" const cliente = new
MongoClient(uri); cliente.on('connectionPoolCreated', evento
=> console.log(e cliente.on('connectionCheckedOut', evento => console.log(ev
cliente.on('connectionCheckedIn', evento => console.log(eve
```

Ver

[https://www.mongodb.com/docs/drivers/node/current/monitoreo-y-registro/monitoreo/
#connection-](https://www.mongodb.com/docs/drivers/node/current/monitoreo-y-registro/monitoreo/#connection-)

[pool-events](#) para obtener más información sobre el seguimiento de estas métricas de conexión mediante el controlador Node.js.

Mientras que las métricas del controlador capturan la visión de la aplicación sobre sus interacciones con la base de datos, los servidores MongoDB proporcionan métricas complementarias sobre las conexiones entrantes desde su perspectiva. Estas métricas pueden ayudarle a comprender el uso general de las conexiones de su implementación en todas las aplicaciones cliente:

- **connections.current** Número de conexiones de cliente activas a El servidor. Acercarse al máximo indica potencial saturación de la conexión.
- **connections.available** Cuántas conexiones más puede aceptar el servidor antes de alcanzar su límite configurado.
- **connections.totalCreated** Número total de conexiones Creado desde el inicio del servidor. Un aumento rápido puede indicar una pérdida de conexión.

Puede recuperar estas métricas a través del comando `serverStatus` en MongoDB Shell o a través de su controlador MongoDB preferido:

```
db.adminCommand({ estado del servidor: 1 })
```

Al correlacionar las métricas del lado del cliente y del servidor, puede obtener una visión completa del estado de la conexión en toda su pila de aplicaciones.

Consulte la documentación de MongoDB para el comando `serverStatus` en

<https://www.mongodb.com/docs/manual/reference/command/serverStatus/#connections> para obtener más información.

Cuando ocurren problemas de conexión, suelen manifestarse como tipos de error específicos. Aprender a reconocer estos síntomas es crucial para una solución eficaz de problemas:

- **socketTimeout** : operación tarda más que el `socketTimeoutMS` configurado . Suele indicar congestión de la red o servidores sobrecargados.
- **EOF** (Fin de archivo) : El servidor cerró la conexión inesperadamente. Esto puede deberse a reinicios del servidor, interrupciones de la red o políticas de tiempo de espera por inactividad.
- **Broken pipe** : conexión se cortó mientras el cliente aún la usaba. Generalmente indica fallos de red o interrupciones del proxy.
- Errores de autenticación : Credenciales o autenticación incorrectas Configuración. Puede ocurrir después de rotaciones de credenciales o cambios de configuración.

El agotamiento del pool de conexiones es una condición particularmente grave que puede degradar rápidamente el rendimiento de la aplicación. Preste atención a estas señales de advertencia:

- Aumento de la longitud de la cola de espera en las métricas del conductor
- **MongoWaitQueueTimeoutException** o errores similares en los registros de la aplicación
- Aumentos repentinos en la latencia de la operación
- Errores de solicitud durante picos de tráfico

Es importante recordar que el agotamiento del grupo de conexiones puede derivar rápidamente en fallas en todo el sistema a medida que los tiempos de espera activan reintentos, que generan más solicitudes de conexión y aceleran el agotamiento.

En implementaciones que utilizan detección de servicios basada en TLS o DNS, pueden surgir problemas de conexión adicionales debido a problemas de seguridad o configuración de red. Estos problemas pueden ser especialmente complejos, ya que pueden aparecer de forma intermitente o solo en circunstancias específicas. La seguridad de las conexiones depende de una configuración de TLS y una gestión de certificados adecuadas. Si experimenta fallos de conexión relacionados con TLS, verifique lo siguiente:

- Las cadenas de certificados son válidas y confiables para todos los clientes.
- Los nombres de host del certificado coinciden con el nombre de host configurado del servidor
- Las versiones del protocolo TLS y los conjuntos de cifrado son compatibles entre el cliente y el servidor
- Las fechas de vencimiento de los certificados son válidas y no están próximas a vencer.
- Los certificados intermedios están correctamente instalados en todos los sistemas

Un problema común ocurre cuando los clientes y servidores no coinciden

Configuraciones de TLS. Por ejemplo, si su aplicación requiere TLS 1.3, pero su servidor MongoDB solo admite hasta TLS 1.2, las conexiones fallarán aunque ambos sistemas tengan certificados válidos.

MongoDB Atlas y los conjuntos de réplicas suelen depender de los registros SRV de DNS para la detección de servicios. Cuando se producen problemas de conexión en estos entornos, la configuración del DNS suele ser la causa. Para problemas de resolución de SRV de DNS, puede utilizar herramientas de diagnóstico de DNS estándar para verificar la configuración correcta:

```
dig srv _mongodb._tcp.su-nombre-de-host-del-clúster.mongodb.net
dig txt su-nombre-de-host-del-clúster.mongodb.net
```


Estos comandos ayudan a verificar que su DNS esté configurado correctamente para soportar los mecanismos de descubrimiento de servicios de MongoDB, que son especialmente importantes en implementaciones de conjuntos de réplicas y clústeres fragmentados. Al trabajar con MongoDB Atlas, recuerde que los retrasos en la propagación del DNS pueden causar problemas de conexión temporales tras modificar la configuración del clúster. Si modificó recientemente su clúster Atlas, espere el tiempo suficiente para que las actualizaciones del DNS se propaguen por la red.

Prácticas recomendadas para la monitorización de conexiones. Tras comprender cómo diagnosticar problemas de conexión específicos, implementar una estrategia integral de monitorización es esencial para mantener el buen estado del sistema a lo largo del tiempo. Para una monitorización eficaz de las conexiones, siga estos pasos:

1. Establecer líneas de base : comprender los patrones de conexión normales para su aplicación
2. Establecer umbrales significativos : configure alertas basadas en el uso de la conexión en relación con los límites
3. Correlacionar métricas : conectar las métricas del lado del controlador y del lado del servidor para una imagen completa
4. Monitorear tendencias : estar atento a cambios graduales que podrían indicar problemas en evolución.
5. Automatice las alertas : configure notificaciones proactivas para tiempos de espera altos o recuentos de conexiones que se acercan al límite.

Al combinar estas prácticas de monitoreo con los enfoques de resolución de problemas descritos anteriormente, puede construir una conexión sólida

estrategia de gestión que ayuda a prevenir problemas antes de que afecten a sus usuarios.

Optimización de la gestión de conexiones

Ahora que comprende cómo lograr y mantener un sistema resiliente mediante la gestión de conexiones, así como cómo supervisar y solucionar problemas de conexión, profundicemos en el tema y analicemos cómo optimizar la gestión de conexiones. Lograr un rendimiento y una resiliencia óptimos en MongoDB requiere un ajuste meticuloso de su estrategia de gestión de conexiones. Esta sección abarca los enfoques clave para optimizar los grupos de conexiones e implementar estrategias para diferentes patrones de carga de trabajo.

Optimización del pool de conexiones. Para optimizar el pool de conexiones, es fundamental garantizar que su tamaño sea el ideal para la aplicación. El tamaño ideal del pool de conexiones varía según las características de la carga de trabajo de la aplicación:

- Cargas de trabajo OLTP : comience con tamaños de grupo moderados (50 a 100 conexiones) para muchas operaciones cortas
- Cargas de trabajo OLAP : utilice grupos más pequeños para que coincidan con las operaciones analíticas simultáneas esperadas
- Procesamiento por lotes : dimensione el grupo para que coincida con el paralelismo de sus operaciones por lotes

Una práctica recomendada es ajustar el tamaño del pool de conexiones según las métricas de rendimiento de la aplicación y el comportamiento observado, en lugar de depender de un porcentaje de utilización fijo. Supervise indicadores clave como el tamaño de la cola de espera, la latencia de la operación y las métricas de conexión del servidor para determinar si es necesario ajustar el tamaño del pool.

Tenga en cuenta estas pautas al ajustar los grupos de conexiones:

- Si su aplicación dedica demasiado tiempo a crear nuevas conexiones, aumente `minPoolSize` para garantizar que haya suficientes conexiones disponibles al inicio.
- Si el uso de la CPU de la base de datos es mayor de lo esperado con muchos intentos de conexión, considere reducir `maxPoolSize` para disminuir la carga.
- Si la carga es baja pero el rendimiento de la aplicación es menor al esperado, es posible que deba aumentar `maxPoolSize` o ajustar los subprocesos de la aplicación.

Recuerde que cada controlador MongoDB crea un grupo de conexiones separado para cada servidor en su topología, por lo que las conexiones totales de una aplicación serán `maxPoolSize` multiplicado por la cantidad de servidores MongoDB.

Configuración del tiempo de espera de conexión. Para crear aplicaciones resilientes y de alto rendimiento que gestionen eficazmente las interrupciones de la red y la degradación del servicio, es necesario configurar los tiempos de espera adecuados. Una configuración incorrecta de los tiempos de espera puede provocar fallos en cascada, agotamiento de recursos o problemas de rendimiento de los usuarios.

MongoDB proporciona varios parámetros de tiempo de espera configurables que controlan diferentes aspectos del comportamiento de la conexión:

- **connectTimeoutMS** Tiempo de espera del controlador al establecer una nueva conexión. Establézcalo en un valor mayor que la latencia de red máxima que tenga con cualquier miembro de su implementación.
Tiempo de
- **socketTimeoutMS** Tiempo de espera para una respuesta a una operación específica. Establézcalo en dos o tres veces la duración de la operación más lenta que ejecute su controlador.
Tiempo de espera de las conexiones
- **maxIdleTimeMS** (equilibrio entre la pérdida de conexiones y el uso de recursos).

Para ajustar correctamente estos parámetros de tiempo de espera es necesario comprender tanto el comportamiento de su aplicación como la topología de su implementación de MongoDB.

Importante

No utilice `socketTimeoutMS` para evitar operaciones de larga duración en el servidor. En su lugar, utilice `maxTimeMS()` o `timeoutMS()` para controladores que admitan el tiempo de espera de los operadores del lado del cliente con sus consultas, de modo que el servidor pueda cancelar operaciones de larga duración. Por ejemplo, si las operaciones agotan el tiempo de espera correctamente en el controlador, la aplicación puede intentar reintentos significativos sin saturar el servidor. Sin tiempos de espera adecuados, los reintentos podrían saturar los grupos de conexiones, aumentar el agotamiento de recursos y provocar tiempos de inactividad en todo el sistema.



Optimización del lado del servidor. En implementaciones autogestionadas, es necesario configurar tanto los parámetros de conexión de MongoDB como los límites del sistema operativo subyacente para garantizar un rendimiento óptimo. Estos ajustes se combinan para determinar cuántos clientes simultáneos pueden conectarse a la base de datos.

```
red:
  maxIncomingConnections: 65536
```

La configuración `maxIncomingConnections` controla el número máximo de conexiones de cliente simultáneas que MongoDB aceptará. Aumentar este límite permite que más aplicaciones cliente simultáneas se conecten a su servidor MongoDB, lo cual es esencial para aplicaciones con mucho tráfico o entornos con muchos microservicios. Sin embargo, aumentar este parámetro no es suficiente; también debe ajustar los límites del sistema operativo.

Configuración del sistema operativo

En sistemas Unix, MongoDB utiliza un descriptor de archivo por conexión de cliente. Cuando MongoDB alcanza el límite de `maxIncomingConnections` o el límite de descriptors de archivo del sistema operativo (el que sea menor), rechaza conexiones de cliente adicionales. Esto crea un cuello de botella que puede afectar gravemente la disponibilidad de la aplicación durante periodos de alta demanda.

Para los sistemas Linux que usan `systemd`, debe configurar el límite del descriptor de archivo en el archivo de servicio MongoDB:

```
[Servicio]
# En el Archivo de servicio systemd de MongoDB
LímiteNOFILE=64000
```

Para entornos que no sean systemd, establezca los límites para MongoDB usuario del servicio en el archivo `/etc/security/limits.conf`:

```
nofile suave de mongodb 64000
archivo nofile duro de mongodb 64000
```

Después de cambiar estos límites, MongoDB debe reiniciarse para Los cambios surtan efecto. Para confirmar que los cambios de configuración están beneficiando su implementación, puede utilizar estas verificaciones métodos:

1. Verifique los límites actuales del sistema operativo con lo siguiente:

```
ulimit -n          # Para el # sesión de shell actual
gato /proc/$(pidof mongod)/límites          Para el funcionamiento
```

2. Supervise la utilización de la conexión MongoDB a través de Comando `serverStatus`:

```
db.adminCommand({ estado del servidor: 1 }).connections
```

El resultado mostrará:

```
{
  "conexiones": {
    "actual": 123,          // Conexión activa actual
    "el bl " 63877        // R ii el bl
```

```

    "disponibles": 63877, "total
    creado": 1578
  }
}

```

Conexión disponible restante // //
Total de conexiones creadas

3. Esté atento a los errores de rechazo de conexión en los registros de MongoDB, que Indica que has alcanzado los límites de conexión:

```
[conn12345] Demasiadas conexiones abiertas
```

Debería observar una mejora en la estabilidad y el rendimiento bajo carga tras configurar correctamente el límite de `maxIncomingConnections` de MongoDB y los límites de descriptores de archivos del sistema operativo. Las aplicaciones podrán establecer conexiones de forma fiable incluso durante picos de tráfico, y evitará los fallos en cascada que se producen al alcanzar los límites de conexión. MongoDB Atlas gestiona estos ajustes automáticamente según el nivel del clúster, escalando tanto la configuración de la base de datos como los recursos de la infraestructura subyacente para adaptarlos a las necesidades de su carga de trabajo.

Optimización del rendimiento aprovechando la compresión de la red

La compresión de red es una técnica de optimización importante en MongoDB que reduce la cantidad de datos transmitidos entre clientes y servidores. Al comprimir los datos antes de la transmisión y descomprimirlos al recibirlos, MongoDB puede reducir significativamente los requisitos de ancho de banda de la red, a la vez que mejora potencialmente el rendimiento general. Esta función es especialmente valiosa cuando

operando sobre conexiones de alta latencia o en entornos con restricciones de ancho de banda.

Beneficios y desventajas de la compresión de red Al implementar la compresión de red en su implementación de MongoDB, es importante comprender tanto las ventajas como las posibles desventajas para tomar decisiones informadas.

Una de las principales ventajas de habilitar la compresión es la reducción significativa del consumo de ancho de banda de la red. Al comprimir los datos antes de la transmisión, MongoDB puede reducir significativamente la cantidad total de datos transferidos a través de la red.

Esto es especialmente efectivo para documentos con mucho texto, como datos JSON con nombres de campo o valores de cadena extensos, que suelen comprimirse muy bien. Otra ventaja notable es la posibilidad de reducir la latencia y los tiempos de respuesta en ciertos entornos.

Dado que es necesario que viajen menos datos a través de la red, las operaciones pueden completarse más rápidamente, lo que genera una experiencia de aplicación más receptiva para los usuarios.

Sin embargo, hay consideraciones importantes que deben tenerse en cuenta antes de habilitar la compresión en un entorno de producción. La utilización de los recursos de la CPU es un factor importante.

Las operaciones de compresión y descompresión requieren recursos computacionales tanto del lado del cliente como del servidor. En entornos donde

Los recursos de la CPU ya están limitados y los requisitos de procesamiento adicionales podrían compensar los beneficios de la red.

También es importante reconocer que la compresión no siempre produce mejoras significativas en el rendimiento y, en algunos casos, incluso puede generar ineficiencias. En operaciones pequeñas o al trabajar con datos binarios ya comprimidos (como imágenes o vídeos), la sobrecarga de la compresión y la descompresión podría superar los beneficios de la reducción del tráfico de red. En estos casos, habilitar la compresión podría reducir el rendimiento general.

Algoritmos de compresión disponibles

MongoDB ofrece varias opciones de algoritmos de compresión, cada una con diferentes características que las hacen adecuadas para diversos usos.

casos:

- Snappy ofrece un excelente equilibrio entre velocidad y ratio de compresión. Normalmente comprime los datos a entre un 30 % y un 50 % de su tamaño original con un consumo mínimo de recursos de CPU. Esto lo convierte en la opción predeterminada ideal para la mayoría de las implementaciones de MongoDB, ya que proporciona un ahorro significativo de ancho de banda sin una sobrecarga de procesamiento significativa.
- Zlib ofrece ratios de compresión más altos que Snappy, lo que podría reducir el tamaño de los datos entre un 20 % y un 40 % del original. Sin embargo, esto implica un mayor uso de CPU tanto para la compresión como para la descompresión. Zlib puede ser adecuado cuando el ancho de banda es extremadamente limitado y los recursos de CPU son abundantes.
- Zstd (disponible en las versiones más recientes de MongoDB) proporciona excelentes ratios de compresión a la vez que mantiene una eficiencia de CPU razonable. A menudo logra una mejor compresión que zlib.

menor costo computacional, lo que lo convierte en una opción cada vez más popular para los entornos que lo admiten.

La elección de un algoritmo de compresión depende de las compensaciones específicas que su sistema puede permitirse entre el uso de la CPU y el ancho de banda de la red, y lo ideal es que esté guiada por una evaluación comparativa en el contexto de su carga de trabajo.

Implementar la compresión de red. Habilitar la compresión en MongoDB es sencillo y requiere cambios mínimos de configuración. Puede especificar sus algoritmos de compresión preferidos mediante la cadena de conexión al establecer una conexión con la base de datos:

```
mongodb://nombredehost/?compresores=snappy,zlib
```

Al establecerse una conexión, el cliente y el servidor negocian para seleccionar el mejor algoritmo de compresión compatible entre ambos de la lista proporcionada. Si se especifican varios algoritmos (como en el ejemplo anterior), se evalúan en orden de preferencia, de izquierda a derecha.

De forma predeterminada, las implementaciones de MongoDB tienen la compresión habilitada; sin embargo, los mensajes entre los clientes y el clúster no se comprimirán a menos que el cliente especifique una configuración de compresión .

Habilitar la compresión de red con el algoritmo adecuado a sus requisitos específicos puede reducir la utilización de la red de su implementación de MongoDB, manteniendo al mismo tiempo un rendimiento excelente.

Estrategias de conexión para entornos sin servidor

La gestión de conexiones en entornos sin servidor presenta desafíos únicos debido a su naturaleza efímera y su modelo de ejecución distintivo. Esta sección explora estrategias integrales de conexión de MongoDB para funciones sin servidor (AWS Lambda, Azure Functions, Google Cloud Functions), abarcando los desafíos fundamentales de los contextos de ejecución efímera, los arranques en frío, las ráfagas de conexión durante eventos de escalado y las limitaciones de recursos. A continuación, se presentan patrones de implementación prácticos que incluyen la optimización del grupo de conexiones, técnicas de inicialización fuera de los controladores de funciones y métodos para mantener la persistencia de la conexión entre invocaciones. Todo ello diseñado para ayudarle a crear aplicaciones sin servidor resilientes y eficientes con conectividad MongoDB fiable, a la vez que minimiza los costes y maximiza el rendimiento.

Las funciones sin servidor son efímeras por diseño, lo que genera desafíos únicos en la gestión de conexiones:

- Ejecución efímera : las funciones se inician, procesan solicitudes y pueden finalizar en cualquier momento.
- Arranques en frío : las nuevas instancias de función deben establecer nuevas conexiones
- Ráfagas de conexión : muchas invocaciones de funciones simultáneas pueden crear picos de conexión
- Restricciones de recursos : las instancias de función tienen memoria y limitaciones de la CPU

El patrón más crítico para entornos sin servidor es la inicialización su cliente MongoDB y el grupo de conexiones fuera de la función manejador. Echemos un vistazo al siguiente ejemplo para que puedas obtener una Mejor comprensión:

```
constante { MongoClient } = require('mongodb');  
// MongoClient ahora se conecta automáticamente // promete en cualquier lugar y hace referencia  
const cliente = nuevo MongoClient(proceso.env.MONGODB_URI);  
exportar const handler = async() => {  
  const bases de datos = await cliente.db('admin').comando({ li  
  devolver {  
    Código de estado: 200,  
    bases de datos: bases de datos  
  };  
};
```

Este código demuestra una implementación de MongoDB

Patrón de conexión optimizado específicamente para AWS Lambda, pero

Podría adaptarse a otros entornos sin servidor.

La instancia de MongoClient se crea fuera del alcance de la manejador de funciones, lo que permitirá su reutilización. El intencional

La exclusión de client.close() al final del controlador debe mantenerse

El grupo de conexiones permanece activo para invocaciones posteriores, lo que reduce latencia para llamadas de función “calientes”.

Los grupos de conexiones para entornos sin servidor generalmente requieren configuraciones diferentes a las aplicaciones tradicionales:

- Tamaño de piscina más pequeño : comience con una piscina de tamaño máximo pequeño (3-5 conexiones) ya que cada instancia de función tiene su propio grupo

- Tiempo de espera inactivo más bajo : establezca un límite de `maxIdleTimeMS` más corto para liberar conexiones no utilizadas más rápidamente
- Evite grupos mínimos excesivos : utilice `minPoolSize: 0` para evitar mantener conexiones que podrían no usarse
- Establecer tiempos de espera de cola adecuados : que coincidan con los límites de tiempo de ejecución de la función

El uso de estrategias de conexión adecuadas en entornos sin servidor puede ayudar a mitigar algunos de los desafíos típicos de estas funciones efímeras. Al inicializar estratégicamente los grupos de conexiones fuera de los controladores de funciones con parámetros optimizados, los desarrolladores pueden mitigar los arranques en frío y las ráfagas de conexión, garantizando interacciones persistentes y de baja latencia con la base de datos.

Resumen

Una gestión eficaz de las conexiones de MongoDB es esencial para crear aplicaciones resilientes y de alto rendimiento. A lo largo de este capítulo, hemos explorado cómo la pila de red constituye una base fundamental para la arquitectura de su base de datos, desde los factores fundamentales de la red: latencia, pérdida de conexiones y saturación, hasta los detalles técnicos del protocolo de conexión de MongoDB y el ciclo de vida de las conexiones.

La agrupación de conexiones se erige como la técnica de optimización más eficaz, ya que mejora significativamente el rendimiento de las aplicaciones al reutilizar las conexiones TCP y distribuir los costos de autenticación entre múltiples operaciones. Una implementación adecuada requiere comprender tanto los parámetros de configuración de la agrupación de conexiones del lado del cliente como los límites correspondientes del lado del servidor. La relación crítica entre MongoDB...

Los límites del parámetro `maxIncomingConnections` y del descriptor de archivo del sistema operativo merecen especial atención, ya que estas configuraciones funcionan juntas para determinar la capacidad de conexión durante los períodos pico de tráfico.

Los diferentes entornos informáticos requieren enfoques específicos de gestión de conexiones. Para implementaciones sin servidor, inicialice los grupos de conexiones fuera de los controladores de funciones y utilice grupos más pequeños con valores de tiempo de espera adecuados. En arquitecturas distribuidas, ajuste la configuración de tiempo de espera según las características de la red y utilice el enrutamiento con reconocimiento de localidad siempre que sea posible.

El monitoreo regular de las métricas del lado del cliente, como la utilización del grupo y los tiempos de espera en la cola, combinado con las métricas del estado del servidor, lado del servidor, crea una vista integral del estado de la conexión. Esta capacidad de observación le permite evitar fallas en cascada mediante la detección temprana y ajustes de tamaño adecuados.

Al implementar estos principios técnicos en toda su pila de tecnología, desarrollará aplicaciones MongoDB que mantienen un rendimiento constante incluso en condiciones de red difíciles, lo que garantiza tanto la confiabilidad como la utilización eficiente de los recursos a medida que sus volúmenes de datos y su base de usuarios se expanden.

El siguiente capítulo profundiza en los patrones de indexación y consulta de MongoDB, examinando el ciclo de vida de la ejecución de consultas, analizando la salida de `Explain` y diagnosticando consultas lentas mediante registros. También aborda el dominio de la semántica MQL con el uso adecuado de índices y el aprovechamiento de colecciones y funciones especializadas para optimizar el rendimiento general de la base de datos.

12

Conceptos avanzados de consulta e indexación

Hay muchas razones por las que sus consultas podrían ser `slow`, un término subjetivo que usualmente significa más lento de lo esperado o necesario. Muchos factores que pueden contribuir a esto incluyen la falta de índices, índices subóptimos, recursos de hardware insuficientes, el uso innecesario de operaciones ineficientes o demasiadas operaciones que compiten por los mismos recursos.

En este capítulo, analizaremos algunos conceptos avanzados sobre la ejecución de consultas, incluyendo cómo las consultas usan índices, cómo el optimizador de consultas selecciona un índice, cómo profundizar en el uso del comando `explain` y técnicas para mejorar el rendimiento si es posible. También presentaremos algunos tipos de índices y colecciones especiales que pueden ser útiles cuando las estrategias de indexación habituales resulten inadecuadas. Describiremos también otros puntos donde la ineficiencia puede afectar a su sistema y ofreceremos otros consejos para un rendimiento óptimo.

Los temas tratados en este capítulo son los siguientes:

- El ciclo de vida de la ejecución de consultas
- El proceso de evaluar y analizar la producción `explain`
- Uso de mensajes de registro para detectar, analizar y diagnosticar las causas fundamentales de las consultas lentas
- Comprensión de la semántica del lenguaje de consulta MongoDB y el uso de índices
- Colecciones especiales adicionales, tipos de índices y características que pueden mejorar el rendimiento

Comprender la ejecución de consultas

Es importante comprender qué sucede cuando se envía una operación desde la aplicación a la base de datos. Hay muchos aspectos donde se puede mejorar el rendimiento, y comprender completamente cada etapa permite determinar correctamente dónde las cosas no funcionan de la mejor manera posible y cómo solucionarlas.

Cuando se envía una consulta al motor de base de datos, ya sea como parte de un filtro de búsqueda, actualización o eliminación, o de agregados `$match`, ocurren varias cosas. Primero, la consulta se analiza en un formato estandarizado.

A continuación, el planificador de consultas debe determinar si existe un índice utilizable y, si hay más de uno, cuál sería el mejor. El lote inicial de resultados se devuelve al cliente. Si el cliente solicita más resultados, el comando `getMore` devolverá lotes adicionales.

Dado que seleccionar el índice correcto implica cierta sobrecarga, MongoDB utiliza un proceso que garantiza que, una vez elegido un plan para una forma de consulta específica, este se almacene en la caché de planes para su uso futuro. Una forma de consulta es la consulta que se ejecuta ignorando los valores, por lo que `{x:5}` y `{x:25}` se consideran la misma forma de consulta. También existe un mecanismo para garantizar que, si un índice no es el mejor, se reevalúe en caso de que exista uno mejor.

La figura 12.1 muestra el flujo completo del proceso de planificación de consultas:

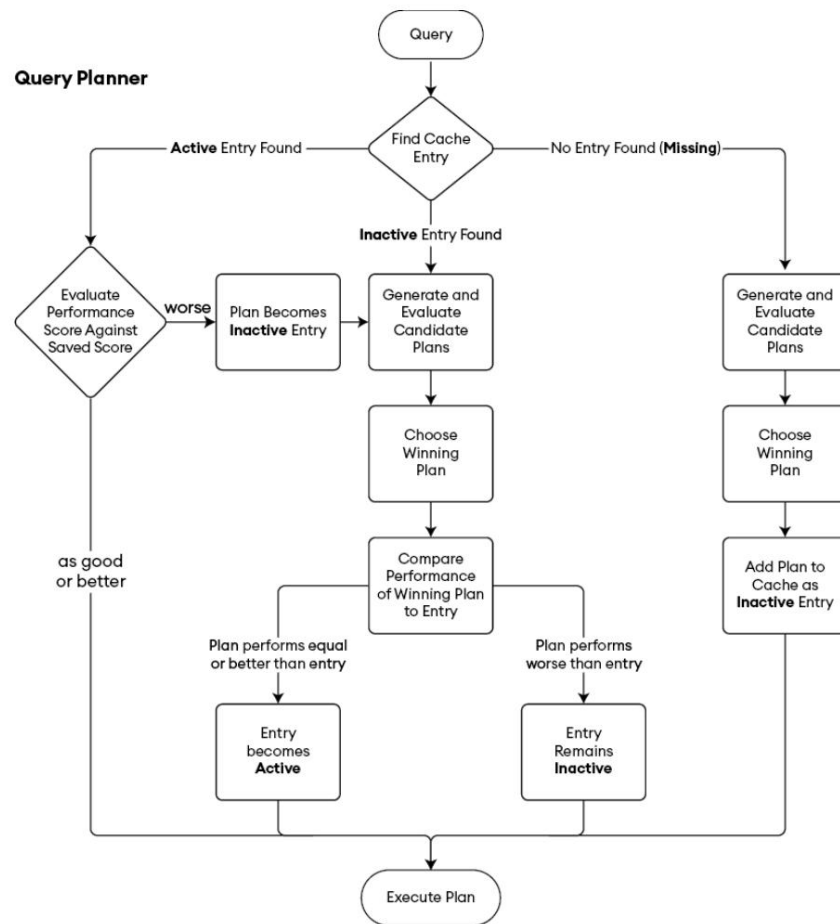


Figura 12.1: Proceso de planificación de consultas

Pero puedes verlo de una forma un poco más simple: primero, preguntamos: "¿Ya hay un plan en caché?". Si es así, úsalo; si no, genera y evalúa planes candidatos, elige el plan ganador y agrégalo a la caché para su uso futuro. Esta es una descripción del flujo un poco simplificada, ya que no tiene en cuenta que el plan ganador se guarda primero como inactivo y solo se activa después de confirmar su buen rendimiento. Sin embargo, como simplificación, es una buena aproximación.

¿Cuáles son los posibles planes candidatos para una consulta específica? Sabemos que, si no hay un índice elegible, el plan consistirá en un análisis completo de la colección (COLLSCAN). Si hay uno o varios índices disponibles, se pueden usar de diferentes maneras, según los requisitos de la consulta. En la siguiente sección, analizaremos las diferentes maneras de usar los índices.



Planifique el uso de la caché con un único índice elegible

Si solo hay un plan disponible (por ejemplo, si solo hay un índice elegible), este no se almacenará en la caché de planes. Esta caché se utiliza exclusivamente para casos en los que varios índices pueden satisfacer la consulta.

Dependiendo de la eficiencia del índice, así como de si cubre la consulta o admite filtros de ordenación y consulta, se asignará una puntuación. Los detalles exactos de cómo se califican los planes quedan fuera del alcance de este libro. Sin embargo, como principio general, la puntuación más alta representaría el plan más eficiente durante la ejecución de prueba y se convertiría en el ganador provisional. Este plan se almacena en caché junto con su puntuación, al menos hasta que una ejecución ineficiente de la misma forma de consulta con el plan obtenga una puntuación significativamente inferior a la guardada. En ese caso, el plan almacenado en caché se elimina de la caché de planes. Cuando esto sucede, la consulta se replanifica, lo que significa que pasa de nuevo por la etapa de evaluación del plan para intentar encontrar un índice mejor.

La planificación no es independiente de la entrega de resultados al cliente. Los documentos encontrados durante la planificación de la ejecución de la consulta se devuelven al cliente como el primer lote de resultados.

Etapas del plan o “cómo se pueden utilizar los índices”

Aunque acabamos de mencionar que " las consultas usan índices ", resulta que existen múltiples maneras de usarlos. En un caso estándar, al buscar un campo en un documento que coincida con un valor único en un árbol B, recorremos el árbol B y encontramos el ID de registro del nodo hoja correspondiente. Sin embargo, existen otras maneras de usar índices en consultas que no buscan documentos, sino recuentos de documentos. Existen otras optimizaciones en la forma en que se recorre el índice cuando buscamos el primer valor distinto en todo el árbol B o en una subsección del mismo. Todas estas optimizaciones tienen nombres internos diferentes.

que puede ver en el resumen del plan de consulta en los registros o en la salida de explicación .

Aquí se muestran los diferentes planes de consulta indexados y lo que significan:

- **IXSCAN** : un escaneo de índice significa que estamos recorriendo (generalmente un subconjunto de) el árbol B en algún orden
- **IDHACK o EXPRESS_IXSCAN** : escaneo de índice optimizado especial para índices _id u otros índices únicos
- **DISTINCT_SCAN** : escaneo de índice especial para ayudar a encontrar todos los valores distintos en el árbol B
- **COUNT_SCAN** : escaneo de índice especial para permitir obtener recuentos únicamente de un subconjunto del árbol B

Si no hay un índice elegible para la operación, entonces el plan de consulta será **COLLSCAN** , lo que significa una consulta completa. escaneo de colección.



Preferencia de índice sobre COLLSCAN

Si hay al menos un índice elegible para una consulta, entonces COLLSCAN no se seleccionará a menos que el usuario lo fuerce.

Si bien la eficiencia del plan de consulta se encuentra aproximadamente en el orden creciente de esta lista, esa no es una afirmación absoluta, ya que depende del tamaño del subconjunto del árbol B que el escaneo está examinando (un **COUNT_SCAN** de todo el índice probablemente será más lento que un **IXSCAN** de una pequeña porción del índice).

Verá otras etapas o pasos en el resultado del planificador de consultas. Su significado es el siguiente:

- **FETCH** : Se debe leer el documento completo y, opcionalmente, aplicar un filtro para garantizar que coincida
- **SHARD_FILTER** : Generalmente se aplica al final para garantizar que solo se devuelvan los documentos propiedad del fragmento.
- **ORDENAR** : Indica una ordenación de resultados en memoria
- **SORT_MERGE** : Se utiliza para fusionar múltiples secuencias de documentos ya ordenados

Hay algunas otras etapas más oscuras, pero no nos preocuparemos por ellas por ahora.



Las ordenaciones admitidas por índice omiten la etapa SORT

Una clasificación respaldada por un índice no tiene ninguna etapa SORT en el plan ya que no se requiere ningún paso adicional para devolver resultados ordenados.

Las métricas y los diagnósticos que surgen de la planificación y ejecución de consultas aparecen en varios lugares:

- Explicar la salida
- Mensajes registrados para consultas lentas en la colección `system.profile` de
- registros de mongod y mongos cuando habilita la creación de
- perfiles Paneles en Atlas, como la salida del comando
- `currentOp` de Query Insights

Estas métricas incluyen qué plan se seleccionó, cuántas entradas de índice y documentos se revisaron, si hubo una clasificación en memoria y varias otras que exploraremos a través de ejemplos más adelante en el capítulo.

La herramienta más detallada y útil para depurar el rendimiento del planificador de consultas será el comando `explain`.

Uso del comando `explain`. Ya explicamos cómo usar el

comando `explain` para examinar la ejecución de consultas y agregaciones. Ahora, veamos en detalle cómo se ejecuta y qué muestra. Al ejecutar un comando con un filtro de consulta, podemos invocar `explain` insertándolo entre el nombre de la colección y el método auxiliar del comando en mongosh, como en el siguiente ejemplo:

```
db.collection.explain(<opción>).find({<consulta>})
```

La `<opción>` para explicar determina qué tan detallada será la salida y puede ser una de las siguientes:

- `"queryPlanner"`: Valor predeterminado. Solo ejecuta el optimizador de consultas y devuelve el plan de consulta que se seleccionará.
- `"executionStats"`: ejecuta el optimizador de consultas, ejecuta el plan ganador elegido y devuelve estadísticas que describen la ejecución. `"allPlansExecution"`:
- igual que `"executionStats"`, pero además de devolver estadísticas sobre la ejecución del plan ganador, también devuelve estadísticas de los planes rechazados capturados durante la selección y evaluación del plan.

La salida del comando `explain` incluye varias secciones. La sección `queryPlanner` siempre está presente. Si usa la opción `"executionStats"` o `"allPlansExecution"`, verá la sección `"executionStats"`, que es muy informativa.

La sección queryPlanner

La sección queryPlanner es donde puede ver información básica sobre la consulta, incluyendo cómo se analizó en forma canónica o normalizada, queryHash que es una forma abreviada de identificar las mismas formas de consulta con diferentes valores, planCacheKey del plan estará presente si el plan de consulta estaba en la memoria caché, indicación de si la selección del plan de consulta fue influenciada por alguna configuración especial, y los planes rechazados en el caso de que hubiera opciones de plan adicionales. winningPlan ,

La sección de estadísticas de ejecución

El comando explain que ejecutó la consulta incluirá la sección "ExecutionStats" , donde se capturan todas las métricas observadas durante la ejecución. Para el filtro de la consulta, incluye campos como "totalKeysExamined " (número de claves de índice examinadas), "totalDocsExamined " (número de documentos examinados), "nReturned" (número de documentos que se devolverían al cliente si no se hubiera especificado " explain ") y los mismos detalles para cada etapa de ejecución observada.

Para las ordenaciones en memoria, el comando explain incluye una sección SORT con información sobre la cantidad de memoria utilizada. Esta sección aparece cuando no hay un índice válido para obtener los datos ordenados. También indica si los datos tuvieron que volcarse al disco y, de ser así, cuánto espacio de disco se utilizó para facilitar la ordenación.

En el caso de las agregaciones, explain también enumera cada etapa posterior a la etapa inicial \$cursor (responsable de recuperar documentos de la colección para su transmisión al resto de la canalización). Incluye el número de documentos devueltos en cada etapa, así como el tiempo acumulado de todas las etapas hasta esta.

A continuación se muestra una parte de un ejemplo de salida para una sección de estadísticas de ejecución de agregación que enumera las etapas:

```
etapas: [ {
  '$cursor': { queryPlanner: { executionStats: { ... } } } nDevuelto: Long('10805'),
  estimación de tiempo de ejecución
  en millones: Long('21') }, {
  '$set': { ... },
  nDevuelto: Long('10805'),
  EstimaciónMillisDeTiempoDeEjecución: Long('21') },
  { '$unwind': { ... },
    nDevuelto: Long('14804'),
    EstimaciónMillisDeTiempoDeEjecución: Long('22') },
  {
    '$grupo': { ... },
    maxAccumulatorMemoryUsageBytes: { recuento: Long('1768') },
    totalOutputDataSizeBytes: Long('3534'), usedDisk:
    falso, nReturned:
    Long('13'),
    executionTimeMillisEstimate: Long('23')
  }
}
```

```

    },
    {
      '$sort': { sortKey: { count: -1 } },
      totalDataSizeSortedBytesEstimate: Long('3638'), usedDisk:
      false, spills: Long('0'),
      nReturned: Long('13'),
      executionTimeMillisEstimate:
      Long('23')
    }
  }

```

La primera etapa se llama \$cursor e indica que el origen de los documentos para la canalización es una colección. Las primeras etapas de \$match y otras etapas que se ejecutan de la misma manera que el comando find se pueden observar dentro de la etapa \$cursor . Esta etapa devolvió una cierta cantidad de documentos, que luego fluyeron a través de la canalización. La estimación del tiempo de ejecución es acumulativa, por lo que vemos que la parte de consulta tardó 21 milisegundos, mientras que la mayoría de las demás etapas tardaron menos de 1 milisegundo, excepto \$unwind , que tardó 1 milisegundo cada una. y \$group .

En colecciones fragmentadas, el enrutador mongos reenvía una consulta con la opción de explicación a cada fragmento, fusiona sus resultados e incluye su propio trabajo en la salida de explicación , que tiene secciones separadas para el trabajo de los fragmentos y el trabajo de mongos .

A continuación se muestra un ejemplo de los campos de salida de explicación para una consulta de agregación en un clúster fragmentado:

```

"mergeType" : "mongos",
"splitPipeline" :
{ "shardsPart" : [ ... ],
  "mergerPart" : [ ... ]
}

```

Esto muestra que la fusión de las salidas de cada fragmento se realiza en mongos . También muestra cómo el flujo de trabajo general se divide en dos partes. La primera parte se envía a los fragmentos relevantes, mientras que la parte de fusión se ejecuta en mongos .

A continuación se muestra un ejemplo de explicación de un fragmento que necesita filtrar documentos huérfanos:

```

Plan ganador:
{ etapa: 'SHARD_MERGE',
  fragmentos:
  [ {
    shardName: 'shard01',
    winningPlan:
    { etapa: 'FETCH',
      etapa de entrada:
      { etapa: 'SHARDING_FILTER',
        etapa de entrada:
        { etapa: 'IXSCAN',
          patrón de clave: { id: 1 },
          ...
        }
      }
    }
  }
]
}

```

Este ejemplo muestra una consulta sobre un campo específico que utiliza un índice de forma eficiente. Sin embargo, dado que la clave de fragmento no está en el índice, se requiere un paso adicional para obtener el documento y comprobarlo.

Si el valor de su clave de fragmento pertenece a este fragmento en particular. Esta consulta se puede optimizar añadiendo un índice compuesto que incluya los campos de clave de fragmento (normalmente como campos finales).



Etapas SHARD_MERGE versus SINGLE_SHARD

SHARD_MERGE indica que la consulta está siendo gestionada por varios fragmentos. Si la consulta puede dirigirse a un solo fragmento, el nombre de la etapa será SINGLE_SHARD.

Análisis de mensajes de registro. Por muy útil

que sea la salida de explain, hay mucha información disponible en los registros de cada ejecución real de un comando que no se puede ver en explain. A continuación, se muestra un ejemplo formateado para facilitar la lectura y sin campos no específicos de la ejecución del filtro de consultas:

```

"tipo": "comando", "ns" :
"prueba.coll", "tipoColección" :
"normal", "comando" : { "agregado": "coll",
"tubería" :
  [ { "$coincidencia" : { "$o" : [ { "x" :
    { "$gt" : 5, "$lt" : 10 } },
    { "a" : { "$gt" : 5, "$lt" : 10 } } ] } },
    { "$sortByCount" : "$b" }
  ],
  "$clusterTime" : { ... }, "$db" : "prueba"
},
"planSummary" : "IXSCAN { x: 1 }, IXSCAN { a: 1 }", "planningTimeMicros" :
2572, "keysExamined" : 567,
"docsExamined" : 567,
"hasSortStage" : verdadero,
"nreturned" : 40, "planCacheKey" :
"DAC1B215",
"queryFramework" : "sbe", "locks" :
{ "Global" : { "acquireCount" : { "r" : 4 } } },
"storage" : { }, "workingMillis" : 12, "durationMillis" : 12
}

```

Puede ver el plan de consulta utilizado (dos índices, uno para cada cláusula \$or) y que hubo una etapa de SORT (hasSortStage: true), lo que indica que se realizó una ordenación en memoria de los resultados. También puede ver la selectividad de los índices según el número de claves y documentos examinados, así como la duración de la operación y un desglose de su tiempo empleado. Si se ejecutó el proceso de evaluación de varios planes, también verá "fromMultiPlanner":true en el mensaje de registro.

Los mensajes en los registros están en formato json, por lo que si desea analizarlos programáticamente, puede cargarlos en su propia colección MongoDB, o puede usar grep y otras herramientas de línea de comandos para buscarlos.

para patrones problemáticos.

Identificación de patrones problemáticos

Así como no todas las consultas rápidas son óptimamente eficientes, a veces las consultas bien escritas y con soporte de índice pueden ser lentas. Es importante comprender por qué son lentas para evitar perder tiempo ajustando algo que no es la causa raíz del problema de rendimiento.

¿Las consultas problemáticas siempre serán lentas? No necesariamente. Ya sabemos que la lentitud es relativa y, por defecto, las consultas que son más lentas de 100 milisegundos se registran en los registros de MongoDB. Este umbral se puede configurar para que sea mayor, menor o se base en algo distinto a un umbral de tiempo absoluto; consulte [Capítulo 14](#) Monitoreo y Observabilidad para más detalles. Sin embargo, las consultas ineficientes aún pueden fallar. Pasan desapercibidas si caen justo por debajo del umbral, aunque podrían ser significativamente más rápidas con índices óptimos. Podemos detectar estas consultas lentas ajustando el umbral de registro y analizando la tasa de destino de las consultas, el uso del disco u otra contención de recursos.

Relación de segmentación de consultas Una

de las métricas más importantes para determinar la eficiencia de sus consultas se denomina relación de segmentación de consultas. Ahora que hemos visto los detalles disponibles en los mensajes de explicación y registro, veamos qué implica esta proporción. Al consultar todos los documentos que cumplen un criterio específico, cuantos menos documentos tenga que revisar el sistema para determinar cuáles devolver, más rápido se completará la consulta.

La relación de segmentación de consultas se refiere a la cantidad de claves de índice leídas en comparación con la cantidad de documentos devueltos o a la cantidad de documentos leídos en comparación con la cantidad de documentos devueltos. El número ideal para cualquiera de estos sería 1, ya que nos gustaría consultar solo las entradas de índice o los documentos que deben devolverse al cliente. Si existe un índice óptimo para una consulta y cada entrada de índice analizada termina en el conjunto de resultados, la proporción será de 1:1. Las consultas de índice cubierto son un caso especial en el que ni siquiera necesitamos consultar ningún documento y se indicarán con `"docsExamined":0`, Pero dejémoslos de lado para poder entender los siguientes ejemplos.

El ejemplo más simple es la consulta `{a: 5}` y un índice en `a`. Cada entrada de índice que revisemos corresponderá al valor que queremos encontrar, y solo visitaremos los documentos que deban devolverse al cliente. Esta es la proporción ideal. Si no hay índice en `a`, debemos realizar un análisis de la colección, lo que significa que revisamos todos los documentos de la colección. Si hay 50 documentos que cumplen nuestra condición y deben devolverse, y hay 50 millones de documentos en la colección, nuestra proporción objetivo será de 1 millón a uno, lo cual no es muy bueno, dado que la proporción ideal sería de uno a uno.

A continuación se muestran varios fragmentos de mensajes de registro de consultas que utilizaron un índice para satisfacer varias cláusulas de predicado de consulta, pero luego tuvieron que examinar campos adicionales en el documento para determinar si debía ser devuelto al cliente:

```
"claves examinadas": 977, "documentos examinados": 977, "n devueltos":  
4, "claves examinadas": 69070, "documentos examinados": 147, "n devueltos": 0
```

Si bien la proporción de `keysExamined` a `docsExamined` indica la eficiencia de los índices para limitar el número de documentos que la consulta debía examinar, un valor de `nreturned` significativamente menor indica que se debieron aplicar filtros adicionales directamente al documento. Esto solo aplica a las consultas, no a las agregaciones, que pueden devolver un número menor de documentos al agregarlos en un número menor de grupos totales.

Esperando disco u otros recursos. Una de las métricas más útiles en nuestros registros puede ser el almacenamiento, que estará vacío en el caso de una consulta rápida. Sin embargo, si alguna de las páginas de índice o datos no estaba en la caché del motor de almacenamiento, las métricas relacionadas aparecerán en el campo de almacenamiento. Las consultas muy lentas, incluso aquellas que utilizan índices eficazmente, suelen ser lentas porque los datos no están ya en la caché de WiredTiger, y este es el campo que indicará si ese es el caso.

Así es como podrían verse los mensajes de registro parciales para tres consultas:

```
almacenamiento:{datos:{bytesRead:8674010,timeReadingMicros:6297}},duraciónMillis:105
almacenamiento:{datos:{bytesRead:15353,timeReadingMicros:918415}},duraciónMillis:919
almacenamiento:{datos:{bytesRead:31033716,timeReadingMicros:82948}},duraciónMillis:145
```

Desde la perspectiva de la base de datos, esto indica que los datos no provienen de su caché. Aun así, podrían haberse leído desde la caché del sistema de archivos.

En diferentes casos, la causa raíz puede ser un disco muy lento (segunda línea de ejemplo), una gran cantidad de datos no presentes en la caché o una ordenación en memoria que tuvo que transferir datos al disco. Puede rastrear esto buscando `"usedDisk":true` en el mensaje de registro o examinando la información más detallada en la salida de la explicación.

Del mismo modo, los mensajes de registro pueden indicar que una consulta lenta utilizó un plan de ejecución óptimo, pero seguía siendo lenta debido a la falta de recursos, como la espera de otras operaciones lentas. No tiene sentido intentar ajustar esa consulta, ya que es el sistema en general, u otras operaciones lentas, las que deben ajustarse.

Cómo influir en la ejecución de consultas

A veces, por mucho que intente crear un índice perfecto para una consulta, el planificador de consultas elige un índice diferente. Hay dos maneras de influir en la ejecución: forzar la selección del índice preferido o bloquear por completo la ejecución de consultas no deseadas.

Uso de sugerencias:

Puede pasar una `directiva` de sugerencias a cualquier consulta con el nombre o la definición de un índice. Esto indica al motor de consultas que omita por completo la caché de consultas y el proceso de planificación, y que, en su lugar, utilice el índice especificado para la consulta. Hay algunos aspectos a tener en cuenta al usar sugerencias. Especificar un índice inexistente devolverá un error. Especificar un índice disperso, parcial o incompatible puede devolver resultados incompletos si el índice no incluye todos los documentos que satisfacen el predicado de la consulta.

Al usar sugerencias en el código de la aplicación, es importante proceder con precaución: compruebe periódicamente que sigan siendo necesarias. Normalmente es preferible eliminar los índices adicionales que hacen que el motor de consultas tome decisiones deficientes, pero si esto no es posible, usar sugerencias puede ser una solución rápida.

A continuación se muestra un ejemplo de código que fuerza a `find` a utilizar un índice por su nombre a través de una pista:

```
db.coll.find({<filtro>}).hint("nombreíndice")
```

Utilice las sugerencias con cuidado. Valide siempre su relevancia a medida que sus datos e índices evolucionan.

Uso de la configuración de consultas

La versión 8.0 de MongoDB introdujo una nueva función llamada configuración de consultas persistentes. Esta función permite controlar la ejecución de consultas añadiendo automáticamente sugerencias en el servidor sin necesidad de modificar el código de la aplicación, definiendo filtros de rechazo ni configurando otros campos. En lugar de depender de que una sugerencia se añada explícitamente a una forma de consulta específica en el código de la aplicación (lo que conlleva el riesgo de omitir algunas secciones si la consulta se invoca en varias partes del sistema), se puede configurar el servidor para que aplique automáticamente una sugerencia al observar una forma de consulta específica.

Este comando establece una sugerencia de índice para una base de datos, una colección y una forma de consulta específicas:

```
db.adminCommand( {
  setQuerySettings: { buscar:
    "collName", filtro:
    {<filtro>}, $db:"dbName" },
  configuraciones:

    { indexHints: {

      ns: {db: "nombre_db", coll: "nombre_coll"},
      índices_permitidos: ["nombre_índice"] }

    }
})
```

Si desea bloquear por completo la ejecución de una consulta problemática, modifique el campo de configuración para incluir la directiva `reject:true`.

Para obtener más información sobre el comando `setQuerySettings`, visite [https://www.mongodb.com/docs/manual/reference/command/](https://www.mongodb.com/docs/manual/reference/command/setquerysettings/)

[setquerysettings/](#).

Otras opciones

Otra configuración importante, que funciona como un martillo gigante, aunque generalmente no se recomienda para entornos de producción, es la opción `--notablescan` al iniciar mongod. Esta configuración puede bloquear todas las consultas que tengan un filtro de consulta pero que no tengan un índice válido para usar. Esto podría ser seguro de habilitar en un sistema de prueba o de pruebas para detectar si alguna consulta está ejecutando análisis de colección inesperadamente, pero puede que no sea una buena idea usar una herramienta tan indiscriminada en producción. Puede

Monitorear dichas consultas en producción y decidir bloquearlas del sistema mediante consultas.
configuraciones o agregar índices para respaldarlos.

Semántica e índices de MQL

Como se discutió en [Capítulo 3](#), Índices , Los índices fueron diseñados para soportar el lenguaje de consulta MongoDB (MQL) encuentra semántica, pero ¿qué significa eso? MQL tiene varios componentes, algunos específicos de agregación, algunas específicas para actualizaciones y otras específicas para buscar filtros de comandos, que se denominan Expresiones de coincidencia. Esta es la sintaxis que se utiliza al especificar un predicado de consulta para las funciones `find` y `update` , y eliminar comandos, así como la sintaxis que espera la etapa de agregación `$match` . Índices Admiten algunas de las peculiaridades de las expresiones de coincidencia, y esto puede afectar la eficiencia con la que se ejecuta el índice. Puede ser utilizado por una consulta específica. Comprender esto puede ayudarle a identificar, depurar y reescribir consultas difíciles de optimizar, así como a escribir consultas más eficientes desde el principio.

Desafíos con matrices e índices multiclave

Los campos que almacenan matrices presentan desafíos únicos para la planificación de consultas y el uso de índices. Documentos con Los campos que podrían ser matrices necesitan una consideración especial para evitar convertirse en una fuente de errores no deseados. Problemas de rendimiento.

La igualdad no es sólo igualdad

En una consulta MQL, especificar la igualdad (`$eq`) con un valor (explícito o implícito) significa igual o igual a una matriz. donde uno de los elementos es igual al valor .

Por ejemplo, `db.coll.findOne({a:5})` coincide con `{a:5}` así como con `{a:[1, 5, 10]}` .

Esto significa que un índice en el campo `a` que contiene la entrada 5 debe apuntar a documentos donde `a` es 5 , así como a documentos donde `a` es una matriz que contiene 5. Las entradas se ven exactamente iguales, pero tan pronto como A medida que se indexa cualquier elemento de la matriz, el índice adquiere la propiedad multiclave , que le indica al motor de consulta Que `a` podría ser una matriz. Por lo tanto, al devolver solo el valor `a` , se debe , No se puede devolver desde el índice obtener el documento para devolver el valor real del campo.

Tenga en cuenta que, en la agregación, la expresión `$eq` no tiene este matiz. En `{ $eq:[v1, v2]}` , el resultado Es verdadero si los dos valores son iguales y falso en caso contrario. Las expresiones de agregación no alcanzan el interior. Matrices para que coincidan con valores individuales.

\$elemMatch

La forma en que indexamos las matrices no solo tiene implicaciones para devolver campos de matriz potenciales, sino que también afecta Cómo podemos usar índices para verificar las condiciones de `$elemMatch` . El operador `$elemMatch` verifica si al menos un elemento de la matriz satisface los criterios de consulta .

Por ejemplo, la consulta `{a:{ $elemMatch:{ $eq:5}}}` coincidirá con `{a: [1, 4, 5]}` pero no coincidirá con `{a: 5}` porque 5 aquí no es un elemento de la matriz.

De manera similar, `{a:{$gt:5, $lt:10}}` coincidirá con `{a: [1, 4, 11]}` porque ambas condiciones son verdaderas; `a` tiene un elemento que es menor que 10 y también tiene un elemento que es mayor que 5 .

Para buscar un solo elemento que cumpla ambas condiciones, tendríamos que escribir `{a: {$elemMatch: {$gt:5, $lt:10}}}` . Ahora, el significado es: `a` debe tener un solo elemento que sea mayor que 5 y menor que 10 .

Dado que indexamos valores de array en el árbol B, existen varias situaciones en las que el índice podría no responder a la consulta con la eficiencia esperada. Una simple consulta `{a:{$elemMatch:{Seq:5}}}` puede usar el índice para encontrar todos los documentos donde `"a"` sea un escalar de 5 o un array que contenga 5. Sin embargo, sin examinar el documento, no es posible determinar si `"a"` es un elemento de array, ya que un esquema flexible permite incluir escalares y arrays en el mismo campo de una colección.

Otro caso en el que el uso de índices en una consulta puede sorprenderle es cuando no utiliza `$elemMatch` para múltiples condiciones, específicamente cuando no desea que un solo elemento satisfaga varias condiciones, sino que desea encontrar una matriz donde diferentes elementos puedan satisfacer estas condiciones.

Un ejemplo es la consulta `{a:{$gt:5, $lt:10}}` anterior , que puede utilizar un índice en un índice multiclave (es decir, si se sabe que `a` puede ser una matriz en algunos de los documentos), entonces no se puede utilizar de la misma manera que un índice normal que no sea una matriz.

Consideremos el ejemplo que hemos estado viendo y cómo podría utilizar un índice multiclave y un índice no multiclave:

db.coll.explain().find({a:{\$gt:5, \$lt:10}})

<pre>// índice sin claves múltiples en un "Plan ganador" : { "etapa" : "OBTENER", "etapaDeEntrada": { "etapa" : "IXSCAN", "patrónDeClave":{"a":1}, "isMultiKey" : falso, "límitesDeÍndice": {"a":["(5.0,10.0)"]}} }</pre>	<pre>// índice multiclave en un "winningPlan" : { "etapa" : "FETCH", "filtro" : { "a" : { "\$lt" : 10}}, "inputStage" : { "etapa" : "IXSCAN", "keyPattern" : { "a" : 1}, "isMultiKey" : verdadero, "indexBounds" : { "a" :["(5.0,inf.], "rejectedPlans" : [{ "etapa" : "FETCH", "filtro" : { "a" : { "\$gt" : 5}}, "inputStage" : { "etapa" : "IXSCAN", "keyPattern" : { "a" : 1}, "indexName" : "a_1", "isMultiKey" : verdadero, "indexBounds": {"a" :["-inf.0, 10.</pre>
---	--

Tabla 12.1: La consulta utiliza diferentes límites de índice dependiendo de si el índice es multiclave

A la izquierda, el índice sin múltiples claves utiliza límites estrictos para encontrar todos los documentos donde el campo `a` esté entre 5 y 10. A la derecha, el plan utiliza el índice para encontrar todos los documentos donde `a` esté entre 5 e infinito y luego recupera el documento para comprobar que también exista un valor de `a` menor que 10.

¿Por qué un índice con múltiples claves no puede utilizar los mismos límites que un índice sin múltiples claves y, en su lugar, debe elegir entre usar un límite en el índice y comprobar el otro manualmente en la etapa `FETCH`? Esto se debe a que la semántica de `Un elemento puede coincidir con una condición y un elemento diferente puede coincidir con la misma.` matriz indica "otra condición", y si usáramos límites estrictos, no encontraríamos documentos coincidentes como `{a:[1, 11]}`, ¡donde ninguno de los elementos está entre 5 y 10!

Desduplicación Dado que

cada elemento de una matriz se indexa por separado en el árbol B, se requiere trabajo adicional para desduplicar los documentos coincidentes cuando una consulta indexada de múltiples claves se parece a `{a:{$in:[1, 5, 10]}}`.

· En este caso, realizamos tres búsquedas en el árbol B, una para cada valor, pero las tres pueden apuntar al mismo `recordId`. La consulta siempre debe realizar un trabajo adicional con los campos multiclave de los índices multiclave para garantizar que no se devuelvan registros duplicados (ni se cuenten más de una vez).

Veamos la parte de `executionStats` de la salida de explicación para la consulta `{a:{$in:[500, 501, 502]}}`, donde varios documentos tienen matrices con varios de estos valores: `502}}`,

```
Estadísticas de
ejecución:
  { Etapa de entrada:
    { Etapa:
      'IXSCAN',
      nReturned: 1,
      Works: 4,
      Advanced: 1, IsEOF: 1,
      Patrón de clave: { a:
        1}, IsMultiKey: true, Rutas de múltiples
        claves: { a: [ 'a' ] }, Límites de índice: { a: [ '[500, 500]', '[501, 501]', '[502, 502]' ] },
        Claves examinadas:
          3,
        Búsquedas: 1,
        Dups probados: 3, Dups eliminados: 2
```

El documento único devuelto contenía los tres valores de la lista de búsqueda en la matriz, Por eso vemos "a", por lo que, de los tres resultados probados, dos se descartaron como duplicados. Esto muestra el trabajo adicional realizado, que ya se puede apreciar porque `nReturned` es 1 y `keysExamined` es 3. Este trabajo adicional no se debe a que el índice no almacene eficientemente todos los valores que queremos devolver, sino al trabajo adicional realizado para eliminar los resultados duplicados. Si bien las matrices son una forma eficaz de representar estructuras complejas en los documentos, es importante tener en cuenta el coste adicional que pueden tener algunas consultas al trabajar con campos de matriz indexados.

Desafíos con null y \$exists

En MQL, la consulta `{a: null}` coincide con todos los documentos donde "a" está ausente o "a" está presente y es igual a "null". Esto incluye casos donde "a" es una matriz que contiene "null", así como algunos escenarios más complejos.

Pero en todos estos casos, dado que esta consulta devuelve tanto "missing" como "null", los índices del árbol B almacenan el valor "null" para indicar que un valor es explícitamente nulo o que falta; es decir, que no existe. Por lo tanto, los índices se pueden usar para comprobar fácilmente que un campo no es igual a "null", pero no para responder a las consultas `"$exists:true"` o `"$exists:false"`. Esto se debe a que el documento aún necesitaría ser revisado para ver si el campo existe con un valor "null" o si no existe. La única excepción a esto son los índices dispersos, que solo indexan valores que existen (incluido "null") y, por lo tanto, pueden responder al predicado `"$exists:true"` directamente desde el índice sin necesidad de verificar detalles adicionales en el propio documento.

A continuación se explican varios campos cuando se utiliza un índice en `a` para verificar `db.coll.find({a:$ne:null})`:

```
etapa: 'IXSCAN',
patrón de clave: { a: 1 },
límites de índice: { a: [ '[MinKey, null)', '(null, MaxKey]' ] }
```

Los límites del índice muestran que la consulta se maneja como una consulta de rango, que coincide con valores menores que null y mayores que null.

Es fundamental comprender todo esto para escribir consultas correctas, pero también es importante comprender cuándo se pueden usar los índices con la eficacia esperada. En resumen, si su aplicación tiene una consulta que debe comprobar la existencia de un campo, se puede usar un índice disperso.

Prácticas recomendadas adicionales. Aquí

encontrará algunos consejos adicionales sobre las mejores maneras de escribir sus aplicaciones y sobre qué debe tener en cuenta, así como qué debe evitar como regla general si desea tener un sistema eficiente y escalable. MongoDB ya ha planeado la discontinuación y eliminación de algunas de las funciones que debe evitar, mientras que otras están diseñadas para casos de uso muy específicos, y usarlas fuera de estos casos puede generar una sobrecarga innecesaria en su sistema.

Actualizaciones

Al crear las modificaciones de actualización, siempre es preferible usar modificaciones de campos individuales en lugar de reemplazar el documento completo. Desde la perspectiva del desarrollador de aplicaciones, usar `updateOne` suele ser mejor que `replaceOne`, ya que reemplazar el documento completo cuando solo se han modificado algunos campos será menos eficiente para el subsistema de replicación.

En algunos casos, cuando la nueva versión de un documento se recibe desde fuera de la aplicación y determinar los cambios exactos resulta prohibitivamente costoso o poco práctico, existe un truco que puede utilizar. No utilice la siguiente sintaxis:

```
db.coll.replaceOne({_id:X}, {documento nuevo completo})
```

En su lugar, utilice esta sintaxis actualizada para lograr el mismo efecto:

```
db.coll.updateOne({_id:X}, [ {$replaceWith:{$literal:documento-nuevo-entero}}])
```

Si bien a simple vista esto parece utilizar la sintaxis de agregación para indicarle al sistema que realice el mismo proceso, documento existente con este nuevo documento . en la práctica, lo que sucede internamente es reemplazar el MongoDB compara los documentos existentes y nuevos, y genera deltas para registrar en el registro de operaciones de replicación, lo que resulta en una entrada mucho más pequeña. Si nuestro cálculo determina que estos deltas son más extensos que el nuevo documento completo, lo escribiremos en su totalidad en el registro de operaciones.

Para mayor precisión, en sistemas multihilo, asegúrese de crear sus actualizaciones con modificadores de actualización, que permiten que varios hilos realicen cambios en el mismo documento simultáneamente sin sobrescribir el trabajo de los demás. Esto implica usar \$inc para incrementar los contadores en lugar de leer el documento, modificarlo en código y luego volver a escribirlo. Si el patrón de lectura-modificación-escritura es inevitable por razones específicas de la aplicación, utilice el control de versiones para garantizar que no se haya realizado ningún otro cambio en el documento entre la lectura y la escritura, o utilice el valor anterior del campo que está modificando en el filtro para la escritura, de modo que esta solo se ejecute correctamente si el valor no ha cambiado desde la lectura.

buscar y modificar

El comando `findAndModify` se conoce en los métodos del controlador como `findOneAndUpdate` y su propósito suele malinterpretarse. Es esencialmente el mismo comando que `updateOne` , pero en lugar de devolver el documento resultante que informa al cliente exactamente qué hizo la actualización (por ejemplo, si hubo una actualización, si coincidió con algún documento, etc.), `findOneAndUpdate` realiza la misma operación de actualización , pero devuelve el documento actualizado (si lo hubo), ya sea la versión anterior o posterior a la actualización.

¿Por qué es necesaria esta funcionalidad? Porque a veces es necesario saber qué efecto tuvo una actualización o qué documento se actualizó, y el filtro de actualización está estructurado de tal manera que no se conoce ni se puede determinar esta información de ninguna otra manera. Consideremos un ejemplo con una sola colección de documentos que simplemente registra un contador que se incrementa en 1 cada vez que se usa un valor:

```
{_id:1, conteo: 35}
```

Si un cliente desea utilizar el siguiente valor de conteo disponible e incrementar el contador para que esté en el valor correcto Estado para el siguiente hilo; no puede hacerlo con dos comandos de actualización y búsqueda independientes , ya que no hay garantía de que otro hilo no se infiltre entre ambas operaciones. Aquí es donde `findOneAndUpdate` resulta útil: permite al cliente enviar una actualización con `{ $inc: {count: 1} }` para incrementar el contador y, al mismo tiempo, recuperar el valor correcto que tenía el documento antes o inmediatamente después de la actualización.

Otro ejemplo sería un cliente que busca el siguiente trabajo en una colección que representa una cola. Además de recuperar el valor `_id` del siguiente documento que se procesará, también necesita escribir en el documento para indicar que ahora se está trabajando en él, evitando que otro hilo lo intente para trabajar en el mismo elemento. Aquí, se devuelve el documento y, al mismo tiempo, se configura automáticamente un campo para bloquearlo lógicamente de otros hilos que están tratando de encontrar su próximo trabajo, puede ser `$lock`. Se logra fácilmente con el comando `findOneAndUpdate` (también conocido como `findAndModify`). Aplica un filtro y ordena para encontrar el documento que está listo para trabajar (normalmente con el valor de marca de tiempo más bajo o más alto) y no está bloqueado por otro hilo, establece el campo para indicar "Voy a trabajar en este documento", y devuelve el documento para que el cliente sepa en qué está trabajando. Mediante una operación `updateOne` independiente. Seguido de una operación `findOne` no solo introduciría un viaje de ida y vuelta adicional innecesario, sino también No deja ninguna forma confiable de saber cómo encontrar el documento que acaba de modificarse en una llamada separada.

Entonces, ¿por qué `findAndModify` es más costoso que `update`, ya que ambos comandos actualizan un documento? ¿Y devolver un documento (ya sea el documento resultante o el documento modificado)? Resulta que...

La razón principal es una característica descrita en [Capítulo 10](#), Bibliotecas de clientes, Las llamadas escrituras, donde un reintentables hacen que el controlador MongoDB vuelva a intentar automáticamente una operación de escritura que puede o no haber tenido éxito durante conmutación por error principal en un conjunto de réplicas. MongoDB rastrea internamente si cada operación de escritura ha tuvo éxito y puede devolver fácilmente una copia duplicada del resultado si se vuelve a intentar una operación después de que ya se ha realizado. Sin embargo, para devolver la versión correcta del documento cuando se realiza una operación `findAndModify` Una vez reintentado, el sistema tiene que hacer el trabajo adicional de escribir una copia del documento modificado antes del cambio en un Colección lateral especial. Esto genera una sobrecarga que puede ralentizar el sistema si se utiliza habitualmente. `findOneAndUpdate` innecesariamente en lugar de `updateOne`, y el impacto es aún mayor y más problemático si sus documentos son bastante grandes.

Por lo tanto, es perfecto usar `findAndModify` cuando sea necesario implementar correctamente su lógica de la aplicación, pero es mejor evitarlo cuando una simple actualización puede hacer el trabajo.

Agregación y consulta

Algunas etapas y expresiones de agregación son adecuadas cuando se usan con moderación y solo en casos específicos. Fueron diseñados, pero no se deben usar en exceso cuando existe una alternativa más simple y eficiente. basta. Esta sección enumerará algunas de esas etapas, operadores y comandos que deben evitarse o utilizados con moderación, junto con sus alternativas potencialmente más eficaces.

\$sample

La etapa `$sample` está diseñada para devolver un número, `N`, en `size` de documentos seleccionados al azar desde el punto el pipeline donde aparece. Sin embargo, solo funciona eficientemente en un escenario muy específico: Cuando se trata de la primera etapa del proceso de una colección con más de 100 documentos, los `size`, y el número de documentos solicitados representan menos del 5% del total de documentos de la colección. En ese caso, se utiliza un cursor pseudoaleatorio para seleccionar esos `N` documentos. En todos los demás escenarios, `$sample` realiza un costoso ordenar aleatoriamente todos los documentos solo para devolver el primer `N`. Si realmente necesitas un subconjunto verdaderamente aleatorio de documentos, entonces, por supuesto, use `$sample`, pero en la mayoría de los casos, donde simplemente desea limitar el número de documentos devueltos, es mucho más rápido usar `$limit`.

\$facet

`$facet` procesa múltiples pipelines de agregación en una sola etapa sobre el mismo conjunto de documentos de entrada. Existe la idea errónea de que la etapa `$facet` realiza una magia de agregación para procesar en paralelo un pipeline en múltiples ramas. Esto no es así. La forma más sencilla de entender `$facet` es la siguiente: cada documento que fluye por el pipeline hasta ese punto se envía a cada rama de `$facet`. Esto significa que si una rama necesita solo los primeros 10 documentos, pero otra necesita un millón, entonces el millón de documentos se enviará a cada rama de `$facet`.

`$facet` es una excelente opción cuando se desea realizar un procesamiento extremadamente similar en cada rama; por ejemplo, al calcular los recuentos por categoría y por rango de precio de todos los documentos entrantes. No es una buena opción cuando el procesamiento que se desea realizar en sus ramas es muy diferente. Un ejemplo sería devolver los primeros 10 documentos, así como el recuento total. En tales casos, es mucho mejor usar `$unionWith`, conservando, que ejecuta dos (o más) tuberías todas las optimizaciones que se pueden realizar en cada uno de ellos.

\$donde, \$función, \$acumulador y mapReduce

Al ejecutarse en el servidor, JavaScript siempre será más lento que al expresar la misma lógica con la sintaxis nativa de MQL. Además, JavaScript representa un riesgo inherente para la seguridad de los servidores MongoDB y se ha descontinuado con la intención de eliminarlo por completo para eliminar dicho riesgo.

Para una ejecución más rápida y una mejor seguridad, evite utilizar JavaScript en sus consultas o agregaciones de MongoDB.



Ejecución de JavaScript: Servidor versus shell mongosh

Esto no aplica al shell de Mongosh, que expande e interpreta todo el JavaScript antes de enviarlo al servidor. Esta advertencia se aplica específicamente a cualquier JavaScript que deba ejecutarse en el servidor de base de datos.

`$text`. Si

querías realizar una búsqueda de texto completo antes de la introducción de la función de búsqueda de texto completo de MongoDB Atlas, tenías opciones: usar el operador `$text`, que dependía de los índices de texto heredados de MongoDB, o enviar los datos a otro sistema. Desde el lanzamiento de Atlas Search, se ha detenido el desarrollo de índices de texto heredados nativos, y se espera que esta función quede obsoleta en favor de la búsqueda basada en Lucene. Esta nueva función de búsqueda estará disponible en la versión de MongoDB Community, así como en MongoDB Atlas.

\$regex e índices

Dado que los índices almacenan valores ordenados, no es sorprendente que, al usar un operador `$regex` para hacer coincidir parte de una cadena, el índice solo se pueda usar eficientemente si `$regex` es el equivalente del patrón " inicia con ", lo que llamamos una expresión regular anclada o de raíz izquierda. Además, el índice

solo se puede usar de manera eficiente si la expresión regular no especifica la opción que no distingue entre mayúsculas y minúsculas . Para Por ejemplo, si su consulta es algo como `{s:{$regex:/^Abc/}}` , que dice: " todas las cadenas que comienzan con exactamente Abc ", entonces se usará un índice en `s` como una consulta de rango eficiente. Para cadenas que no distinguen entre mayúsculas y minúsculas Para coincidencias, considere usar índices con intercalación apropiada.

Expresiones de agregación versus expresiones de coincidencia

Hay algunas expresiones que comparten el mismo nombre tanto en `find` como en `added` pero se comportan de forma ligeramente diferente. De forma diferente. Siempre sea cauteloso y asegúrese de expresar la semántica exacta de su aplicación. necesidades y comprobar que los índices se utilizan correctamente. Por ejemplo, `$eq` en Agregación significa literalmente igual ; no analiza dentro de los arrays. Por otro lado, `$in` en agregación medio " ¿Hay algún elemento de la matriz igual a este valor? " mientras que en `find` `$in` se verifica si alguno de los Los valores especificados son iguales al campo que estamos buscando (donde "igual" en la búsqueda significa " es igual al campo que estamos buscando"). valor o contenido dentro de una matriz ").

`$expr` e índices

Porque usar `$expr` en `find` o `$match` significa semántica de " Estoy a punto de usar una expresión de agregación ", implica agregación, y el uso de índices, que están diseñados para soportar la semántica de `find` , se vuelve complicado. Por eso, muchas consultas que usan `$expr` no usan índices cuando se podría esperar que lo hicieran. a, o utilizan índices pero tienen pasos adicionales para garantizar que se aplique la semántica correcta a su Consulta. Si es posible expresar la consulta sin usar `$expr` , asegúrese de que esté escrita así. y solo use `$expr` cuando sea la única forma de lograr la semántica de consulta exacta que necesita.

cláusula `$or` e índices

Normalmente, una sola consulta solo puede usar un índice. En este contexto, «consulta» se refiere a un único filtro de consulta. pasado al comando `find` o a la etapa `$match` ; una canalización con muchas etapas o subcanalizaciones puede utilizar Diferentes índices para distintos componentes. Por ejemplo, cada subcanalización en una etapa `$unionWith` es planificado independientemente del resto de la canalización, y las canalizaciones `$lookup` hacen su propia planificación utilizando índices apropiados independientemente de la canalización de nivel superior.

La única excepción en las versiones actuales de MongoDB es cuando tienes una consulta con un `$or` de nivel superior . En este caso, cada rama de `$or` puede usar un índice diferente ya que los resultados son una unión de todas las ramas. Resultados. En algunos casos, un `$or` podría incluso estar dentro de una cláusula `$and` y puede reescribirse para estar en la parte superior. nivel para que se puedan usar múltiples índices. A veces, el optimizador de consultas reescribe esto automáticamente. Pero para simplificar las cosas, coloque la cláusula `$or` en el nivel superior si es posible. Lo que aprendiste sobre La planificación de consultas aún se aplica, ya que cada cláusula de la declaración `$or` se planifica por separado, pero la de cada cláusula El plan aparecerá en la salida de explicación como una etapa SUBPLAN especial .

Colecciones especiales, tipos de índices y características

Hay casos de uso que son difíciles de implementar utilizando documentos básicos e índices de árbol B regulares. Para patrones muy específicos, como series temporales, coordenadas geoespaciales o incrustaciones vectoriales, MongoDB

Ofrece colecciones e índices especializados. Los tipos de colecciones especiales y las funciones avanzadas de indexación de MongoDB pueden mejorar considerablemente el rendimiento de su aplicación para casos de uso específicos.

Colecciones de series temporales

Imagina que estás construyendo una plataforma de IoT que recopila lecturas de temperatura cada segundo de miles de sensores, o que rastrea precios de acciones que cambian cada milisegundo. Tendrás un flujo constante de puntos de datos, cada uno con una marca de tiempo. Las actualizaciones son poco frecuentes y las consultas suelen abarcar porciones de... tiempo.

En MongoDB, cualquier conjunto de documentos donde cada uno incluya una marca de tiempo y compartan campos de metadatos puede almacenarse en una colección especial de series temporales. Para más detalles, consulte <https://www.mongodb.com/docs/manual/core/timeseries-collections/>.

En lugar de tratar cada documento con marca de tiempo por separado, las colecciones de series temporales de MongoDB los agrupan internamente en contenedores, de modo que los datos de la misma fuente se almacenan junto con otros puntos de datos de un momento similar. Esto reduce los requisitos de almacenamiento y la ampliación de escritura, además de agilizar las consultas de rango típicas en intervalos de tiempo.

Las mejoras de rendimiento se deben a varios factores. MongoDB comprime y organiza automáticamente los datos de series temporales en un formato interno de columnas, lo cual resulta increíblemente eficiente para las mediciones repetitivas típicas de sensores o datos financieros. Esto genera un ahorro de almacenamiento considerable, a menudo de entre el 70 % y el 90 %. Se crea automáticamente un índice agrupado en el campo de tiempo, lo que permite realizar consultas por rango de tiempo de forma altamente eficiente sin necesidad de configuración adicional. Esto se traduce en velocidades de consulta que suelen ser de 3 a 5 veces más rápidas que las que se obtendrían con las mismas consultas en una colección estándar. La reducción del espacio de almacenamiento también implica un menor uso de memoria, copias de seguridad más pequeñas y menores costos de infraestructura. Por ejemplo, una startup fintech vio cómo sus tiempos de consulta se reducían de 5 segundos a tan solo 200 ms tras migrar a una colección de series temporales, mientras que un cliente del sector manufacturero redujo sus costos de almacenamiento de datos en más del 80 %.

Índices geoespaciales

MongoDB admite operaciones de consulta sobre datos geoespaciales. Al almacenar datos como objetos GeoJSON o pares de coordenadas y luego crear índices geoespaciales a partir de ellos, se accede a múltiples operadores de consulta geoespacial para encontrar objetos dentro de una geometría específica, formas que se intersectan y puntos cercanos a una ubicación específica. Puede encontrar una descripción general completa de las consultas geoespaciales y sus capacidades en <https://www.mongodb.com/docs/manual/geospatial-queries/>.

Al trabajar con datos geoespaciales, un índice 2dsphere bien configurado puede mejorar considerablemente el rendimiento de las consultas, especialmente para conjuntos de datos grandes donde las consultas sin índices serían extremadamente lentas. Sin embargo, tenga en cuenta que los índices geoespaciales son más grandes que los índices B-tree estándar, por lo que conviene supervisar su uso de memoria, sobre todo si indexa polígonos grandes. La densidad de los datos también es importante; si el área de consulta contiene muchos puntos, MongoDB tendrá más trabajo. En estos casos, conviene usar filtros adicionales para refinar los resultados. Para consultas que suelen combinar la ubicación con otros criterios, como buscar restaurantes abiertos en un radio de 5 km, un índice compuesto tanto en el campo geoespacial como en los demás campos de filtro es la mejor opción para un rendimiento óptimo.

Búsqueda en Atlas

Si bien las funciones de consulta habituales de MongoDB son potentes, a veces se necesitan funciones de búsqueda de texto completo más avanzadas. Atlas Search integra Apache Lucene directamente en su clúster de MongoDB.

Puede obtener más [información en https://www.mongodb.com/docs/atlas/atlas-search/](https://www.mongodb.com/docs/atlas/atlas-search/).

Atlas Search ofrece una búsqueda de texto altamente optimizada, un avance significativo respecto a lo que ofrecen los índices convencionales. Destaca en búsquedas en lenguaje natural, coincidencias parciales y autocompletado, con resultados ordenados por relevancia para ayudar a los usuarios a encontrar lo que necesitan más rápidamente. Esto no solo mejora la experiencia del usuario, sino que también aumenta la eficiencia de la infraestructura. Sin Atlas Search, probablemente necesitaría mantener un motor de búsqueda independiente junto con MongoDB, lo que añade complejidad operativa y problemas de sincronización de datos. Dado que Atlas Search está basado en Lucene, mantiene un rendimiento constante incluso a medida que crece su conjunto de datos.

Búsqueda de vectores en Atlas

Atlas Vector Search permite búsquedas de similitud eficientes mediante incrustaciones vectoriales, una tarea que requiere un alto volumen de cálculo y es lenta con los métodos de consulta estándar. Al almacenar sus datos vectoriales junto con otros datos de MongoDB, puede usarlos como una base de datos vectorial. Esto le permite consultar datos según su significado semántico y potenciar sus aplicaciones de IA. Para obtener

más información, visite <https://www.mongodb.com/docs/atlas/atlas-vector-search/vector-search-overview/>.

Al implementar la búsqueda vectorial, existe un equilibrio entre la velocidad y la precisión de la consulta y su elección del algoritmo de similitud (como la similitud euclídea, o dotProduct) afectará a ambos. Coseno de coseno, a menudo proporciona un buen equilibrio entre los dos). Para colecciones muy grandes, el uso de búsquedas de vecino más cercano aproximado (ANN) puede ser mucho más rápido que las búsquedas exactas con solo un equilibrio mínimo en la precisión.

Intercalaciones

Cada lenguaje tiene reglas diferentes para ordenar y comparar cadenas. La intercalación permite especificar reglas específicas del lenguaje para la comparación de cadenas. En lugar de tener que lidiar con el complejo proceso de almacenar cadenas normalizadas para una simple comparación binaria, se pueden almacenar los datos tal como están y usar la intercalación adecuada para influir en el comportamiento de la ordenación y las comparaciones en la aplicación sin sacrificar el rendimiento. La intercalación es una opción en los índices B-tree estándar. No solo admite diferentes caracteres en idiomas como el francés o el alemán, sino que también permite la comparación de cadenas sin distinguir entre mayúsculas y minúsculas, entre otras opciones. Puede

[leer más sobre su funcionamiento en https://www.mongodb.com/docs/manual/reference/collation/](https://www.mongodb.com/docs/manual/reference/collation/).

Para obtener el mejor rendimiento, es fundamental que sus consultas utilicen la misma configuración de intercalación que sus índices. Una discrepancia impedirá que MongoDB utilice el índice, lo que provocará un análisis de recopilación mucho más lento. Aunque los índices con intercalación pueden consumir un poco más de memoria que los índices estándar, la compensación suele merecer la pena. Ordenar grandes conjuntos de resultados en memoria, con o sin intercalaciones complejas, consume mucho CPU, por lo que usar ordenaciones indexadas siempre es la mejor opción cuando sea posible.

La principal ventaja de usar una herramienta especializada para un caso de uso específico es que puede ahorrarle mucho tiempo en comparación con el mantenimiento de las funciones usted mismo utilizando las funciones básicas de MongoDB como bloques de construcción. Las colecciones e índices especiales de MongoDB están optimizados a nivel de motor de almacenamiento para un rendimiento superior al que puede implementar en su aplicación. No piense en estas funciones solo cuando prevea alcanzar la escala empresarial. La facilidad de uso de la implementación predefinida le permite aprovecharla para pequeñas colecciones de series temporales, búsquedas geográficas o de texto completo limitadas, así como para cargas de trabajo de petabytes.

Resumen

Optimizar el rendimiento de una base de datos comienza con una comprensión profunda de cómo el motor de consultas de MongoDB procesa las consultas y utiliza los índices. Conocer el ciclo de vida de la ejecución de las consultas, es decir, cómo se analizan, planifican, ejecutan y devuelven, proporciona a los desarrolladores la base necesaria para diseñar operaciones eficientes. Herramientas de diagnóstico como el comando `explain` y los mensajes de registro son invaluable para examinar los resultados de las consultas e identificar ineficiencias causadas por planes de consulta deficientes, limitaciones de recursos u otros problemas. Esta información permite a los desarrolladores determinar si los problemas de rendimiento se deben a la propia consulta o a factores externos como la E/S del disco o la contención de memoria. La evaluación adecuada de métricas como las tasas de destino de las consultas y las estadísticas de ejecución permite a los desarrolladores diseñar consultas más inteligentes que minimicen la sobrecarga innecesaria y reduzcan la latencia.

Más allá del ciclo de vida de ejecución, evitar operaciones ineficientes y aprovechar las funciones especializadas de MongoDB puede mejorar significativamente el rendimiento del sistema y, al mismo tiempo, reducir el esfuerzo de desarrollo. El uso de colecciones de series temporales para datos financieros o de IoT, índices geoespaciales para consultas basadas en la ubicación y herramientas como Atlas Search o Atlas Vector Search para búsquedas de texto completo y similitud semántica ofrece soluciones a medida para casos de uso únicos. Estas funciones, optimizadas a nivel del motor de almacenamiento, proporcionan eficiencia y escalabilidad sin necesidad de implementaciones personalizadas. Además, la implementación de estrategias como sugerencias de consulta, configuraciones de consultas persistentes e índices bien ajustados garantiza un control preciso de los planes de ejecución y mantiene los sistemas funcionando de forma óptima. Al combinar un diseño de consultas minucioso con prácticas de diagnóstico robustas y seleccionar las herramientas adecuadas para las cargas de trabajo adecuadas, los desarrolladores pueden crear aplicaciones escalables y de alto rendimiento que satisfacen diversos requisitos con menos esfuerzo y menos... recursos.

En el próximo capítulo, describiremos cómo los sistemas operativos y los recursos del sistema pueden influir en el rendimiento y el costo de sus sistemas de producción.

13

Sistemas operativos y sistemas Recursos

Al ejecutar una implementación autogestionada de MongoDB, tiene la flexibilidad de configurar el kernel y el sistema operativo según sus necesidades específicas. Sin embargo, esta flexibilidad también introduce variables adicionales y posibles cuellos de botella en el rendimiento que deben considerarse cuidadosamente durante el ajuste.

Este capítulo es principalmente relevante para entornos autogestionados, ya que MongoDB Atlas no permite el acceso directo al sistema operativo subyacente. En Atlas, la configuración se gestiona según el nivel de instancia, y las actualizaciones del sistema operativo se gestionan automáticamente para evitar problemas de rendimiento inesperados debido a cambios en el kernel o el sistema. Además, Atlas puede escalar automáticamente el cómputo y el almacenamiento en respuesta a los cambios en la carga de trabajo, una capacidad que exploraremos más adelante en este capítulo.

En este capítulo aprenderá cómo hacer lo siguiente:

- Optimice el rendimiento del sistema comprendiendo el impacto de las opciones de infraestructura
- Ajuste las configuraciones del sistema operativo y del almacenamiento para mejorar la confiabilidad y el rendimiento.

- Identificar y abordar los cuellos de botella de rendimiento en entornos autogestionados y gestionados (Atlas)

Requisitos técnicos

Este capítulo se centrará principalmente en Linux y sistemas operativos de alto nivel. conceptos del sistema. Para obtener detalles sobre sistemas operativos específicos y Versiones de MongoDB, consulte la lista de verificación de operaciones de la documentación en <https://www.mongodb.com/docs/manual/administration/>

producción-lista-de-verificación-operaciones/ y la producción notas en

<https://www.mongodb.com/docs/manual/administration/> notas de producción/ para implementaciones autogestionadas.

Para supervisar la utilización de todo tipo de recursos del sistema, puede utilizar Comandos del sistema operativo, monitoreo de MongoDB Atlas y otros Herramientas GUI diseñadas específicamente para monitorear múltiples Servidores. Se proporcionarán más detalles en [Capítulo 14](#) Monitoreo Observabilidad .

Para los recursos del sistema, los comandos de Linux como `top` , `vmstat` , `iostat` , y `netstat` se puede utilizar para monitorear la CPU, la memoria, almacenamiento y red, respectivamente, en un punto en el tiempo en un solo Servidor. Se pueden utilizar herramientas como `dstat` y `sar` para combinar datos. desde herramientas más simples y visualizarlo a lo largo de un período de tiempo.

La monitorización de MongoDB Atlas proporciona un sistema histórico y en tiempo real. datos de recursos, que se integran con métricas específicas de la base de datos.

Para implementaciones autogestionadas, puede utilizar herramientas gráficas como Netdata, Prometheus + Grafana, Zabbix, Munin u Observium.

Por último, puede utilizar herramientas comerciales de monitorización del rendimiento de aplicaciones (APM), como AppDynamics, Datadog, Dynatrace, LogicMonitor, New Relic, SolarWinds y Splunk Observability.

Para probar los cambios de configuración o los comandos mencionados en este capítulo, necesitará usar un servidor Linux e instalar MongoDB localmente. Puede descargar MongoDB Community Edition desde [mongodb.com](https://www.mongodb.com). También necesitarás descargar los controladores oficiales para el lenguaje que elijas (Ruby o Python): <https://www.mongodb.com/docs/drivers/> .

Gestión de recursos para un rendimiento óptimo

Hasta ahora, a lo largo de este libro, hemos explorado situaciones en las que el rendimiento está limitado por el uso de un recurso a su máxima capacidad. Por ejemplo, estos recursos pueden incluir ciclos de CPU, memoria, almacenamiento o ancho de banda de red. Para mejorar el rendimiento, es necesario aumentar la disponibilidad del recurso limitante o reducir su uso. Esta sección ofrece una descripción general de cada tipo de recurso y describe cómo puede afectar el rendimiento de MongoDB.

Utilización de la CPU

Si la CPU del servidor principal de MongoDB es el cuello de botella actual de su aplicación, es posible que deba escalar a instancias más rápidas con más núcleos. Si ejecuta MongoDB como un conjunto de réplicas y ya ha escalado las instancias o sistemas al máximo,

Quizás sea el momento de fragmentar. Esto se explicó en detalle en [Capítulo 6](#),
Fragmentación .

Alternativamente, puede optimizar las consultas de su aplicación y operaciones de base de datos para minimizar el consumo de CPU. Esto incluye utilizando índices apropiados, diseñando modelos de datos eficientes y evitando canales de agregación ineficientes u operaciones de clasificación que pueden consumen demasiados ciclos de CPU. Estas técnicas se analizaron en más detalles en los capítulos 2 , 3 , 4 , y 12 .

De forma predeterminada, el servidor MongoDB está configurado para utilizar todos los recursos disponibles. Núcleos de CPU en el sistema en el que se ejecuta. Esta es una decisión deliberada para... Maximizar la capacidad de la base de datos para procesar múltiples datos simultáneos. consultas, manejar conexiones simultáneas y administrar internamente operaciones de manera eficiente. El motor de almacenamiento WiredTiger, por ejemplo, se beneficia significativamente del paralelismo multinúcleo.

Si administra usted mismo MongoDB, se recomienda implementar el Servidores MongoDB en equipos dedicados. Se desaconseja encarecidamente ejecutar otro software que consuma mucha CPU en el mismo sistema. El diagnóstico de problemas de rendimiento se vuelve significativamente más complejo cuando varias aplicaciones compiten por los mismos recursos.

En MongoDB, la eficiencia del servidor de base de datos se mejora continuamente. siendo mejorado mediante el ajuste del código y la adopción de compiladores modernos, enlazadores y optimizadores que hacen un mejor uso de los ciclos de CPU. Esto El proceso implica un ajuste cuidadoso del código para reducir la sobrecarga y eliminar cuellos de botella y optimizar operaciones críticas. MongoDB también evalúa nuevas arquitecturas de hardware, conjuntos de instrucciones y herramientas para generar un código de máquina más optimizado, lo que resulta en un menor consumo de recursos consumo. Este enfoque doble (a nivel de software y

La optimización a nivel tecnológico es fundamental para la misión de MongoDB de ofrecer una base de datos de alto rendimiento y eficiente en el uso de los recursos.

Más adelante en el capítulo, en la sección Configuración de sistemas para el rendimiento de MongoDB , exploraremos ejemplos prácticos de cómo garantizar Las CPU se están utilizando de manera eficaz y cómo identificar si otras aplicaciones están afectando la disponibilidad de la CPU para MongoDB servidor.

Gestión de memoria. La Figura 13.1 muestra un diagrama de alto nivel de la distribución de la memoria de un sistema que ejecuta un servidor MongoDB. Es útil tener esto en cuenta al optimizar el rendimiento, ya que algunas funciones consumen más datos que otras, lo que, a su vez, reduce la cantidad de memoria disponible para la caché del sistema de archivos (página), lo que genera más solicitudes de E/S dirigidas a los dispositivos de almacenamiento.

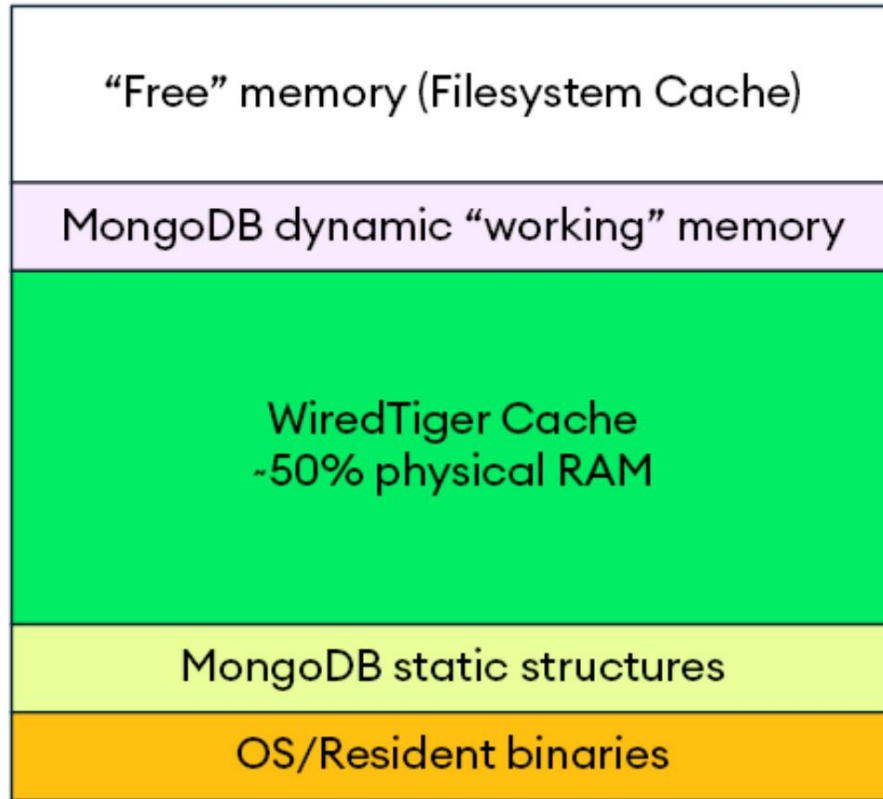


Figura 13.1: Diseño de memoria de un sistema que ejecuta el servidor MongoDB

En un nivel fundamental, el uso de memoria del servidor se puede clasificar en varios componentes clave, cada uno de los cuales desempeña un papel importante en su funcionalidad general.

Comenzando desde la parte inferior del diagrama, tenemos lo siguiente:

- Binarios residentes/del sistema operativo : este es el código en memoria para el sistema operativo y el binario del servidor MongoDB.
- Estructuras estáticas de MongoDB : metadatos que el servidor utiliza y asigna cuando se inicia por primera vez.
- Caché de WiredTiger : Por defecto, representa el 50 % de la memoria (menos 1 GB). La caché de WiredTiger almacena en RAM datos e índices de acceso frecuente para minimizar la E/S del disco. Ya explicamos esta caché en

Algunos detalles en [Capítulo 7](#) Motores de almacenamiento . Algo de Atlas .

Las instancias pueden usar diferentes tamaños relativos para el caché de WiredTiger.

- Memoria de trabajo dinámica de MongoDB : MongoDB utiliza búferes de memoria para construir, analizar y manipular documentos BSON durante las operaciones. Al ejecutar consultas, MongoDB asigna memoria para diversas operaciones, como la ordenación, las etapas de agregación y las operaciones de proyección.

Las consultas complejas o las que implican grandes ordenaciones o agregaciones pueden consumir temporalmente cantidades significativas de memoria. Las operaciones de ordenación muy extensas pueden sobrescribir el disco.

Cada conexión de cliente activa al servidor MongoDB consume cierta cantidad de memoria, y un gran número de conexiones simultáneas puede utilizar, en conjunto, una parte sustancial de la RAM.

- Memoria libre (caché de página) : Se refiere a la memoria restante, que el kernel suele usar para almacenar en caché los datos del sistema de archivos en la RAM y evitar la repetición de operaciones de E/S en el disco. Proporciona una segunda capa de caché, además de la caché de WiredTiger, y es especialmente útil en escenarios donde la caché de WiredTiger se utiliza intensivamente.

Comprender esta distribución de memoria le ayudará a diagnosticar mejor los problemas de rendimiento y a tomar decisiones informadas sobre dónde optimizar los recursos del sistema.

Almacenamiento.

Los diferentes tipos de dispositivos de almacenamiento suelen tener sus propias limitaciones de rendimiento. Los dispositivos de almacenamiento físico se dividen en dos categorías principales: discos duros mecánicos y unidades de estado sólido (SSD). Los discos en red y las unidades virtuales en entornos de nube tienen otras limitaciones.

El rendimiento de los dispositivos de almacenamiento físico está limitado por los siguientes factores:

- Velocidad de la interfaz de bus : el ancho de banda de la conexión del dispositivo, como SATA, PCIe o NVMe, puede limitar el rendimiento máximo disponible.
- Tamaño de memoria y caché : La mayoría de los dispositivos de almacenamiento modernos contienen memoria interna y caché. Un almacenamiento en caché insuficiente puede generar cuellos de botella al gestionar grandes cantidades de datos.
- Fragmentación : los datos dispersos en sectores o bloques pueden aumentar los tiempos de búsqueda y reducir el rendimiento de los discos duros y los SSD.
- Operaciones de lectura vs. escritura : las operaciones de escritura a menudo requieren más procesamiento que las operaciones de lectura, especialmente para SSD, debido a los requisitos de borrado antes de escritura.

Borrar antes de escribir

Los SSD utilizan memoria flash NAND, que requiere que se borren los datos antes de poder escribir datos nuevos.

a la misma ubicación. Esto puede ralentizar

Operaciones de escritura sucesivas, especialmente si es necesario borrar muchos bloques. Para mitigar esto, el firmware del SSD y la función TRIM del sistema operativo pueden limpiar preventivamente las celdas no utilizadas para garantizar el espacio disponible, evitando así retrasos de borrado en futuras escrituras.



- E/S secuencial vs. aleatoria : las lecturas y escrituras secuenciales generalmente tienen un mayor rendimiento en comparación con la E/S aleatoria debido al movimiento mecánico reducido (para HDD) o la sobrecarga reducida (para SSD).
- Acceso concurrente : los altos niveles de solicitudes de E/S simultáneas pueden saturar las capacidades del dispositivo de almacenamiento, especialmente si carece de una gestión de cola eficiente o de optimizaciones para el acceso paralelo. acceso.
- Degradación media : tanto los HDD como los SSD pueden experimentar una reducción del rendimiento con el tiempo: los HDD debido al desgaste mecánico y los SSD debido al desgaste de las celdas de memoria flash NAND.
- Sobrecarga de almacenamiento : Los dispositivos de almacenamiento suelen tener un rendimiento inferior cuando están cerca de su capacidad máxima. Esto es especialmente cierto en el caso de los SSD, donde se requiere un sobreaprovisionamiento de espacio para mantener un rendimiento de escritura constante.

Los discos duros se usan con menos frecuencia en sistemas de alto rendimiento. Al momento de escribir este artículo, el costo por GB de los discos duros sigue siendo menor que el de los SSD, pero la diferencia se está acortando. En el caso de los discos duros, los factores que limitan el rendimiento incluyen los siguientes:

- Velocidad del disco : la velocidad de giro de los platos (medida en revoluciones por minuto (RPM)) afecta la latencia y el rendimiento sostenido.
- Movimiento del cabezal de lectura/escritura : El movimiento y la posición del cabezal de lectura/escritura introducen retrasos mecánicos (tiempo de búsqueda).

Las unidades SSD ofrecen diversas ventajas sobre los discos duros en cuanto a rendimiento, durabilidad y eficiencia energética. En el caso de las unidades SSD y las matrices de almacenamiento flash, los factores limitantes incluyen los siguientes:

- Limitaciones de NAND : el tiempo de programación de las celdas NAND y la cantidad de operaciones simultáneas en una matriz NAND pueden limitar el rendimiento.
- Nivelación de desgaste : el rendimiento puede degradarse si los algoritmos de nivelación de desgaste se vuelven ineficientes o no logran gestionar las escrituras de manera efectiva en las celdas NAND.
- Optimizaciones de firmware : los controladores SSD utilizan algoritmos internos (como recolección de basura, nivelación de desgaste y sobreaprovisionamiento) que pueden afectar el rendimiento, especialmente bajo cargas de trabajo pesadas.
- Sobrecalentamiento : el calor excesivo puede reducir el rendimiento de los SSD modernos, ya que los mecanismos de gestión térmica interna reducen el rendimiento para evitar daños.

Para los discos en red y las unidades virtuales en un entorno de nube, los siguientes factores pueden restringir o limitar el rendimiento:

- Ancho de banda : la velocidad a la que se pueden transmitir los datos a través de la red puede limitar el rendimiento del almacenamiento en redes de área de almacenamiento (SAN) o almacenamiento conectado a red (NAS).
- Latencia : la latencia de la red puede introducir retrasos que son críticos para las aplicaciones que requieren E/S de baja latencia.
- E/S de red concurrente : grandes volúmenes de tráfico de red simultáneo que acceden al almacenamiento pueden generar problemas de contención y cuellos de botella.

Un sistema de archivos conecta el sistema operativo y los dispositivos de almacenamiento en bloque, proporcionando una interfaz abstracta unificada entre discos físicos, discos en red y unidades virtuales en un entorno de nube.

Elegir el sistema de archivos correcto es fundamental, ya que puede tener consecuencias significativas.

Implicaciones en el rendimiento. Con una implementación autogestionada de MongoDB, puede usar dispositivos de almacenamiento separados para los archivos de diario y de datos. Al separar los archivos de diario y de datos en diferentes dispositivos de almacenamiento, minimiza la contención de E/S entre las escrituras de diario y las lecturas/ escrituras de datos. Esto puede mejorar el rendimiento en cargas de trabajo con alta demanda de escritura.



Consejo

Coloque los archivos de registro en una unidad SSD o NVMe rápida. Esto permite que MongoDB gestione escrituras secuenciales frecuentes con mayor rapidez, sin competir por el ancho de banda de E/S con los archivos de datos principales.

Los archivos de datos, que suelen ser más grandes y requieren almacenamiento masivo, pueden almacenarse en soluciones de almacenamiento de mayor capacidad y más rentables (por ejemplo, discos duros más lentos). Sin embargo, existen algunas desventajas a considerar.

La administración de dispositivos de almacenamiento separados para archivos de diario y de datos introduce complejidad y aumenta la probabilidad de falla:

- Tiene dos dispositivos para configurar, monitorear e investigar posibles cuellos de botella.
- El uso de dos dispositivos aumenta la probabilidad de fallo. Por ejemplo, la tasa de fallo anualizada (TAA) típica para los SSD empresariales se sitúa actualmente entre el 0,3 % y el 0,8 % anual. Si se utiliza el punto medio de ese rango (0,55 %), y el servidor utiliza dos SSD de forma independiente para el registro y los datos, la TAA para el almacenamiento sería del 1,1 %. Como medida de seguridad, el mecanismo de replicación de MongoDB protege contra el riesgo de estos fallos.

- En entornos de nube, separar los archivos de registro y de datos en diferentes discos virtuales aún puede generar contenciones ocultas.
- Puede resultar difícil realizar una instantánea consistente tanto del diario como de los archivos de datos. Las instantáneas se utilizan a menudo para realizar copias de seguridad instantáneas de la base de datos, y la consistencia de las instantáneas depende de la atomicidad tanto del diario como de los archivos de datos.

Una plataforma de servicios de datos, como una base de datos, depende de la tecnología de almacenamiento subyacente. Esto significa que es importante, tanto desde el punto de vista del rendimiento como del ciclo de vida, comprender las características de la tecnología que se utiliza.

Red

La red juega un papel crucial en el rendimiento de una implementación de MongoDB. Los siguientes son factores clave de red que pueden limitar el rendimiento de MongoDB:

- Alta latencia de red : esto provoca retrasos en la comunicación entre la aplicación y la base de datos MongoDB, o entre los nodos del clúster en una configuración fragmentada o replicada.
- Ancho de banda de red insuficiente : esto limita el flujo de datos entre los componentes de MongoDB, especialmente en entornos de alto rendimiento, lo que afecta las operaciones de lectura y escritura.
- Pérdida de paquetes : Esto ocurre cuando los paquetes de datos no llegan a su destino debido a congestión de la red, fallos de hardware o una configuración incorrecta. Esto puede provocar lo siguiente:
 - Aumento de reintentos en operaciones fallidas, lo que ralentiza el rendimiento general de la base de datos
 - Posible retraso de replicación en los nodos secundarios de un conjunto de réplicas

- Interrupciones en la comunicación entre fragmentos de un clúster distribuido
- Resolución de DNS lenta o poco confiable : esto provoca demoras en la conexión a los servidores MongoDB, particularmente durante las conexiones iniciales o eventos de conmutación por error.
- Alto tráfico de red : la competencia por recursos compartidos puede degradar la capacidad de MongoDB para transmitir o recibir datos de manera eficiente.
- Cortafuegos mal configurados : las políticas de seguridad demasiado restrictivas pueden bloquear o retrasar el tráfico legítimo, lo que afecta el rendimiento de MongoDB.
- Topología de red inconsistente : una infraestructura de red mal diseñada puede provocar enrutamiento asimétrico, latencia inconsistente o flujo de datos ineficiente.
- Latencia geográfica : la implementación de nodos MongoDB en regiones geográficamente distantes puede generar retrasos de comunicación significativos.

Las conexiones de red lentas o poco confiables pueden afectar gravemente el rendimiento, la latencia y la escalabilidad de la base de datos.

En resumen, los cuellos de botella de rendimiento suelen surgir al utilizar un recurso a su máxima capacidad. Los ejemplos más comunes son los ciclos de CPU, la memoria, el almacenamiento y la red. Cabe destacar que, con MongoDB Atlas, muchos de estos recursos se gestionan automáticamente.

Configuración de sistemas para el rendimiento de MongoDB

En esta sección, veremos las acciones que puede tomar para diagnosticar y solucionar problemas de ejemplo con ciclos de CPU, memoria, almacenamiento y red.

Comprender la proporción ideal de operaciones simultáneas por núcleo de CPU

El servidor MongoDB ejecuta múltiples operaciones simultáneamente y los distribuye entre subprocesos que se ejecutan en múltiples núcleos de CPU. Como vimos en [Capítulo 1](#), el servidor de aplicaciones de sistemas y el [arquitectura MongoDB](#) puede convertirse en un cuello de botella si no envía suficiente operaciones en paralelo para utilizar completamente el servidor MongoDB.

Por ejemplo, si tuvieras que realizar una serie de experimentos usando tu la carga de trabajo de rendimiento crítica de la aplicación y graficar los resultados en un En la hoja de cálculo, es posible que vea un gráfico similar a la Figura 13.2, [cual](#) que muestra las tendencias de utilización y rendimiento de la CPU como la cantidad de Aumentan los subprocesos o las operaciones paralelas.

DB ops/s and CPU Utilization

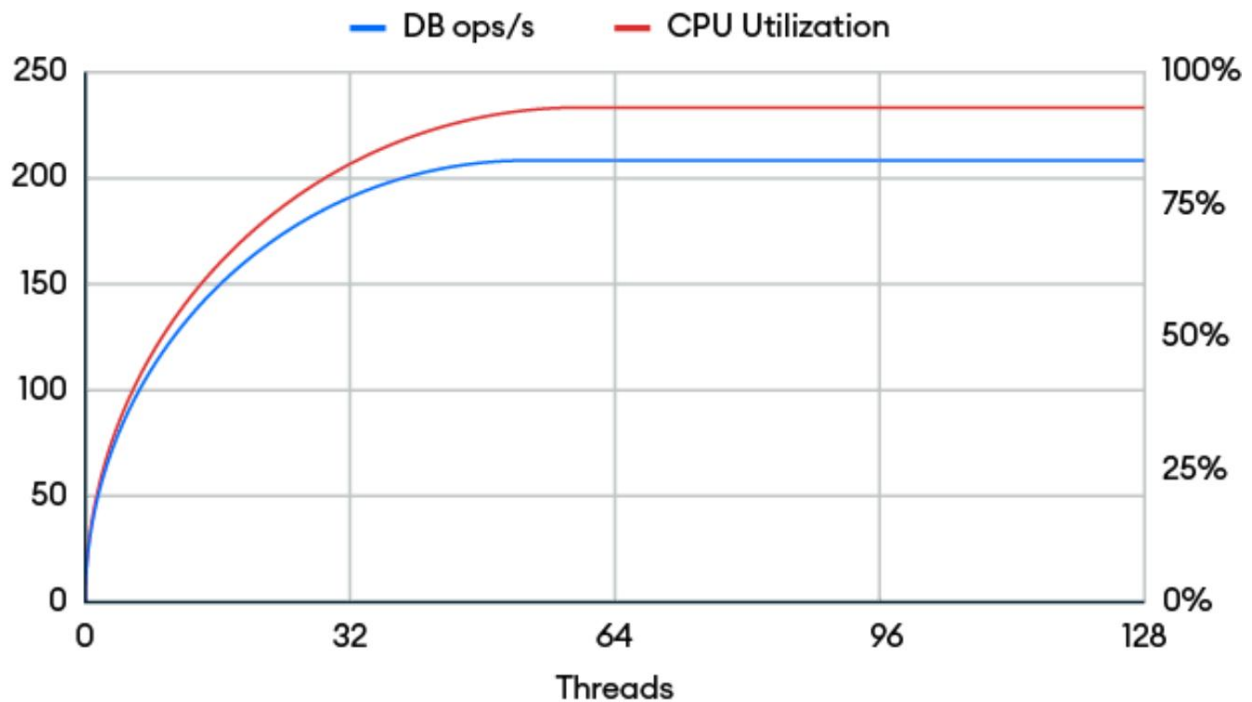


Figura 13.2: Los efectos sobre la utilización de la CPU y las tendencias de rendimiento a medida que aumenta el paralelismo

En este ejemplo, el servidor de base de datos se ejecuta en una CPU de 16 núcleos.

El rendimiento alcanza su punto máximo con 64 subprocesos e incluso puede disminuir ligeramente a partir de ese punto a medida que se añaden más. Este es un patrón típico en muchas aplicaciones. En este caso, el rendimiento óptimo se alcanza con 64 subprocesos en 16 núcleos, lo que sugiere una proporción de 4 subprocesos o conexiones activas por núcleo. Es importante realizar pruebas con una carga de trabajo y un conjunto de datos de producción para determinar la proporción ideal de subprocesos por núcleo.

Sus cargas de trabajo de rendimiento críticas. Por ejemplo, no querrá ejecutar solo ocho operaciones en paralelo en una máquina de 16 núcleos, dejando la mitad de los núcleos inactivos, ya que esto puede generar costos innecesarios. costos de recursos.

Busque otros programas que consumen mucha CPU

procesos durante las caídas de rendimiento

Las caídas periódicas del rendimiento pueden ser causadas por otros procesos que hacen un uso intensivo de la CPU.

Procesos que compiten por los recursos del sistema. Para diagnosticar estos problemas,

Puede utilizar varias herramientas y comandos para identificar qué procesos

consumen recursos importantes. Puedes usar herramientas de Linux como

`top` , `htop` , `ps` , `pidstat` , `iotop` , en el sistema y `perf` para ver qué se está ejecutando

durante caídas de rendimiento.

Por ejemplo, al ejecutar el comando `top` se muestran los procesos principales por

Uso de recursos:

```
arriba - 18:09:15 activo 10 min, 1 usuario, carga promedio: 1.01, 0
%Cpu(s): 87,0 us, 13,0 sy, 0,0 ni, 0,0 id, 0,0 wa, 0
MiB Mem: 31373.4 total, 30190.5 libres, 428.5 usados,
Intercambio de MiB: 0,0 total, 0,0 libres, 0,0 usados.
  PID  USUARIO  RES  SHR  S  %CPU  %MEM
 11108  python    11108  7464  S   16.1   0.2
 1910  ubuntu    500288  99191  6436  R   16.1
 1  raíz      20  0  167004  11108  7464  S   6.0
 4  raíz      0  -20      0      0      0  0 yo 0.0
589  raíz      20  0  82100  3204  2796  S   0.3
1968  ubuntu    5  -15      7148  3296  2448  R   0.3
raíz    20  0      0      0      0  0 S 0.0
```

En esta salida, puede ver que no hay ciclos de CPU inactivos (0.0 id

), y un proceso Python utiliza el 16,1% de la CPU, tiempo que

De lo contrario, es posible que se pueda utilizar MongoDB (`mongod`).

Si MongoDB se ejecuta en una máquina virtual, pueden producirse caídas periódicas

También pueden ser resultado de procesos externos a la máquina virtual, como otros

Máquinas virtuales o tareas de hipervisor. En este caso, verifique el uso de recursos a nivel de host con herramientas de hipervisor como `vmstat` o las `vmstat`, `virt-top`, `virt-top`, herramientas de monitorización de su proveedor de nube.

Seleccione el sistema de archivos adecuado para su aplicación . Esta sección se centra en los sistemas

de archivos de Linux. Abordaremos con más detalle las dos opciones más relevantes: el Sistema de Archivos eXtent (XFS) y el cuarto sistema de archivos extendido (ext4), y abordaremos otros sistemas de archivos en una sección posterior.

Sistema de archivos de extensión (XFS)

Las notas de producción de MongoDB recomiendan XFS para implementaciones de producción. XFS generalmente ofrece un mejor rendimiento para cargas de trabajo con alto rendimiento y operaciones de lectura/escritura simultáneas. Gestiona archivos grandes y datos fragmentados con mayor eficiencia que ext4 y suele gestionar mejor un gran número de operaciones de vaciado, presentando menos bloqueos de E/S durante los puntos de control. XFS también admite archivos y sistemas de archivos de mayor tamaño.

Desarrollado originalmente por Silicon Graphics, Inc. (SGI) en 1994, XFS se adaptó a Linux en 2001 y se integró en el kernel principal en 2004 (versión 2.6). Red Hat adoptó XFS como sistema de archivos predeterminado para RHEL 7 en 2014, gracias a su escalabilidad y rendimiento para cargas de trabajo empresariales.

Cuarto sistema de archivos extendido (ext4)

Si la distribución de Linux que usa es compatible con XFS, úsela. De lo contrario, ext4 sigue siendo una opción sólida. Ext4 es un sistema de archivos consolidado y ampliamente utilizado, y ha sido durante mucho tiempo el predeterminado en muchas distribuciones de Linux. Si bien ext4 solía tener mayor disponibilidad que XFS, la diferencia se ha reducido en los últimos años. La diferencia de rendimiento entre ext4 y XFS también se ha reducido con el tiempo, a medida que los sistemas operativos, los kernels y el propio MongoDB han evolucionado.

Otros sistemas de archivos

Si bien MongoDB puede ejecutarse en otros sistemas de archivos, sus características a veces se superponen con las que ya ofrece MongoDB y WiredTiger, lo que genera ineficiencias:

- Sistema de archivos de red (NFS) : no recomendado para MongoDB.
Consulte las notas de producción de MongoDB para obtener más información.
- Sistema de archivos B-tree (Btrfs): Puede presentar dificultades con cargas de trabajo con alta carga de escritura. Su diseño de copia en escritura (CoW) puede causar amplificación de escritura, donde un documento de 100 B a nivel de base de datos podría resultar en la escritura de muchos más datos en el dispositivo de almacenamiento. Btrfs también implementa mecanismos ya existentes en WiredTiger.
- Sistema de archivos Zettabyte (ZFS): Más adecuado para datos grandes y secuenciales, como copias de seguridad o medios. Para las lecturas/escrituras pequeñas y aleatorias de MongoDB, ZFS puede ser más lento que XFS o ext4.
- Sistema de archivos Ceph (CephFS): Diseñado para clústeres de almacenamiento distribuido y aplicaciones nativas de la nube. MongoDB ya admite estos casos de uso mediante fragmentación y replicación.

En general, XFS o ext4 son las opciones más confiables para las implementaciones de MongoDB, ya que ofrecen el mejor equilibrio entre rendimiento, estabilidad y compatibilidad.

Configuración del sistema de archivos. De

forma predeterminada, antes del kernel 2.6.30, Linux actualizaba los metadatos de tiempo de acceso (`atime`) de los archivos cada vez que se leían. Esto implicaba que las cargas de trabajo con uso intensivo de lectura también activaban una operación de escritura cada vez que se accedía a un archivo en el disco. Desde la versión 2.6.30, el tiempo de acceso relativo (`relatime`) se convirtió en el comportamiento predeterminado. Esto reduce la frecuencia de las actualizaciones de `atime` y evita escrituras innecesarias en el disco, lo que mejora significativamente el rendimiento de E/S de archivos, a la vez que conserva suficiente información de `atime` para la mayoría de los casos de uso.

Puede desactivar todas las actualizaciones de `atime` añadiendo la opción `noatime` a la configuración de montaje del sistema de archivos. El archivo `/etc/fstab` es un archivo de configuración de Linux que define cómo deben montarse los sistemas de archivos, las particiones y los dispositivos de almacenamiento durante el arranque.

Para deshabilitar las escrituras `atime` , agregue la opción `noatime` en `/etc/fstab` en , como este ejemplo:

```
/dev/sdX /ruta/a/mongodb_data xfs valores predeterminados,noatime 0 0
```

Otras opciones del sistema de archivos también pueden mejorar el rendimiento, pero muchas reducen la durabilidad de las escrituras. En general, MongoDB no recomienda usar estas opciones a menos que se comprendan claramente los riesgos y las desventajas.

Evite RAID con paridad Un dispositivo de almacenamiento de matriz redundante de discos independientes (o económicos) (RAID) se refiere a un sistema que utiliza múltiples discos duros físicos o SSD combinados para mejorar el rendimiento, la redundancia de datos o ambos. RAID se utiliza comúnmente en servidores, estaciones de trabajo y sistemas de almacenamiento para ofrecer mayor fiabilidad y rendimiento en comparación con un solo disco. Sin embargo, su uso es cada vez menor a medida que la industria avanza hacia soluciones de almacenamiento más modernas, distribuidas y definidas por software que ofrecen mayor escalabilidad, redundancia y tolerancia a fallos.

MongoDB no requiere RAID para la tolerancia a fallos. Si su organización exige el uso de RAID, se recomienda usar RAID 0 o RAID 10. RAID 0 divide los datos en varias unidades. Escribe datos simultáneamente en todas las unidades de la matriz, lo que mejora el rendimiento de lectura y escritura al aprovechar el paralelismo.

Sin embargo, RAID 0 no almacena datos reflejados ni de paridad, lo que significa que no ofrece protección de datos. Si una unidad falla, se pierden todos los datos. RAID 10 (también llamado RAID 1+0) combina la duplicación y la segmentación. Crea pares de unidades reflejadas (RAID 1) para redundancia y, a continuación, segmenta los datos entre los pares reflejados (RAID 0) para un mejor rendimiento.

Las notas de producción de MongoDB recomiendan evitar RAID 5 o RAID 6 para bases de datos con mucha escritura porque sus cálculos de paridad pueden ralentizar las operaciones de escritura.

Ajustar la configuración de lectura anticipada

La función de lectura anticipada de Linux es una optimización a nivel de kernel diseñada para mejorar la eficiencia de las lecturas secuenciales de disco. Anticipa el acceso futuro a los datos leyendo bloques de datos adicionales a los solicitados y almacenándolos en caché. En cargas de trabajo donde el acceso al disco y la distribución de datos son secuenciales, es posible que el siguiente bloque ya esté en la caché cuando la base de datos emita una solicitud de E/S. Esto puede mejorar el rendimiento al reducir la latencia de E/S del disco.

Con MongoDB, el impacto del ajuste de lectura anticipada depende de la naturaleza de la carga de trabajo, el diseño de los datos en el disco y la versión del kernel.

Las consultas de MongoDB normalmente acceden a los datos siguiendo un patrón aleatorio.

Sin embargo, si su carga de trabajo accede a los documentos en el orden en que fueron escritos, un valor de lectura anticipada más alto puede ser beneficioso.

De forma predeterminada, el motor WiredTiger de MongoDB lee bloques de datos de aproximadamente 28 KB la mayor parte del tiempo (con la configuración predeterminada `leaf_page_max=32 KB`). En Linux, el valor de lectura anticipada se especifica en bloques, cada uno de los cuales ocupa 512 bytes. Por ejemplo, un valor de lectura anticipada de 32 significa que el kernel precargará 32 bloques, lo que equivale a 16 KB.

En los kernels Linux 5.x y 6.x, se mejoró el manejo de la lectura anticipada con un valor de 0 para evitar las pérdidas de rendimiento observadas en los kernels 4.x. El kernel 6.x también incluye mejores heurísticas de escalado, que determinan de forma más eficiente el tamaño de la lectura anticipada según el comportamiento de la carga de trabajo y las características del dispositivo. Además, simplifica la lógica de la lectura anticipada al eliminar rutas de código redundantes. Cambiar la versión del kernel puede afectar significativamente el rendimiento, por lo que puede ser útil volver a ejecutar los experimentos de ajuste de la lectura anticipada después de un cambio o actualización del kernel.

Se recomienda comenzar con un valor de lectura anticipada de 32, luego prueba con 16 y 64. Si el rendimiento mejora con cualquiera de los dos, continúe.

Sintonización en esa dirección. Generalmente no se recomiendan valores de 0 o superiores a 256 .

Cambiar la configuración de caché del sistema de archivos Ajustar la configuración del sistema como `vm.dirty_background_ratio` puede mejorar el rendimiento y la confiabilidad de MongoDB al controlar cómo el kernel de Linux maneja las páginas sucias (modificadas pero aún no escritas en el disco) en la memoria.

La configuración `vm.dirty_background_ratio` define el porcentaje de la memoria total del sistema que puede llenarse con páginas sucias antes de que el kernel comience a vaciarlas al disco en segundo plano. Las páginas sucias representan cambios en el sistema de archivos (p. ej., de escrituras en MongoDB) que aún no se han almacenado.

MongoDB escribe frecuentemente en el disco y depende del sistema operativo para el almacenamiento en búfer de escritura. Si `vm.dirty_background_ratio` se establece demasiado alto, el kernel puede retrasar el vaciado, permitiendo la acumulación de una gran cantidad de páginas sucias. Esto puede provocar picos repentinos e intensos de E/S que aumentan la latencia. Reducir este valor resulta en vaciados más pequeños y frecuentes, lo que facilita las escrituras en disco.

Otra configuración relacionada, `vm.dirty_ratio` , define el porcentaje máximo de memoria que puede llenarse con páginas sucias. Una vez superado este umbral, los procesos que generan nuevas escrituras se bloquean hasta que las páginas sucias se vacían en el disco.



Escritura anticipada de diario versus escritura de archivos de datos



Estas configuraciones afectan la escritura en archivos de datos. MongoDB vacía inmediatamente las operaciones de escritura anticipada en el diario para mayor durabilidad (en lugar de depender de mecanismos a nivel de kernel).

Podría considerar ajustar `vm.dirty_background_ratio` en los siguientes escenarios:

- Picos de E/S o alta latencia : Si observa ráfagas irregulares de E/S de disco al vaciar grandes lotes de páginas sucias simultáneamente, reduzca la tasa de fondo sucia para activar vaciados más pequeños y frecuentes. Utilice el comando `iostat` para detectar valores altos de `await` . La medida de `await` es el tiempo total de respuesta (en milisegundos) que observan las aplicaciones al realizar solicitudes de E/S.

Incluye tanto el tiempo de espera en la cola del programador de E/S como el tiempo de acceso al disco. Mida el tiempo de espera antes y después de un cambio para comprobar si influye.

- Uso excesivo de memoria por páginas sucias : Si la memoria sucia ocupa constantemente una parte significativa de la memoria, más del 10-20 % de la RAM, reduzca `vm.dirty_background_ratio` para activar vaciados en segundo plano más tempranos. Puede usar el siguiente comando para monitorizar la memoria sucia en Linux:

```
$ cat /proc/meminfo | grep -E "Sucio|Mem"
```

Aquí está el ejemplo de salida:

```
MemTotal:      32126312 kB
MemFree:       3067873 kB
```

MemDisponible:	3134261 kB
Sucio:	6425262 kB

La parte `grep` del comando busca las subcadenas "Dirty" y "Mem" en la salida. En este ejemplo, el sistema tiene aproximadamente un 10 % de memoria libre y casi un 20 % de memoria sucia.

- Puntos de control lentos o alta latencia de escritura : las duraciones prolongadas de los puntos de control de MongoDB o la latencia durante las operaciones de escritura pueden indicar una mala sincronización entre MongoDB y el comportamiento de vaciado del kernel.
- Bloqueos de escritura debido a una tasa de escritura sucia excedida : Si las escrituras en MongoDB se bloquean cuando la memoria sucia supera el umbral definido por `vm.dirty_ratio` , es necesario ajustar tanto `vm.dirty_ratio` como `vm.dirty_background_ratio` . Puede usar el siguiente comando en Linux para ver las operaciones de escritura y los procesos bloqueados:

```
dstat -c --top-io
```

- El rendimiento del sistema disminuye durante una alta actividad de escritura : el alto uso de la CPU o la degradación del rendimiento del disco durante picos de escritura sugieren una limpieza ineficiente de páginas sucias.

Estos escenarios resaltan cuándo se hace necesario ajustar `vm.dirty_background_ratio` , pero comprender el ajuste es solo el primer paso; aplicarlo de manera inteligente, junto con configuraciones complementarias y monitoreo, es clave para mejorar el rendimiento de escritura general.

Para ello, comience por reducir moderadamente `vm.dirty_background_ratio` para controlar la acumulación de páginas sucias y activar vaciados más tempranos.

El valor predeterminado suele ser del 10 %. Se recomienda empezar con un valor inferior, entre el 5 % y el 7 %.

```
sudo sysctl -w vm.dirty_background_ratio=7
```

Reduzca `vm.dirty_ratio` para reducir el riesgo de bloqueo. El valor predeterminado suele ser del 20 al 30 %. Se recomienda reducir este valor al 15-18 %.

```
sudo sysctl -w vm.dirty_ratio=18
```

Los dos comandos anteriores realizan cambios en el sistema en ejecución. Si desea que los cambios persistan al reiniciar el servidor, debe modificar el archivo `/etc/sysctl.conf`. Al experimentar con estas opciones de configuración, le recomendamos:

- Monitoreo del rendimiento después de realizar cada ajuste
- Uso de `iostat` o `sar` para garantizar una entrada/salida fluida y predecible sin ráfagas
- Medición de la latencia de escritura de MongoDB con `mongostat`
- Búsqueda en los registros de la base de datos de los tiempos de finalización de los puntos de control para garantizar duraciones predecibles

Si su carga de trabajo presenta picos de latencia de escritura, un alto uso de memoria por páginas sucias o escrituras bloqueadas, reducir `vm.dirty_background_ratio` puede ayudar a optimizar las operaciones de vaciado de disco y reducir los cuellos de botella en el rendimiento. Combine esto con la monitorización y los ajustes de otras configuraciones, como `vm.dirty_ratio`, la frecuencia de los puntos de control de MongoDB y el tamaño de la caché de WiredTiger, para una optimización máxima.

Comprobar el estado del SSD

Los SSD utilizan sofisticadas técnicas de firmware, como la nivelación del desgaste, la recolección de elementos no utilizados, el sobreaprovisionamiento y TRIM, para reducir el impacto en el rendimiento del borrado previo a la escritura. En el sistema operativo, asegúrese de que TRIM y la recolección de elementos no utilizados estén habilitados. Estas funciones hacen que los SSD modernos sean rápidos y fiables en una amplia gama de casos de uso. Sin embargo, es importante supervisar su estado para detectar cuándo el desgaste empieza a afectar al rendimiento o cuándo la unidad está llegando al final de su vida útil.

En Linux, la mayoría de los SSD son compatibles con los informes de datos SMART, que proporcionan métricas detalladas de estado y diagnóstico, como el estado de nivelación de desgaste, la vida útil restante y el recuento de bloques defectuosos. Muchos proveedores de SSD ofrecen herramientas propietarias para supervisar sus unidades. Estas herramientas suelen revelar datos relacionados con el desgaste y permiten configurar alertas para problemas de estado o degradación del rendimiento. Si utiliza SSD de nivel empresarial en un servidor o centro de datos, puede que tenga disponibles herramientas de supervisión adicionales, como Nagios, Datadog, Dynatrace, Zabbix o Prometheus con Node Exporter.

Los SSD también registran advertencias o errores a nivel de sistema cuando el estado o el rendimiento comienzan a disminuir. En Linux, revise `/var/log/syslog` o ejecute `journalctl` en sus SSD. `dmesg` para buscar errores de E/S relacionados con el uso de

Evite el doble cifrado y la compresión

Algunas tecnologías de almacenamiento admiten compresión o cifrado, y otras admiten ambos. Por ejemplo, las matrices de almacenamiento empresariales y los SSD suelen incluir ambos. Los servicios de almacenamiento en la nube no suelen admitir la compresión, pero sí el cifrado para proteger los datos en reposo, en tránsito y en instantáneas.

Los datos comprimidos o cifrados son más difíciles de comprimir. Si la base de datos comprime o cifra los datos, la lógica de compresión del dispositivo de almacenamiento puede encontrar menos patrones redundantes, lo que resulta en una compresión mínima o nula. En algunos casos, intentar comprimir datos ya comprimidos puede generar sobrecarga de almacenamiento, lo que aumenta los recursos computacionales necesarios.

Los datos procesados tanto por la base de datos como por el dispositivo de almacenamiento podrían comprimirse y cifrarse dos veces. Esto suele ser ineficiente y puede afectar al rendimiento, sin ofrecer ventajas significativas. Para evitarlo, debe determinar dónde se aplican la compresión y el cifrado con mayor eficacia: en la capa de base de datos o en la de almacenamiento.

En algunas industrias, el cifrado doble puede proporcionar una capa adicional de defensa al garantizar que el cifrado se aplique de forma independiente en múltiples capas (por ejemplo, cifrado de base de datos para seguridad lógica y cifrado de dispositivo de almacenamiento para seguridad física).

Asegúrese de que el uso de la memoria residente se mantenga por debajo del 80 %

Como regla general, debe intentar mantener el uso de memoria residente por debajo del 80% porque la memoria baja puede tener un gran impacto en

Rendimiento. Por ejemplo, el código podría ser paginado (expulsado de la RAM física) porque puede recargarse desde su fuente original.

Sin embargo, si necesita volver a ejecutarse, deberá volver a paginarse y esperar la E/S. Si se configura el intercambio, el sistema mueve las páginas de memoria menos utilizadas al disco para liberar RAM para los procesos activos. Esto provoca una escritura y una posterior lectura desde el almacenamiento al acceder de nuevo a la memoria. Si la memoria se agota críticamente, el fallo de memoria insuficiente (OOM) podría interrumpir el servidor. proceso.

Si gestiona usted mismo un sistema MongoDB y no puede agregar memoria, debe supervisar periódicamente el uso de memoria de su clúster para determinar si es necesario realizar ajustes. Podría experimentar reduciendo el tamaño de la caché de WiredTiger, como comentamos en

[Capítulo 7](#), Motores de almacenamiento .

En Atlas, puede configurar alertas específicas para notificarle cuando la memoria residente se acerca a umbrales críticos. Puede solucionar esto escalando a un nivel superior o habilitando el escalado automático, que se describe con más detalle en la sección "Uso del escalado automático para el rendimiento de Atlas" más adelante en este capítulo.

Tanto en implementaciones Atlas como autoadministradas, si una gran cantidad de memoria es utilizada por agregaciones complejas que ejecutan cargas de trabajo analíticas, se pueden usar las características de MongoDB para aislar la carga de trabajo.

El aislamiento de cargas de trabajo minimiza la contención de memoria y garantiza que cada carga de trabajo se ejecute eficientemente. Al crear un clúster Atlas, puede crear nodos de análisis o de búsqueda para aislar las cargas de trabajo.

Utilice nodos de análisis para aislar las consultas que no desea que compitan con su carga de trabajo operativa. Por ejemplo, los nodos de análisis pueden gestionar operaciones de análisis de datos, como consultas de informes de BI Connector for Atlas. El nivel de clúster de sus nodos de análisis puede ser diferente al de los nodos operativos. Si selecciona un nodo de análisis en un nivel significativamente inferior, deberá supervisar el retardo de replicación para asegurarse de que no tenga recursos insuficientes.

De igual forma, puede usar nodos de búsqueda dedicados para aislar las cargas de trabajo de búsqueda de su carga de trabajo operativa. Por último, si aún no utiliza la fragmentación, esta podría ser una buena razón para empezar. Consulte para [Capítulo 6](#), Fragmentación, obtener más información sobre cuándo y cómo añadir la fragmentación.

Mejores prácticas de redes Como con todos los sistemas

distribuidos, seguir las mejores prácticas de redes es esencial para lograr implementaciones de MongoDB estables y de alto rendimiento.

Si ha determinado que la red es el factor limitante actual, la siguiente lista de verificación puede ayudar:

- Implemente clústeres de MongoDB cerca de sus servidores de aplicaciones (por ejemplo, en la misma región si utiliza infraestructura en la nube).
- Utilice conexiones de red de baja latencia con interconexiones de alta velocidad.
- Utilice la compresión de red y monitoree/optimize los patrones de consulta para reducir la transferencia de datos. Esta función es compatible con los controladores de MongoDB, pero no está habilitada por defecto.
- Asigne suficiente ancho de banda a los servidores MongoDB, especialmente en entornos de producción de alto tráfico.

- Utilice interfaces de red optimizadas para un mayor rendimiento (por ejemplo, Ethernet gigabit o multigigabit).
- Confíe en una infraestructura de red estable y bien mantenida para minimizar la pérdida de paquetes.
- Priorice el tráfico de MongoDB utilizando configuraciones de calidad de servicio (QoS).
- Establezca valores de tiempo de vida (TTL) cortos y consistentes para las entradas DNS a fin de evitar problemas de caché obsoleto.
- Implemente clústeres MongoDB con direcciones IP fijas o nombres DNS que se resuelvan rápidamente; utilice servidores DNS locales para una resolución más rápida.
- Aísle el tráfico de MongoDB de las cargas de trabajo no críticas mediante VLAN o QoS.
- Escale la infraestructura de red o migre MongoDB a un entorno menos congestionado si es necesario.
- Evite la inspección profunda de paquetes del tráfico de MongoDB siempre que sea posible.
- Utilice nubes privadas virtuales (VPC), redes confiables y VPN para proteger el tráfico sin sobrecargar los sistemas de filtrado de paquetes.
- Implementar políticas de enrutamiento consistentes en todos los nodos.
- Pruebe la topología de la red periódicamente para garantizar una distribución del tráfico equilibrada y eficiente.
- Implemente clústeres de MongoDB dentro de la misma región geográfica cuando sea posible.
- Aproveche los clústeres globales de MongoDB Atlas para cargas de trabajo distribuidas mientras minimiza la latencia.
- Elija proveedores de nube con interconexiones regionales optimizadas.

Además, considere ajustar los siguientes parámetros de red a nivel del sistema operativo:

- **somaxconn** En Linux, este parámetro define el número máximo de solicitudes de conexión pendientes que se pueden poner en cola en un socket. Aumentar el valor de somaxconn permite a MongoDB poner en cola más conexiones entrantes, lo que reduce la probabilidad de errores de conexión durante períodos de tráfico de red intenso. Controla cuántas solicitudes SYN pendientes se pueden poner en cola durante el protocolo de enlace TCP inicial.
- **tcp_max_syn_backlog** Previene ataques de inundación SYN cuando el backlog de solicitudes SYN pendientes se llena.
- **tcp_syncookies** Este parámetro controla el tiempo (en segundos) que un socket permanece en el estado TIME_WAIT después de cerrar la conexión. El estado TIME_WAIT es un mecanismo de seguridad que garantiza que todos los paquetes relacionados con la conexión cerrada se hayan transmitido y confirmado correctamente antes de que el socket se libere para su reutilización. Si bien esto es fundamental para
- **tcp_fin_timeout**

Para evitar errores de retransmisión de paquetes, un valor tcp_fin_timeout innecesariamente alto puede provocar el agotamiento de los recursos en entornos con mucha actividad, ya que el sistema mantiene una gran cantidad de estados de conexión cerrados durante un período prolongado.

Aunque los cuellos de botella en la red suelen ser menos comunes que los relacionados con la CPU, la memoria o el almacenamiento, aun así es recomendable monitorear activamente el estado de la red y abordar los problemas de forma proactiva con las herramientas y la configuración adecuadas.

Uso del escalado automático para mejorar el rendimiento de Atlas

MongoDB Atlas ofrece escalado automático, una potente función diseñada para optimizar la gestión de recursos y garantizar un rendimiento óptimo para las cargas de trabajo de su clúster. Al ajustar dinámicamente el nivel del clúster y la capacidad de almacenamiento, el escalado automático le ayuda a mantener una alta disponibilidad y rendimiento, a la vez que minimiza la intervención manual.

Se puede escalar el cómputo hacia arriba o hacia abajo; el almacenamiento solo se escala automáticamente hacia arriba.

Los usuarios se benefician de la simplicidad y eficiencia de Atlas. Si no se satisfacen sus necesidades específicas, consulte con el soporte de MongoDB para obtener orientación adaptada a su aplicación específica.

Cuando las cargas de trabajo cambian y los datos crecen, escalar recursos manualmente puede ser complicado e ineficiente. El escalado automático aborda estos desafíos aprovechando la automatización, lo que garantiza que los recursos de su clúster se ajusten a las demandas de su aplicación en tiempo real. Esto elimina el sobreaprovisionamiento o la infrautilización, lo que se traduce en rentabilidad y un rendimiento constante.

Los beneficios clave de rendimiento del escalamiento automático incluyen lo siguiente:

- Gestión fluida de la carga de trabajo : ajusta automáticamente los recursos para adaptarse a picos o caídas repentinas en la carga de trabajo sin requerir tiempo de inactividad.
- Mayor confiabilidad : garantiza que haya suficientes recursos (CPU, RAM y almacenamiento) están disponibles para evitar cuellos de botella durante el uso pico
- Optimización de costos : evita el aprovisionamiento innecesario cuando las demandas de carga de trabajo son bajas, lo que reduce los gastos operativos.

- Escalamiento proactivo : prepara su aplicación para picos de uso rápidos o grandes eventos de ingesta de datos para brindar un servicio ininterrumpido.

Con el escalado automático, MongoDB Atlas monitorea automáticamente el rendimiento de su clúster y las métricas de utilización, como el uso de CPU y el espacio en disco.

Estos son los dos componentes principales del escalado automático en MongoDB Atlas:

- Escalado automático a nivel de clúster : ajusta el nivel del clúster (por ejemplo, M10, M20 o M40) para garantizar recursos suficientes para el uso de la CPU y la memoria. Escalado
- automático de almacenamiento : escala la capacidad del disco para adaptarse al crecimiento de los datos, lo que evita errores relacionados con el almacenamiento insuficiente.

El escalado automático está habilitado de forma predeterminada al aprovisionar un nuevo clúster y puede deshabilitarse si es necesario. Para mantenerse informado sobre los eventos de escalado, configure alertas en la suite Atlas Monitoring. Las alertas le notificarán sobre los cambios en los recursos, lo que le permitirá realizar un seguimiento de la actividad y garantizar que los ajustes se ajusten a sus expectativas. El escalado automático no sustituye otras prácticas recomendadas de rendimiento, como la optimización de consultas. Es una medida de seguridad temporal, pero a largo plazo, debería revisar periódicamente el rendimiento y el historial de escalado de su clúster para identificar patrones y aplicar otras optimizaciones.

Resumen

Este capítulo ha proporcionado una guía detallada para optimizar el Rendimiento de MongoDB mediante la configuración del sistema operativo y

Otros recursos del sistema, con un enfoque principal en implementaciones autogestionadas. Si bien MongoDB Atlas automatiza muchas de estas tareas, comprender los principios subyacentes sigue siendo valioso.

Abordamos las categorías clave de recursos (CPU, memoria, almacenamiento y redes) y ofrecemos orientación práctica sobre temas como la relación núcleo-operación, la elección del sistema de archivos adecuado (con una recomendación para XFS) y el ajuste de la configuración de la caché del sistema de archivos. También advertimos contra el uso de cifrado/compresión doble y el uso de memoria residente superior al 80 %.

Además, describimos una lista de verificación para la optimización de la red, que incluye garantizar la proximidad de los clústeres, mantener conexiones de baja latencia y planificar la asignación de ancho de banda. Finalmente, explicamos cómo el escalado automático de MongoDB Atlas ajusta dinámicamente los niveles del clúster y la capacidad de almacenamiento para satisfacer la demanda.

El siguiente capítulo examina la monitorización y la observabilidad en MongoDB, mostrando cómo realizar un seguimiento de métricas críticas, aprovechar herramientas integradas y externas y establecer prácticas que garanticen la confiabilidad y el rendimiento a escala.

14

Monitoreo y observabilidad

En entornos de bases de datos de alto rendimiento, comprender lo que sucede bajo el capó no solo es beneficioso, sino que es esencial.

El monitoreo y la observabilidad forman la base para mantener y optimizar las implementaciones de MongoDB, especialmente a medida que los sistemas escalan y las cargas de trabajo se vuelven más complejas.

Este capítulo proporciona una guía práctica para la supervisión y la observabilidad en MongoDB, brindándole el conocimiento necesario para mantener un alto rendimiento y confiabilidad a medida que sus sistemas crecen. Aprenderá a interpretar métricas clave, a aprovechar herramientas integradas y externas, y a responder proactivamente a las primeras señales de problemas antes de que afecten a sus aplicaciones. Tanto si ejecuta MongoDB en la nube con Atlas como si gestiona su propia infraestructura, este capítulo le ayudará a desarrollar una estrategia de observabilidad robusta y adaptada a sus necesidades.

Los temas clave tratados en este capítulo incluyen los siguientes:

- La diferencia entre monitoreo y observabilidad
- Por qué tanto la monitorización como la observabilidad son importantes para el rendimiento de la base de datos
- Las métricas y señales principales de MongoDB que se deben seguir para lograr un estado óptimo y la resolución de problemas

- Cómo usar MongoDB Atlas y herramientas autogestionadas para monitoreo y alertas en tiempo real
- Mejores prácticas para integrar métricas de MongoDB en plataformas de observación más amplias

Diferencias clave entre el seguimiento y la observabilidad

Aunque a menudo se usan indistintamente, la monitorización y la observabilidad representan prácticas distintas pero complementarias. La monitorización responde a la pregunta "¿Hay algún problema?" mediante la recopilación y alertas sobre métricas predefinidas, contadores de operaciones, presión de caché, fallos de página y recuentos de conexiones en comparación con umbrales conocidos. Se centra en el seguimiento de los modos de fallo conocidos de un sistema.

La observabilidad responde a la pregunta "¿Por qué sucede?", enriqueciendo esas métricas con datos contextuales como registros, seguimientos y planes de explicación, para que pueda analizar en detalle desde una alerta hasta la causa raíz. Permite explorar problemas desconocidos al proporcionar una visión completa del estado interno del sistema.

En entornos de alto rendimiento, ya sea ejecutando un único conjunto de réplicas o un clúster fragmentado global, las degradaciones sutiles del rendimiento pueden derivar en interrupciones importantes. Sin una visibilidad completa del uso de recursos, el rendimiento de las consultas y el estado del sistema, identificar cuellos de botella se convierte en una tarea ardua.

La observabilidad transforma estas conjeturas en información basada en datos. Al rastrear continuamente las señales clave, puede optimizar el rendimiento y

Detectar señales tempranas de problemas antes de que afecten sus aplicaciones.

A continuación se muestran algunos ejemplos:

- Correlacionar las operaciones registradas con el uso del índice (o la falta de él) y los planes de ejecución para identificar índices faltantes o ineficientes
- Superponga las profundidades de la cola de discos con los tiempos de confirmación para descubrir la contención del almacenamiento bajo cargas de trabajo máximas
- Retraso de replicación de gráficos en relación con el rendimiento de escritura para detectar cuellos de botella en la red durante eventos de conmutación por error
- Supervisar la presión de la memoria para identificar tamaños de caché óptimos y evitar fallas de página
- Realizar un seguimiento de los patrones de conexión para dimensionar adecuadamente los grupos de conexiones y evitar las colas

Estas prácticas son importantes porque convierten los datos de monitoreo sin procesar en información útil. Al correlacionar las consultas lentas con el uso del índice, superponer la profundidad de las colas de disco con los tiempos de confirmación y rastrear el retardo de replicación, la presión de memoria y los patrones de conexión, puede ir más allá de simplemente detectar un problema; puede identificar la causa raíz de los problemas de rendimiento. Esto le permite optimizar proactivamente su implementación de MongoDB, evitar interrupciones y garantizar que su base de datos siga ofreciendo un rendimiento alto y confiable a medida que las cargas de trabajo crecen y cambian. En resumen, estas técnicas le ayudan a pasar de la resolución de problemas reactiva a la mejora continua basada en datos.

Para optimizar verdaderamente el rendimiento, se necesita más que una simple instantánea del estado del sistema; se necesitan señales claras y oportunas que revelen cuándo y dónde se produce la degradación. Una estrategia robusta de monitoreo y observabilidad proporciona estas señales esenciales, lo que facilita...

Es posible identificar problemas emergentes como consultas lentas, limitaciones de recursos, retrasos en la replicación, cuellos de botella de almacenamiento o problemas de conexión. Con esta visibilidad, obtiene el contexto y la confianza para diagnosticar, priorizar y resolver problemas antes de que se agraven, convirtiendo lo que de otro modo sería una simple extinción de incendios en un proceso de mejora continua y resiliencia.

Métricas y señales principales de MongoDB

Comprender el estado interno de su implementación de MongoDB es crucial para mantener un rendimiento óptimo y diagnosticar problemas.

MongoDB expone un conjunto completo de métricas que brindan información sobre su estado, utilización de recursos y características operativas. Al monitorear y correlacionar estas señales clave, puede identificar proactivamente posibles cuellos de botella, tomar decisiones informadas sobre la planificación de la capacidad y garantizar que su base de datos soporte eficazmente las cargas de trabajo de sus aplicaciones.

Estas métricas generalmente se dividen en dos categorías: métricas operativas, que brindan una visión en tiempo real de la actividad actual de la base de datos, y métricas específicas de rendimiento, que profundizan en el comportamiento de la carga de trabajo y las posibles áreas de contención.

Métricas operativas. Las métricas

operativas responden a la pregunta "¿Cómo está funcionando mi base de datos actualmente?" . Estas se exponen mediante el comando `serverStatus` y se pueden recopilar mediante el shell, los controladores o la monitorización externa.

Soluciones como Prometheus, como se analiza más adelante en la sección Integración con sección de sistemas de monitoreo externo.

El siguiente ejemplo demuestra cómo extraer algunos de los más Métricas operativas importantes mediante MongoDB Shell. Este código recupera el estado actual del servidor e imprime una selección de métricas clave que son esenciales para monitorear la salud y la carga de trabajo en tiempo real:

```

Ejecutar // Shell de MongoDB
Ejemplo: Extraer la clave métricas operativas en // MongoDB
constante estado = db.serverStatus();
imprimirjson({
  opcounters:          estado.opcounters,          // lee, residente
  memUsageMB:          estado.mem.resident,         // //
  cacheStats:          estado.wiredTiger.cache,     WiredTig
  conexiones: });      estado.connections          // activo v

```

El código anterior revela varias de las métricas más procesables para Monitoreo diario, pero es importante tener en cuenta que `serverStatus` Expone un conjunto mucho más amplio de métricas que cubren casi todos los aspectos de Funcionamiento de MongoDB. En la siguiente lista, destacamos solo algunos de los campos más relevantes para la observabilidad práctica y continua. Aquí está qué representa cada campo y dónde encontrar métricas relacionadas no mostrado en el código:

- `opcounters` (de `status.opcounters`) Estos rastrean el número de operaciones de base de datos (lecturas, escrituras, comandos) desde que se inició el servidor. Monitorear esto le ayuda a comprender patrones de carga de trabajo y detectar picos repentinos que pueden indicar Tormentas de consultas o problemas de aplicación. Desequilibrios entre lecturas.

Las escrituras pueden sugerir oportunidades de escalado u optimización.

- `memUsageMB` (de `status.mem.resident`) : Muestra la cantidad de RAM física (en megabytes) utilizada actualmente por el proceso MongoDB. Controlar esto ayuda a garantizar que el conjunto de trabajo se ajuste a la memoria, minimizando el acceso al disco y manteniendo el rendimiento. Para una visión más detallada, la sección "mem" de `serverStatus` también incluye memoria virtual, memoria asignada y fallos de página. `cacheStats`
- (de `status.wiredTiger.cache`) Proporciona detalles sobre el uso de la caché interna del motor de almacenamiento WiredTiger, incluyendo la cantidad de datos almacenados en caché y la proporción de páginas sucias (modificadas) y limpias. Una alta presión de caché o expulsiones frecuentes pueden indicar la necesidad de más memoria o un ajuste de la caché. La sección completa `wiredTiger.cache` contiene métricas adicionales, como tasas de desalojo y tamaño máximo de caché.
- conexiones (de `status.connections`) : Esto Informa sobre el número de conexiones de cliente activas y las ranuras de conexión disponibles. Monitorear esto ayuda a prevenir la saturación de las conexiones, lo que puede provocar errores en la aplicación o una respuesta deficiente. La sección de conexiones también muestra el número de conexiones disponibles y el total de conexiones creadas desde el inicio.
- Métricas de replicación (no se muestran en el código) : para las implementaciones que utilizan conjuntos de réplicas, las métricas como el retraso de replicación, la ventana de registro de operaciones, los tiempos de latido y las métricas de elección están disponibles en las secciones `repl` y `oplog` de `serverStatus` o a través de comandos específicos del conjunto de réplicas como `rs.status()` y

`rs.printReplicationInfo()` . Son vitales para garantizar la durabilidad de los datos y la preparación para la conmutación por error.

- Métricas de E/S de disco (no se muestran en el código) : Las métricas de rendimiento del almacenamiento, como la profundidad de la cola, la latencia, el rendimiento y las IOPS, se encuentran en las secciones `wiredTiger` y métricas de `serverStatus` o , mediante herramientas de monitorización externas. Estas son cruciales para identificar cuellos de botella en el almacenamiento y garantizar que el subsistema de discos pueda gestionar la carga de trabajo.

Al recopilar y revisar periódicamente estas métricas, obtendrá una visión integral y en tiempo real del estado operativo de su base de datos, lo que le permitirá detectar problemas emergentes y optimizar el rendimiento antes de que los problemas se agraven.

Métricas específicas de rendimiento. Si bien las métricas operativas ofrecen una visión general de la actividad de la base de datos, las métricas específicas de rendimiento profundizan en las causas de las ralentizaciones y las ineficiencias. Estas métricas ayudan a identificar cuellos de botella y a responder a la pregunta crucial: ¿ dónde se producen exactamente los problemas de rendimiento? Las métricas específicas de rendimiento incluyen estadísticas de consultas lentas, estadísticas de ejecución de consultas y métricas de uso del índice.

Para capturar y revisar operaciones lentas, utilice el generador de perfiles de bases de datos de MongoDB (mediante `db.setProfilingLevel()` y la colección `system.profile`) o Query Insights en Atlas. Se puede acceder a las métricas de bloqueo y contención mediante el comando `serverStatus` y visualizarlas con herramientas externas como Prometheus y Grafana.

Para obtener estadísticas de ejecución de consultas, como documentos examinados versus devueltos y desgloses del tiempo de ejecución, utilice el método `explain()` en el shell o los controladores, el generador de perfiles de base de datos y Atlas Query Insights. Para la monitorización programática, extraiga los campos relevantes de la salida del generador de perfiles o de `system.profile`. Para comprender el uso y la eficacia del índice, utilice el método `explain()`, el comando `collStats`, Atlas Performance Advisor y Query Insights para identificar patrones de uso y oportunidades de optimización.

Por ahora, familiarícese con las herramientas integradas (generador de perfiles de base de datos MongoDB, planes de explicación, `serverStatus`, y la interfaz de usuario Atlas) e integraciones externas (Prometheus, Grafana y Datadog) que hacen posible este nivel de observabilidad.

Monitoreo con MongoDB Atlas

MongoDB Atlas ofrece un servicio de base de datos en la nube totalmente gestionado que muestra señales críticas de la base de datos en tiempo real y le guía hacia optimizaciones basadas en datos. En entornos de bases de datos de alto rendimiento, una monitorización eficaz no se limita a recopilar métricas, sino a transformarlas en información práctica que le ayude a mantener el máximo rendimiento de sus clústeres.

Características de la interfaz de usuario de Atlas

La interfaz de usuario de Atlas ofrece un conjunto completo de funciones para la monitorización y el análisis del rendimiento. El panel de rendimiento en tiempo real ofrece visualizaciones en vivo del estado del clúster, incluyendo el consumo de recursos, las tasas de operación y los tiempos de ejecución de consultas, ideal para la gestión inmediata.

Supervisión. Performance Advisor analiza continuamente las cargas de trabajo y sugiere índices para mejorar el rendimiento, mientras que Query Insights proporciona una visión general al agrupar consultas similares, supervisar los percentiles de latencia e identificar colecciones con alta demanda . Por último, la integración del perfilador en Query Insights simplifica el diagnóstico de cuellos de botella en el rendimiento al ofrecer una interfaz intuitiva para el perfilador de bases de datos de MongoDB.

Panel de rendimiento en tiempo real (RTPP)

MongoDB Atlas ofrece una visualización en vivo y de alto nivel del estado y el rendimiento operativo de su clúster, actualizándose cada pocos segundos.

Disponible para clústeres M10+, RTPP es especialmente útil para la monitorización de entornos en vivo, pruebas de carga y validación del rendimiento posterior a la implementación. Le permite identificar visualmente picos, cuellos de botella y tendencias en diversas métricas críticas, lo que le ayuda a correlacionar los cambios de rendimiento con el uso de recursos o la carga operativa en tiempo real.

RTPP ofrece dos vistas principales, Gráfico y Tabla, un . Cada superficie es un conjunto completo de métricas, incluidas las siguientes:

- Consumo de recursos : Uso de CPU, memoria del sistema y operaciones de E/S de disco por segundo (IOPS), así como tráfico de red (bytes de entrada/salida). Estas métricas le ayudan a detectar la saturación de recursos o picos repentinos que suelen estar relacionados con la degradación del rendimiento.
- Tasas de operación : la cantidad de operaciones de lectura, escritura, comando y agregación procesadas por segundo, lo que proporciona información sobre la intensidad y los patrones de la carga de trabajo.

- Tiempos de ejecución de consultas : estadísticas de latencia para operaciones de lectura, escritura y comandos actuales, que le permiten identificar rápidamente ralentizaciones.
- Segmentación de consultas : la relación entre los documentos escaneados y los documentos devueltos en todas las operaciones durante un período de muestreo, lo que ayuda a evaluar la eficiencia del índice y las necesidades de optimización de consultas.
- Retraso de replicación : para los conjuntos de réplicas, el retraso entre los nodos primarios y secundarios es fundamental para la coherencia de los datos y la preparación para la conmutación por error.
- Lecturas y escrituras : la cantidad de operaciones de lectura/escritura activas y en cola, lo que indica posibles cuellos de botella o contención de recursos si las colas crecen.
- Conexiones : Conexiones de cliente actuales y red
rendimiento, útil para detectar la saturación de la conexión.
- Colecciones más activas : colecciones con la mayor cantidad de operaciones, que resaltan posibles puntos críticos para una mayor investigación (aprovechando estadísticas similares a mongotop).
- Operaciones más lentas : las operaciones más lentas que se ejecutan actualmente, con la capacidad de explorar en detalle o finalizar consultas problemáticas.

Puede pausar la vista de gráficos para inspeccionar los valores exactos de las métricas en un momento específico, incluyendo las operaciones más lentas y las recopilaciones más activas. RTPP está habilitado de forma predeterminada, pero los usuarios con acceso de propietario del proyecto pueden activarlo en la configuración del proyecto . RTPP es especialmente útil durante pruebas de carga, incidentes de producción o después de las implementaciones para correlacionar rápidamente los cambios de rendimiento con la utilización de recursos o la carga operativa.

Asesor de rendimiento

Atlas Performance Advisor analiza continuamente la carga de trabajo de su clúster, buscando específicamente patrones de consulta ineficientes. Ofrece información útil mediante lo siguiente:

- Sugerencias de índice : recomienda índices específicos que podrían mejorar el rendimiento de las consultas lentas identificadas, a menudo proporcionando un porcentaje de impacto estimado.
- Gestión de índices : Permite revisar, crear o eliminar índices sugeridos directamente desde la interfaz de usuario de Atlas. También resalta los índices potencialmente no utilizados que podrían eliminarse.

La integración de las comprobaciones de Performance Advisor en los flujos de trabajo de desarrollo o en las canalizaciones de CI/CD le ayuda a identificar y abordar de forma proactiva posibles problemas de rendimiento de consultas antes de que lleguen a producción.

Query Insights Query

Insights expone una observabilidad mejorada y una telemetría granular con estadísticas de latencia a nivel de espacio de nombres y de nivel de operación.

Monitoreo de información sobre

espacios de nombres Supervise la latencia de consultas a nivel de colección con Namespace Insights .

La página Información sobre espacios de nombres muestra dos gráficos y una tabla con información para cada espacio de nombres principal o anclado. Esta información incluye métricas y estadísticas para determinados hosts y tipos de operación.

Puede administrar espacios de nombres anclados y elegir hasta cinco espacios de nombres para mostrar en los gráficos de latencia de consulta correspondientes. Esto

Permite monitorizar la diferencia de rendimiento entre múltiples espacios de nombres mediante latencias percentiles (P50, P95 y P99), lo que revela no solo el rendimiento promedio, sino también la latencia de cola que experimentan los usuarios. Las latencias P95 o P99 altas indican que algunos usuarios podrían estar experimentando respuestas lentas.

Revisar esta información puede ayudarle a identificar colecciones activas, que pueden combinarse con otras herramientas para proporcionar una vista de alto nivel de la distribución de la carga de trabajo.

Formas de consulta y perfilador de consultas. En Query

Insights, Atlas ofrece una interfaz intuitiva para obtener información sobre las operaciones registradas. Las operaciones se registran en los registros de mongos / mongod cuando superan el umbral de slowms (predeterminado en 100 milisegundos).

La pestaña Formas de consulta agrupa estas operaciones por forma y permite ver sus métricas de rendimiento. El Perfilador de consultas permite obtener más detalles para diagnosticar y resolver cuellos de botella de rendimiento con mayor precisión mediante las siguientes funciones:

- Identificación de consultas lentas : Detecta consultas que superan constantemente el umbral de slowms . Esto puede ocurrir durante un análisis de colección. (leer colecciones completas sin usar un índice) u operaciones que no están indexadas de forma óptima. Si observa una consulta lenta con `planSummary: COLLSCAN` , Considere un índice apropiado.
- Operaciones lentas recientes : Lista de operaciones que superan el umbral configurado, junto con detalles clave como el tiempo de ejecución, el espacio de nombres (base de datos y colección) y los documentos escaneados frente a los devueltos. Si el tiempo de ejecución es alto y la proporción...

de documentos examinados versus documentos devueltos se eleva, optimice la consulta o agregue índices para reducir escaneos innecesarios.

- Estadísticas de ejecución : Analice en detalle cada operación lenta para examinar su plan de ejecución, uso del índice, latencia de lectura/escritura y tamaño de respuesta. Añada o ajuste los índices según sea necesario. Un tamaño de respuesta elevado puede indicar la necesidad de realizar proyecciones para limitar los campos devueltos.
- Filtrar y explorar en profundidad : limite la salida del generador de perfiles por rango de tiempo, tipo de operación o espacio de nombres para aislar problemas específicos. Use esto para correlacionar ralentizaciones con cambios en la aplicación, eventos de implementación o cargas de trabajo específicas.

En la práctica, utilice Query Insights junto con Performance Advisor para confirmar que los nuevos índices o cambios de consultas produzcan las mejoras de latencia esperadas.

Alertas

Atlas emite alertas para las condiciones de la base de datos y del servidor configuradas en su configuración de alertas. Cuando una condición activa una alerta, Atlas muestra un símbolo de advertencia en el clúster y envía notificaciones de alerta. Su configuración de alertas determina los métodos de notificación.

Atlas continúa enviando notificaciones a intervalos regulares hasta que la condición se resuelva o usted elimine o deshabilite la alerta.

Atlas cuenta con varias alertas predefinidas, tanto a nivel de organización como de proyecto; sin embargo, también puede crear alertas personalizadas. Analicemos algunas categorías de alertas comunes y cómo abordar los problemas subyacentes.

Problemas con la búsqueda en Atlas

Atlas Search activa alertas cuando la cantidad de CPU y memoria utilizada por los procesos de Atlas Search alcanza un umbral especificado. Si el proceso de búsqueda (mongot) se queda sin memoria, la indexación y las consultas fallan.

Las alertas de búsqueda de Atlas suelen aparecer al intentar crear un índice de búsqueda grande o complejo. Estos índices permanecen en la fase de sincronización inicial hasta que se resuelva el problema de memoria.

Si el proceso de búsqueda agota la memoria o el espacio en disco, puede actualizar su clúster para solucionar el problema inmediato. Puede seleccionar un nivel de clúster con más memoria, almacenamiento e IOPS, o considerar un nodo de búsqueda dedicado para obtener mayor disponibilidad, rendimiento y equilibrio de carga.

Problemas de conexión

Las alertas de conexión generalmente ocurren cuando se excede el número máximo de conexiones permitidas a un proceso MongoDB.

Una vez superado el límite, no se podrán abrir nuevas conexiones hasta que el número de conexiones abiertas descienda por debajo del límite.

Las alertas de conexión suelen ser síntoma de un problema mayor. Si estas alertas se activan con frecuencia, una solución adecuada probablemente consista en lo siguiente:

- Examinar sus aplicaciones de base de datos en busca de código de conexión defectuoso. Las situaciones en las que las conexiones se abren pero nunca se cierran pueden permitir que las conexiones antiguas se acumulen y, con el tiempo,

Exceder el límite de conexiones. Además, es posible que deba implementar algún tipo de agrupación de conexiones.

- Actualizar a un nivel de clúster Atlas más grande, que permite una mayor cantidad de conexiones, si su base de usuarios es demasiado grande para su nivel de clúster actual.

Al identificar la causa raíz y abordarla, ya sea en el código de su aplicación o en la configuración del clúster, puede evitar problemas de conexión recurrentes y garantizar un rendimiento de la base de datos más fluido y confiable.

Problemas de consulta. La

alerta de orientación de consulta suele aparecer cuando no hay un índice que admita una o varias consultas, o cuando un índice existente solo admite parcialmente una o varias consultas. Algunos ejemplos de estas alertas son los siguientes:

- **Query Targeting: Scanned Objects / Returned** Estas alertas se activan cuando el promedio de documentos escaneados, en relación con el promedio de documentos devueltos en todo el servidor en todas las operaciones durante un período de muestreo, supera un umbral definido. La alerta predeterminada utiliza un umbral de 1000:1 .

Lo ideal sería que la proporción entre documentos escaneados y documentos devueltos fuera cercana a 1.. Una proporción alta impacta negativamente en la consulta actuación.

- **Query Targeting: Scanned / Returned** Estas alertas se producen si la cantidad de claves de índice examinadas para completar una consulta en relación con la cantidad real de documentos devueltos cumple o supera un valor determinado.

Umbral definido por el usuario. Esta alerta no está habilitada de forma predeterminada.

La fuente más común de este problema es un índice faltante, que el Performance Advisor ofrece la forma más fácil y rápida de Investigar. El Asesor de Rendimiento monitorea las consultas que MongoDB considera lento y recomienda índices para mejorar Rendimiento. Identificar problemas mediante estas alertas, junto con la Los conceptos introducidos en este [Capítulo 12](#), Consulta e indexación avanzadas Conceptos, documento pueden ayudarle a solucionar problemas y abordarlos de manera más eficaz. consultas de bajo rendimiento.

Problemas de oplog

En [Capítulo 5](#), Replicación, Aprendimos sobre el registro de operaciones (oplog) y el papel que desempeña para garantizar que las operaciones de escritura se realicen se propaga a todos los miembros del conjunto de réplicas. También aprendimos que el registro de operaciones es una colección limitada de un tamaño fijo (ya sea por tamaño de disco o ventana) tamaño en el tiempo).

Si se realizan una gran cantidad de operaciones de escritura en el disco duro principal nodo del conjunto de réplicas y los nodos secundarios no tienen suficiente Es hora de reproducir todas las operaciones contenidas en el registro de operaciones, esto Por lo general, resultan en una “caída del oplog”, lo que requiere una intervención inicial. sincronizar para recuperar y garantizar que los datos sean consistentes en todos los todos los nodos.

Atlas ofrece monitoreo y alerta de eventos relevantes para este escenario en la forma siguiente:

- La ventana del registro de operaciones de replicación es (X) : esto ocurre si Cantidad aproximada de tiempo disponible en la replicación primaria El registro de operaciones alcanza o no alcanza el umbral especificado. Esto se refiere a

la cantidad de tiempo que el servidor principal puede continuar registrando, dada la velocidad actual a la que se generan los datos de oplog.

- Los datos del oplog por hora son (X) : esto ocurre si la cantidad de datos por hora que se escriben en el oplog de replicación de un servidor principal cumple o supera el umbral especificado.

El desencadenante más común de estas alertas son las operaciones intensivas de escritura y actualización en un corto periodo de tiempo. Cuando esto ocurre, algunas medidas de mitigación comunes son las siguientes:

- Aumente el tamaño del registro de operaciones editando la configuración de su clúster para asegurarse de que sea mayor que el valor máximo del gráfico GB/hora de registro de operaciones en la vista de métricas del clúster.
- Asegúrese de que todas las operaciones de escritura especifiquen una preocupación de escritura mayoritaria para garantizar que las escrituras se repliquen en al menos un nodo antes de pasar a la siguiente operación. Esto controla la velocidad del tráfico de su aplicación al evitar que el nodo principal acepte escrituras a una velocidad mayor que la que pueden gestionar los secundarios.

Para sacar el máximo provecho de su sistema de alertas, revise y ajuste periódicamente los umbrales de alerta para que se ajusten a su carga de trabajo y a las necesidades de su negocio. Esto ayuda a evitar la saturación de alertas y la omisión de incidentes. Integre las notificaciones de alerta con las herramientas de comunicación y gestión de incidentes preferidas de su equipo para garantizar una respuesta rápida. Trate las alertas como señales accionables: cuando se activen, investigue rápidamente la causa raíz, tome medidas correctivas (como escalar recursos, optimizar consultas o ajustar los umbrales) y documente la resolución para futuras consultas.

Audite periódicamente las alertas configuradas para garantizar que sigan siendo relevantes a medida que su aplicación e infraestructura evolucionan. Finalmente, utilice el historial de alertas y las tendencias para fundamentar la planificación de la capacidad, identificar problemas recurrentes e impulsar la mejora continua de su estrategia de monitoreo.

Para obtener una lista completa de las condiciones de alerta disponibles, consulte la [documentación de MongoDB Atlas en https://www.mongodb.com/docs/atlas/reference/alert-conditions/](https://www.mongodb.com/docs/atlas/reference/alert-conditions/).

Herramientas de monitorización autogestionadas

Al operar MongoDB fuera del entorno administrado de Atlas, usted es responsable de configurar su propio flujo de trabajo de observabilidad. Afortunadamente, MongoDB ofrece un conjunto de utilidades de línea de comandos integradas y comandos de base de datos diseñados para ofrecer información detallada sobre la actividad en tiempo real y el rendimiento histórico del servidor. Dominar estas herramientas es esencial para diagnosticar problemas, comprender las características de la carga de trabajo y garantizar el buen estado de sus implementaciones alojadas.

`mongostat`: instantánea de actividad en tiempo real. Piense en `mongostat` como una utilidad de monitorización específica de MongoDB que proporciona una vista dinámica y en tiempo real del estado de una instancia de `mongod` o `mongos` en ejecución. Al muestrear el estado del servidor periódicamente (por defecto, cada segundo), presenta métricas clave en un formato tabular conciso, lo que permite evaluar rápidamente la carga operativa actual y el uso de recursos.

Para ejecutar `mongostat`, puede conectarse a su instancia de MongoDB como

entonces:

```
// desde el terminal Comando del sistema Línea
mongostat --host localhost:27017 --rowcount 2
```

Verá un resultado similar al siguiente, actualizándose cada segundo:

	insertar	consulta	actualizar	eliminar	obtener más	comando sucio usado	fl
0			0	0	0	3 0	3,4% 7,4%
1	8	12	0	0	0	5 0	3,7% 2,5%

Los campos clave incluyen los siguientes:

- **Contadores de operaciones** : Muestra el número de operaciones por segundo, indicando la forma e intensidad de su carga de trabajo (`insertar`, `consulta`, `actualización`, y `eliminar`).
- **Memoria** : revela detalles de utilización de la memoria, incluida la memoria asignada. Memoria, tamaño virtual y memoria residente. Cambios significativos. o una memoria residente alta podría indicar presión de memoria (`mapeado`, `vsize`, y `res`).
- **Comandos** : Muestra la cantidad de comandos ejecutados por segundo (`comando`).

Para los conjuntos de réplicas, puede usar la opción `--discover` para recuperarlas automáticamente.

Realizar un seguimiento de todos los miembros y su estado:

```
mongostat --descubrir
```

Utilice `mongostat` para realizar una comprobación rápida del estado de salud y observar el estado inmediato.

impacto de los cambios en la aplicación o durante incidentes de rendimiento para obtener

una vista de alto nivel de la actividad de la base de datos.

mongotop: lectura y escritura a nivel de colección tiempos

Mientras que mongostat proporciona una descripción general de todo el servidor, mongotop

Se centra específicamente en el tiempo empleado en realizar lecturas y escrituras.

operaciones por colección. Esto es invaluable para identificar

¿Qué colecciones están experimentando la mayor actividad o potencialmente?

provocando cuellos de botella de E/S.

Al igual que mongostat , conecta mongotop a una instancia en ejecución:

```
// Correr desde el Comando del sistema Línea  
mongotop --host localhost:27017 --rowcount 2
```

El resultado se ve así:

ns	lectura total escritura	2025-04-30T16:30:27
mydb.usuarios	0,1 ms 0,1 ms 0,0 ms	
mydb.pedidos	5,2 ms 2,8 ms 2,4 ms	

Las columnas muestran lo siguiente:

- ns : El espacio de nombres (<database>.<collection>) que se está creando monitoreado
- total : Tiempo total que el servidor pasó realizando operaciones en Esta colección durante el intervalo
- lectura : Tiempo dedicado a operaciones de lectura
- escribir : Tiempo dedicado a operaciones de escritura

Utilice `mongotop` para identificar colecciones populares que podrían beneficiarse de Optimización del esquema, mejoras de indexación o fragmentación si hay actividad es constantemente alto y afecta el rendimiento general. Por ejemplo, En la salida anterior, puede ver que la colección `mydb.orders` experimenta una actividad de lectura y escritura significativamente mayor en comparación con `mydb.usuarios`, lo que podría justificar una mayor investigación.

serverStatus: Métricas completas a través del shell

Para obtener la instantánea más detallada del estado de un servidor MongoDB, El comando `serverStatus` es la fuente definitiva. Devuelve un gran... Documento JSON que contiene cientos de métricas que cubren casi todos los aspectos del funcionamiento del servidor:

```
// Correr desde el Shell de MongoDB
db.adminCommand({ estado del servidor: 1 })
// O usando el ayudante de shell
db.serverStatus()
```

Si bien el resultado es demasiado extenso para detallarlo completamente aquí, es importante Las secciones incluyen lo siguiente:

- `opcounters` : recuentos granulares de todas las operaciones de la base de datos desde El servidor se inició
- `Conexiones` : información sobre conexiones actuales, disponibles y en cola. conexiones de cliente
- `bloqueos` : información sobre la contención de bloqueos (base de datos, colección, o global)

- `wiredTiger` : métricas profundas específicas del motor de almacenamiento

`WiredTiger`, incluidas las siguientes:

- Uso de caché (páginas leídas en caché, páginas escritas desde caché, bytes sucios)
 - Puntos de control de transacciones
 - Estadísticas del administrador de
- `bloques mem` : Métricas de estadísticas de memoria residente y
 - `virtual` : Varias métricas operativas como tasas de inserción/actualización/eliminación de documentos, estadísticas del ejecutor de consultas y detalles de replicación

`serverStatus` es menos adecuado para el monitoreo en tiempo real debido al volumen de datos, pero es esencial para el análisis profundo, el establecimiento de líneas de base de rendimiento y el monitoreo programático donde se extraen métricas específicas y se rastrean a lo largo del tiempo mediante sistemas externos.

Perfilador de bases de datos: Análisis a fondo de las operaciones lentas. Si necesita comprender

las características de

rendimiento de cada consulta, el perfilador de bases de datos es la herramienta ideal. Captura información detallada sobre las operaciones de una base de datos específica, lo que le permite identificar consultas lentas o ineficientes que podrían afectar el rendimiento de la aplicación.

Para empezar, puede habilitar el perfilador para una instancia específica de `Mongod` mediante el método `setProfilingLevel()` desde `MongoDB Shell`. Este método ofrece tres niveles de perfilado, cada uno adaptado a diferentes necesidades:

- Nivel (0 desactivado) : el generador de perfiles está desactivado de forma predeterminada y no recopilar cualquier dato
- Nivel (1 operaciones lentas) : captura datos solo para operaciones más lento que un umbral especificado (slowms predeterminado: 100 ms)
- Nivel (2 todas las operaciones) : captura datos para todas las operaciones (uso con precaución en producción debido a la sobrecarga de rendimiento)

Además, puedes configurar slowms , que es la operación lenta.

umbral. Cualquier operación del usuario que se complete más rápido que esto no se completará.

se registrará en el registro de mongod . sampleRate también se puede configurar, lo que

Le permite controlar qué fracción de operaciones lentas debe ser

registrado.

El siguiente comando, cuando se ejecuta a través de MongoDB Shell,

Habilitar el generador de perfiles de base de datos para la base de datos actual con una velocidad lenta.

umbral de operación de 20 milisegundos y una frecuencia de muestreo del 42%:

```
db.setProfilingLevel(1, { slowms: 20, sampleRate: 0.42 })
```

Una vez habilitado, puede consultar los datos del generador de perfiles para analizar los recientes

operaciones. Por ejemplo, este comando busca y muestra los cinco

Operaciones perfiladas más recientes:

```
// desde el Shell de MongoDB  
// Encuentra las 5 operaciones perfiladas más recientes  
db.system.profile.find().sort({ ts: -1 }).limit(5).forEach
```

Las operaciones perfiladas se almacenan en el system.profile capped

colección en cada base de datos para la que se ha habilitado la creación de perfiles. Cada

El documento de esta colección contiene detalles como los siguientes:

- `millis` : La duración de la operación en milisegundos `ns` : El
- `espacio de nombres ( database.collection )` comando de destino :
- El documento de comando completo asociado con la operación
- `execStats` : Estadísticas de ejecución detalladas para operaciones de consulta (similares a la salida de una operación de explicación)

A continuación se muestra un ejemplo de una entrada del generador de perfiles para una consulta lenta. Dado que especificamos un valor de `appName` en la cadena de conexión de la aplicación conectada al clúster cuando se registró esta operación, el nombre de la aplicación se muestra junto con la operación:

```
{
  "op": "consulta", "ns":
    "mydb.orders", "comando":
    { "buscar":
      "pedidos", "filtro": { "estado":
        "enviado", "fechadelpedido": { "$gt" "ordenar": { "nombredelcliente": 1 }, "límite": 100 },
        "clavesexaminadas": 0, "documentosexaminados":
          1000000,

        "ndevuelto": 53,
        "longitudderespuesta": 24601, "milis":
          2543, "resumendelplan":
            "COLLSCAN", "estadísticasejecutivas":
              {

                // Estadísticas de ejecución detalladas
              },
            "cliente": "192.168.1.50", "usuario":
              "usuario de la aplicación",
            "nombreDeLaAplicación" : "mi-microservicio-personalizado-01"
          }
    }
```

En este ejemplo de una aplicación que llamamos my-custom- podemos microservicio-01 , identificar inmediatamente varios problemas.

La operación tardó 2,5 segundos ("milis": 2543), realizó una escaneo de colección ("planSummary": "COLLSCAN"), y lo examinó 1000000 documentos (docsExamined) para devolver solo 53 documentos (ndevuelto).

Estos detalles del generador de perfiles muestran que la consulta está escaneando todo el sitio.

colección en lugar de utilizar un índice, lo cual es altamente ineficiente,

Especialmente considerando el filtro de "estado" y "fecha de pedido" . Esto

El patrón indica claramente que no hay un índice adecuado en estos campos.

Para mejorar el rendimiento, debe crear un índice compuesto en "orderDate" en "estado" , "nombredelcliente" , ese orden, lo que permite

MongoDB para localizar eficientemente documentos coincidentes sin escanearlos Toda la colección.

Mejores prácticas para MongoDB Profiler en producción



Habilitación del generador de perfiles de base de datos MongoDB en el nivel 2 (perfilando todas las operaciones) en entornos de producción puede introducir una sobrecarga de rendimiento significativa. En

En la mayoría de los escenarios de producción, la mejor práctica es utilizar

Perfil de nivel 1, configure sampleRate o habilite la configuración completa

elaboración de perfiles únicamente durante períodos cortos y específicos durante

Solución de problemas. Siempre monitoree el impacto del sistema cuando

Ajustar la configuración del generador de perfiles en entornos en vivo.

Utilice los datos del generador de perfiles para identificar consultas ineficientes y detectar información faltante.

índices y guían el refinamiento del esquema. Al analizar patrones en

Operaciones lentas: puede realizar optimizaciones específicas que mejoren significativamente el rendimiento de la aplicación.

Integración con sistemas de monitorización externos

Si bien MongoDB ofrece potentes herramientas integradas, como los comandos `mongo` y `mongoostat`, y MongoDB Atlas y `mongotop` ofrecen una suite integral de monitoreo administrado, los entornos de aplicaciones modernos rara vez operan de forma aislada. Para lograr una verdadera observabilidad de extremo a extremo, es crucial integrar las señales de rendimiento de MongoDB en plataformas externas de monitoreo y observabilidad más amplias.

Esta integración le permite hacer lo siguiente:

- Correlacionar el comportamiento de la base de datos con el rendimiento de la aplicación y el estado de la infraestructura
- Cree paneles unificados que abarquen toda su pila tecnológica
- Implementar alertas consistentes en todos los componentes
- Obtenga contexto histórico para el análisis del rendimiento
- Apoyar el análisis de causa raíz a través de los límites del sistema

En esencia, al introducir los datos de rendimiento de MongoDB en las herramientas de observación más amplias de su organización, obtiene una visión más integral, lo que facilita la conexión del rendimiento de la base de datos con el estado general de la aplicación, la gestión centralizada de alertas y la realización de análisis exhaustivos de causa raíz en toda su pila de tecnología.

Prometeo + Grafana

Prometheus y Grafana forman una popular solución de monitorización de código abierto. Prometheus recopila y almacena métricas, mientras que Grafana las visualiza mediante paneles personalizables.

MongoDB Atlas proporciona integración nativa con esta pila, lo que le permite monitorear sus implementaciones de MongoDB junto con otros componentes de infraestructura.



Nota

La integración de Prometheus solo está disponible en clústeres M10+.

Integración de Atlas con Prometheus. MongoDB Atlas

ofrece una integración sencilla con Prometheus que elimina la necesidad de exportadores externos. Aquí te explicamos cómo configurarlo.

arriba:

- Configurar la autenticación : en la interfaz de usuario de Atlas, navegue a la pestaña Integraciones y configure las credenciales de Prometheus diseñadas específicamente para la recopilación de métricas.
- Elija el método de descubrimiento : seleccione Descubrimiento de servicio HTTP (recomendado) o Descubrimiento de servicio de archivos , según su infraestructura y preferencia.
- Actualizar la configuración de Prometheus : agregue la configuración de raspado adecuada a su servidor Prometheus.

En resumen, la integración de MongoDB Atlas con Prometheus implica tres pasos clave:

1. Configure las credenciales de autenticación dentro de la interfaz de usuario de Atlas.
2. Seleccione un método de descubrimiento de servicio (HTTP o basado en archivos).
3. Actualice la configuración de su servidor Prometheus para incluir la configuración de raspado necesarias.

A continuación se muestra un ejemplo de cómo podría verse la configuración de scrape en su archivo `prometheus.yml` :

```
scrape_configs: -  
  job_name: "<insertar-nombre-de-trabajo>"  
  scrape_interval: 10s  
  metrics_path: /metrics scheme:  
  https basic_auth:  
  
  nombre_de_usuario: <nombre-de-usuario-de-  
    prometheus> contraseña: <contraseña- de-prometheus>  
  http_sd_configs: - url:  
    <url-de-la-configuración-de-descubrimiento-de-servicio>  
    intervalo_de_actualización: 60  
    s  
    autenticación_básica: nombre_de_usuario:  
      <nombre_de_usuario-de-prometheus> contraseña: <contraseña_de-prometheus>
```

Una vez configurado, Prometheus detectará y extraerá automáticamente diversas métricas de todos sus clústeres Atlas. Estas métricas incluyen tasas de operación (como consultas, inserciones, actualizaciones y eliminaciones), utilización de recursos (como CPU, memoria y conexiones), métricas de replicación (incluidos el retardo y la ventana de registro de operaciones) y métricas de almacenamiento (como el uso del disco y las IOPS).

Visualizando con Grafana

Para completar su solución de monitoreo y obtener información más profunda sobre el rendimiento de su implementación, puede importar paneles de MongoDB prediseñados en Grafana:

1. En Grafana, navegue hasta la pantalla Importar (haga clic en el ícono + y luego seleccione Importar).
2. Sube un archivo JSON de ejemplo del panel (p. ej., `mongo-metrics.json` o `hardware-metrics.json`). Normalmente se encuentran en la interfaz de usuario de Atlas o en la documentación oficial.
3. Asegúrese de que el panel importado esté configurado para usar su Fuente de datos de Prometeo.
4. Comience a monitorear su implementación de MongoDB utilizando estas visualizaciones especialmente diseñadas.



Figura 14.1: Ejemplo de panel de métricas de MongoDB en Grafana

Para obtener instrucciones detalladas paso a paso y acceder a los archivos JSON del panel de muestra, consulte la documentación oficial de MongoDB Atlas en

<https://www.mongodb.com/docs/atlas/tutorial/prometheus-integration/#import-a-sample-grafana-dashboard> .

Esta integración funciona con MongoDB Atlas, así como con implementaciones de Cloud Manager y Ops Manager, y proporciona capacidades de monitoreo consistentes ya sea que su infraestructura MongoDB esté completamente administrada o autohospedada.

Perro de datos

Datadog es una plataforma SaaS comercial integral de observabilidad que ofrece monitorización de infraestructura, SAPM, gestión de registros y mucho más.

MongoDB Atlas ofrece una integración de primera clase con Datadog que permite una monitorización fluida de las implementaciones de bases de datos.

La integración de MongoDB Atlas con Datadog le permite monitorear sus clústeres Atlas directamente dentro de sus paneles de Datadog, brindando una vista unificada del rendimiento de su base de datos junto con otras métricas de aplicaciones e infraestructura.

Las características clave de esta integración incluyen las siguientes:

- Integración nativa : Configure la conexión entre Atlas y Datadog con solo unos clics en la interfaz de usuario de Atlas
- Recopilación automática de métricas : una vez conectado, Atlas envía automáticamente métricas de rendimiento a su cuenta Datadog
- Paneles prediseñados : la integración incluye paneles preconfigurados que muestran métricas clave de rendimiento de MongoDB
- Alertas unificadas : cree alertas en Datadog basadas en métricas de Atlas que se integren con sus flujos de trabajo de notificación existentes

- Correlación entre plataformas : correlacione fácilmente los problemas de rendimiento de la base de datos con las métricas de la aplicación y la infraestructura



Nota

La integración con Datadog solo está disponible en clústeres M10+.
Necesita una cuenta de Datadog y una clave API.

Para configurar la integración, siga estos pasos:

1. Asegúrese de tener acceso de propietario del proyecto en Atlas.
2. En Atlas, navegue hasta su proyecto y luego haga clic en Integraciones .
3. Haga clic en el mosaico Datadog .
4. Ingrese su clave API de Datadog. Puede encontrarla o crearla en su Cuenta Datadog en Integraciones > API .
5. Configure qué clústeres y métricas específicas desea que Datadog Para monitorear.
6. Opcionalmente, para realizar un seguimiento de las métricas de alta cardinalidad, habilite `sendDatabaseMetrics` y `sendCollectionLatencyMetrics` .
Esto se puede hacer a través de la interfaz de usuario de Atlas o de la administración de Atlas API.
7. Una vez configurado, puede acceder a su panel de control y métricas de MongoDB Atlas dentro de Datadog.

Para obtener pasos e información más detallados, consulte la documentación oficial de MongoDB Atlas sobre la integración de Datadog en <https://www.mongodb.com/docs/atlas/tutorial/datadog-integration/#procedure> .

Métricas del proyecto Atlas enviadas automáticamente a Datadog



Si configura su proyecto Atlas para enviar alertas y eventos a Datadog, no necesita seguir este procedimiento por separado. Atlas envía las métricas del proyecto a Datadog mediante la misma integración que se utiliza para alertas y eventos.

Una vez completada la integración, puede empezar a explorar los paneles prediseñados de Datadog para obtener una visión general del rendimiento de su clúster Atlas.

Considere configurar alertas para métricas clave como el uso de la CPU, el espacio en disco y el retraso de replicación para recibir notificaciones sobre posibles problemas. También puede empezar a correlacionar las métricas de MongoDB con los datos de su aplicación e infraestructura para obtener una visión integral de su sistema. En las siguientes secciones, profundizaremos en las estrategias de monitorización del rendimiento y las mejores prácticas para las alertas.

Plataformas de monitorización del rendimiento de aplicaciones

Plataformas como New Relic , AppDynamics y Dynatrace se especializan en la monitorización del rendimiento de aplicaciones (APM). Su principal fortaleza reside en el seguimiento integral de las solicitudes de las aplicaciones, lo que proporciona una visibilidad completa de cómo el código de la aplicación interactúa con la base de datos.

Los agentes APM suelen integrarse en el código de la aplicación mediante bibliotecas específicas del lenguaje. Cuando la aplicación realiza una llamada a la base de datos, el agente APM realiza lo siguiente:

1. Intercepta la llamada

2. Mide su duración
3. Captura el contexto relevante (como la propia consulta, a menudo normalizada)
4. Correlaciona esto con el contexto más amplio de la solicitud.

Esta integración permite que las plataformas APM hagan lo siguiente:

- Rastrear solicitudes individuales desde el navegador del usuario o el cliente API a través de múltiples servicios hasta consultas de bases de datos específicas
- Identificar llamadas lentas a la base de datos dentro del contexto de una transacción de usuario completa
- Correlacionar errores de aplicación o picos de latencia con operaciones de base de datos específicas
- Proporcionar vistas agregadas que muestren cómo el rendimiento de la base de datos afecta el rendimiento general de la aplicación

En última instancia, las plataformas APM proporcionan una lente poderosa para entender cómo las interacciones de la base de datos encajan en el panorama más amplio del comportamiento de la aplicación, lo que facilita la identificación de cuellos de botella relacionados con la base de datos que afectan la experiencia del usuario final.

OpenTelemetry

OpenTelemetry es un marco de observabilidad de código abierto que ofrece API, bibliotecas e instrumentación independientes del proveedor para capturar rastros, métricas y registros distribuidos. Representa el futuro de la instrumentación de observabilidad, con una creciente adopción en la industria.

OpenTelemetry ofrece bibliotecas de instrumentación para lenguajes de programación populares y controladores MongoDB. Al incorporar...

estas bibliotecas en su aplicación, las llamadas del controlador MongoDB son capturados automáticamente como "spans" dentro de trazas distribuidas. Cada span registra lo siguiente:

- La duración de la operación de la base de datos
- El tipo de operación (consulta, actualización, inserción, etc.)
- La colección o base de datos de destino
- Cualquier error que haya ocurrido

Estos intervalos proporcionan un registro detallado de cada interacción con la base de datos, Capturar qué operación se realizó, cuánto tiempo tomó, dónde se realizó. se ejecutó y si tuvo éxito. Estos datos granulares son cruciales para comprender el rendimiento de la base de datos y diagnosticar problemas.

Para una aplicación Node.js, implementar OpenTelemetry

La instrumentación para MongoDB podría verse así:

```
// rastreo.js
const opentelemetry = require('@opentelemetry/api');
const { NodeTracerProvider } = require('@opentelemetry/sdk
const { MongoDBInstrumentation } = require('@opentelemetry
const { registerInstrumentations } = require('@opentelemet
// Configurar el proveedor del rastreador
const proveedor = nuevo NodeTracerProvider();
proveedor.register();
// Registrar la instrumentación de MongoDB
registrarInstrumentaciones({
  instrumentaciones: [nueva MongoDBInstrumentation()],
});
// Exporta rastros usando tu exportador preferido
// (OTLP, Jaeger, Zipkin, etc.)
```

Con esta instrumentación en funcionamiento, las operaciones de MongoDB son capturado automáticamente y se puede enviar a cualquier OpenTelemetry-

backend compatible.

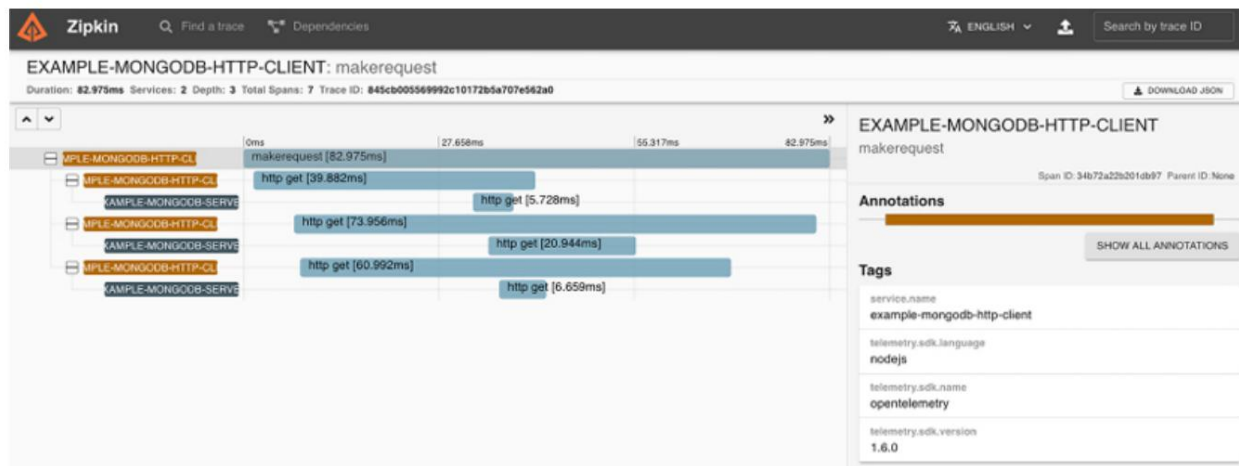


Figura 14.2: Visualización de los intervalos de operaciones de MongoDB capturados por OpenTelemetry

La captura de pantalla [anterior es del ejemplo de instrumentación](#) .

[OpenTelemetry MongoDB disponible en https://github.com/open-telemetry/opentelemetry-js-contrib/tree/main/examples/mongodb](https://github.com/open-telemetry/opentelemetry-js-contrib/tree/main/examples/mongodb) .

Esta instrumentación crea intervalos detallados para cada operación de MongoDB y captura información crucial como el tipo de operación, los nombres de las bases de datos y colecciones, y el tiempo de ejecución, lo que permite un monitoreo preciso del rendimiento y la resolución de problemas en toda la pila de aplicaciones.

Consideraciones para herramientas externas

Al integrar MongoDB con sistemas de monitorización externos, se deben considerar varios factores importantes para garantizar una monitorización eficaz y segura. Proteger el acceso a las instancias de MongoDB y a los datos que fluyen a los sistemas de monitorización es fundamental.

Implica implementar mecanismos de autenticación robustos, cifrar los datos en tránsito y adherirse al principio de mínimo privilegio para los usuarios de monitoreo. Prestar atención a estos aspectos de seguridad ayudará a proteger sus datos y a mantener la integridad de su configuración de monitoreo.

Las prácticas de seguridad clave incluyen las siguientes:

- Usuarios de monitoreo dedicados : cree usuarios de MongoDB específicamente para herramientas de monitoreo con los privilegios mínimos requeridos (normalmente, el rol `clusterMonitor`)
- Autenticación fuerte : utilice contraseñas fuertes y considere la autenticación basada en certificados, cuando sea compatible.
- TLS/SSL : Cifrar la comunicación entre las herramientas de monitorización y MongoDB
- Seguridad de la red : Restrinja el acceso de la herramienta de monitoreo a MongoDB mediante reglas de firewall adecuadas

Más allá de la seguridad, otro aspecto crítico de la integración de herramientas de monitorización externa es la gestión de su consumo de recursos.

Los sistemas de monitorización, por su naturaleza, interactúan con su implementación de MongoDB para recopilar datos. Esta interacción, si no se configura correctamente, puede suponer una carga adicional para sus servidores de bases de datos, lo que podría afectar su rendimiento. Por lo tanto, es fundamental tener en cuenta cómo estas herramientas consumen recursos para garantizar su correcto funcionamiento.

El monitoreo no degrada el rendimiento del sistema que estás intentando observar.

Las consideraciones clave para gestionar el consumo de recursos incluyen las siguientes:

- Frecuencia de sondeo/scraping : configure intervalos adecuados para la recopilación de métricas (normalmente entre 10 y 60 segundos) para lograr un equilibrio entre la puntualidad de los datos y la minimización de la sobrecarga.
- Sobrecarga de recursos : supervise el consumo de recursos de los propios agentes de monitoreo para asegurarse de que no afecten
Rendimiento de MongoDB
- Tamaño de la carga útil : tenga en cuenta el volumen de datos que se recopilan, especialmente de comandos como `serverStatus` que devuelven documentos grandes
- Punto de referencia en la puesta en escena : Pruebe las configuraciones de monitoreo en entornos que no sean de producción antes de implementarlas en producción

En resumen, la integración de herramientas de monitorización externa con MongoDB requiere un enfoque equilibrado, priorizando tanto la seguridad como el rendimiento.

Al proteger el acceso y gestionar cuidadosamente el uso de los recursos, puede garantizar una monitorización fiable sin comprometer la estabilidad ni la integridad de su implementación.

Patrones de rendimiento comunes y qué monitorear

Comprender los cuellos de botella de rendimiento comunes en MongoDB y saber qué métricas los detectan es crucial para una gestión proactiva del rendimiento. Al supervisar los indicadores clave, puede detectar y diagnosticar problemas antes de que afecten significativamente la capacidad de respuesta y la fiabilidad de su aplicación. Esta sección explora los desafíos de rendimiento recurrentes y las señales esenciales que debe monitorizar.

Cuellos de botella de E/S de disco

El subsistema de almacenamiento suele ser un factor limitante para el rendimiento de la base de datos, especialmente con cargas de trabajo intensas de lectura o escritura. Cuando el disco no puede satisfacer la demanda de MongoDB, las operaciones se ralentizan mientras esperan que se lean o escriban los datos en el disco.

Para identificar y diagnosticar cuellos de botella de E/S de disco, es fundamental monitorear varios indicadores clave:

- **Latencia de disco** : Mide el tiempo que tardan las operaciones de lectura y escritura en completarse a nivel de disco. Una latencia constantemente alta (normalmente superior a unos pocos milisegundos, según el medio de almacenamiento) es un indicador principal de un cuello de botella de E/S.

Esto a menudo se monitorea a través de herramientas a nivel del sistema operativo o métricas del proveedor de la nube, y Atlas muestra la latencia del disco en sus vistas de métricas.
- **Profundidad de la cola de E/S** : Monitorea el número de operaciones de E/S pendientes que esperan ser atendidas por el disco. Una profundidad de cola persistentemente alta indica que el sistema de almacenamiento está saturado.
- **Rendimiento del disco (bytes/seg) e IOPS** : supervisa la velocidad de transferencia de datos real y la cantidad de operaciones de E/S por segundo.

Compare estos datos con los límites previstos de su hardware de almacenamiento. Alcanzar estos límites confirma un cuello de botella de E/S.

Para abordar los cuellos de botella de E/S de disco se puede optimizar las consultas para reducir el acceso al disco, mejorar la indexación, aumentar el rendimiento/las IOPS aprovisionadas, actualizar el hardware de almacenamiento o garantizar que el conjunto de trabajo quepa en la RAM para minimizar las lecturas del disco.

Presión de caché (utilización de caché de WiredTiger)

El motor de almacenamiento WiredTiger de MongoDB utiliza una caché interna para almacenar en memoria los datos e índices de acceso frecuente, minimizando así la necesidad de un acceso lento al disco. La presión de caché se produce cuando el conjunto de trabajo activo (los datos e índices requeridos por la carga de trabajo actual) excede el tamaño de la caché WiredTiger configurada. Esto provoca un aumento de la actividad de desalojo de caché y fallos de página, lo que reduce el rendimiento.

Varias métricas clave pueden ayudarle a detectar y diagnosticar la presión de caché de WiredTiger en MongoDB. Monitorear estos indicadores le proporciona señales tempranas de cuellos de botella en el rendimiento causados por limitaciones de memoria:

- Utilización de caché : Monitorea la cantidad de caché de WiredTiger actualmente en uso (`serverStatus().wiredTiger.cache["bytes actualmente en la caché"]`). Si bien una alta utilización suele ser buena, alcanzar el límite máximo constantemente indica presión.
- Tasa de expulsión de caché : Supervise la velocidad a la que se expulsan las páginas de la caché para liberar espacio para nuevos datos (`serverStatus().wiredTiger.cache["unmodified pages evicted"]`). Una tasa de expulsión alta o un rápido aumento indica que la caché se está renovando con frecuencia. Atlas suele visualizar la actividad de la caché, destacando las expulsiones.
- Páginas leídas en caché : rastrea la velocidad a la que se leen los datos desde el disco hacia la caché (`serverStatus().wiredTiger.cache["pages read into cache"]`)

). Las tasas altas a menudo se correlacionan con tasas altas de desalojo e indican que con frecuencia los datos requeridos no se encuentran en la memoria.

- Errores de página : si bien no están vinculados únicamente con la caché de WiredTiger (sistema operativo)

También pueden ocurrir errores de página), errores de página frecuentes (

`serverStatus().extra_info.page_faults` o métricas del sistema operativo) a menudo

acompañar la presión de caché, lo que indica que los procesos de MongoDB

Están solicitando datos que no se encuentran actualmente en la RAM física.

La monitorización de estas métricas proporciona una imagen clara del estado de la memoria caché.

La alta utilización no es inherentemente mala, pero cuando se combina con una alta tasas de desalojo, aumento de lecturas de páginas desde el disco y fallos de página,

Indica que la caché tiene dificultades para seguir el ritmo de la carga de trabajo.

forzando lecturas frecuentes desde un almacenamiento de disco más lento.

El siguiente comando `db.serverStatus()` se puede utilizar en el

MongoDB Shell para recuperar algunas de estas métricas de caché clave:

```
// Correr desde el Shell de MongoDB
// Consulte las métricas de uso de caché y desalojo de WiredTiger
constante wtCache = db.serverStatus().wiredTiger.cache;
const bytesInCache = wtCache["bytes actualmente en la caché"]
const maxBytes = wtCache["máximo de bytes configurados"];
const evictedPages = wtCache["páginas no modificadas expulsadas"];
print(`Utilización de caché: ${bytesInCache/maxBytes*100}.to
print(`Páginas expulsadas: ${evictedPages}`);
```

La resolución de la saturación de caché generalmente implica aumentar la memoria caché disponible.

RAM y el tamaño de caché de WiredTiger configurado, optimizando las consultas

e índices para reducir el tamaño del conjunto de trabajo o escalar el clúster

horizontalmente.

Retraso de replicación y desviación de la ventana del registro de operaciones El retraso de replicación es una métrica común que puede querer monitorear, ya que un retraso excesivo puede afectar la consistencia de la lectura (si se lee desde secundarios), aumentar el tiempo de conmutación por error y poner en riesgo la durabilidad de los datos.

La salud del conjunto de réplicas se puede rastrear monitoreando la siguiente clave métrica:

- Retraso de replicación : Monitoree el retraso en segundos de cada nodo secundario. Esta métrica importante está disponible en la salida de `rs.status()` y se muestra claramente en la interfaz de usuario de Atlas. Se deben configurar alertas cuando el retraso supere los umbrales aceptables.
- Ventana de registro de operaciones : Monitorea la ventana de tiempo estimada que cubre el registro de operaciones (`rs.printReplicationInfo()` o métricas de Atlas). Una ventana de registro de operaciones cada vez más corta aumenta el riesgo de que las instancias secundarias deban resincronizarse si se retrasan.
- Tasa de registros de operaciones (GB/hora) : Supervise la velocidad a la que se genera el registro de operaciones en el servidor principal. Una tasa alta puede contribuir a un retraso en la replicación si los servidores secundarios no pueden aplicar escrituras con la suficiente rapidez o si el ancho de banda de la red es insuficiente. Atlas proporciona métricas para el rendimiento del registro de operaciones.

En conjunto, estas métricas proporcionan una visión integral del estado de la replicación.

Monitorear el retardo garantiza la consistencia de los datos y la preparación para la conmutación por error; el seguimiento de la ventana de registros de operaciones ayuda a evitar costosas resincronizaciones completas de los secundarios; y observar la tasa de generación de registros de operaciones permite identificar si la carga de escritura del primario supera la capacidad de replicación.

Puede utilizar los siguientes comandos en MongoDB Shell para recuperar el estado de replicación actual y la información del registro de operaciones:

```
// desde el Shell de MongoDB
// Verificar el retraso de replicación en todos los nodos secundarios
const rsStatus = rs.status();
const primaria = rsStatus.members.find(m => m.state === 1)
rsStatus.members.forEach(miembro => {
  si (miembro.estado === 2) { // secundaria
    const lagSeconds = Math.abs((primaria.optimeDate.getTime()
                                - miembro.optimeDate.getTime()) / 1000);
    print(`Retraso secundario ${member.name} : ${lagSeconds.toFixed(2)} s`);
  }
});
// Comprobar el tamaño de la ventana del registro de operaciones
const oplogInfo = db.getReplicationInfo();
print(`Ventana de registro de operaciones: ${oplogInfo.timeDiff / 3600}h`);
```

Este script utiliza varios comandos y conceptos clave. `rs.status()`

Obtiene un documento detallado para cada miembro del conjunto de réplicas, incluido estado y `optimeDate` (la marca de tiempo de la última operación aplicada).

A continuación, el script compara el valor de `optimeDate` del valor principal con el de cada secundario para calcular el retraso de replicación en segundos.

`db.getReplicationInfo()` se utiliza para recuperar información sobre

oplog, específicamente la duración de , que indica la ventana de registro de operaciones

`timeDiff`. Finalmente, el script imprime el retardo calculado para cada

secundaria y la ventana de registro de operaciones en horas, lo que proporciona una visión clara instantánea de la salud de la replicación.

La ejecución de este script podría producir un resultado similar al siguiente:

```
Retraso secundario mongo-secondary-0.example.com:27017: 2,35 s
Retraso secundario mongo-secondary-1.example.com:27017: 1,80 s
```

Ventana de registro de operaciones: 48,25 h

Esta salida permite a los administradores evaluar rápidamente si los secundarios están al día con el principal y si la ventana de registro de operaciones es suficiente para las necesidades operativas.

Las causas del retraso en la replicación incluyen un alto volumen de escritura en el servidor primario, recursos insuficientes (CPU, RAM o E/S) en los servidores secundarios, latencia de red entre nodos u operaciones de larga duración en los servidores secundarios que bloquean la replicación.

Resumen

La monitorización y la observabilidad eficaces de MongoDB transforman los datos sin procesar en información práctica que mantiene sus implementaciones en buen estado, con un buen rendimiento y fiables. A lo largo de este capítulo, hemos explorado la distinción fundamental entre la monitorización (el seguimiento de métricas conocidas para detectar posibles fallos) y la observabilidad (el uso de métricas, registros y seguimientos para comprender por qué ocurre algo).

Hemos examinado diversas herramientas disponibles, desde la completa suite que ofrece MongoDB Atlas, que incluye el Panel de Rendimiento en Tiempo Real, exploradores de métricas y el Asesor de Rendimiento para recomendaciones de optimización automáticas, hasta herramientas autogestionadas como mongostat , mongotop y el generador de perfiles de bases de datos para obtener información detallada sobre implementaciones autoalojadas. Además, sistemas externos como Prometheus+Grafana, Datadog y las plataformas APM amplían estas capacidades, lo que permite una observabilidad unificada de toda la pila tecnológica.

Al implementar un flujo de trabajo de observabilidad sólido y combinar paneles en tiempo real, alertas escalonadas, registros detallados, planes de ejecución de consultas y seguimientos distribuidos, puede detectar problemas emergentes antes de que afecten a los usuarios, diagnosticar las causas raíz de manera eficiente y tomar decisiones basadas en datos para optimizar el rendimiento.

Recuerde que los equipos más exitosos no solo recopilan datos; establecen una cultura donde la observabilidad impulsa la optimización continua.

Con los principios y prácticas descritos en este capítulo, estará bien posicionado para mantener un rendimiento óptimo de MongoDB, incluso a medida que sus aplicaciones escalan y evolucionan.

Después de haber establecido una base sólida en el monitoreo y la observación de su entorno MongoDB, el próximo capítulo cambiará el enfoque a otro aspecto crítico del rendimiento de la base de datos: cómo optimizar y ajustar de manera proactiva su implementación de MongoDB para prevenir problemas antes de que surjan.

15

Depuración de problemas de rendimiento

Los problemas de rendimiento en las bases de datos pueden suponer un reto incluso para los desarrolladores y administradores de bases de datos más experimentados. Estos problemas suelen surgir de forma inesperada, provocando ralentizaciones en las aplicaciones, quejas de los usuarios y, en ocasiones, incluso interrupciones. El difunto experto en MongoDB, William Zola, identificó tres razones fundamentales para la lentitud.

Rendimiento de MongoDB: operaciones inherentemente lentas que consumen recursos naturaleza; operaciones bloqueadas que esperan a que se completen significativos por otras operaciones lentas; y recursos de hardware insuficientes o mal ajustados que carecen de la capacidad para manejar la carga de trabajo.

Aunque MongoDB ha evolucionado significativamente desde que Zola formuló estos principios, estos siguen siendo notablemente relevantes. La mayoría de los problemas de rendimiento aún se pueden atribuir a una de estas tres causas fundamentales, siendo un diseño deficiente del esquema el factor subyacente.

Este capítulo proporciona un enfoque estructurado para identificar, comprender y resolver problemas de rendimiento de MongoDB. Exploraremos ejemplos reales que ilustran problemas comunes y sus soluciones, basándonos en las técnicas de capítulos anteriores. Al finalizar este capítulo, contará con una base sistemática.

Metodología para abordar problemas de rendimiento en sus propias implementaciones de MongoDB.

A través de estudios de casos prácticos, en este capítulo aprenderás a hacer lo siguiente:

- Identificar los síntomas de los diferentes tipos de problemas de rendimiento.
- Diagnosticar las causas raíz utilizando las herramientas de monitorización de MongoDB
- Implementar soluciones específicas para resolver los problemas
- Prevenir que problemas similares ocurran en el futuro

Identificación de operaciones inherentemente lentas

Las operaciones inherentemente lentas son aquellas que requieren que MongoDB realice un trabajo significativo, que a menudo implica la lectura de grandes cantidades de datos o la realización de cálculos complejos. Estas operaciones se vuelven especialmente problemáticas a medida que crece el volumen de datos.

Las operaciones inherentemente lentas más comunes incluyen las siguientes:

- Escaneos de colecciones (consultas sin índices)
- Consultas que examinan muchos más documentos de los que devuelven
- Tuberías de agregación complejas
- Consultas que devuelven grandes conjuntos de resultados
- Operaciones que provocan la creación de índices en colecciones grandes

Cuando estas operaciones ocurren con frecuencia, pueden consumir recursos sustanciales del sistema y ralentizar el rendimiento general de la base de datos.

Estudio de caso: Solución de problemas de rendimiento del clúster con Atlas

MongoDB Atlas, nuestra plataforma para la gestión de bases de datos, se basa en clústeres internos para supervisar y administrar las implementaciones de los clientes.

Entre estos, un clúster clave se encarga de recopilar datos de servidores y configuración de todas las implementaciones de Atlas. Estos datos son vitales para comprender el estado, el rendimiento y el funcionamiento de las bases de datos de los clientes, lo que nos ayuda a identificar y solucionar problemas de forma proactiva.

Sin embargo, a medida que Atlas escalaba rápidamente, este clúster central de recopilación de datos comenzó a mostrar signos de tensión. A partir del monitoreo, observamos lo siguiente:

- La CPU del sistema normalizado en el fragmento 0 aumentó de un 45-55 % constante a alrededor del 70 %.
- Las tasas de segmentación de consultas comenzaron a aumentar
- La latencia también aumentó
- Los recuentos de operaciones de consulta aumentaron de aproximadamente 21.000 operaciones de base de datos por segundo a más de 30.000 en un período de cinco meses

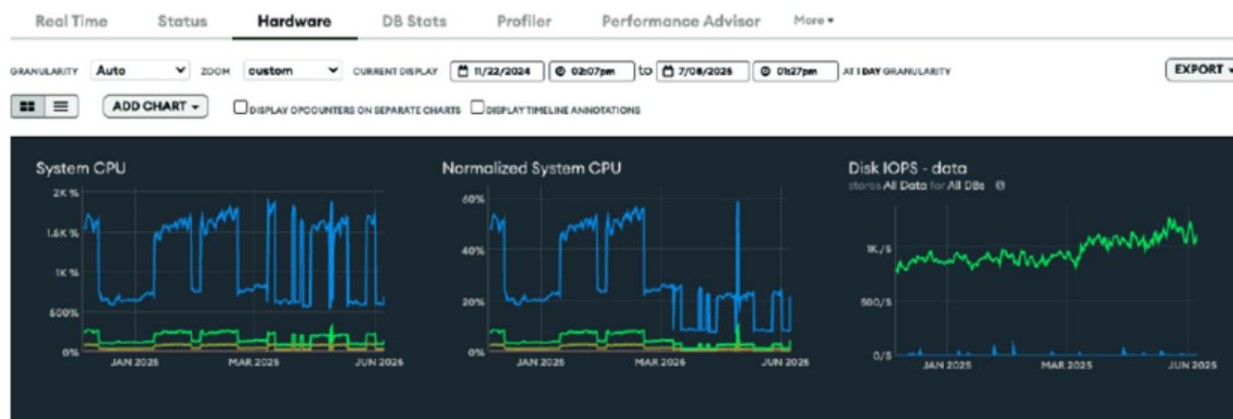


Figura 15.1: Panel de hardware del clúster que muestra la CPU del sistema, la CPU del sistema normalizada y las IOPS del disco

Para mantener el sistema funcionando, ampliamos el clúster a un nivel superior. Al mismo tiempo, iniciamos un proyecto para descubrir cómo podíamos ajustar el rendimiento para manejar la mayor carga de trabajo mientras permanecíamos en el nivel anterior.

Esta degradación gradual del rendimiento se manifestó en varias métricas clave que proporcionaron indicadores claros de problemas subyacentes. Si bien se espera un mayor uso de la CPU a medida que las aplicaciones crecen, en este caso, la tasa fue significativamente mayor de lo previsto para la carga de trabajo.

Las elevadas métricas de segmentación de consultas eran especialmente preocupantes, ya que indicaban la posibilidad de índices subóptimos, con consultas que procesaban muchos documentos para devolver solo unos pocos resultados. Estas ineficiencias suelen pasar desapercibidas en sistemas con exceso de aprovisionamiento, ocultando los problemas de diseño subyacentes. Sin embargo, a medida que estas operaciones de consulta subóptimas se acumulaban, acababan produciendo aumentos de latencia mensurables que ponían de manifiesto los problemas fundamentales. Resolver estos problemas de escalado se convirtió en una prioridad absoluta para garantizar la fiabilidad, el rendimiento y la monitorización exhaustiva de las implementaciones de los clientes en Atlas.

Diagnosticar la causa

El equipo de MongoDB inició una investigación utilizando el monitoreo de Atlas herramientas como se describe en [Capítulo 14](#) Monitoreo y observabilidad . Dado que se muestra en la Figura , hubo picos en los contadores operativos durante los períodos 15.2 cuando el servidor era principal, con caídas cuando cambió a

Secundario. El aumento en la segmentación de consultas a finales de marzo fue particularmente notable.

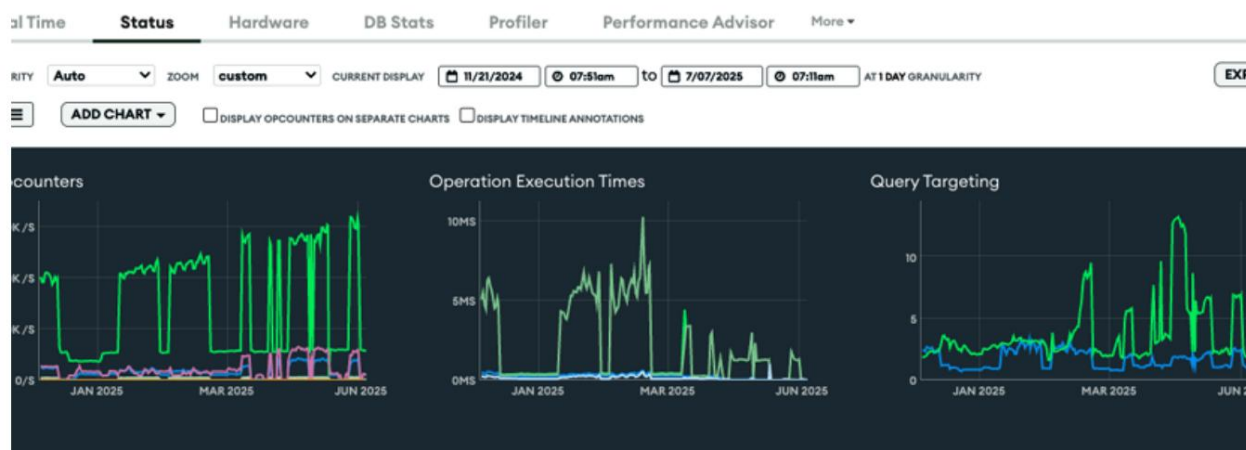


Figura 15.2: Panel de estado del clúster que muestra contadores de operaciones, tiempos de ejecución de operaciones y orientación de consultas

Para mitigar la presión constante sobre los recursos del clúster, el equipo actualizó temporalmente el nivel del clúster. Esto nos proporcionó el aumento inmediato de CPU que necesitábamos y, como era de esperar, la métrica normalizada de CPU del sistema disminuyó notablemente. Si bien esto solucionó el problema de rendimiento en aquel momento, fue solo una medida temporal. El objetivo principal del equipo seguía siendo identificar la causa subyacente e implementar una solución más rentable y a largo plazo para el alto uso de CPU, garantizando así la estabilidad del sistema y la eficiencia financiera.

Identificación de la causa raíz

Con nuestra solución temporal implementada, el equipo de ingeniería comenzó un análisis de causa raíz de los problemas de rendimiento del sistema.

Primero utilizaron los 19 índices potenciales del [Atlas Performance Advisor](#), que recomendó un claro indicador de que muchas consultas estaban utilizando

Índices subóptimos. Además, el análisis de registros reveló consultas lentas con marcas de tiempo directamente correlacionadas con períodos de alta utilización de la CPU . Esta combinación proporcionó evidencia sólida de que patrones de consulta específicos causaban la degradación del rendimiento, en particular aquellos que realizaban análisis completos de colecciones o agregaciones complejas sin un soporte de indexación adecuado.

Tras confirmar que algunas consultas probablemente causaban la degradación del rendimiento, Atlas Query Insights y Query Profiler fueron los pasos lógicos para comprender la situación. Query Insights y Query Profiler revelaron que ciertos tiempos de ejecución de consultas, tanto agregados como promedio, eran extremadamente largos , y algunas consultas tardaban hasta 8 segundos, como se muestra en la Figura 15.3 .

Esta información detallada por espacio de nombres permitió al equipo comprender con exactitud dónde se producían los cuellos de botella, lo que les permitió desarrollar estrategias de optimización específicas, como la creación de nuevos índices o la reescritura de consultas ineficientes. Este enfoque combinado proporcionó una visión clara de los problemas de rendimiento del sistema y condujo a una base de datos más eficiente.

La página Información de consultas presenta una lista que detalla la cantidad de operaciones y sus tiempos totales de ejecución para cada espacio de nombres.

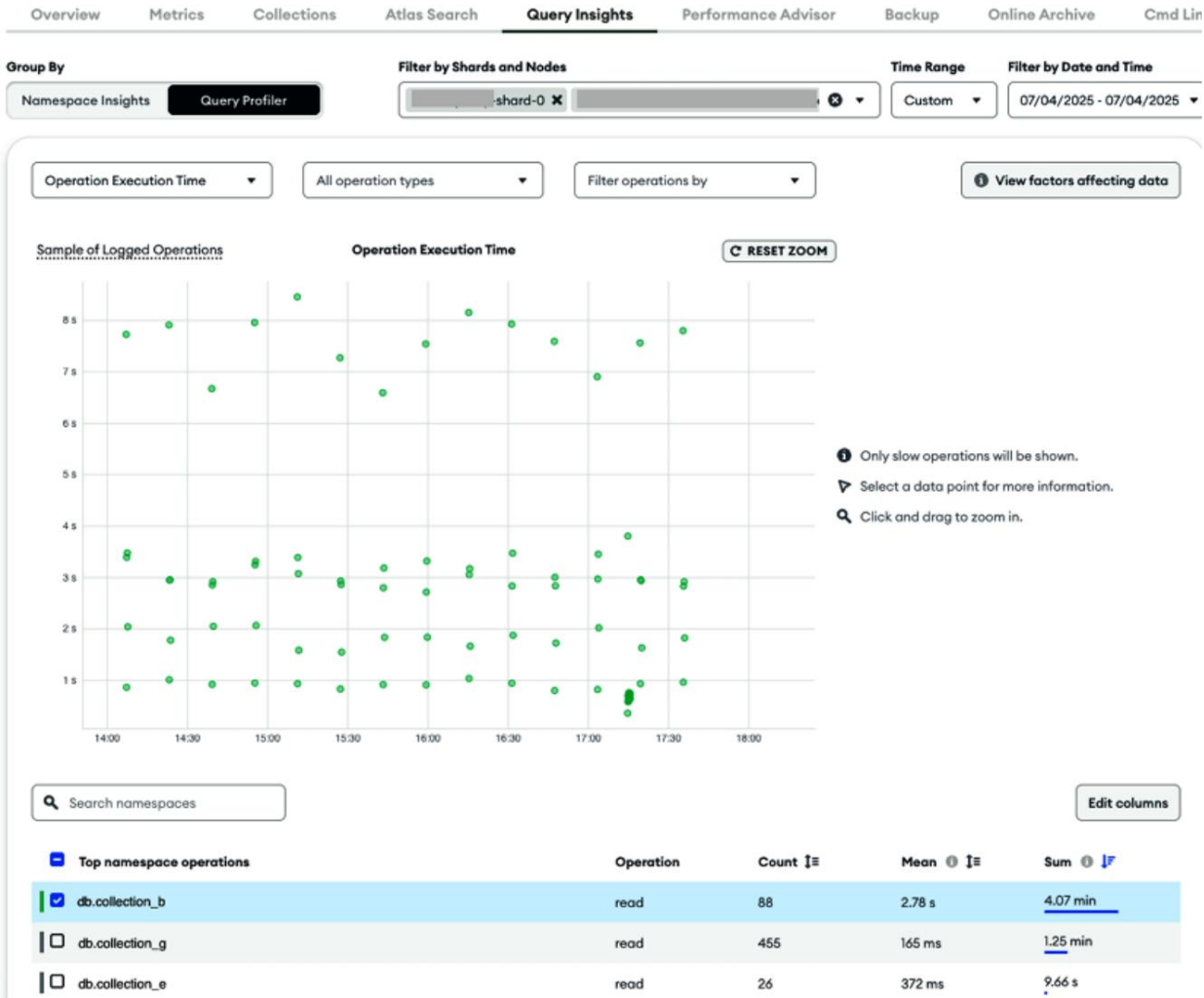


Figura 15.3: Panel de información de consultas de clúster que muestra consultas lentas

Mediante correlación, utilizando la misma técnica mencionada anteriormente, el equipo identificó que durante los períodos en que la base de datos procesaba aproximadamente 20.000 operaciones por segundo, los registros indicaban alrededor de 13.000 consultas lentas diariamente.

Al utilizar tanto los registros como el Analizador de Consultas, pudimos correlacionar las consultas lentas con períodos de alto uso de la CPU. Esta correlación nos ayudó a identificar las formas de consulta que probablemente contribuían más a nuestros problemas de rendimiento. Por ejemplo, la más

La forma de consulta lenta registrada con frecuencia, que ocurre aproximadamente 9.000 veces al día, apareció de la siguiente manera:

```
{
  "Buscar": "db.collection_g", "Filtro":
  { "Atributo1_Id":
    { "$oid":
      "621094e7bdbed43d3f408384" }, "Atributo1_Nombre":

      "aws-use1-1-v2-1", "Atributo2_Id": { "$oid":
        "65bd5e7ccff98a5722037dc6"

    } }, "límite": 1
}
"planSummary": "COLLSCAN", "planningTimeMicros": 259, "keysE
```

Este patrón de consulta era particularmente problemático. Analizaba más de 34 000 documentos y devolvía un solo documento coincidente, consumiendo una cantidad considerable de recursos de CPU. El equipo identificó que esta colección y consulta se habían añadido como nueva función a finales de marzo sin un índice correspondiente. Esto coincidió con el aumento repentino en la segmentación de consultas que habían observado.

Implementando la solución

Con una comprensión más clara de los cuellos de botella en el rendimiento, el equipo se centró en optimizaciones específicas y de bajo riesgo. Su objetivo era mejorar la eficiencia sin sobrecargar el sistema. recursos.

El equipo implementó varias soluciones específicas:

- Creación de índices recomendados : Se creó un índice para las consultas lentas más frecuentes, lo que mejoró inmediatamente el rendimiento y redujo la métrica de segmentación de consultas. Para evitar una mayor sobrecarga de los recursos de CPU, ya de por sí saturados, durante este período crítico, el equipo planificó cuidadosamente la creación del índice, programándola durante las horas de menor actividad, con monitorización continua de los recursos del sistema para evitar una mayor degradación del rendimiento.
- Configurar la monitorización preventiva : Crearon una alerta de segmentación de consultas desde la página Alertas del proyecto para monitorizar las altas tasas de objetos escaneados en relación con los documentos devueltos. Esto les avisaría cuando las cargas de trabajo, las aplicaciones o los conjuntos de datos cambiaran y redujeran la eficiencia de las consultas con el tiempo.

Create a New Alert ✕

- Select Category and Condition**

Category
Host

Type
of type Primary

Condition/Metric
targ

Severity
Default (Error)

Query Targeting: Scanned Objects / Returned is...
Query Targeting: Scanned / Returned is...
- Select Target**

Any Host

Hosts where...
- Add Notification Method**

Atlas Internal Project

Project:
☒ All Roles
17 current users are included in this selection.

Notification via
☒ Email ☐ SMS

Recurrence
Send if condition lasts at least 0 minutes.
Resend after 5 minutes.

Add Notifier

Save

Figura 15.4: Crear una nueva alerta para las consultas lentas

- Índices redundantes eliminados** : Al revisar las recomendaciones de Performance Advisor, el equipo identificó nueve índices en campos que eran prefijos de otro índice compuesto y que podrían eliminarse. Sin embargo, eliminar índices requiere una cuidadosa planificación. El equipo primero validó cada recomendación en un entorno de prueba, evaluó el posible impacto en las consultas e implementó los cambios gradualmente con una supervisión minuciosa. Solo después de una evaluación exhaustiva, procedieron a eliminar estos índices redundantes, ocultándolos inicialmente y confirmando posteriormente que no tenían efectos adversos en el...

Sistema y eliminarlos. Esto, en última instancia, mejoró el rendimiento de escritura sin afectar el de lectura.

Estas acciones, en conjunto, redujeron el uso de la CPU, mejoraron la eficiencia de las consultas y estabilizaron el rendimiento del sistema. Y lo que es más importante, sentaron las bases para la optimización continua y la monitorización proactiva en el futuro.

Resultados y aprendizajes

Este caso ilustra varios principios importantes sobre el diagnóstico de operaciones inherentemente lentas:

- Los problemas de rendimiento a menudo coinciden con cambios en la aplicación o nuevas funciones.
- Utilizando múltiples herramientas de diagnóstico como Performance Advisor, El generador de perfiles de consultas y los registros proporcionan una vista completa
- La creación de índices apropiados suele ser la solución más eficaz para los análisis de colecciones.
- Eliminar índices redundantes puede mejorar el rendimiento de escritura
- La configuración de alertas para métricas clave permite la identificación proactiva de problemas futuros

Finalmente, este riesgo se identificó inicialmente mediante el monitoreo de métricas clave a lo largo del tiempo. Esto nos ayudó a identificar la degradación gradual del rendimiento y a actuar antes de que se viera afectado por ello en los clientes. Este ejercicio de optimización del rendimiento nos permitió reducir los niveles del clúster y los costos.

Caso práctico: Una consulta administrativa inesperada provocó un escaneo de colección. Las consultas lentas suelen ser el primer indicador de problemas de rendimiento en las implementaciones de MongoDB. Un problema importante afectó las implementaciones autogestionadas de MongoDB de un cliente. La monitorización de sus aplicaciones mostró un aumento repentino en la latencia de las operaciones de la base de datos, lo que afectó a los usuarios de todo el sistema. La rapidez del problema obligó a investigarlo y solucionarlo de inmediato para evitar más interrupciones y que los servicios volvieran a la normalidad.

Diagnosticar la causa

Un ingeniero de atención al cliente contactó con el soporte de MongoDB. Este se dio cuenta de que la situación era crítica y, en primer lugar, buscó una solución inmediata pero segura para reducir la gravedad del impacto.

El soporte le pidió al cliente que ejecutara un comando de shell para mostrar las operaciones que se habían estado ejecutando durante 10 segundos o más:

```
db.currentOp({"segundos_en_ejecución":{"$gte": 10}}).inprog [
  { tipo: 'op', opid:
    201, host:
    'somewhere.net:27017', connectionId:
    1063017, cliente:
    '555.111.222.333:47140', currentOpTime:
    '2024-12-31T00:29:08.230+00:00', subprocesado: verdadero,
    segundos_en
    ejecución: Long('5410'),
    microsegundos_en ejecución: Long('5410001296'),
    op: 'comando',
```

```
ns: 'admin.$cmd',  
comando: {  
  /* Redactado */  
},  
"planSummary": "COLLSCAN",  
numYields: 0,  
bloqueos: {},  
}  
]
```

Esto reveló una operación que llevaba más de 90 minutos ejecutándose, según el campo `secs_running`. Una investigación más profunda reveló que un administrador de la base de datos había iniciado sesión y ejecutado una consulta compleja. El campo `host` de los registros reveló que esta consulta se ejecutó desde otra oficina, mientras que el campo `ns admin` confirmó que el usuario tenía permisos administrativos. Esta consulta inició un análisis de la colección, lo que provocó la eliminación de las colecciones almacenadas en caché.

Implementación de la solución. El cliente, ante una operación no crítica que consumía demasiados recursos y ponía en peligro la estabilidad del sistema, tomó medidas decisivas. Para recuperar el control y evitar una mayor degradación, inició una secuencia de terminación del proceso problemático. Para ello, utilizó el siguiente comando:

```
db.killOp(201)
```

En este caso, el equipo canceló la operación ad hoc problemática y el problema se resolvió. Durante el análisis de seguimiento, el equipo de soporte de MongoDB descubrió que la consulta compleja emitida por el administrador había...

provocó una expulsión masiva de caché. Esto ocurrió porque el análisis cargó cantidades masivas de documentos raramente utilizados en la memoria, expulsando los datos a los que se accede con frecuencia desde el espacio de caché limitado. Esto explica por qué el rendimiento seguía afectado incluso después de finalizar la consulta; el daño a la caché ya estaba causado. Tras aproximadamente 2-3 minutos, una vez recargados los datos de acceso frecuente (conjunto de trabajo) en la caché de MongoDB desde el almacenamiento en disco, las métricas de rendimiento de la base de datos volvieron a sus valores iniciales. Si bien este período de recuperación podría parecer una degradación adicional del rendimiento tras la intervención, es importante tener en cuenta que el sistema ya estaba bloqueado por la consulta problemática, por lo que esta breve fase de recuperación fue una consecuencia inevitable de abordar el problema de raíz.

Resultados y aprendizajes. Este caso destaca

varios principios importantes. En primer lugar, los análisis de recopilación de datos de larga duración pueden afectar drásticamente el rendimiento de todos los usuarios.

Además, la expulsión de datos almacenados en caché es un efecto secundario común de los análisis de colecciones grandes.

Las consultas ad hoc realizadas por administradores en bases de datos de producción representan un riesgo importante que requiere una gestión y gobernanza cuidadosas. Si bien estas investigaciones improvisadas pueden brindar información valiosa, también pueden afectar gravemente el rendimiento y la disponibilidad del sistema si se ejecutan sin considerar adecuadamente sus necesidades de recursos. Para evitar esto, las organizaciones deben implementar políticas estrictas sobre quién puede ejecutar consultas ad hoc en los sistemas de producción, cuándo pueden ejecutarse y qué limitaciones deben aplicarse.

Se prioriza su alcance y complejidad. Idealmente, este trabajo exploratorio debería realizarse en servidores secundarios o durante ventanas de mantenimiento, con tiempos de espera de consultas y límites de recursos configurados para evitar procesos descontrolados. Sin estas medidas de seguridad, incluso administradores de bases de datos bienintencionados pueden, sin darse cuenta, desencadenar problemas de rendimiento en cascada que afecten a miles de aplicaciones. usuarios.

Gestión de operaciones bloqueadas

Las operaciones bloqueadas pueden ocurrir cuando un proceso debe esperar a que se complete otra operación o a que un recurso esté disponible, lo que crea un efecto dominó que puede afectar gravemente la capacidad de respuesta de la aplicación.

Comprender las causas del bloqueo de las operaciones es esencial para mantener un rendimiento óptimo de la base de datos. Reconocer estos patrones a tiempo permite implementar medidas preventivas antes de que afecten la experiencia de los usuarios.

Las causas típicas de operaciones bloqueadas en MongoDB incluyen las siguientes:

- Operaciones de larga duración con bloqueos exclusivos, como creaciones de índices o entregas de colecciones
- Contención de recursos, donde múltiples operaciones intentan acceder a las mismas colecciones o documentos, especialmente durante las horas pico uso
- Agotamiento del grupo de conexiones, que ocurre cuando todas las conexiones disponibles están en uso y las nuevas operaciones deben esperar, lo que crea una cola artificial.



Síntomas similares a bloqueos por problemas no bloqueantes

Algunas condiciones, como fugas del cursor o alta latencia de red, pueden producir síntomas que se asemejan a un bloqueo.

Por ejemplo, alta latencia o consultas lentas, pero lo hacen.
no implica una contención de bloqueo real.

Estudio de caso: Investigación de fugas del cursor

Un cliente experimentó operaciones más lentas y un alto número de consultas que se agotaron. El problema afectó directamente a los usuarios, y algunos experimentaron tiempos de espera en las aplicaciones. El monitoreo de Atlas reveló un aumento significativo en el número de cursores abiertos, en concreto, más de 600 000 a partir de las 4:00 a. m. en la zona horaria local del cliente.



Seguimiento de cursores abiertos mediante `serverStatus`

Para las instancias de MongoDB autoadministradas, esta información también está disponible en el contador `cursor.open.total` del comando `serverStatus`.

Diagnóstico de la causa. El equipo de soporte de

MongoDB se centró inicialmente en optimizar las operaciones lentas y sugirió crear un índice, según lo recomendado por Performance Advisor, para acelerar las consultas. Sin embargo, esto no mejoró la situación. El elevado número de cursores abiertos, incluso después de la optimización del índice, sugería una causa subyacente más compleja.

Problema. Esto podría deberse a una lógica de aplicación ineficiente, a cursores que no se cerraban correctamente o a problemas de agrupación de conexiones.



Gestión automática del ciclo de vida del cursor en MongoDB

MongoDB gestiona automáticamente los ciclos de vida de los cursores en la mayoría de las operaciones normales. Los cursores se cierran cuando se completa su iteración tras el tiempo de espera (normalmente después de 10) o al finalizar una sesión de cliente. En aplicaciones que utilizan controladores y patrones estándar de MongoDB, las fugas de cursores son relativamente poco frecuentes.

La implementación del cliente continuó degradándose, y cada vez más usuarios experimentaban tiempos de respuesta lentos y errores. Indicadores clave del sistema, como CPU, memoria y E/S, mostraban un grave cuello de botella. Incluso después de una investigación exhaustiva por parte de nuestro equipo de soporte, la causa exacta no estaba clara, pero necesitábamos estabilizar el sistema rápidamente para evitar una interrupción. El equipo de soporte recomendó al cliente reiniciar el nodo principal. Esto activaría una conmutación por error programada, convirtiendo uno de los nodos secundarios en el nuevo principal, garantizando así la continuidad de los servicios. Tras la promoción exitosa del secundario, el rendimiento del sistema mejoró de inmediato, lo que confirmó que esta acción resolvió temporalmente los problemas operativos urgentes.

El equipo de soporte continuó monitoreando el sistema durante la noche. El número de cursores abiertos aumentaba, pero la actividad de los usuarios era menor y las latencias operativas aún no habían alcanzado un nivel crítico. Cuando el ingeniero de atención al cliente comenzó a trabajar a la mañana siguiente, el número de cursores abiertos había alcanzado los 260 000 y seguía aumentando.

El equipo de soporte y el cliente trabajaron juntos para diagnosticar el problema en tiempo real e identificaron varios indicadores clave:

- Análisis del generador de perfiles : el cliente habilitó el generador de perfiles de base de datos durante 40 minutos para capturar consultas que demoraban más de 20 ms.
- Análisis de registros : Al analizar los registros, el ingeniero de soporte observó que el número de cursores abiertos se disparó a 600 000 y luego descendió casi a cero. A partir del estado del servidor, se observó lo siguiente: un
 - gran número de `cursor.lifespan.greaterThanOrEqualTo10Minutes` ; más de 500
 - `cursor.totalOpened` por segundo, pero menos de 250 `cursor.moreThanOneBatch`.
 - Más de 300.000 mensajes de registro con `{"ctx":"LogicalSessionCacheRefresh", "msg": "Eliminar el cursor como parte de eliminar sesiones"}`
- Análisis de patrones de consulta : utilizando comandos `grep` , analizaron los patrones del archivo de registro:

```
grep 'aCollection' mongodb.log* | grep -v 'obtenerMás' |  
122784  
grep 'aCollection' mongodb.log* | grep '"cursorEscape"  
46954
```

El primer comando contabiliza todas las operaciones en 'aCollection' , excluyendo las operaciones 'getMore' , lo que revela 122 784 creaciones iniciales de cursores. El segundo comando contabiliza solo los cursores que se agotaron correctamente (se iteraron completamente o se cerraron explícitamente), mostrando solo 46 954 casos. Esta significativa discrepancia (solo alrededor del 38 % de los cursores se cerraron correctamente) confirmó nuestra sospecha.

de una fuga de cursor, ya que la mayoría de los cursores creados por la aplicación se abandonaban sin una limpieza adecuada, lo que provocaba que se acumularan hasta que transcurriera su período de tiempo de espera.

Identificación de la causa raíz

El problema se originó a partir de un error específico en la aplicación del cliente que había permanecido oculto hasta hace poco. Cuando las consultas empezaron a devolver más de los 101 documentos predeterminados en un solo lote debido al crecimiento de los datos, el problema finalmente se hizo evidente. La aplicación estaba diseñada para procesar conjuntos de resultados pequeños y, en esos casos, gestionaba correctamente los cursores.

Este problema solo se manifiesta cuando un cliente mantiene una sesión activa abierta sin agotar completamente el cursor (ya sea por no iterar todos los resultados o por no ejecutar explícitamente la función `close()` en el cursor). En circunstancias normales, con un manejo del cursor correctamente implementado o conjuntos de resultados más pequeños, este problema no ocurriría. La mayoría de las implementaciones de controladores de MongoDB y los patrones de consulta estándar gestionan automáticamente la gestión del ciclo de vida del cursor, lo que hace que este tipo de fuga sea relativamente poco común en aplicaciones bien diseñadas.

Para obtener más información sobre cómo se comportan los cursores, visite MongoDB Documentación sobre cursores en

<https://www.mongodb.com/docs/manual/core/cursors/#behavior>.

Implementando la solución

Mientras el equipo de desarrollo del cliente trabajaba en una solución permanente, el equipo de soporte de MongoDB proporcionó una medida temporal clave: un comando de shell para terminar los cursores inactivos en el espacio de nombres afectado. Esto ayudó a evitar la acumulación de cursores inactivos, lo que podría haber empeorado el rendimiento y causado inestabilidad en el sistema. Este comando ofreció un alivio inmediato y le dio al equipo de desarrollo tiempo valioso para solucionar el problema principal.

El cliente desarrolló una solución completa y eficaz para el error. Esta solución se implementó en colaboración con el soporte de MongoDB en una llamada activa, lo que garantizó orientación en tiempo real y una resolución de problemas inmediata.

La implementación tuvo un impacto inmediato y significativo. Antes de la corrección, el número de cursores abiertos alcanzaba habitualmente más de 600 000 antes de sufrir caídas temporales. Para esta aplicación en particular, las operaciones normales deberían haber requerido solo unos 100 cursores simultáneos. Tras la implementación, el sistema se estabilizó. El número máximo de cursores abiertos se mantuvo consistentemente en un nivel mucho más bajo y manejable de 100. Esta reducción mejoró considerablemente la eficiencia del sistema y el uso de recursos.

Como confirmación adicional del éxito de la solución implementada, los informes de estado del servidor mostraron que el problema se había resuelto.

En concreto, la métrica `cursor.lifespan.greaterThanOrEqual10Minutes` reportó constantemente 0. Esta métrica es útil para identificar cursores de larga duración y potencialmente problemáticos. Un valor de 0 indicaba que ningún cursor permanecía abierto durante 10 minutos.

Resultados y aprendizajes

La situación del cliente representaba un caso extremo inusual en el que el código de su aplicación creaba repetidamente nuevos cursores sin cerrarlos correctamente.

En este ejemplo se aplicaron varios principios de resolución de problemas:

- La degradación gradual del rendimiento puede ser causada por fugas del cursor
- Diagnosticar la causa raíz a menudo requiere múltiples enfoques
- Mientras se desarrolla una solución permanente, podría ser necesaria una mitigación temporal.
- Las métricas de monitoreo permiten la detección temprana

Finalmente, esta situación proporciona un ejemplo vívido de cómo los problemas en el código de aplicación de la fase de desarrollo pueden aparecer como problemas de rendimiento de la base de datos en producción.

Caso de uso: Explosión de consultas mal optimizadas

Un cliente notó un aumento repentino de operaciones con tiempo de espera agotado. El monitoreo de Atlas mostró un aumento en la utilización normalizada de la CPU al 100 % y una velocidad de lectura de más de 3,7 GB/s en la caché del motor de almacenamiento, como se muestra en la Figura 15.5 .

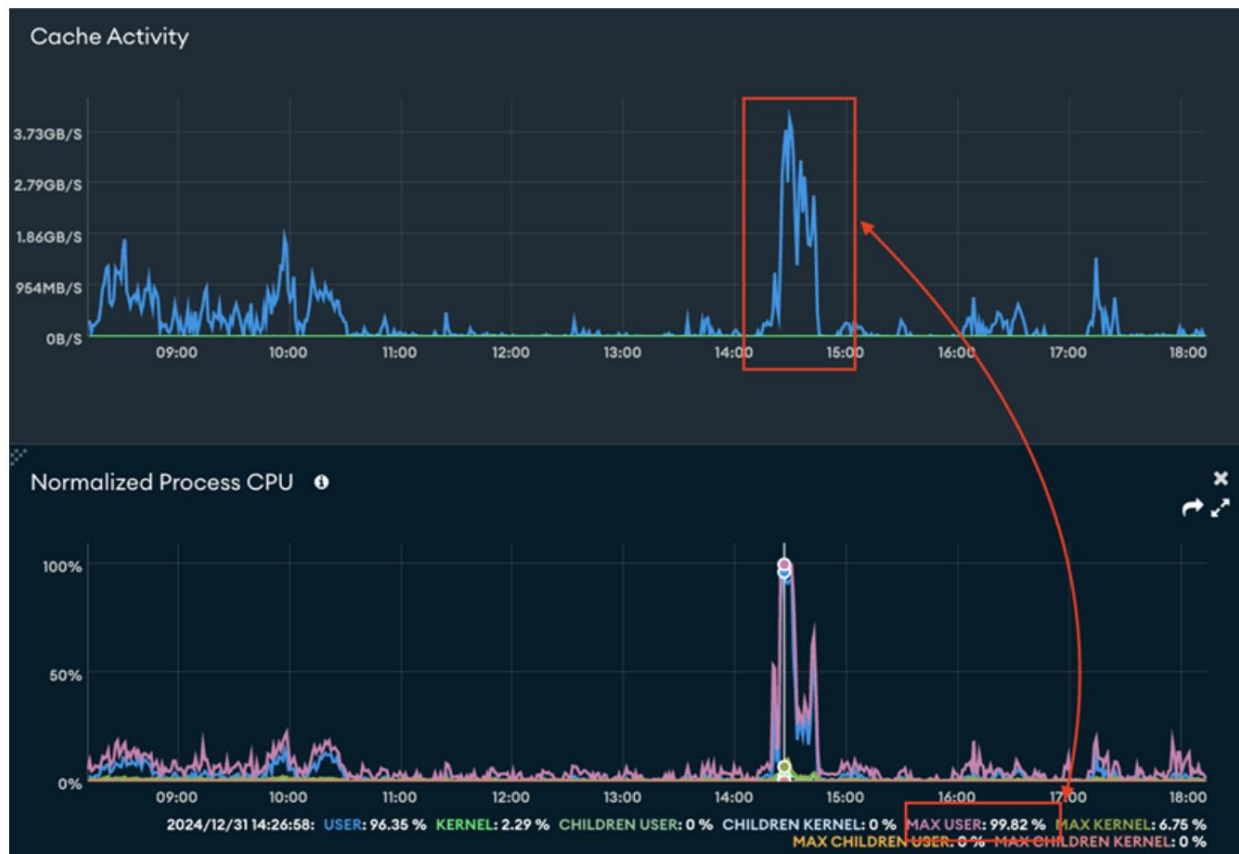


Figura 15.5: Actividad de caché y utilización de CPU del usuario

Las consultas de la aplicación se vieron gravemente afectadas. Los usuarios experimentaron tiempos de espera, respuestas lentas y, en algunos casos, fallos totales de las solicitudes. El sistema quedó prácticamente bloqueado, ya que las operaciones normales no pudieron continuar mientras estos procesos, que consumían muchos recursos, monopolizaban la capacidad disponible de CPU y E/S. Pero ¿qué causó esta carga repentina y abrumadora? Dado que la base de datos atiende principalmente consultas de clientes, estas métricas de rendimiento sugerían firmemente que una o más consultas problemáticas probablemente estaban detrás del pico. Para identificar a los responsables específicos de esta actividad inusual, necesitábamos analizar a fondo los patrones de consulta. Esto nos llevó a examinar Atlas Query Insights para comprender mejor lo que sucedía a nivel de consulta.

Diagnóstico de la causa: Query Insights/

Query Profiler reveló un número significativo de consultas en una colección que examinaban más de un millón de documentos cada una, como se muestra en la Figura 15.6 , representada por puntos verdes en la parte superior del gráfico. En la parte inferior de la captura de pantalla, también podemos ver que, en total, 233 consultas analizaron aproximadamente 144 millones de documentos durante este período de 16 minutos. Analizamos los 20 minutos anteriores y observamos que solo se analizaron 8 millones de documentos en total.

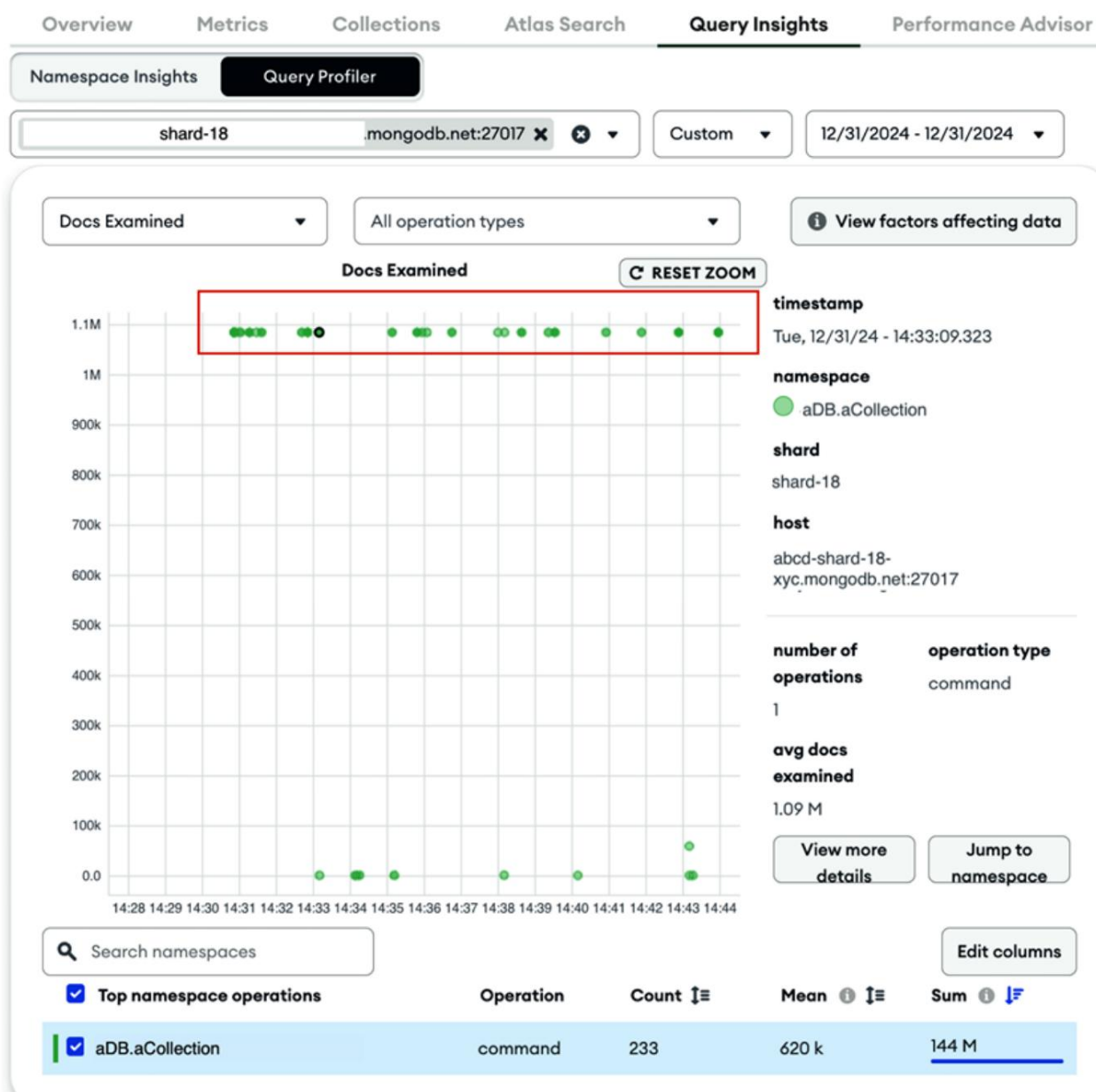


Figura 15.6: Generador de perfiles de consultas que muestra una gran cantidad de documentos examinados

Al hacer clic en uno de los puntos verdes que se muestran en el Generador de perfiles de consultas, como se muestra en la Figura 15.6, reveló conocimientos más profundos sobre cada individuo consulta. Al hacer clic en una de esas consultas, el generador de perfiles mostró la siguiente:

```
"replanReason": "el plan en caché fue menos eficiente de lo esperado"
```

Esto indicaba que el planificador de consultas tenía dificultades para encontrar un plan de ejecución eficiente. Al revisar la colección, se encontraron 25 índices aptos para su uso, pero ninguno se ajustaba bien a estas consultas, lo que indicaba que podría ser necesario un nuevo índice.

Además, un análisis de registros más detallado también reveló que algunas de las consultas lentas en esta colección intentaban hacer coincidir documentos en un rango de seis meses.

Implementación de la solución. Para abordar las ineficiencias causadas por la falta de un índice óptimo, el equipo revisó el Asesor de Rendimiento, que, como se esperaba, recomendó un nuevo índice compuesto diseñado para resolver el problema, como se muestra en la Figura 15.7 . Siguiendo esta recomendación, el equipo creó rápidamente el índice compuesto, lo que mejoró significativamente el rendimiento de las consultas.

aDB.aCollection

FIELD_ARN: 1 FIELD_AN: 1 FIELD_PD: 1 FIELD_UR: 1 FIELD_RL: 1 👍 👎 CREATE INDEX	QUERIES IMPROVED BY THIS INDEX			
	Execution Count	Avg. Execution Time	Avg. Query Targeting ⓘ	In Memory Sort ⓘ
	3944.67/hour	8491 ms	43019	3944.67 ops/hr
	Avg. Docs Scanned	Avg. Docs Returned	Avg. Object Size	
	43019	0	2.1 KB	

Figura 15.7: Recomendaciones de Performance Advisor

Además, el equipo mejoró la aplicación modificando su lógica de consulta para que coincida con los documentos en un lapso de tiempo más corto. Este ajuste optimizó aún más la ejecución de la consulta, reduciendo tanto la

tamaño de los datos devueltos y latencia de la consulta, lo que resulta en un rendimiento más rápido y un uso más eficiente de los recursos del sistema.

Resultados y aprendizajes Este caso subraya

varios principios clave de la optimización de consultas.

Los problemas repentinos de rendimiento suelen estar relacionados con cambios en el comportamiento de las aplicaciones, por lo que es fundamental supervisar de cerca sus patrones. Un indicador crítico de consultas ineficientes es cuando la base de datos examina una cantidad desproporcionadamente grande de documentos en comparación con los resultados reales devueltos. Esto puede indicar un problema subyacente con el diseño o la indexación de las consultas. Además, el Planificador de Consultas puede experimentar dificultades al operar con amplios rangos de datos, lo que destaca la importancia de limitar el alcance de las consultas siempre que sea posible.

Abordar los desafíos de rendimiento suele requerir una combinación de estrategias, que incluyen cambios en la base de datos, como la creación de índices optimizados, y ajustes a nivel de aplicación, como la reducción del lapso de consulta de los datos. Herramientas como el Asesor de Rendimiento son invaluable para identificar índices faltantes, incluso para patrones de consulta poco frecuentes. Ofrece recomendaciones prácticas para mejorar la eficiencia y resolver cuellos de botella.

Abordar las limitaciones de recursos de hardware

Las limitaciones de recursos de hardware representan un desafío crítico para mantener un rendimiento óptimo de MongoDB. Incluso con...

Consultas optimizadas y esquemas cuidadosamente diseñados. El hardware subyacente determina en última instancia la eficiencia con la que su base de datos puede gestionar las cargas de trabajo. Las limitaciones de recursos surgen cuando...

La capacidad computacional, de memoria, de red o de almacenamiento de su implementación es insuficiente para satisfacer las demandas, lo que genera problemas de rendimiento en cascada. Imagine una línea de producción donde las máquinas funcionan a su máxima capacidad. Cualquier aumento en la demanda o ineficiencia sobrecarga los sistemas, lo que genera cuellos de botella y retrasos. De igual manera, las limitaciones de recursos en MongoDB pueden afectar significativamente la capacidad de respuesta y la escalabilidad de su aplicación.

Estas restricciones suelen manifestarse en cuatro áreas principales: CPU, memoria, E/S de disco y red. Un uso elevado de la CPU puede limitar la capacidad de procesamiento disponible para consultas, agregaciones y tareas en segundo plano, lo que obliga a las operaciones a colarse y retrasa los resultados.

Las limitaciones de memoria se producen cuando la RAM disponible es insuficiente para almacenar el conjunto de trabajo, lo que aumenta la dependencia de lecturas de disco más lentas y reduce el rendimiento. Las limitaciones de E/S de disco, como un subsistema de almacenamiento lento o sobrecargado, pueden dificultar drásticamente las operaciones que requieren lecturas y escrituras frecuentes o de gran volumen de datos. Las restricciones de red pueden afectar significativamente las operaciones de bases de datos distribuidas, lo que provoca problemas como mayor retraso en la replicación, sincronizaciones iniciales más lentas, tiempos de respaldo extendidos y elecciones demoradas en conjuntos de réplicas, todo lo cual puede comprometer la disponibilidad y la consistencia de los datos en toda su implementación.

El impacto de las limitaciones de recursos suele ser gradual y surge junto con el crecimiento del volumen de datos, el aumento de las cargas de trabajo o los cambios en los patrones de uso. Sin embargo, el problema también puede aparecer repentinamente.

cuando los recursos se distribuyen de forma desigual en un clúster, lo que crea puntos críticos o desequilibrios que agravan los cuellos de botella en el rendimiento.

Reconocer y abordar estas limitaciones de hardware es esencial para garantizar que MongoDB siga siendo receptivo y confiable a escala.

Comprender cómo la CPU, la memoria y el almacenamiento interactúan con las cargas de trabajo de MongoDB permite a los equipos mitigar de forma proactiva las limitaciones y optimizar la asignación de recursos en las implementaciones.

Caso práctico: Optimización de recursos del clúster Atlas. Siguiendo con el ejemplo anterior del clúster

de monitorización Atlas, el equipo observó que, incluso tras abordar las operaciones inherentemente lentas con mejoras de indexación, la utilización de recursos en el fragmento 0 seguía siendo mayor que en otros fragmentos. Este desequilibrio representaba un problema importante, ya que creaba un cuello de botella en el rendimiento del sistema distribuido. Cuando un fragmento opera de forma constante con niveles de utilización más altos que otros, se convierte en un factor limitante del rendimiento general del clúster. Esto no solo supone el riesgo de agotamiento de los recursos en el fragmento sobrecargado, lo que podría provocar fallos operativos, sino que también representa un uso ineficiente de la capacidad de todo el clúster. Además, este desequilibrio socava la tolerancia a fallos y los beneficios de escalabilidad que la fragmentación está diseñada para proporcionar, ya que el sistema se ve limitado de forma desproporcionada por el rendimiento de un solo fragmento.

Al observar la CPU del proceso no normalizado en los núcleos primarios de cada fragmento, observaron que el fragmento 0 tenía una mayor utilización y tamaño de datos que todos los demás fragmentos, como se muestra en la Figura 15.8 .



Figura 15.8: Gráfico que muestra la CPU del proceso no normalizado en los primarios de cada fragmento

Además, la Figura 15.9 ilustra una observación importante: durante el último año, las IOP de escritura para el fragmento 0 aumentaron a un ritmo más rápido en comparación con otros fragmentos del clúster.

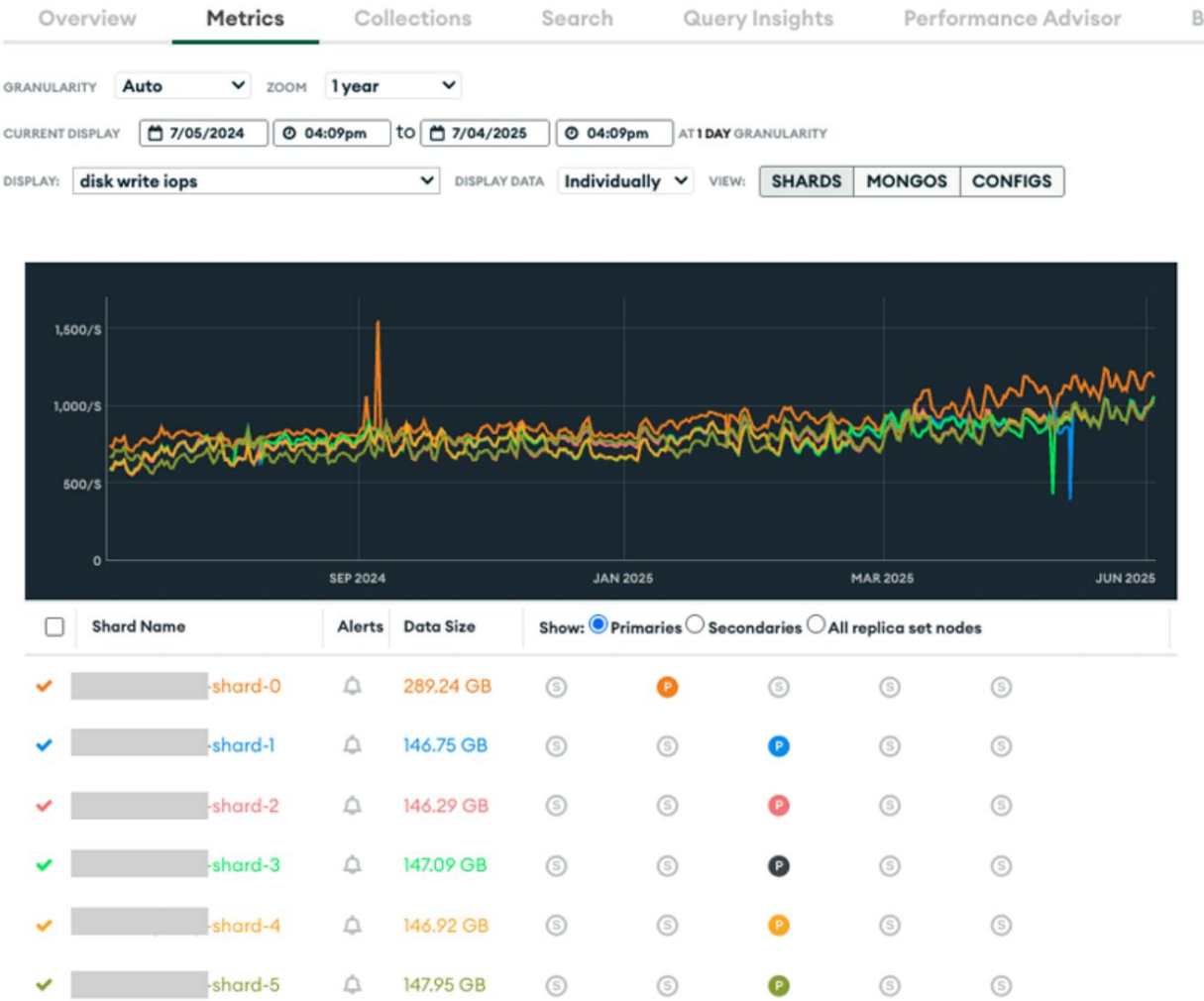


Figura 15.9: Gráfico que muestra el último año de IOPS de escritura en el fragmento 0

Diagnosticar la causa

La causa fundamental del desequilibrio surgió del desequilibrio inicial del clúster.

Configuración. El clúster contenía una única base de datos con numerosas colecciones, y al crearla, el fragmento 0 se designó como principal. Como resultado, todas las colecciones no fragmentadas se ubicaron en el fragmento 0 de forma predeterminada. Con el tiempo, a medida que aumentaba el número de colecciones no fragmentadas y sus respectivos volúmenes de datos, el fragmento 0 se sobrecargó con cargas de trabajo desproporcionadas.

Esta distribución desigual provocó una mayor concentración de operaciones y datos en el fragmento 0, lo que provocó un uso elevado de CPU, memoria y disco en comparación con los demás fragmentos. El uso intensivo del fragmento 0 también incrementó la latencia de las consultas y limitó el rendimiento general del clúster, ya que las operaciones en colecciones no fragmentadas se vieron obstaculizadas por las limitaciones de recursos del fragmento principal.

Como se discutió en [Capítulo 1](#) [Sistemas y arquitectura MongoDB](#), La optimización del rendimiento rara vez se realiza en un solo paso. Los cuellos de botella en los sistemas de bases de datos son dinámicos. Al resolver un problema, suele hacerse evidente la siguiente restricción del sistema. Este principio se demostró claramente en nuestro caso. Si bien las mejoras de indexación solucionaron con éxito las operaciones lentas identificadas inicialmente, simplemente revelaron otro cuello de botella subyacente que había quedado parcialmente enmascarado por los problemas de indexación más graves.

Implementando la solución

El equipo tuvo que determinar cuál de las siguientes opciones sería la mejor para equilibrar la carga de trabajo:

- Ampliar el fragmento principal (Atlas admite el escalado individual de cada fragmento)
- Mover colecciones no fragmentadas del fragmento 0 a otros fragmentos
- Compartir más colecciones

El equipo decidió mover las colecciones no fragmentadas del fragmento 0 a Otros fragmentos, ya que este enfoque era más sencillo de implementar y gestionar en comparación con las alternativas. Este método no requería cambios en el código de la aplicación ni en el esquema de datos y podía ejecutarse.

Sin tiempos de inactividad significativos. Además, le dio al equipo control directo sobre qué colecciones se movían a dónde, lo que permitió un equilibrio preciso según el tamaño de las colecciones y los patrones de acceso.

Pero primero, verificaron dos veces qué colecciones no estaban fragmentadas y cuáles estaban fragmentadas.

1. Para ello, ejecutaron los siguientes comandos en mongosh :

```
// Lista de colecciones no fragmentadas
db.getCollectionNames().forEach(nombreDeColección => { if (!
  db[nombreDeColección].stats().sharded)
    { print(nombreDeColección);
  }
});
// Lista de colecciones fragmentadas
db.getCollectionNames().forEach(nombreDeColección => { if
  (db[nombreDeColección].stats().sharded)
    { print(nombreDeColección);
  }
});
```

Encontraron una base de datos con 163 colecciones. De ellas, 20 estaban fragmentadas y 143 no.

2. A continuación, el equipo analizó el panel en tiempo real del fragmento 0 para obtener una vista rápida de su carga de trabajo:



Figura 15.10: Panel que muestra qué tan popular es una colección en tiempo real

La Figura 15.10 muestra la vista en tiempo real de las colecciones más populares en términos de operaciones por segundo. Esta vista muestra el uso de CPU y memoria, junto con el número de conexiones, operaciones por segundo y las IOP de lectura y escritura subyacentes dirigidas a los discos. Este desglose detallado resultó fundamental para determinar qué colecciones debían migrarse a diferentes fragmentos.

Al identificar las colecciones no fragmentadas con mayor número de operaciones, mayor consumo de memoria y mayor consumo de CPU, el equipo pudo priorizar el traslado de las colecciones que consumen más recursos. Este enfoque específico garantizó que el esfuerzo de reequilibrio tuviera el máximo impacto en la reducción de la carga del fragmento 0, a la vez que minimizaba el número de colecciones que debían reubicarse. Las colecciones con menor actividad podían permanecer en el fragmento 0 o trasladarse en fases posteriores si fuera necesario.

El equipo también analizó la vista Query Insights para obtener una imagen a más largo plazo de las colecciones más activas, como se muestra en la Figura 15.11 .

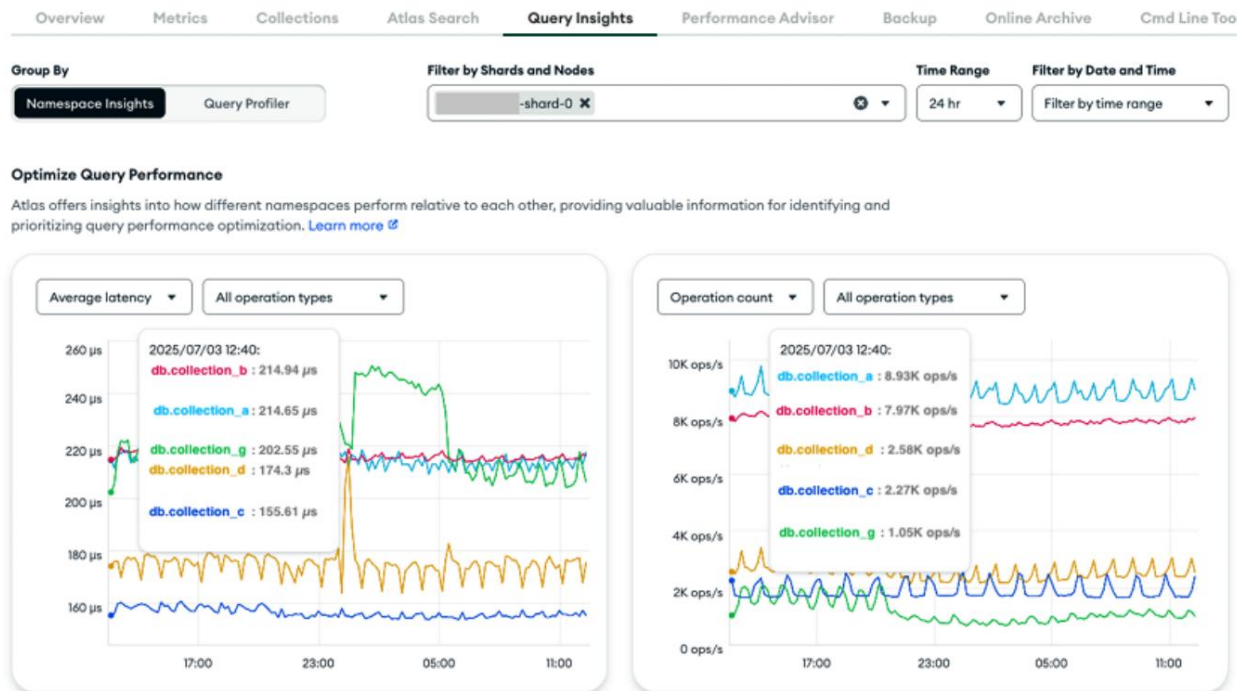


Figura 15.11: Gráfico que muestra tanto el recuento de operaciones por segundo como la latencia promedio durante las últimas 24 horas

El gráfico muestra tanto el número de operaciones por segundo como la latencia promedio de las últimas 24 horas. Con esta información sobre la carga de trabajo del fragmento 0, el equipo identificó las 10 colecciones sin fragmentar con el mayor consumo de CPU y con conjuntos de datos de tamaño manejable para garantizar una migración rápida.

Planearon distribuir estas colecciones del fragmento 0 a otros fragmentos:

- La colección no fragmentada más concurrida hasta el fragmento 1
- La segunda colección más concurrida hasta shard-2
- Colecciones ocupadas 3 y 4 al fragmento 3

- Colecciones ocupadas 5, 6 y 7 hasta el fragmento 4
- Colecciones ocupadas 8, 9 y 10 hasta el fragmento 5

Este plan se probó primero en un entorno de control de calidad antes de ejecutarlo en producción. Esta precaución fue importante porque mover colecciones entre fragmentos implica operaciones pesadas que pueden afectar significativamente el rendimiento del clúster.

Primero trasladaron las dos colecciones más pequeñas y verificaron las métricas para verificar que los cambios funcionaran. Para cada traslado de colección, siguieron este proceso:

1. Supervisar el fragmento 0 y las métricas de hardware del fragmento de destino y tiempos de ejecución de operaciones.
2. Verifique que el fragmento de destino tenga espacio libre en disco, IOP, recursos de red y margen de CPU.
3. Conéctese a mongos y mueva una colección:

```
db.adminCommand( { moverColección: "db.colección_b",
```

4. Si algo sale mal durante el traslado, utilice el comando de administración `abortMoveCollection` .
5. Supervisar el fragmento 0 y las métricas de hardware del fragmento de destino y tiempos de ejecución de operaciones.



MoveCollection provoca un bloqueo de escritura corto

El comando `moveCollection` bloquea las escrituras en el Recolección por hasta 2 segundos durante su sección crítica . [Puede encontrar más detalles en el](#)

documentación en <https://mongodb.com/docs/manual/tutorial/move-a-collection/> .

Resultados y aprendizajes El equipo esperaba

que la migración de las colecciones no fragmentadas que consumen más recursos, junto con las optimizaciones de esquema e indexación anteriores, aliviaría las demandas de recursos del clúster, lo que les permitiría reducir la infraestructura, reducir significativamente los costos y mantener los niveles de rendimiento.

Este ejemplo demuestra que abordar las limitaciones de recursos de hardware requiere más que simplemente escalar; requiere un enfoque estratégico que integre la monitorización, el diagnóstico, el análisis de la carga de trabajo y la planificación operativa. Herramientas como la monitorización de Atlas y Query Insights son cruciales para identificar cuellos de botella en los recursos y analizar tanto las métricas a corto plazo como las tendencias de la carga de trabajo a largo plazo. Con una clara visibilidad de estas métricas, las organizaciones pueden implementar soluciones basadas en datos adaptadas a las necesidades específicas de su clúster, garantizando que las acciones correctivas sean precisas y eficaces.

Caso de uso: Diagnóstico de IOP insuficientes en MongoDB autogestionado

En sistemas autogestionados, es común observar un patrón similar en las implementaciones de los clientes: las cargas de trabajo crecen naturalmente con el tiempo, a menudo superando las capacidades de la infraestructura subyacente. Con MongoDB Atlas, esto es fácil de detectar a tiempo mediante herramientas de monitorización integradas que brindan visibilidad en tiempo real de métricas clave como la inactividad.

CPU, capacidad de IOP de disco y número de operaciones en ejecución. Sin embargo, al gestionar clústeres de MongoDB de forma independiente, los equipos deben recurrir a scripts o herramientas de monitorización externas para monitorizar estas métricas.

En este ejemplo, repasamos el caso de un cliente que autogestionaba MongoDB en su infraestructura en la nube. Al surgir problemas de rendimiento, el equipo de soporte de MongoDB recurrió a los datos de diagnóstico del servidor.

Diagnosticar la causa

Los servidores MongoDB capturan información del sistema y contadores internos cada segundo mediante la Captura de Datos de Diagnóstico a Tiempo Completo (FTDC). Estos datos no contienen información confidencial, como consultas, predicados de consulta, resultados ni credenciales.

Al analizar las solicitudes de lectura de almacenamiento por segundo (que en Linux provienen del comando `iostat`), descubrieron que el servidor de base de datos emitía 16 600 IOP. Los datos de FTDC de `vmstat` mostraron un 39 % de espera de `iowait`. `iowait`, lo que indica que el sistema estaba perdiendo un tiempo significativo esperando operaciones de E/S.

El cliente había aprovisionado el almacenamiento subyacente con 15 000 IOP. Los proveedores de nube suelen permitir la carga explosiva por encima del límite aprovisionado durante un periodo antes de la limitación. Esta carga explosiva, seguida de la limitación, se percibía como bloqueos en la aplicación del cliente.

Al analizar la asignación de IOPS, nos dimos cuenta de que estábamos utilizando más IOPS de las previstas. El rendimiento fue aceptable durante un tiempo porque

Se proporcionaron mayores IOPS en ráfagas. Sin embargo, una vez agotada la capacidad de ráfagas, se produjo una marcada degradación del rendimiento. El sistema ya no tenía suficientes IOPS y las operaciones se ralentizaron, a la espera de E/S.

Implementando la solución

El cliente aumentó las IOPS aprovisionadas de 15 000 a 18 000, lo que resolvió de inmediato los problemas de rendimiento.

Resultados y aprendizajes Este caso

destaca varios principios importantes con respecto a las limitaciones de recursos de hardware en Atlas y las implementaciones autogestionadas de MongoDB. Las limitaciones de E/S con frecuencia se presentan como altos porcentajes de `iowait`, lo que indica posibles cuellos de botella en el almacenamiento. En entornos de nube, vale la pena señalar que puede ser posible una explosión temporal por encima de los límites aprovisionados, pero esto no siempre es sostenible para cargas de trabajo crecientes. Una comprensión profunda de la relación entre la carga de trabajo de la aplicación y el aprovisionamiento de recursos es fundamental para mantener un rendimiento óptimo. Incluso sin las capacidades de monitoreo avanzadas que proporciona Atlas, las herramientas de diagnóstico como FTDC pueden ofrecer información valiosa para identificar y abordar problemas subyacentes. Finalmente, vale la pena recordar que, a veces, la solución más simple y efectiva a los problemas de rendimiento es simplemente aumentar los recursos disponibles para que coincidan con las demandas de la carga de trabajo.

Este caso sirve como recordatorio de la importancia de la planificación y el monitoreo proactivos p

Enfoque sistemático para la resolución de problemas de rendimiento

Basándonos en los ejemplos que hemos explorado, aquí hay una metodología estructurada para diagnosticar y resolver problemas de rendimiento de MongoDB:

1. Identificar los síntomas y recopilar datos iniciales : El primer paso para la resolución de problemas es identificar los síntomas de rendimiento y recopilar datos relevantes para comprender el problema. Determine cuándo comenzó el problema, identifique qué operaciones o colecciones de bases de datos se ven afectadas y recopile métricas de referencia, como el uso de CPU, las IOPS y el rendimiento de las consultas, para compararlas. Esta información inicial ayuda a comprender mejor la situación y guía los siguientes pasos del diagnóstico.
2. Use múltiples herramientas de diagnóstico : Aproveche diversas herramientas de diagnóstico para comprender mejor el problema. Para los usuarios de Atlas, los paneles de monitoreo ofrecen una visión general de las métricas clave de rendimiento, mientras que quienes utilizan clústeres autogestionados pueden usar herramientas de monitoreo del sistema para obtener resultados similares. El generador de perfiles de base de datos es fundamental para detectar consultas lentas, los archivos de registro ayudan a descubrir errores o patrones relacionados con la degradación del rendimiento, y el comando `currentOp()` identifica las operaciones en ejecución. Además, el Asesor de Rendimiento ofrece recomendaciones para la optimización de índices, que a menudo pueden abordar problemas de eficiencia de las consultas. Recuerde que el ecosistema de observabilidad y monitoreo está en constante evolución, y es importante mantenerse al día con la
3. Categorizar el problema : Una vez recopilados los datos, categorizar la naturaleza del problema para delimitar las posibles soluciones. ¿Es...?

¿Se debe a operaciones inherentemente lentas, como análisis completos de colecciones o estrategias de indexación deficientes? ¿Podría deberse a operaciones bloqueadas, como bloqueos o problemas relacionados con el cursor? ¿O es el resultado de limitaciones de recursos relacionadas con la CPU, la memoria o las E/S del disco? Comprender la causa raíz es fundamental para seleccionar el enfoque adecuado para la remediación.

4. Determinar el alcance : A continuación, evalúe el alcance del problema para Asegúrese de que las soluciones sean eficaces. ¿El problema afecta a consultas o colecciones específicas, o es más amplio y abarca todo el sistema? Además, verifique si el problema solo ocurre en momentos específicos, como durante picos de trabajo o trabajos programados, para determinar si el tiempo o los patrones influyen. Definir el impacto y el alcance garantiza que los esfuerzos se dirijan a donde serán más efectivos.
5. Implementar soluciones específicas : Una vez identificado el problema, implementar soluciones adaptadas a él. Para operaciones inherentemente lentas, considere agregar índices u optimizar las consultas para mejorar la eficiencia. Las operaciones bloqueadas pueden requerir la eliminación de procesos de larga duración o bloqueados, así como la resolución de problemas relacionados con el cursor. Si se identifican limitaciones de recursos, ampliar la CPU, la memoria o las IOPS de disco, o redistribuir las cargas de trabajo entre fragmentos o clústeres, puede aliviar los cuellos de botella en el rendimiento.
6. Verificar la solución : después de implementar los cambios, monitoree las métricas clave para asegurarse de que el problema se haya resuelto con éxito. Verifique que el problema no se repita y confirme que el rendimiento haya mejorado al nivel esperado. Configure alertas o monitorización proactiva para detectar problemas similares con antelación, minimizando así el tiempo de inactividad y el impacto en los usuarios.

7. Documentar los hallazgos y las medidas preventivas : Por último, Documente la causa raíz del problema y las medidas tomadas para resolverlo. Aproveche esta oportunidad para implementar medidas preventivas, como actualizar los flujos de trabajo de monitoreo o cambiar la infraestructura, para evitar problemas similares en el futuro. Comparta estos hallazgos y las mejores prácticas con otros miembros del equipo para asegurar la alineación y fortalecer una cultura de gestión proactiva del rendimiento.

Al seguir un enfoque estructurado, los equipos pueden identificar, abordar y prevenir sistemáticamente problemas de rendimiento en las implementaciones de MongoDB. La monitorización constante y la intervención oportuna son clave para mantener un sistema ágil y escalable.

Resumen

En este capítulo, exploramos ejemplos reales de problemas de rendimiento de MongoDB que coinciden con las tres causas fundamentales de William Zola: operaciones inherentemente lentas, operaciones bloqueadas y recursos de hardware insuficientes. Mediante casos prácticos, vimos cómo identificar problemas de rendimiento con las herramientas integradas de MongoDB.

Diagnosticamos las causas raíz examinando los patrones de consulta, el comportamiento del cursor y el uso de recursos. Posteriormente, implementamos

soluciones específicas mediante la indexación, la optimización de consultas y la gestión de recursos.

Por último, establecemos medidas preventivas para detectar los problemas antes de que se vuelvan críticos.

Las herramientas de depuración para el rendimiento de MongoDB han mejorado significativamente en los últimos años. Atlas ofrece capacidades integrales de monitoreo y alertas, mientras que la autogestión...

Las implementaciones pueden aprovechar herramientas como el generador de perfiles de base de datos, el análisis de registros y los datos FTDC.

Recuerde que la optimización del rendimiento es un proceso continuo, no una tarea puntual. A medida que sus datos aumentan y los patrones de aplicación evolucionan, supervise constantemente las métricas de rendimiento, revise el uso del índice y ajuste su configuración para mantener un rendimiento óptimo.

Al abordar sistemáticamente los problemas de rendimiento utilizando los principios y técnicas delineados en este capítulo, estará mejor preparado para mantener implementaciones de MongoDB de alto rendimiento independientemente de la escala o la complejidad.

Epílogo

Bueno, hemos llegado al final de nuestro análisis del rendimiento de MongoDB.

¡Viaje! Pero si hay una lección que destaca de años de

Optimización de bases de datos, es que el ajuste del rendimiento nunca realmente termina. Justo cuando parece que todo está resuelto, el

La aplicación crece, los patrones de los usuarios cambian o MongoDB lanza una nueva versión. Nueva función para probar.

¿Recuerdan nuestra discusión sobre el diseño de esquemas en Diseño [Capítulo 2](#),

de esquemas para el rendimiento o sobre los índices en Índices ? ¡No [Capítulo 3](#),

fue solo por diversión! Muchos usuarios han tenido dificultades con...

Problemas de rendimiento que en última instancia se debieron a un diseño deficiente.

esquemas o índices subóptimos (o faltantes). No se puede crear un

rascacielos en arenas movedizas, no importa cuánto optimices

En otros lugares, esos cimientos inestables siguen contraatacando.

Hemos cubierto mucho terreno, desde detectar consultas lentas hasta

Dominando la indexación. A lo largo del camino, hemos explorado lo complejo que es

Los sistemas interactúan. Analizamos cómo aprovechar características como la

Marco de agregación, flujos de cambio y transacciones. Nosotros

Revisamos cómo usar la replicación para alta disponibilidad y fragmentación.

escalar horizontalmente. Pero ajustar el rendimiento es una maratón, no un sprint. Es una proceso continuo de aprendizaje y adaptación a medida que evolucionan las aplicaciones.

Conclusiones clave

¿Cuáles son las lecciones más importantes que debemos aprender? Aquí tienes un breve resumen de los principios que mantendrán tu implementación de MongoDB rápida, estable y preparada para el futuro:

- El diseño de esquemas lo es todo : En serio, la gran mayoría de los problemas de rendimiento se deben a esquemas que no se ajustan a cómo la aplicación usa los datos. Al diseñar documentos, tenga en cuenta las lecturas y escrituras. Y no dude en evolucionar el esquema con el tiempo. Un buen esquema es la base de un excelente rendimiento.
- La indexación es tu superpoder : Unos buenos índices son clave para consultas de alto rendimiento. Una estrategia de indexación inteligente simplifica el trabajo de MongoDB. Acostúmbrate a revisar los índices regularmente.
- Si no mides, estás volando a ciegas : No puedes arreglar lo que no ves. Configura un monitoreo constante con métricas y alertas significativas. Esto ayuda a detectar problemas a tiempo y a guiar los esfuerzos de ajuste. La visibilidad de las implementaciones te ayuda a detectar problemas antes que tus usuarios.
- No olvides lo aburrido : Más allá de los esquemas e índices, unas operaciones diarias sólidas mantienen todo en funcionamiento. Esto implica gestionar las configuraciones, tener un plan de respaldo sólido y saber cuándo se necesita más capacidad. Automatizar el mantenimiento puede ahorrarte muchos dolores de cabeza en el futuro.

Ajustar el rendimiento puede parecer como perseguir un objetivo en movimiento, pero con los hábitos correctos y una base sólida, siempre estará listo para lo que viene después.

La mentalidad de rendimiento

Piensa como un detective de rendimiento. Sé curioso. No esperes a que algo falle. Investiga los planes de consulta, supervisa el rendimiento base y cuestiona las suposiciones. Haz del rendimiento un deporte de equipo, algo de lo que desarrolladores, operadores y arquitectos compartan la responsabilidad.

Hemos visto un caso en el que un cambio aparentemente menor en un índice redujo el tiempo de carga de una página en un 40 %. Este tipo de logros solo se consiguen cuando se buscan. Al mantener la curiosidad y desarrollar hábitos de mejora, la optimización se convierte en parte de la cultura, dejando de ser una tarea reactiva.

Próximos pasos prácticos

Saber es genial, pero hacer es aún mejor. Aquí tienes algunos pasos prácticos que puedes poner en práctica ahora mismo para convertir la teoría del rendimiento en resultados:

- Audite su esquema : Analice detenidamente su esquema. ¿Funciona realmente con sus consultas más comunes? Revise los patrones de acceso y considere si la estructura del documento (incrustado o referenciado) aún cumple su función.

- Familiarícese con las herramientas de diagnóstico : familiarícese con `db.collection.explain()` y la pestaña Query Insight en Atlas.

Úsalos para observar el comportamiento real de las consultas. Detecta consultas lentas e índices ineficaces; incluso puede ser divertido.

- Configura una monitorización real : utiliza herramientas como MongoDB Cloud Monitoring o alternativas similares. Crea paneles que resalten métricas críticas como la duración de la operación y el uso de caché.

Tasa y retardo de replicación. Deje que los datos indiquen dónde se necesita el trabajo.

- Pruebas bajo presión : Incorpore pruebas de carga en sus procesos de desarrollo. Asegúrese de que el sistema pueda gestionar el tráfico esperado e inesperado. Así podrá anticiparse a los cuellos de botella antes de que afecten la producción.
- Habla con otros : Conéctate con otros a través de grupos de usuarios locales, eventos o foros en línea. La comunidad de MongoDB es grande y está llena de personas que resuelven desafíos similares y comparten experiencias que pueden revelar información valiosa. Puedes empezar visitando los foros de la comunidad de MongoDB en <https://www.mongodb.com/community/forums/>.

Cuanto más apliques lo que has aprendido, te mantengas observador y conectado, más resistente y receptivo será tu sistema.

Consideraciones futuras

MongoDB sigue evolucionando. Manténgase al día con las notas de la versión en <https://www.mongodb.com/docs/manual/release-notes/> Pruebe nuevas funciones , en entornos seguros y adopte lo que funciona sentido.

Mantén la curiosidad. Experimenta. Mide. Y siempre confirma que los cambios realmente conducen a mejoras.

Reflexiones finales

El ajuste del rendimiento combina arte y ciencia. Con los conocimientos compartidos en este libro, es posible lograr un impacto significativo en el rendimiento de las aplicaciones. Considere cada cuello de botella como un problema y cada solución como una oportunidad de aprendizaje. Una base de datos bien optimizada mejora la experiencia de desarrolladores, operadores y usuarios. Es un proceso, pero ahora hay un mapa que guía el camino.

¡Ahora ve y construye algo increíble y hazlo rápido!

Índice

Símbolos

\$elemMatch [261](#) [263](#)

\$expr [269](#)

o cláusula [269](#)

A

tiempo de acceso (atime) [286](#)

colección de mensajes activos [58](#)

Estándar de cifrado avanzado (AES) administración [150](#)

de fragmentación avanzada [133](#)

Consideraciones sobre el equilibrador [135](#), [136](#)

predivisión [136](#) _____

refragmentación _____, [135](#)

[134](#) fragmentos de colección fragmentados, ubicación [138](#)

- [140](#) colecciones no fragmentadas, movimiento [137](#)

agregación, en entornos distribuidos vistas materializadas, [98](#)

utilizando [101](#) optimización para _____

colecciones fragmentadas [99](#) monitoreo de _____

rendimiento [100](#) creación de [perfiles](#)

de rendimiento [101](#) _____

operaciones locales de fragmentos versus operaciones fusionadas [99](#)

etapas de agregación

\$acumulador [268](#)

\$faceta [268](#)

\$función [268](#)

expresión regular [269](#)

\$muestra [267](#)

\$texto [268](#)

\$donde [268](#)

mapReduce [268](#)

antipatrones [41](#)

marcos de aplicación [211](#)

mejores prácticas [215](#)

ODM y ORM, aprovechando [212](#) marcos, [213](#)

compatibles populares [213](#) _____, [214](#)

valor [211](#), [212](#)

Monitoreo del rendimiento de aplicaciones (APM) [316](#), [317](#)

Consideraciones para herramientas externas [319](#)

OpenTelemetry [317](#), [318](#)

herramientas [25](#)

vecino más cercano aproximado (RNA) [272](#)

componentes arquitectónicos, clúster fragmentado

servidores de configuración [122](#)

procesos mongos [122](#) _____

fragmentos [122](#)

colección de mensajes archivados [58](#)

patrón de archivo [57](#), [58](#)

patrones de programación asincrónica [216](#)

Alertas Atlas [304](#)

Número [305](#) de Atlas Search [_____](#)

problemas de conexión [305](#)

problemas de oplog [306](#), [307](#)

problemas de consulta [306](#)

Estudio de caso de optimización de recursos del clúster Atlas [343](#), [344](#)

diagnóstico [344](#) [_____](#)

Resultados y aprendizajes [348](#) [_____](#)

solución, implementación [345](#) - [348](#) [_____](#)

Búsqueda en Atlas [271](#)

Características de la interfaz de usuario de Atlas [301](#)

Información sobre espacios de nombres [303](#)

Asesor de rendimiento [303](#) [_____](#)

Generador de perfiles de consultas [304](#)

Panel de rendimiento en tiempo real (RTPP) [302](#) [_____](#), [303](#)

Búsqueda vectorial en Atlas [272](#)

escalado automático

beneficios clave de rendimiento [293](#) [_____](#)

utilizando, para el rendimiento de Atlas [293](#), [294](#)

B

Documentos JSON binarios (BSON) [14](#)

operaciones bloqueadas [335](#)

Explosión de operaciones con tiempo de espera agotado, caso de [de](#)

estudio [339](#), investigación de fugas del cursor, caso de [de](#)

estudio [335](#), gestión [335](#)

transmisión [124](#)

Sistema de archivos B-tree (Btrfs) [285](#) [patron](#)

de agrupamiento - ráfaga de [50](#) [53](#)

operaciones con tiempo de espera agotado caso de estudio [339](#)

causa, diagnóstico [340](#) resultados, [341](#)

y aprendizajes [342](#) solución, [de](#)

implementación [341](#) [de](#)

do

API de devolución de llamada [184](#)

Ventajas [184](#) [de](#)

Cardinalidad [73](#)

Flujos de cambio Ciclo de [160](#)

vida de la colección [172](#) [de](#)

Estrategias de búsqueda de documentos [165](#)

limitaciones de tamaño del documento [172](#)

agrupación de eventos [171](#)

Estructura del evento [162](#)

Características [160](#)

Flujos de eventos de gran volumen, manejo [168](#) [de](#), [169](#)

implementación [163](#) [de](#)

ciclo de vida [163](#)

seguimiento y controles de salud [172](#) _____

implicaciones para el rendimiento [165](#)

ajuste del rendimiento [173](#)_____

Servicio de seguimiento de precios, edificio [165 - 167](#)_____

Conjunto de réplicas, consideraciones [172](#)

estrategias de optimización de recursos [167](#)_____, [168](#)

alcances [163](#)

filtrado del lado del servidor, con canales de agregación [164](#) _____

Implementaciones compartidas, consideraciones [170](#)_____, [171](#)

Visibilidad de transacciones [171](#)

trabajando [161](#)

Modo de encadenamiento de bloques de cifrado ([CBC](#)) [150](#)

Tiempo de espera de operación del lado del cliente (CSOT) [218](#)

Rendimiento del clúster, solución de problemas con Atlas causa [326](#)_____, [327](#)

diagnóstico [327](#) _____

resultados y aprendizajes [332](#)_____

identificación de la causa raíz [328 - 330](#) _____

solución, implementando [331](#) _____, [332](#)

colaciones [272](#)

recopilación

fragmentación [123](#)_____, [124](#)

gestión de la complejidad, en plataformas de datos [23](#)

redundancia y resiliencia integradas [23](#) _____

modelo de datos flexible, con capacidades rigurosas [23](#) _____

escalamiento horizontal, con distribución inteligente [24](#)

índices compuestos [70](#), [71](#)

patrón calculado - [53](#) [55](#)

pérdida de conexión [228](#)

Fundamentos de conexión [226](#), [227](#)

 pérdida de conexión [228](#)

 latencia [227](#)

 saturación de la red [228](#)

ciclo de vida de la conexión [229](#)

 estableciendo [229](#), [230](#)

 terminación [231](#), [232](#)

 Utilización y agrupación [230](#), [231](#)

Gestión de conexiones [219](#), [220](#)

 configuración del sistema operativo [241](#), [242](#)

 optimización [239](#)

 optimización de pool [239](#)

 Optimización del lado del servidor [240](#), [241](#)

 configuración de tiempo de espera [240](#)

Monitoreo y agrupación de conexiones (CMAP) [236](#)

API conveniente [184](#)

diseño de copia en escritura (CoW) [285](#)

API principal [182](#), [183](#)

consulta cubierta [80](#)

Operaciones CRUD [198](#)

Estudio de caso sobre la investigación de fugas del cursor [335](#)

causa, diagnóstico [335](#) - [337](#) ____

resultados y aprendizajes [338](#)

identificación de la causa raíz [337](#)

solución, implementando [337](#)____

D

perfilador de base de datos [310](#), [311](#)

Sistema de [315](#), [316](#)

servicios de datos Datadog [18](#), [19](#)

componentes [18](#), [19](#)

bibliotecas [21](#), [22](#)

motor de consulta [20](#)

motor de almacenamiento [21](#)

componentes del sistema [22](#)

Django [214](#)

atomicidad a nivel de documento

versus transacciones ACID multidocumento [178](#) - [180](#) ____

Patrón de versiones documentos [48](#) [50](#)

conductores [198](#)

Características [200](#), [201](#)

trabajando [198](#), [199](#)

mi

índices efectivos, diseño [73](#)

orden de índice ascendente versus descendente [81](#) ____

cardinalidad y selectividad [73](#) [____](#), [74](#)
índices compuestos, construcción [74](#) [____](#), [75](#)
consultas cubiertas [80](#) [____](#), [81](#)
consultas de igualdad [75](#), [76](#)
Directriz ESR [79](#) [____](#)
canalizaciones de indexación y agregación [81](#)
recursos, maximizando con índices parciales [80](#) [____](#)
consultas de ordenamiento y rango [76](#) - [79](#)
patrón de incrustación [43](#) [____](#), [44](#)
Entity Framework [214](#)
Directriz de igualdad, ordenación y rango (ESR) [73](#)
Sección de las estadísticas de ejecución [253](#)
patrón de referencia extendido [44](#)
Integración del sistema de monitoreo externo [312](#)
Datadog [315](#) [____](#), [316](#)
Prometeo + Atlas [313](#) [____](#), [314](#)
Prometeo + Grafana [313](#) [____](#)
visualizando, con Grafana [314](#) [____](#), [315](#)
extraer, transformar y cargar (ETL) [110](#)

F

condiciones de falla

manejo [217](#) [____](#), [218](#)
FastAPI [214](#)

sistemas de archivos

Sistema de archivos de árbol B (Btrfs) [285](#)

Sistema de archivos Ceph (CephFS) [285](#)

sistema de archivos de extensión

(XFS) [285](#) cuarto sistema de archivos extendido (ext4) [285](#)

Sistema de archivos de red (NFS) [285](#)

Sistema de archivos Zettabyte (ZFS) [285](#)

configuración del sistema de [286](#)

archivos configuración de caché, [288](#)

cambiar [287](#) cifrado y compresión dobles, evitar [290](#)

RAID, evitando

configuraciones de lectura anticipada [286](#)

ajustando el uso de memoria residente [287](#), manteniendo por debajo del 80% [291](#)

Estado del SSD, comprobación [290](#)

comando findAndModify primero en [266](#)

entrar, primero en salir (FIFO) [112](#)

Matraz [214](#)

Captura de datos de diagnóstico a tiempo completo (FTDC) [349](#)

GRAMO

índices geoespaciales [271](#)

Grafana

visualizando con [314](#), [315](#)

H

limitaciones de recursos de hardware

Dirigiéndose [a 342](#)

Estudio de caso [343](#) sobre optimización de recursos del clúster [Atlas](#)

Estudio de caso [349](#) sobre sistemas autogestionados

fragmentación hash [131](#)

alta disponibilidad [106](#)

directiva de sugerencia [259](#)

escala horizontal [122](#)

|

índices [65](#) [67](#)

conceptos erróneos [__](#)

[69](#) eficiencia de los recursos y [__](#)

compensaciones [67](#) [__](#)

uso de recursos [35](#)

[68](#) indexación [69](#)

tipos de índice [índices](#) [__](#), [71](#)

compuestos [70](#) [__](#)

índice multiclave [71](#) [__](#)

índices parciales [72](#) [índices](#)

de campo único [70](#) [índices](#) dispersos [71](#)

índices comodín [72](#) [__](#)

operaciones inherentemente lentas [326](#)

estudios de caso [326](#) - [334](#)

Identificando [326](#)

operaciones de entrada/salida por segundo (IOPS) [19](#), [35](#)

Yo

Notación de objetos de JavaScript (JSON) [14](#)

Yo

Laravel [214](#)

grandes conjuntos de datos

límites de la canalización de agregación [97](#)

Restricciones de memoria, gestión con allowDiskUse [97](#), [____](#), [98](#)

trabajo con [97](#) [__](#)

latencia [227](#)

bibliotecas [21](#), [22](#)

análisis de mensajes de registro, ejecución de consultas

realizando [255](#) [__](#)

patrones problemáticos, identificando [257](#)

relación de consulta-segmentación [257](#) [__](#), [258](#)

almacenamiento [258](#)

METRO

Métrica de IOPS de disco máximo [26](#)

métricas [299](#)

métricas operativas [299](#) [__](#), [300](#)

métricas específicas de rendimiento [301](#)

MongoDB

componentes arquitectónicos [16](#), [17](#)

arquitectura	14
modelo de documento	14 , 15
Marco de agregación de MongoDB	84
consideraciones de rendimiento	86
canalización	84 - 86
flujo de canalización	
	87 canalizaciones, optimización 88 - 90
Atlas de MongoDB	
alertas	304
Monitoreo con	301
Funciones de la interfaz de usuario	301
Conexión MongoDB, arquitectura	232
conductor, pooling	234 , 235
Protocolo de cable MongoDB	232 - 234
monitoreo y solución de problemas	236 - 238
monitoreo, mejores prácticas	238
manejo de conexión al servidor	235
Sockets TCP/IP	232 - 234
Proceso electoral de MongoDB	107
Lenguaje de consulta MongoDB (MQL)	71
Proceso de conmutación por error del conjunto de réplicas de MongoDB	106
Diseño de esquema de MongoDB	37
cachés	38
atributos dinámicos	38
fortalezas clave	37

relaciones de uno a muchos [37](#)

optimización, para casos de uso comunes [39](#)

evolución del esquema [39](#)

instantáneas [38](#)

entidades débiles, incrustando [38](#)

Transacción de MongoDB

Consideraciones de rendimiento [186 - 190](#)

Límites de tiempo de ejecución y errores, gestión [188](#), [189](#)

Protocolo conexión MongoDB [232](#) [234](#)

mongostat [307](#), [308](#)

mongotop [308](#)

monitoreo [297](#)

Semántica MQL [260](#)

agregación y consulta [267](#) _____, [268](#)

agregación, versus expresiones de coincidencia [269](#)

Mejores prácticas [265 - 267](#)

desafíos, con matrices e índices multiclave [261](#) _____

desafíos, con nulo y \$exists [264](#) _____

deduplicación [263](#) _____, [264](#)

escenarios [261 - 263](#) _____

Transacciones ACID multidocumento [176](#)

Historia y evolución, en MongoDB [176](#) _____

propiedades, en MongoDB [177](#)

Uso, consideraciones [180 - 182](#) _____

versus atomicidad a nivel de documento [178 - 180](#)

índice multiclave [71](#)

control de concurrencia multiversión (MVCC) [146](#)

norte

Información sobre espacios de nombres [303](#)

compresión de red [222](#), [223](#)

beneficios y compensaciones [242](#), [243](#)

opciones del algoritmo de compresión [243](#)

Implementando [244](#)

Optimización del rendimiento, aprovechando [242](#)

Estrategias para entornos sin servidor [244](#), [245](#)

Sistema de archivos de red (NFS) [285](#)

rendimiento de la red [222](#), [223](#)

saturación de la red [228](#)

patrones sin bloqueo [216](#)

Oh

mapeadores de objetos y documentos (ODM) [197](#), [202](#)

beneficios [210](#)

validación de datos [204](#), [205](#)

características [203](#)

Impacto en la productividad del desarrollador [209](#)

API de consulta intuitiva [205](#), [206](#)

ganchos de ciclo de vida [207](#)

middleware [207](#)

gestión de relaciones [206](#) [_____](#), [207](#)
escenarios [210](#) [_____](#)
cumplimiento del esquema [204](#) [_____](#), [205](#) [_____](#)
seguridad de tipos e integración IDE [208](#) [_____](#), [209](#) [_____](#)
mapeadores objeto-relacionales (ORM) [202](#) [_____](#), [203](#) [_____](#)
observabilidad [298](#) [_____](#)
ejemplos [298](#) [_____](#)
relaciones de uno a muchos [37](#) [_____](#)
OpenTelemetry [317](#) [_____](#), [318](#) [_____](#)
métricas operativas [299](#) [_____](#)
Estadísticas de caché [300](#) [_____](#)
conexiones [300](#) [_____](#)
Métricas de E/S de disco [300](#) [_____](#)
uso de memoria MB [300](#) [_____](#)
contadores de operaciones [300](#) [_____](#)
métricas de replicación [300](#) [_____](#)
técnicas de optimización [90](#) [_____](#)
Operaciones de grupo, diseño [92](#) [_____](#), [93](#) [_____](#)
Problemas de rendimiento de búsqueda, cómo evitarlos [94](#) - [96](#) [_____](#)
\$project y \$addFields, uso eficiente [96](#) [_____](#)
filtrado de datos [90](#) [_____](#), [91](#) [_____](#)
innecesarios \$unwind y \$group, evitando [92](#) [_____](#)
sin memoria (OOM) [291](#) [_____](#)

índices parciales [72](#), [73](#)

patrones, para optimización de consultas y análisis [53](#)

patrón calculado [53](#) - [55](#) _____

patrón polimórfico [56](#) patrón _____, [57](#)

de control de versiones de esquema [55](#), [56](#)

patrones, para optimizar el rendimiento de lectura [43](#)

patrón de incrustación [43](#) _____, [44](#)

patrón de referencia extendido [44](#) _____, [45](#)

patrón de subconjunto [45](#) - [47](#)

patrones, para la optimización del almacenamiento [57](#)

patrón de archivo [57](#) _____, [58](#)

patrones, para optimizar el rendimiento de escritura [48](#)

Patrón de agrupamiento [50](#) _____, [51](#)

Patrón de control de versiones de documentos [48-50](#)

patrón de prefijo de clave [52](#) _____, [53](#)

Asesor de rendimiento [303](#)

desafíos de rendimiento

presión de caché [321](#) _____

cuellos de botella de E/S de disco [320](#)

Desviación de la ventana de registro de operaciones [322](#)

retraso de replicación [322](#)

métricas específicas de rendimiento [301](#)

herramientas de rendimiento [24](#)

cuellos de botella, encontrando [25](#) - [28](#)

proceso incremental, para optimización [29](#) _____, [30](#)

solución de problemas de rendimiento

enfoque sistemático [350](#) [_____](#), [351](#)

patrón polimórfico [compresión](#), [56](#), [57](#)

de prefijo [76](#)

prefijo coincidente [75](#)

Q

configuración de calidad de servicio (QoS) [292](#)

Quarkus [214](#)

motor de consulta [20](#), [21](#)

ejecución de consultas [248](#) [250](#)

Sección [253](#) de las estadísticas de ejecución [_____](#), [254](#)

Explicar el comando, usando [252](#) [_____](#)

influyendo [259](#) [_____](#)

opciones de influencia [260](#) [_____](#)

influir, con insinuación directiva [259](#) [_____](#)

Influir, con configuración de consulta [259](#) [_____](#)

mensajes de registro, analizando [255](#) [_____](#)

Planificación [250](#), [251](#)

consultaPlanificador sección [253](#)

Sección [_____](#) de queryPlanner [253](#)

Generador de perfiles de consultas [304](#)

forma de consulta [248](#)

Relación de consulta-segmentación [257](#), [258](#)

R

fragmentación basada en rangos [131](#)

Estrategias de lectura y escritura [114](#)

preferencia de lectura [115](#) - [117](#)

escribir preocupación [117](#) - [119](#)

leer preocupaciones [221](#)

preferencias de lectura [104](#), [221](#)

Panel de rendimiento en tiempo real (RTPP) [24](#), [302](#), [303](#)

objetivo de punto de recuperación (RPO) [168](#)

tiempo de acceso relativo (relatime) [286](#)

configuración del conjunto de réplicas [107](#)

Nodos de análisis [109](#), [110](#)

Replicación encadenada [108](#), [109](#)

Etiquetas [109](#) [110](#)

conjuntos de [16](#), [103](#)

réplicas nodo árbitro [104](#)

componentes [104](#), [105](#)

nodo primario [104](#)

nodos secundarios [104](#)

replicación [104](#), [105](#)

control de flujo [111](#)

Consideraciones internas y de rendimiento [111](#)

registro de operaciones (oplog) [113](#)

[112](#) retraso de replicación, gestión [114](#)

gestión de recursos, para un rendimiento óptimo

Utilización de la CPU [277](#)

gestión de memoria [277](#) - [279](#) _____

red [282](#) _____

almacenamiento [279](#) - [281](#)

Ruby on Rails [214](#)

S

operación de dispersión-reunión [124](#)

limitaciones [124](#) _____, [125](#) _____

diseño de esquema [34](#)

mitos comunes [36](#) _____

principios básicos [34](#) _____

colocación de datos [34](#) _____, [35](#) _____

compensación entre lectura y escritura [35](#)

documentos pequeños versus grandes [36](#) _____

patrones de diseño de esquemas [42](#)

para la optimización de consultas y análisis [53](#) _____

para optimizar el rendimiento de lectura [43](#) _____

para la optimización del almacenamiento [57](#)

para optimizar el rendimiento de escritura [48](#) _____

validación del esquema [39](#)

Errores comunes en el diseño de esquemas: [40](#) _____, [42](#) _____

beneficios prácticos : [39](#) _____

Patrones de diseño de esquemas, por beneficio [42](#)

patrón de control de versiones de esquema [55](#) _____, [56](#) _____

Estudio de caso de MongoDB autogestionado

causa, diagnóstico [349](#) _____

Resultados y aprendizajes [350](#)_____

solución, implementando [349](#) _____

herramientas de monitorización autogestionadas [307](#)

perfilador de base de datos [310](#) - [312](#)

mongostat [307](#)_____, [308](#)

mongotop [308](#)_____

Estado del servidor [309](#)

clúster fragmentado

componentes arquitectónicos [122](#) _____, [123](#)

fragmentación [131](#)

fragmentación hash [131](#)_____

fragmentación basada en rangos [131](#)

fragmentación basada en zonas [132](#)

arquitectura de fragmentación [122](#)

selección de clave fragmento [123](#) [125](#)

para operaciones de selección de objetivos [126](#)

Los valores de clave de fragmento aumentan o disminuyen, evitando la _____, [130](#)

clave de fragmento [129](#) , con buena granularidad [127](#), [128](#)

índices de campo único [70](#)

INTELIGENTE [29](#)

ágil [243](#)

Sistema de chat de la [42](#) , [59](#)

aplicación Socialite [60](#) - [63](#)

Perfil de usuario y feed de actividad [59](#), [60](#)

sistema de software [10](#), [11](#)

eficiencia algorítmica [11](#) —

Ley [12](#) de Amdahl

Puerta de enlace API [10](#)

almacenamiento en caché [12](#)

Ley [13](#) de Little

localidad [12](#)

cuellos de botella de rendimiento —, [11](#)

[10](#) optimización prematura, evitar [12](#) —

índices dispersos [71](#)

Datos de primavera MongoDB [213](#)

motores de almacenamiento [21](#), [144](#)

motor de almacenamiento orientado a columnas [144](#)

motor de almacenamiento de clave-valor [144](#)

motor de almacenamiento orientado a filas [144](#)

WiredTiger [145](#) —

acuerdos de nivel de servicio (SLA) estrictos [35](#)

patrón de subconjunto [45](#) - [47](#)

Symfony [214](#)

Configuración del sistema para el rendimiento de MongoDB

Procesos que consumen mucha CPU [284](#)

sistema de archivos, seleccionando [284](#)

configuración del sistema de archivos [286](#)

Mejores prácticas de redes [292](#) —, [293](#)

operaciones simultáneas por núcleo de CPU [283](#)
sistemas [2](#)
características [3](#) - [9](#) -

T

Sockets TCP/IP [232](#) [234](#)
características [232](#) , [233](#)
colecciones de series temporales [270](#)
tiempo de vida (TTL) [292](#)
actas
antipatrones y sus costos de rendimiento [190](#) - [195](#) -
transacciones, enfoques
API de devolución de llamada [184](#)
API principal [182](#) , [183](#)
Control de preocupaciones de lectura y escritura [185](#)

Tú

Consulta de administrador inesperada que provoca el escaneo de colección [332](#)
causa, diagnóstico [333](#)
resultados y aprendizajes
[334](#) solución, implementación [334](#)

V

escala vertical [122](#)

Unidades centrales de procesamiento virtuales (vCPU) [19](#)
 máquinas virtuales (VM) [22](#)
nubes privadas virtuales (VPC) [292](#)

O

índices comodín [72](#)

WiredTiger [21](#), [144](#)

 Consideraciones sobre la caché [187](#)

 puntos de control [149](#)

 compresión [149](#)

 opciones de configuración

 150 cifrado [149](#)

 desalojo [148](#)

 operación de inserción [148](#)

 operaciones de búsqueda [147](#)

 visión general [145](#)

 recuperación [149](#)

 operación de actualización [147](#)

Opciones de configuración de WiredTiger

 tamaño de caché, cambiando [151](#)

 desalojo, trabajando [154](#)

 motor de almacenamiento inMemory, cambiando a [155](#), [156](#)

 tamaño máximo de página de hoja, [157](#)

 cambiando [156](#) minSnapshotHistoryWindowInSeconds, cambiando

[153](#) retardo de sincronización, cambiando [152](#)

registro de escritura anticipada (WAL) [21](#), [146](#)

Escribe la preocupación [104](#), [117](#), [221](#)

durabilidad [117](#) - [119](#)

Z

Sistema de archivos Zettabyte (ZFS) [285](#)

zlib [243](#)

fragmentación basada en zonas [132](#)

zstd [243](#)



packtpub.com

Suscríbete a nuestra biblioteca digital en línea para acceder a más de 7000 libros y videos, además de herramientas líderes en la industria que te ayudarán a planificar tu desarrollo personal y a impulsar tu carrera profesional. Para más información, visita nuestro sitio web.

¿Por qué suscribirse?

- Dedique menos tiempo a aprender y más tiempo a codificar con libros electrónicos y videos prácticos de más de 4000 profesionales de la industria.
- Mejora tu aprendizaje con planes de habilidades diseñados especialmente para ti
- Obtenga un libro electrónico o un vídeo gratuito cada mes
- Completamente buscable para un fácil acceso a información vital
- Copiar y pegar, imprimir y marcar contenido

En www.packtpub.com También puede leer una colección de artículos técnicos gratuitos, suscribirse a una variedad de boletines gratuitos y recibir descuentos y ofertas exclusivas en libros y libros electrónicos de Packt.

Otros libros que te pueden gustar

Si te ha gustado este libro, puede que te interesen estos otros libros de Packt:



Arquitecturas para la empresa inteligente preparada para la IA

Boris Bialek, Sebastián Rojas Arbulu, Taylor Hedgecock

ISBN: 978-1-80611-715-4

- Diseñe arquitecturas de datos preparadas para IA que escalen en producción
- Definir sistemas de acción y explicar por qué son importantes para las empresas.
- Modernice los sistemas heredados para lograr arquitecturas unificadas y preparadas para la IA

- Implementar marcos de gobernanza, privacidad y cumplimiento para la IA
- Explore implementaciones de IA en el mundo real para más de seis industrias
- Implementar sistemas RAG y agentic de producción con MongoDB
- Aplicar la protección de datos semánticos en industrias reguladas
- Cree agentes de IA específicos del dominio y copilotos inteligentes
- Aplicar MCP, IA causal y sistemas multiagente para arquitecturas preparadas para el futuro



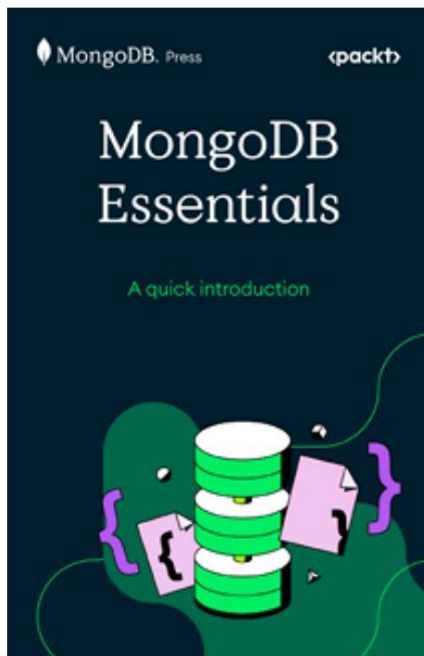
La guía oficial de MongoDB

Rachelle Palmer, Jeffrey Allen, Parker Faucher, Alison Huh, Lander Kerbey, Maya Raman, Lauren Tran

ISBN: 978-1-83702-197-0

- Cree aplicaciones seguras, escalables y de alto rendimiento
- Diseñe modelos de datos e índices eficientes para cargas de trabajo reales
- Escriba consultas potentes para ordenar, filtrar y proyectar datos

- Proteja las aplicaciones con autenticación y cifrado
- Acelere la codificación con herramientas basadas en IDE e impulsadas por IA
- Lanza, escala y administra MongoDB Atlas con confianza
- Desbloquear la búsqueda de Atlas y la búsqueda de vectores de Atlas
- Aplique técnicas probadas de los propios líderes de ingeniería de MongoDB



Fundamentos de MongoDB

ISBN: 978-1-80670-609-9

- Comprender el modelo y la arquitectura de documentos de MongoDB
- Configure implementaciones locales de MongoDB rápidamente
- Esquemas de diseño adaptados a los patrones de acceso de las aplicaciones
- Realice operaciones CRUD y de agregación de manera eficiente
- Utilice herramientas para optimizar el rendimiento y la escalabilidad de las consultas
- Explora funciones impulsadas por IA, como Atlas Search y Atlas Búsqueda de vectores

Packt está buscando autores como tú. Si estás interesado en

convertirte en autor de Packt, visita authors.packt.com Postúlate hoy mismo. Hemos trabajado con miles de desarrolladores y profesionales de la tecnología como tú para ayudarles a compartir sus conocimientos con la comunidad tecnológica global. Puedes enviar una solicitud general, postularte para un tema de actualidad específico para el que estemos reclutando autores o enviar tu propia idea.

Comparte tus pensamientos

Ahora que has terminado " Alto Rendimiento con MongoDB" , ¡nos encantaría conocer tu opinión! Si compraste el libro en Amazon, haz [clic aquí para ir directamente a la página de reseñas de Amazon](#) y compartir tu opinión o dejar una reseña en el sitio web donde lo compraste.

Su reseña es importante para nosotros y para la comunidad tecnológica, y nos ayudará a garantizar que ofrecemos contenido de excelente calidad.

Descargue una copia gratuita en PDF de este libro

¡Gracias por comprar este libro!

¿Te gusta leer mientras viajas pero no puedes llevar tus libros impresos a todas partes?

¿Tu compra de libro electrónico no es compatible con el dispositivo que eliges?

No te preocupes, ahora con cada libro de Packt obtendrás una versión PDF de ese libro sin DRM sin costo.

Lee en cualquier lugar, en cualquier dispositivo. Busca, copia y pega código de tus libros técnicos favoritos directamente en tu aplicación.

Los beneficios no terminan ahí, puedes obtener acceso exclusivo a descuentos, boletines informativos y excelente contenido gratuito en tu bandeja de entrada todos los días.

Siga estos sencillos pasos para obtener los beneficios:

1. Escanea el código QR o visita el siguiente enlace:



<https://packt.link/libro-e-gratuito/9781837022632>

2. Envíe su comprobante de compra.
3. ¡Listo! Te enviaremos tu PDF gratuito y otros beneficios directamente a tu correo electrónico.